

SE4040 Enterprise Application Development

Assignment 1

Smart Solar Microgrid Trading System

Client-server application: web application, native Android application and web service

No.	Name	IT number	Module
M1	De Silva R K D H	IT22001252	Identity & Accounts, Android app shell
M2	Asath M M	IT22633422	Microgrid Nodes, Slots & Maps, IIS hosting
M3	Janukshan S	IT22635266	Energy Reservations & QR Dispatch
M4	Herath D M S T	IT22639776	Booking Views, Dashboards & Operator Verification

Git repository: <https://github.com/DilnukDeSilva/solar-microgrid-trading-system>

Demonstration video: [\[add YouTube / OneDrive link\]](#)

BSc (Hons) in Information Technology Specialising in Software Engineering · Year 4 Semester 2 · 2026

Contents

Right-click the table and choose Update Field if page numbers are missing.

1. Introduction

The Smart Solar Microgrid Trading System lets owners of solar panel arrays (prosumers) reserve energy drop-off and charging slots at shared solar hubs (microgrid nodes). A Backoffice team registers the hubs and keeps their schedules; grid operators keep battery slot availability up to date, approve bookings and complete energy transfers after scanning the prosumer's QR code.

The system follows a client-server design with three parts, as the brief requires:

- **Web service:** a C# ASP.NET Core Web API hosted on Windows IIS with a MongoDB database. It is a FAT service: every business rule lives here.
- **Web application:** an ASP.NET Core MVC application styled with Bootstrap 5, used by Backoffice and Grid Operator staff.
- **Mobile application:** a pure native Android application in Java with a local SQLite database, used by prosumers and grid operators. It shows nearby hubs on Google Maps and scans QR codes with ZXing.

Both clients are user interfaces only. They call the web service over REST/JSON and show its answers, including every rule message, word for word.

1.1 Business rules enforced by the web service

Rule	Where it is enforced	Error returned
Prosumers are identified by NIC; new registrations start as Pending until Backoffice activates them	ProsumerService, AuthService	NIC_EXISTS, ACCOUNT_NOT_ACTIVE
Only Backoffice can reactivate a deactivated prosumer	ProsumersController role check	403 FORBIDDEN
A station cannot be deactivated while it has future Pending or Approved reservations	StationService.DeactivateAsync	409 STATION_HAS_RESERVATIONS
Slots must be in the future, inside the station's opening hours, and may not exceed its battery count when they overlap	SlotService.RequireValidSlot	400 VALIDATION_ERROR, 409 SLOT_OVERLAP
A booked slot cannot be moved, deleted, or reopened	SlotService	409 SLOT_TAKEN
Reservations must be within the next 7 days	ReservationService	RULE_7DAYS
Updates and cancellations need at least 12 hours' notice	ReservationService	RULE_12H
A QR token is issued on approval and can be used once	ReservationService, OperatorService	QR_INVALID, QR_ALREADY_USED

Table 1. Business rules and where they live

1.2 Technologies

Layer	Technology
Web service	C# / ASP.NET Core Web API (.NET 10), MongoDB.Driver 3.12, JWT bearer authentication, Swagger
Hosting	Windows 11, IIS with the ASP.NET Core Module V2 (.NET 10 Hosting Bundle), app pools set to No Managed Code
Database	MongoDB Community (server); SQLite on the Android device
Web application	ASP.NET Core MVC (Razor views), Bootstrap 5.3, jQuery validation
Mobile application	Java, Android SDK 24+, Material Components, Google Maps SDK, Fused Location Provider, ZXing Android Embedded
Tools	Git and GitHub (feature branches and pull requests), Visual Studio / Rider, Android Studio, PlantUML

2. Application high-level diagram

Figure 1 on the next page shows the whole system. The explanation follows below.

The web application and the Android application are thin clients. They send REST requests with a JWT to the web service on IIS and never reach MongoDB directly. Inside the service, controllers only map HTTP to calls, services hold every business rule and throw an `ApiException` with a contract error code, and repositories are the only classes that talk to MongoDB. A global middleware turns any `ApiException` into the JSON body `{ code, message }`, which both clients display as it is.

The Android application keeps a local SQLite database for the session, the prosumer's profile, their own reservations (so the QR code opens without signal), cached booking history and dashboard counts, and cached stations for the offline map. SQLite is a cache only; MongoDB remains the source of truth.

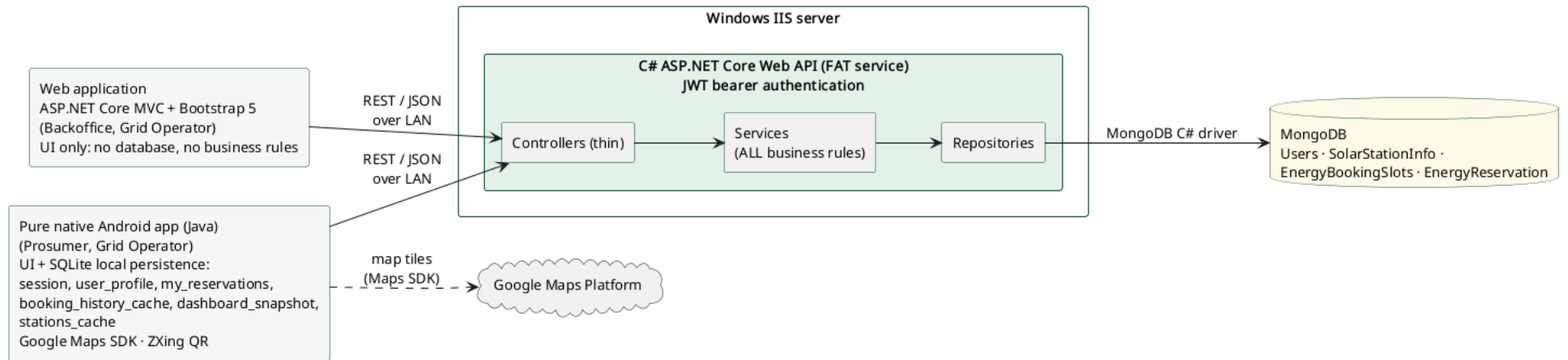


Figure 1. Application high-level diagram

3. Use case diagram

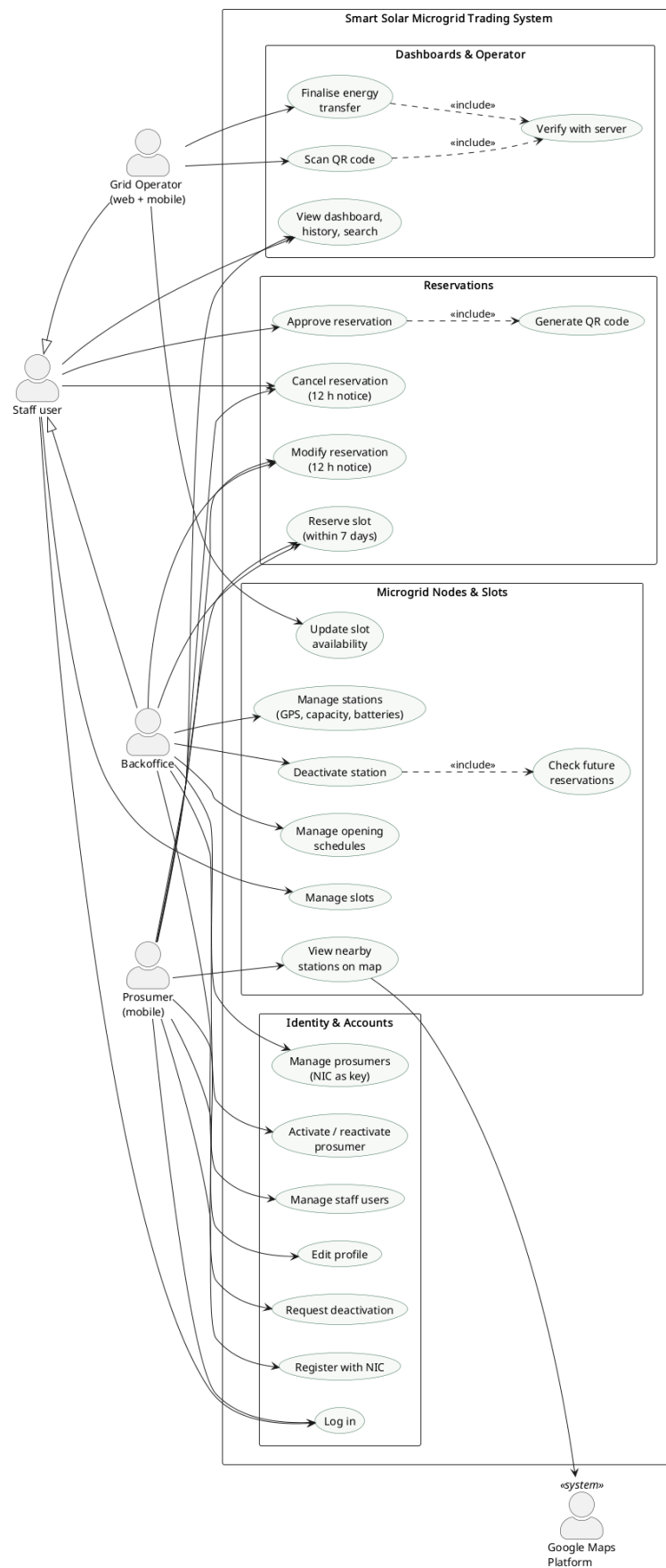


Figure 2. Use case diagram

Actor	Client	Main goals
Backoffice	Web	Manage staff users and prosumer accounts, activate and reactivate prosumers, manage stations and schedules, manage and approve reservations
Grid Operator	Web and Android	Update slot availability, monitor and approve bookings, scan a prosumer's QR code, verify it with the server and finalise the energy transfer
Prosumer	Android	Register with NIC, edit profile, request deactivation, find nearby hubs on the map, reserve, modify and cancel slots, show the QR code, view dashboard and history
Google Maps Platform	External system	Provides the map tiles for the nearby stations screen

Table 2. *Actors*

4. Data flow diagrams

4.1 Level 0 (context diagram)

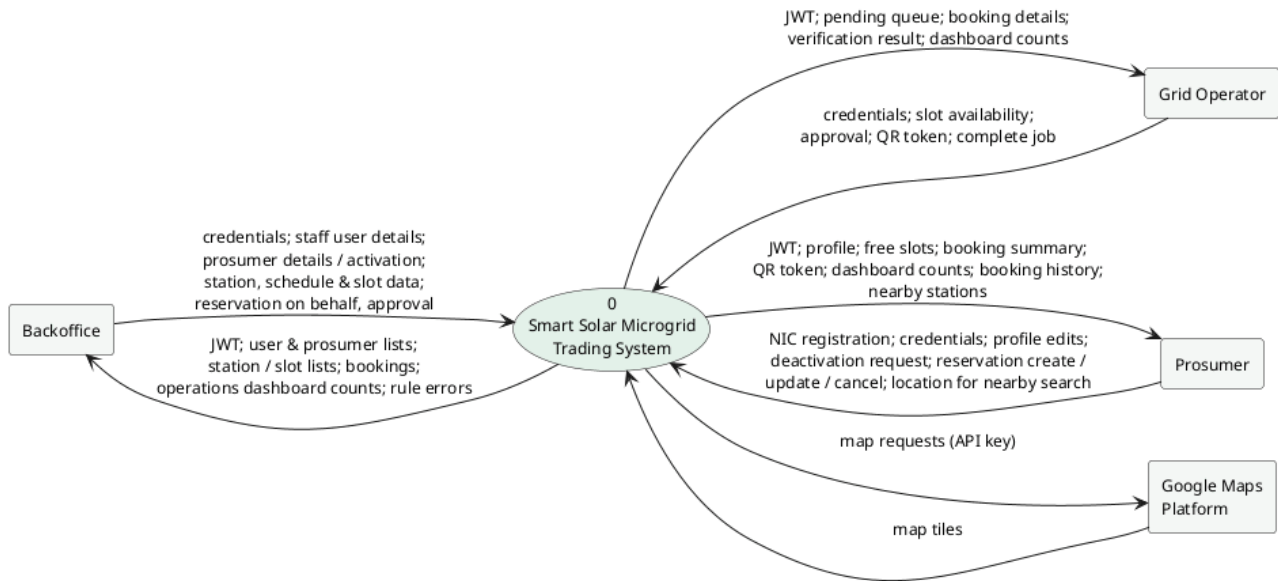


Figure 3. DFD level 0

At level 0 the whole system is one process. Staff and prosumers send credentials, account details, station and slot data, reservation requests and QR tokens, and receive JWTs, lists, dashboard counts, reservation summaries and verification results.

4.2 Level 1

The level 1 diagram is on the next page (landscape).

Process	Owner	Reads	Writes
1.0 Authenticate & manage accounts	M1	D1 Users	D1 Users
2.0 Manage stations, slots & nearby search	M2	D2, D3, D4 (future Pending/Approved check)	D2 SolarStationInfo, D3 EnergyBookingSlots
3.0 Manage reservations & QR	M3	D1 (prosumer status), D2 (station status), D3	D3 (claim/release slot), D4 EnergyReservation
4.0 Dashboards, booking views & operator verification	M4	D2, D4	D4 (mark Completed)

Table 3. Level 1 processes and data stores

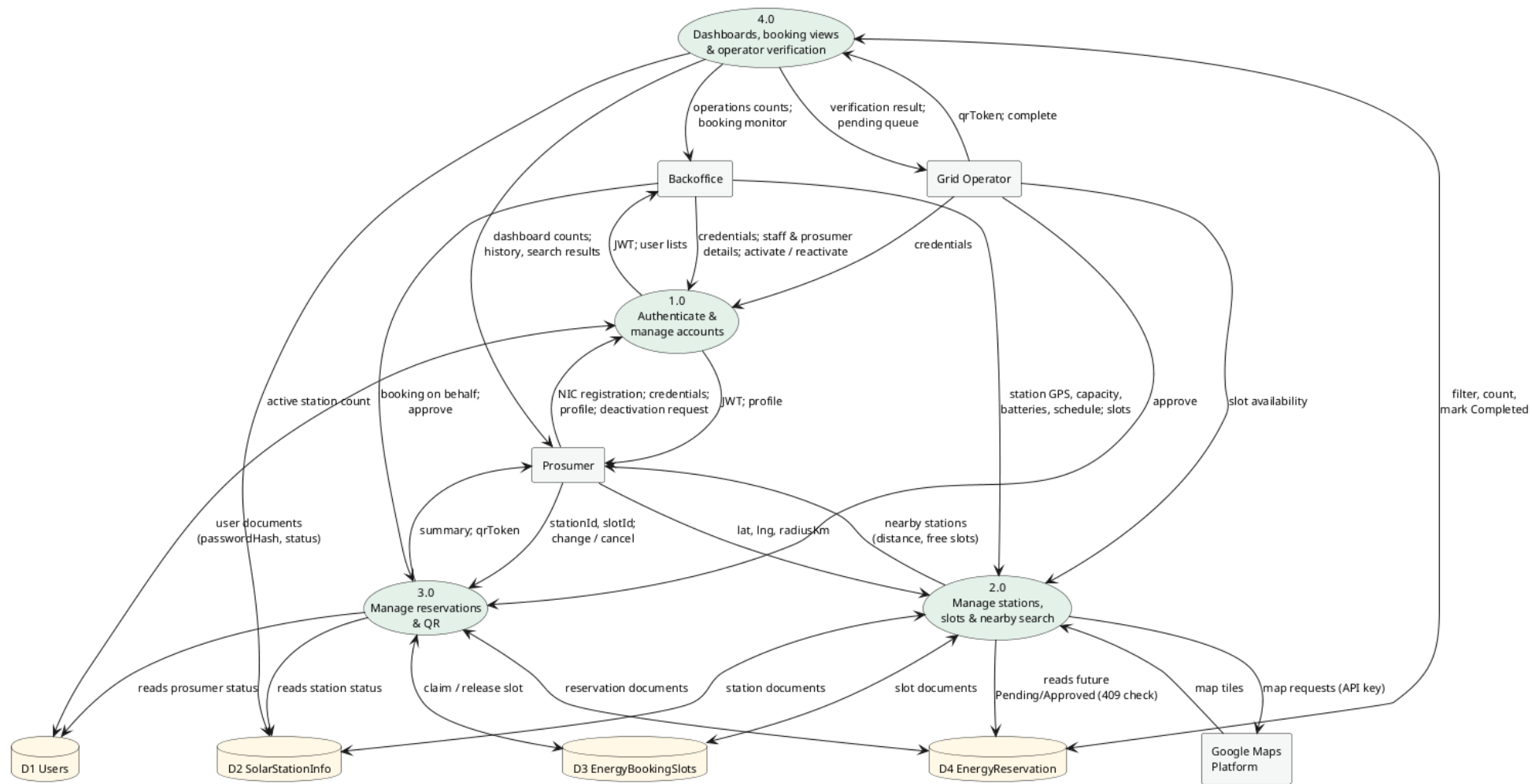


Figure 4. DFD level 1

5. Database design

SmartSolar (MongoDB). Element names are camelCase via CamelCaseElementNameConvention; unknown elements ignored (IgnoreExtraElementsConvention). Source: WebService/SmartSolar.Api/Models, Data/MongoContext.cs, Data/MongoIndexInitializer.cs @ origin/dev.

The document model diagram is on the next page (landscape); each collection is described after it.

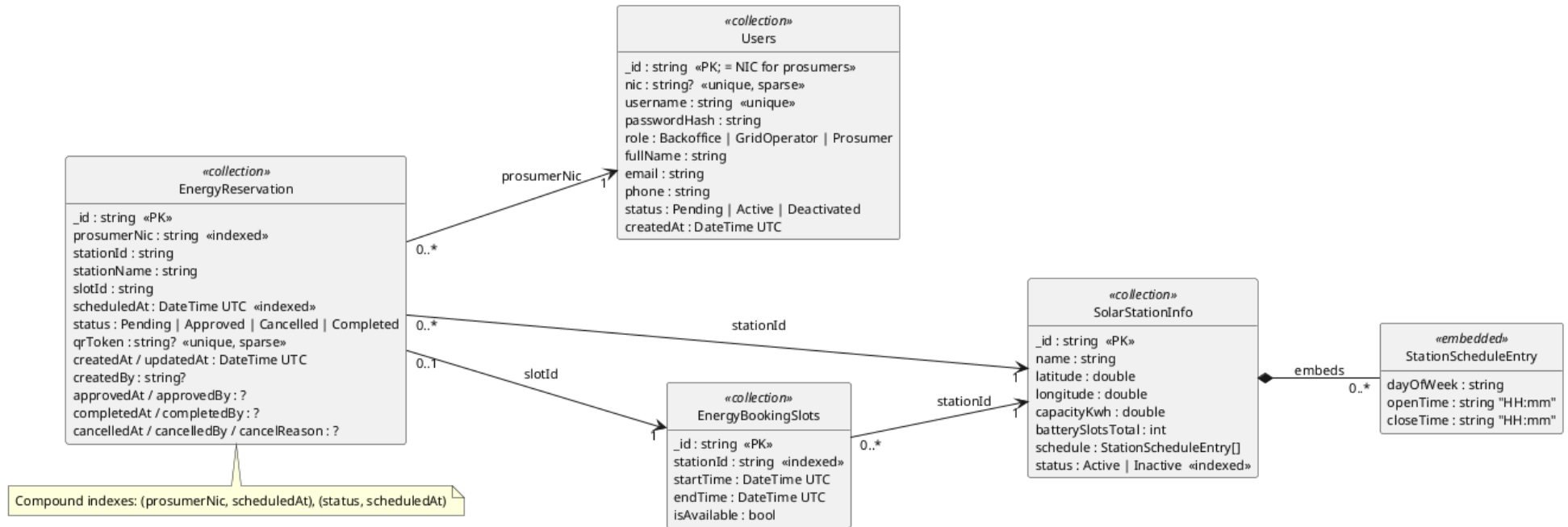


Figure 5. MongoDB document model

5.1 Users

Staff (Backoffice, GridOperator) and Prosumer accounts. Prosumers are keyed by NIC.

Field	Type	Notes
_id	string	BsonId, string representation; equals the NIC for prosumers
nic	string?	BsonIgnoreIfNull; required for Prosumer
username	string	
passwordHash	string	ASP.NET Core Identity PasswordHasher; never returned
role	string	Backoffice GridOperator Prosumer
fullName	string	
email	string	
phone	string	
status	string	Pending Active Deactivated
createdAt	DateTime (UTC)	

Table 4. *Users fields*

Index	Keys	Options
ux_users_nic	nic: 1	unique, sparse
ux_users_username	username: 1	unique

Table 5. *Users indexes*

5.2 SolarStationInfo

Microgrid nodes (solar hubs) with GPS, capacity, battery slots and weekly schedule.

Field	Type	Notes
_id	string	BsonId
name	string	
latitude	double	
longitude	double	
capacityKwh	double	
batterySlotsTotal	int	
schedule	StationScheduleEntry[] (embedded)	{ dayOfWeek: string, openTime: string 'HH:mm', closeTime: string 'HH:mm' }
status	string	Active Inactive

Table 6. *SolarStationInfo fields*

Index	Keys	Options
ix_stations_status	status: 1	

Table 7. *SolarStationInfo indexes*

5.3 EnergyBookingSlots

Bookable time windows per station; isAvailable=false when booked or closed by an operator.

Field	Type	Notes
_id	string	BsonId
stationId	string	→ SolarStationInfo._id
startTime	DateTime (UTC)	
endTime	DateTime (UTC)	
isAvailable	bool	default true

Table 8. *EnergyBookingSlots fields*

Index	Keys	Options
ix_slots_stationId	stationId: 1	

Table 9. *EnergyBookingSlots indexes*

5.4 EnergyReservation

Prosumer energy slot bookings with approval, QR token and completion audit fields.

Field	Type	Notes
_id	string	BsonId
prosumerNic	string	→ Users.nic
stationId	string	→ SolarStationInfo._id
stationName	string	denormalised copy for lists
slotId	string	→ EnergyBookingSlots._id
scheduledAt	DateTime (UTC)	
status	string	Pending Approved Cancelled Completed
qrToken	string?	BsonIgnoreIfNull; single-use token issued on approval
createdAt	DateTime (UTC)	
createdBy	string?	BsonIgnoreIfNull
updatedAt	DateTime (UTC)	
completedAt	DateTime? (UTC)	BsonIgnoreIfNull
completedBy	string?	BsonIgnoreIfNull
approvedAt	DateTime? (UTC)	BsonIgnoreIfNull
approvedBy	string?	BsonIgnoreIfNull
cancelledAt	DateTime? (UTC)	BsonIgnoreIfNull
cancelledBy	string?	BsonIgnoreIfNull
cancelReason	string?	BsonIgnoreIfNull

Table 10. *EnergyReservation fields*

Index	Keys	Options
ix_reservations_prosumerNic	prosumerNic: 1	
ix_reservations_scheduledAt	scheduledAt: 1	
ix_reservations_status	status: 1	
ix_reservations_prosumerNic_scheduledAt	prosumerNic: 1, scheduledAt: 1	

Index	Keys	Options
ix_reservations_status_scheduledAt	status: 1, scheduledAt: 1	
ux_reservations_qrToken	qrToken: 1	unique, sparse

Table 11. EnergyReservation indexes

5.5 Android SQLite (local persistence)

smart_solar.db (SQLite, DB_VERSION 4) on the Android device

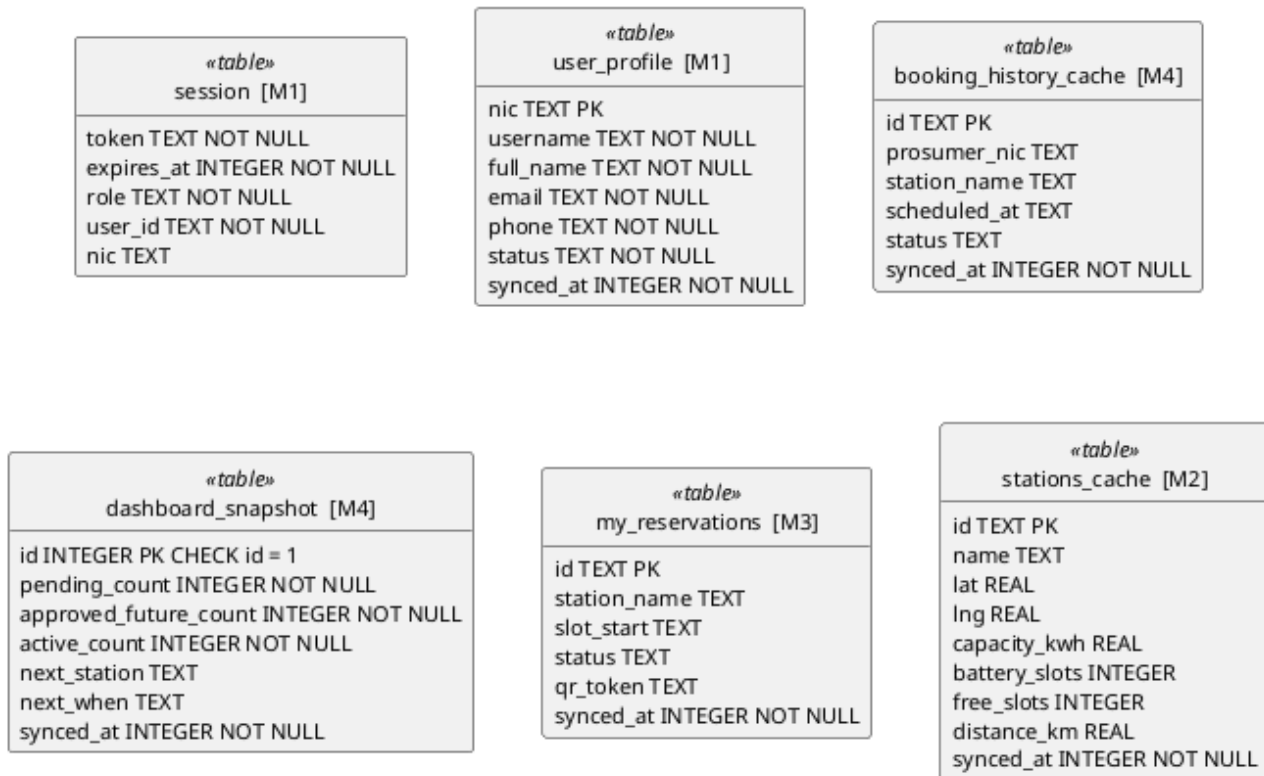


Figure 6. SQLite tables on the Android device

Table	Columns	Owner	Purpose
session	token TEXT NOT NULL, expires_at INTEGER NOT NULL, role TEXT NOT NULL, user_id TEXT NOT NULL, nic TEXT	M1	Keeps the user logged in across restarts; cleared on logout or 401.
user_profile	nic TEXT PRIMARY KEY, username TEXT NOT NULL, full_name TEXT NOT NULL, email TEXT NOT NULL, phone TEXT NOT NULL, status TEXT NOT NULL, synced_at INTEGER NOT NULL	M1	Cached profile shown first, then refreshed from the API.
booking_history_cache	id TEXT PRIMARY KEY, prosumer_nic TEXT, station_name TEXT, scheduled_at TEXT, status TEXT, synced_at INTEGER NOT NULL	M4	Last booking list, shown offline (added in DB version 2).
dashboard_snapshot	id INTEGER PRIMARY KEY CHECK (id = 1), pending_count INTEGER NOT NULL, approved_future_count INTEGER NOT NULL, active_count INTEGER NOT NULL, next_station TEXT, next_when TEXT, synced_at INTEGER NOT NULL	M4	Last prosumer dashboard counts, shown offline (version 2).
my_reservations	id TEXT PRIMARY KEY, station_name TEXT, slot_start TEXT, status TEXT, qr_token TEXT, synced_at INTEGER NOT NULL	M3	Prosumer's own bookings so the QR opens without signal (version 3).
stations_cache	id TEXT PRIMARY KEY, name TEXT, lat REAL, lng	M2	Nearby stations for the offline map

Table	Columns	Owner	Purpose
	REAL, capacity_kwh REAL, battery_slots INTEGER, free_slots INTEGER, distance_km REAL, synced_at INTEGER NOT NULL		(version 4).

Table 12. *SQLite tables*

Each module adds its own table through an additive DbHelper.onUpgrade migration and a DB_VERSION bump, so no module's offline data is dropped on upgrade.

6. Web service endpoints

All routes are under /api and return the contract error body { code, message } on failure. Access is enforced on the server with [Authorize] roles.

Method	Path	Access	Owner
POST	/api/auth/login	anonymous	M1
POST	/api/auth/register	anonymous	M1
GET	/api/dashboard/me	Prosumer	M4
GET	/api/dashboard/operations	Backoffice, GridOperator	M4
GET	/health	anonymous	M1
GET	/api/health	anonymous	M1
GET	/api/me	Prosumer	M1
PUT	/api/me	Prosumer	M1
POST	/api/me/request-deactivation	Prosumer	M1
POST	/api/reservations/verify-qr	GridOperator	M4
POST	/api/reservations/{id}/complete	GridOperator	M4
GET	/api/prosumers	Backoffice, GridOperator	M1
GET	/api/prosumers/pending	Backoffice	M1
GET	/api/prosumers/{nic}	Backoffice	M1
POST	/api/prosumers	Backoffice	M1
PUT	/api/prosumers/{nic}	Backoffice	M1
POST	/api/prosumers/{nic}/deactivate	Backoffice	M1
POST	/api/prosumers/{nic}/reactivate	Backoffice	M1
POST	/api/prosumers/{nic}/activate	Backoffice	M1
GET	/api/reservations	Prosumer, Backoffice, GridOperator	M4
GET	/api/reservations/pending	Backoffice, GridOperator	M4
GET	/api/reservations/bookable-stations	Prosumer, Backoffice, GridOperator	M3
GET	/api/reservations/available-slots	Prosumer, Backoffice, GridOperator	M3
POST	/api/reservations	Prosumer, Backoffice, GridOperator	M3
GET	/api/reservations/{id}	Prosumer, Backoffice, GridOperator	M3
PUT	/api/reservations/{id}	Prosumer, Backoffice, GridOperator	M3
POST	/api/reservations/{id}/cancel	Prosumer, Backoffice, GridOperator	M3
POST	/api/reservations/{id}/approve	Backoffice, GridOperator	M3
GET	/api/stations/{stationId}/slots	any authenticated user	M2
POST	/api/stations/{stationId}/slots	Backoffice, GridOperator	M2
PUT	/api/slots/{id}	Backoffice, GridOperator	M2
DELETE	/api/slots/{id}	Backoffice, GridOperator	M2
PUT	/api/slots/{id}/availability	GridOperator	M2
GET	/api/stations	any authenticated user	M2
GET	/api/stations/nearby	any authenticated user	M2
GET	/api/stations/{id}	any authenticated user	M2
POST	/api/stations	Backoffice	M2
PUT	/api/stations/{id}	Backoffice	M2
POST	/api/stations/{id}/deactivate	Backoffice	M2

Method	Path	Access	Owner
POST	/api/stations/{id}/activate	Backoffice	M2
GET	/api/users	Backoffice	M1
GET	/api/users/{id}	Backoffice	M1
POST	/api/users	Backoffice	M1
PUT	/api/users/{id}	Backoffice	M1
POST	/api/users/{id}/deactivate	Backoffice	M1

Table 13. *API endpoints*

7. Screenshots of all user interfaces

7.1 Web application

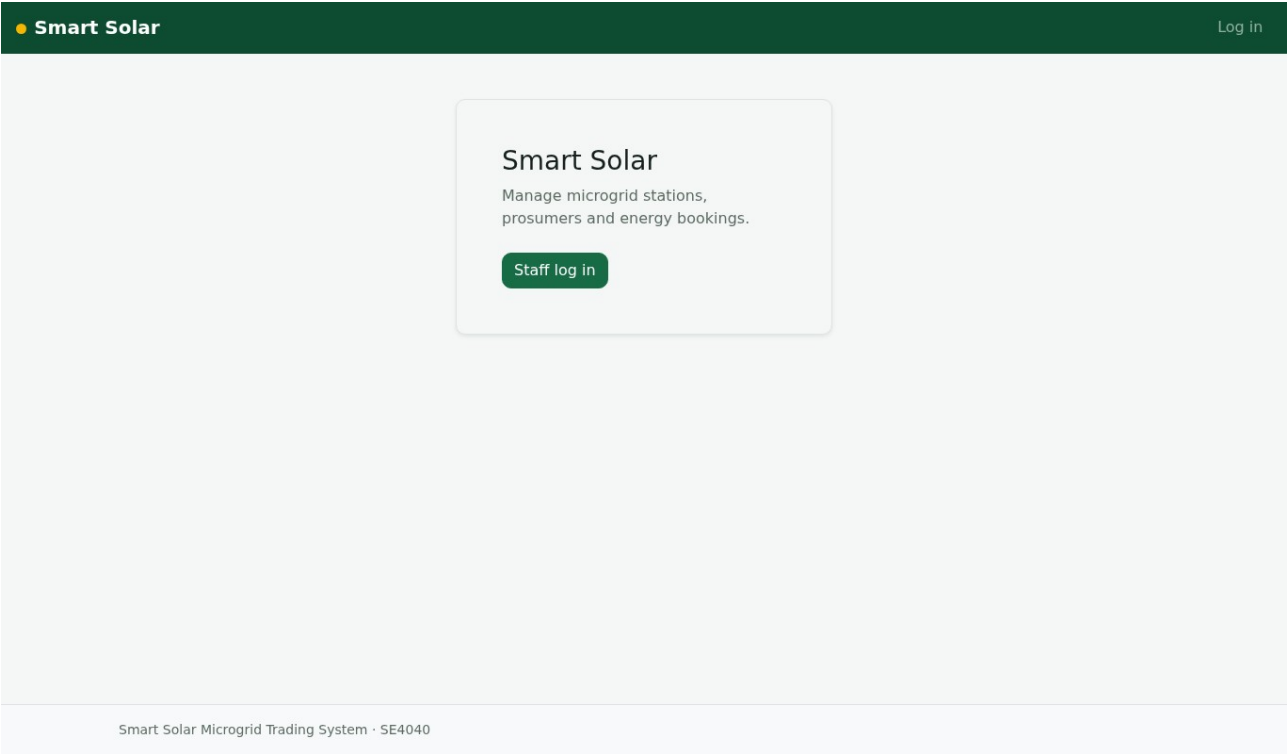


Figure 7. Landing page (M1)

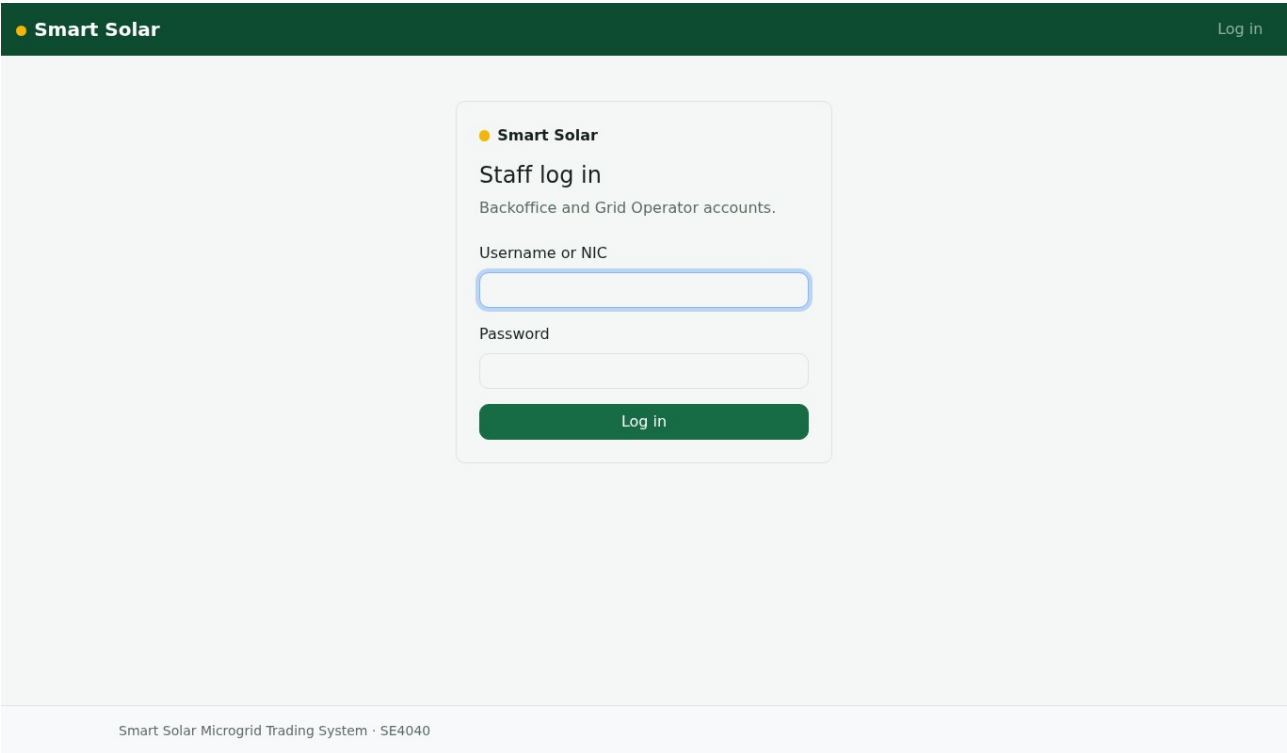


Figure 8. Staff log in (M1)

Smart Solar

Log in

Smart Solar

Staff log in

Backoffice and Grid Operator accounts.

Invalid username or password.

Username or NIC

admin

Password

Log in

Smart Solar Microgrid Trading System · SE4040

Figure 9. Log in with a wrong password (API message shown) (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Backoffice dashboard

Today's bookings

3

Pending approvals

1

Open queue

Active stations

3

Completed today

0

Today's bookings

Approval queue

When	Station	NIC	Status
Thu 01 Oct 2026, 08:00	Malabe Solar Hub	200012345678	Approved
Thu 01 Oct 2026, 10:00	Malabe Solar Hub	200012345678	Approved
Thu 01 Oct 2026, 13:00	Malabe Solar Hub	200012345678	Cancelled

Awaiting activation

1

Open queue

Staff users

Create, update and deactivate staff accounts.

Manage users

Bookings

Filter today's work, the approval queue and booking history.

Open monitor

Reservations

List, edit, cancel or approve bookings.

Open list

Smart Solar Microgrid Trading System · SE4040

Figure 10. Backoffice dashboard with live counts (M4)

Smart Solar

DashboardPeople1StationsBookings

adminBackoffice

Staff users

New staff account

Backoffice and Grid Operator accounts.

Username	Name	Role	Status	Email	
admin	System Administrator	Backoffice	Active	admin@smartsolar.lk	EditDeactivate
operator1	Grid Operator One	GridOperator	Active	operator1@smartsolar.lk	EditDeactivate

Smart Solar Microgrid Trading System · SE4040

Figure 11. Staff users (M1)

Smart Solar

DashboardPeople1StationsBookings

adminBackoffice

New staff account

Username

Password

Required. Hashed by the API, never stored here.

Full name

Email

Phone

Role

GridOperator

Save

Cancel

Smart Solar Microgrid Trading System · SE4040

Figure 12. Create a staff user (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Edit operator1

Username

operator1

Password

Leave blank to keep the current password.

Full name

Grid Operator One

Email

operator1@smartsolar.lk

Phone

0770000002

Role

GridOperator

Save

Cancel

Smart Solar Microgrid Trading System · SE4040

Figure 13. Edit a staff user (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Deactivate operator1?

This sets status to **Deactivated** in the API. The account will no longer be able to log in. You cannot deactivate your own account.

Name

Grid Operator One

Role

GridOperator

Status

Active

Deactivate

Cancel

Smart Solar Microgrid Trading System · SE4040

Figure 14. Deactivate a staff user (confirmation) (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Prosumers

Reactivate is available only to Backoffice.

Create prosumer

Search NIC, name, username or phone

All statuses

Search

Clear

NIC	Name	Username	Phone	Status	Actions
200012345678	Nimal Perera	nimal	0771234567	Active	Edit Deactivate
199912345678	Saman Silva	saman	0777654321	Pending	Edit Deactivate

Smart Solar Microgrid Trading System · SE4040

Figure 15. Prosumer accounts (Backoffice) (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Create prosumer

Back

Staff-created prosumer accounts are active immediately.

NIC

Username

Full name

Email

Phone

Password

Create active account

Smart Solar Microgrid Trading System · SE4040

Figure 16. Create a prosumer with NIC (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Edit prosumer

Back

NIC

200012345678

Username

nimal

Full name

Nimal Perera

Email

nimal@smartsolar.lk

Phone

0771234567

New password (leave blank to keep current)

Save changes

Smart Solar Microgrid Trading System · SE4040

Figure 17. Edit a prosumer (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Pending activations

1 waiting

Registrations waiting for a Backoffice decision.

NIC	Name	Username	Contact	Decision
199912345678	Saman Silva	saman	saman@smartsolar.lk 0777654321	Activate Reject

Smart Solar Microgrid Trading System · SE4040

Figure 18. Pending activations with activate and reject (M1)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Stations

Create station

Name	GPS	Capacity (kWh)	Battery slots	Status	Actions
Malabe Solar Hub	6.9147, 79.9730	150	12	Active	Slots ...
Colombo Fort Node	6.9339, 79.8500	200	16	Active	Slots ...
Rajagiriya Microgrid	6.9090, 79.8910	120	8	Active	Slots ...

Smart Solar Microgrid Trading System · SE4040

Figure 19. Microgrid stations (Backoffice) (M2)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

This station has future reservations and cannot be deactivated.

Stations

Create station

Name	GPS	Capacity (kWh)	Battery slots	Status	Actions
Malabe Solar Hub	6.9147, 79.9730	150	12	Active	Slots ...
Colombo Fort Node	6.9339, 79.8500	200	16	Active	Slots ...
Rajagiriya Microgrid	6.9090, 79.8910	120	8	Active	Slots ...

Smart Solar Microgrid Trading System · SE4040

Figure 20. Deactivation blocked while future reservations exist (409 from the API) (M2)

Smart Solar

DashboardPeople1StationsBookings

adminBackoffice

Create station

Back

New stations are active immediately. The API checks the coordinates, capacity and hours.

Name

Latitude

0

Longitude

0

Capacity (kWh)

0

Battery slots

0

Weekly schedule

Tick a day to include it. Closed days are not sent.

Day	Open	Opens		Closes	
Monday	☐	08:00 AM	🕒	06:00 PM	🕒
Tuesday	☐	08:00 AM	🕒	06:00 PM	🕒
Wednesday	☐	08:00 AM	🕒	06:00 PM	🕒
Thursday	☐	08:00 AM	🕒	06:00 PM	🕒
Friday	☐	08:00 AM	🕒	06:00 PM	🕒
Saturday	☐	08:00 AM	🕒	06:00 PM	🕒
Sunday	☐	08:00 AM	🕒	06:00 PM	🕒

Create station

Smart Solar Microgrid Trading System · SE4040

Figure 21. Create a station with GPS, capacity, batteries and weekly schedule (M2)

Smart Solar

DashboardPeople1StationsBookings

adminBackoffice

Create station

Back

New stations are active immediately. The API checks the coordinates, capacity and hours.

Latitude must be between -90 and 90, and longitude between -180 and 180.

Name

Kaduwela Solar Hub

Latitude

91

Longitude

79.98

Capacity (kWh)

60

Battery slots

4

Weekly schedule

Tick a day to include it. Closed days are not sent.

Day	Open	Opens	Closes
Monday	☒	08:00 AM	06:00 PM
Tuesday	☐	08:00 AM	06:00 PM
Wednesday	☐	08:00 AM	06:00 PM
Thursday	☐	08:00 AM	06:00 PM
Friday	☐	08:00 AM	06:00 PM
Saturday	☐	08:00 AM	06:00 PM
Sunday	☐	08:00 AM	06:00 PM

Create station

Smart Solar Microgrid Trading System · SE4040

Figure 22. Station field rule: latitude out of range (API message) (M2)

Smart Solar

Dashboard

People1

Stations

Bookings

adminBackoffice

Edit station

Back

Name

Malabe Solar Hub

Latitude

6.9147

Longitude

79.973

Capacity (kWh)

150

Battery slots

12

Weekly schedule

Tick a day to include it. Closed days are not sent.

Day	Open	Opens	Closes
Monday	☒	08:00 AM	06:00 PM
Tuesday	☒	08:00 AM	06:00 PM
Wednesday	☒	08:00 AM	06:00 PM
Thursday	☒	08:00 AM	06:00 PM
Friday	☒	08:00 AM	06:00 PM
Saturday	☒	08:00 AM	06:00 PM
Sunday	☒	08:00 AM	06:00 PM

Save changes

Smart Solar Microgrid Trading System · SE4040

Figure 23. Edit a station and its opening schedule (M2)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Malabe Solar Hub

Back to stations

Times are shown in Sri Lanka time. The API decides whether a slot fits the schedule.

Start	End	Status	Actions
Tue 29 Sep 2026, 08:00	Tue 29 Sep 2026, 10:00	Booked	Delete
Thu 01 Oct 2026, 08:00	Thu 01 Oct 2026, 10:00	Booked	Delete
Thu 01 Oct 2026, 10:00	Thu 01 Oct 2026, 12:00	Booked	Delete
Thu 01 Oct 2026, 13:00	Thu 01 Oct 2026, 15:00	Available	Delete
Thu 01 Oct 2026, 15:00	Thu 01 Oct 2026, 17:00	Available	Delete
Fri 02 Oct 2026, 08:00	Fri 02 Oct 2026, 10:00	Available	Delete
Fri 02 Oct 2026, 10:00	Fri 02 Oct 2026, 12:00	Available	Delete
Fri 02 Oct 2026, 13:00	Fri 02 Oct 2026, 15:00	Available	Delete
Fri 02 Oct 2026, 15:00	Fri 02 Oct 2026, 17:00	Available	Delete
Sat 03 Oct 2026, 08:00	Sat 03 Oct 2026, 10:00	Booked	Delete
Sat 03 Oct 2026, 10:00	Sat 03 Oct 2026, 12:00	Available	Delete
Sat 03 Oct 2026, 13:00	Sat 03 Oct 2026, 15:00	Available	Delete
Sat 03 Oct 2026, 15:00	Sat 03 Oct 2026, 17:00	Available	Delete
Sun 04 Oct 2026, 08:00	Sun 04 Oct 2026, 10:00	Available	Delete
Sun 04 Oct 2026, 10:00	Sun 04 Oct 2026, 12:00	Available	Delete
Sun 04 Oct 2026, 13:00	Sun 04 Oct 2026, 15:00	Available	Delete
Sun 04 Oct 2026, 15:00	Sun 04 Oct 2026, 17:00	Available	Delete
Mon 05 Oct 2026, 08:00	Mon 05 Oct 2026, 10:00	Available	Delete
Mon 05 Oct 2026, 10:00	Mon 05 Oct 2026, 12:00	Available	Delete
Mon 05 Oct 2026, 13:00	Mon 05 Oct 2026, 15:00	Available	Delete
Mon 05 Oct 2026, 15:00	Mon 05 Oct 2026, 17:00	Available	Delete

Add a slot

Start

mm/dd/yyyy, --:-- --

End

mm/dd/yyyy, --:-- --

Add slot

Smart Solar Microgrid Trading System · SE4040

Figure 24. Battery slots of a station (Sri Lanka time) (M2)

Sun 04 Oct 2026, 13:00

Sun 04 Oct 2026, 15:00

Available

Delete

Sun 04 Oct 2026, 15:00

Sun 04 Oct 2026, 17:00

Available

Delete

Mon 05 Oct 2026, 08:00

Mon 05 Oct 2026, 10:00

Available

Delete

Mon 05 Oct 2026, 10:00

Mon 05 Oct 2026, 12:00

Available

Delete

Mon 05 Oct 2026, 13:00

Mon 05 Oct 2026, 15:00

Available

Delete

Mon 05 Oct 2026, 15:00

Mon 05 Oct 2026, 17:00

Available

Delete

Add a slot

The slot must fall inside the station schedule.

Start

10/04/2026, 06:00 AM

End

10/04/2026, 08:00 AM

Add slot

Smart Solar Microgrid Trading System · SE4040

Figure 25. Slot before opening time rejected by the API (M2)

Smart Solar

DashboardPeople1StationsBookings

adminBackoffice

This slot has an active reservation.

Malabe Solar Hub

Back to stations

Times are shown in Sri Lanka time. The API decides whether a slot fits the schedule.

Start	End	Status	Actions
Tue 29 Sep 2026, 08:00	Tue 29 Sep 2026, 10:00	Booked	Delete
Thu 01 Oct 2026, 08:00	Thu 01 Oct 2026, 10:00	Booked	Delete
Thu 01 Oct 2026, 10:00	Thu 01 Oct 2026, 12:00	Booked	Delete

Figure 26. A booked slot cannot be deleted (409 SLOT_TAKEN) (M2)

Smart Solar

DashboardPeople1StationsBookings

adminBackoffice

Reservations

Book a slot

Status

Prosumer NIC

Search

All

Station or reference

☐ Include past

Filter

1 booking(s), upcoming only

Slot time	Station	Prosumer NIC	Status	Reference	
Sat 03 Oct 2026, 08:00	Malabe Solar Hub	200012345678	Pending	6abe36b46f535345f2173163	ViewEditCancelApprove

Smart Solar Microgrid Trading System · SE4040

Figure 27. Manage reservations (M3)

Smart Solar

Dashboard

People1

Stations

Bookings

admin · Backoffice

Book a slot for a prosumer

For prosumers who call or visit. The API checks the booking rules and tells you if something is not allowed.

1. Prosumer and station

Prosumer NIC

200012345678

Station

Malabe Solar Hub

Date

10/04/2026

Optional. Leave empty to see every free slot you can book.

Show free slots

2. Pick a slot and confirm

Sunday, 04 October

08:00 – 10:00

10:00 – 12:00

13:00 – 15:00

15:00 – 17:00

Confirm booking

Smart Solar Microgrid Trading System · SE4040

Figure 28. Book a slot on behalf of a prosumer (M3)

Smart Solar

Dashboard

People1

Stations

Bookings

admin · Backoffice

Change booking

Changes need at least 12 hours notice before the booked slot. An approved booking goes back to pending and needs approval again.

Current booking

Reference

6abe36b46f535345f2173163

Prosumer

200012345678

Station

Malabe Solar Hub

Slot time

Sat 03 Oct 2026, 08:00

Status

Pending

1. New station and date

Station

Malabe Solar Hub

Date

mm/dd/yyyy

Optional. Leave empty to see every free slot you can book.

Show free slots

2. Pick the new slot

Friday, 02 October

08:00 – 10:00

10:00 – 12:00

13:00 – 15:00

15:00 – 17:00

Saturday, 03 October

10:00 – 12:00

13:00 – 15:00

15:00 – 17:00

Sunday, 04 October

08:00 – 10:00

10:00 – 12:00

13:00 – 15:00

15:00 – 17:00

Monday, 05 October

08:00 – 10:00

10:00 – 12:00

13:00 – 15:00

15:00 – 17:00

Save change

Back

Smart Solar Microgrid Trading System · SE4040

Figure 29. Change a reservation to another slot (M3)

SE4040 · Smart Solar Microgrid Trading System · 30

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Cancel this booking?

Bookings can only be cancelled at least 12 hours before the slot starts. The slot becomes free for others straight away.

Reference

6abe36b46f535345f2173163

Prosumer

200012345678

Station

Malabe Solar Hub

Slot time

Sat 03 Oct 2026, 08:00

Status

Pending

Reason (optional)
e.g. Prosumer called to cancel

Cancel booking

Keep it

Smart Solar Microgrid Trading System · SE4040

Figure 30. Cancel a reservation (12-hour rule applies) (M3)

Smart Solar

Dashboard

People 1

Stations

Bookings

admin · Backoffice

Booking details

Reference

6abe36b46f535345f2173163

Status

Pending

Prosumer NIC

200012345678

Station

Malabe Solar Hub

Slot time

Sat 03 Oct 2026, 08:00

Last updated

Thu 01 Oct 2026, 16:02

Back to reservations

Change

Cancel booking

Approve

Smart Solar Microgrid Trading System · SE4040

Figure 31. Reservation summary (M3)

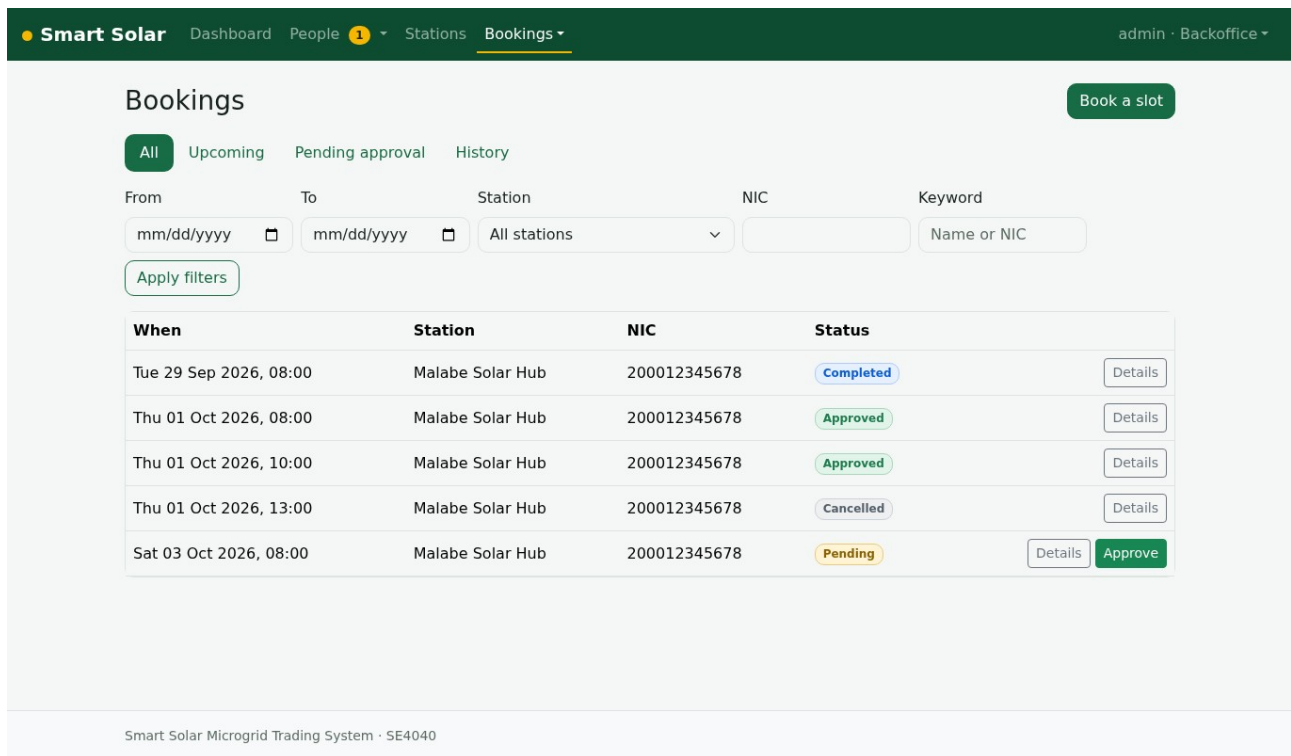


Figure 32. Booking monitor with filters and history (M4)

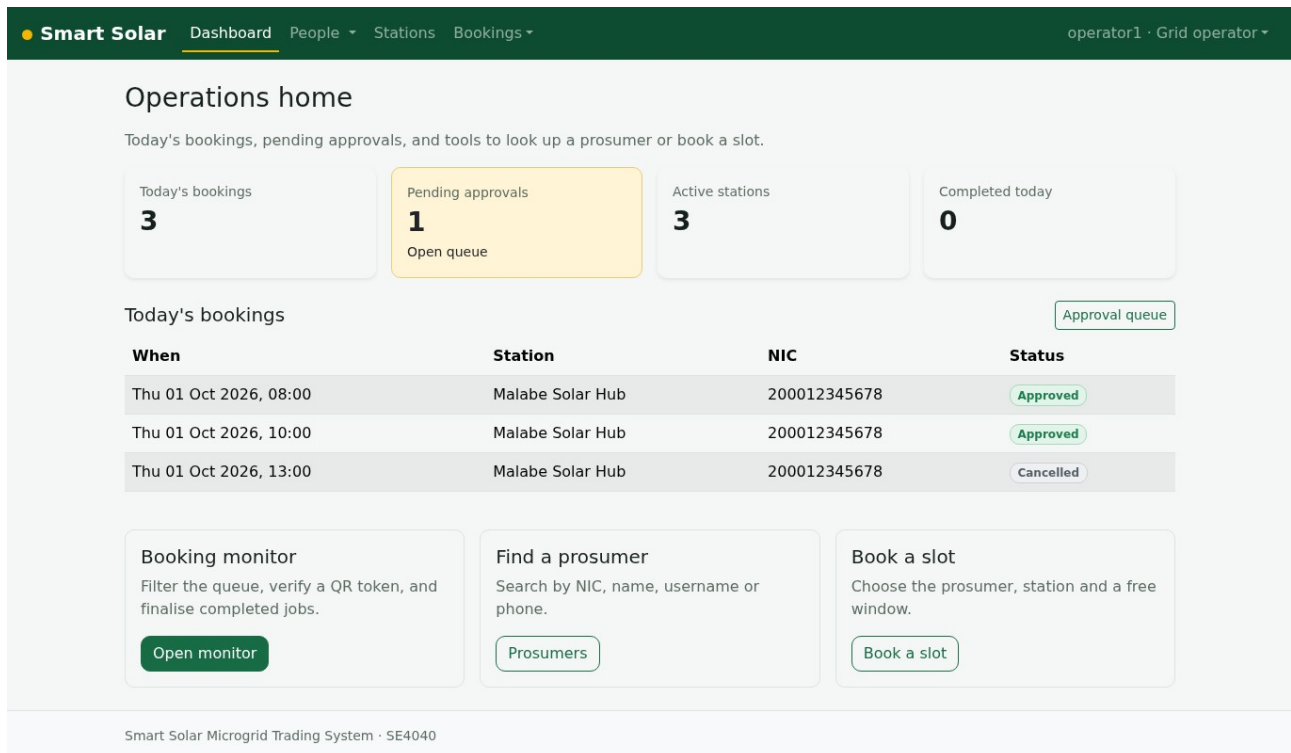


Figure 33. Grid Operator home with today's work (M4)

Smart Solar

Dashboard

People

Stations

Bookings

operator1 · Grid operator

Malabe Solar Hub

Back to stations

Times are shown in Sri Lanka time. The API decides whether a slot fits the schedule.

Start	End	Status	Actions	
Tue 29 Sep 2026, 08:00	Tue 29 Sep 2026, 10:00	Booked	Open	Delete
Thu 01 Oct 2026, 08:00	Thu 01 Oct 2026, 10:00	Booked	Open	Delete
Thu 01 Oct 2026, 10:00	Thu 01 Oct 2026, 12:00	Booked	Open	Delete
Thu 01 Oct 2026, 13:00	Thu 01 Oct 2026, 15:00	Available	Close	Delete
Thu 01 Oct 2026, 15:00	Thu 01 Oct 2026, 17:00	Available	Close	Delete
Fri 02 Oct 2026, 08:00	Fri 02 Oct 2026, 10:00	Available	Close	Delete
Fri 02 Oct 2026, 10:00	Fri 02 Oct 2026, 12:00	Available	Close	Delete
Fri 02 Oct 2026, 13:00	Fri 02 Oct 2026, 15:00	Available	Close	Delete
Fri 02 Oct 2026, 15:00	Fri 02 Oct 2026, 17:00	Available	Close	Delete
Sat 03 Oct 2026, 08:00	Sat 03 Oct 2026, 10:00	Booked	Open	Delete
Sat 03 Oct 2026, 10:00	Sat 03 Oct 2026, 12:00	Available	Close	Delete
Sat 03 Oct 2026, 13:00	Sat 03 Oct 2026, 15:00	Available	Close	Delete
Sat 03 Oct 2026, 15:00	Sat 03 Oct 2026, 17:00	Available	Close	Delete
Sun 04 Oct 2026, 08:00	Sun 04 Oct 2026, 10:00	Available	Close	Delete
Sun 04 Oct 2026, 10:00	Sun 04 Oct 2026, 12:00	Available	Close	Delete
Sun 04 Oct 2026, 13:00	Sun 04 Oct 2026, 15:00	Available	Close	Delete
Sun 04 Oct 2026, 15:00	Sun 04 Oct 2026, 17:00	Available	Close	Delete
Mon 05 Oct 2026, 08:00	Mon 05 Oct 2026, 10:00	Available	Close	Delete
Mon 05 Oct 2026, 10:00	Mon 05 Oct 2026, 12:00	Available	Close	Delete
Mon 05 Oct 2026, 13:00	Mon 05 Oct 2026, 15:00	Available	Close	Delete
Mon 05 Oct 2026, 15:00	Mon 05 Oct 2026, 17:00	Available	Close	Delete

Add a slot

Start

mm/dd/yyyy, --:-- --

End

mm/dd/yyyy, --:-- --

Add slot

Smart Solar Microgrid Trading System · SE4040

Figure 34. Grid Operator opens and closes battery slots (M2)

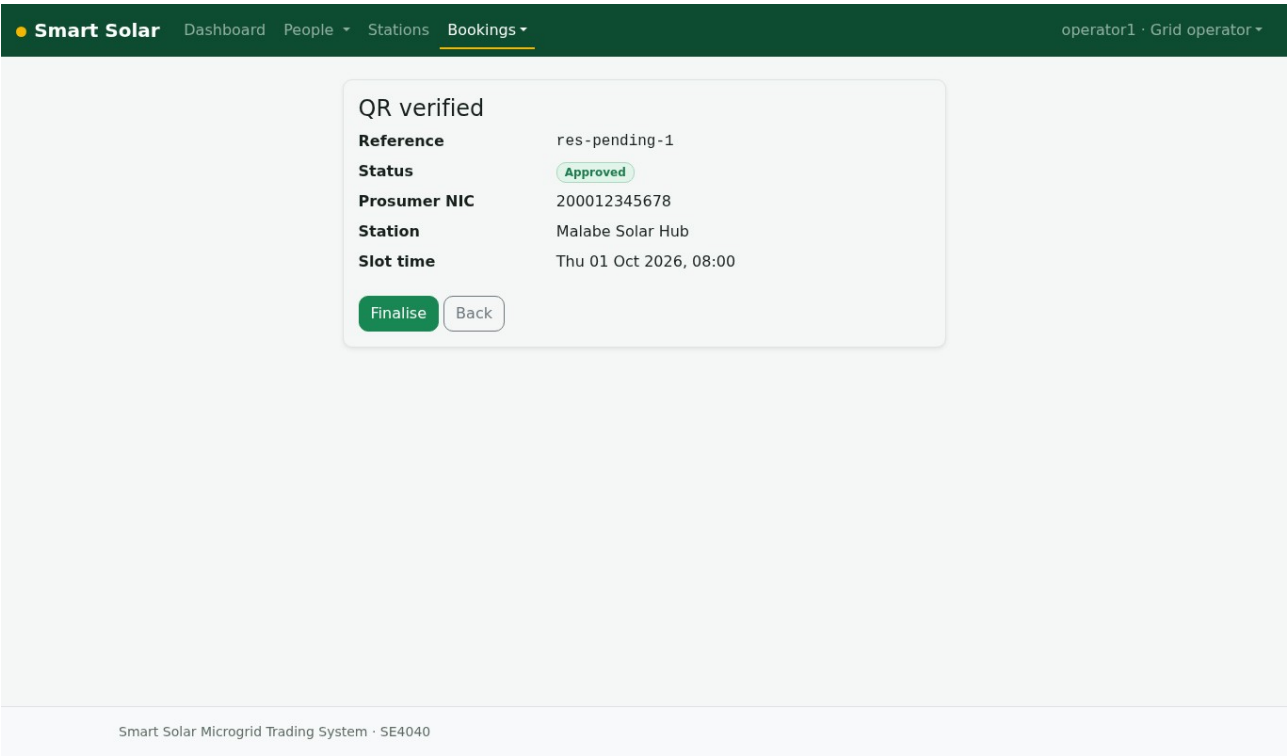


Figure 35. Grid Operator verifies a prosumer's QR token with the server (M4)

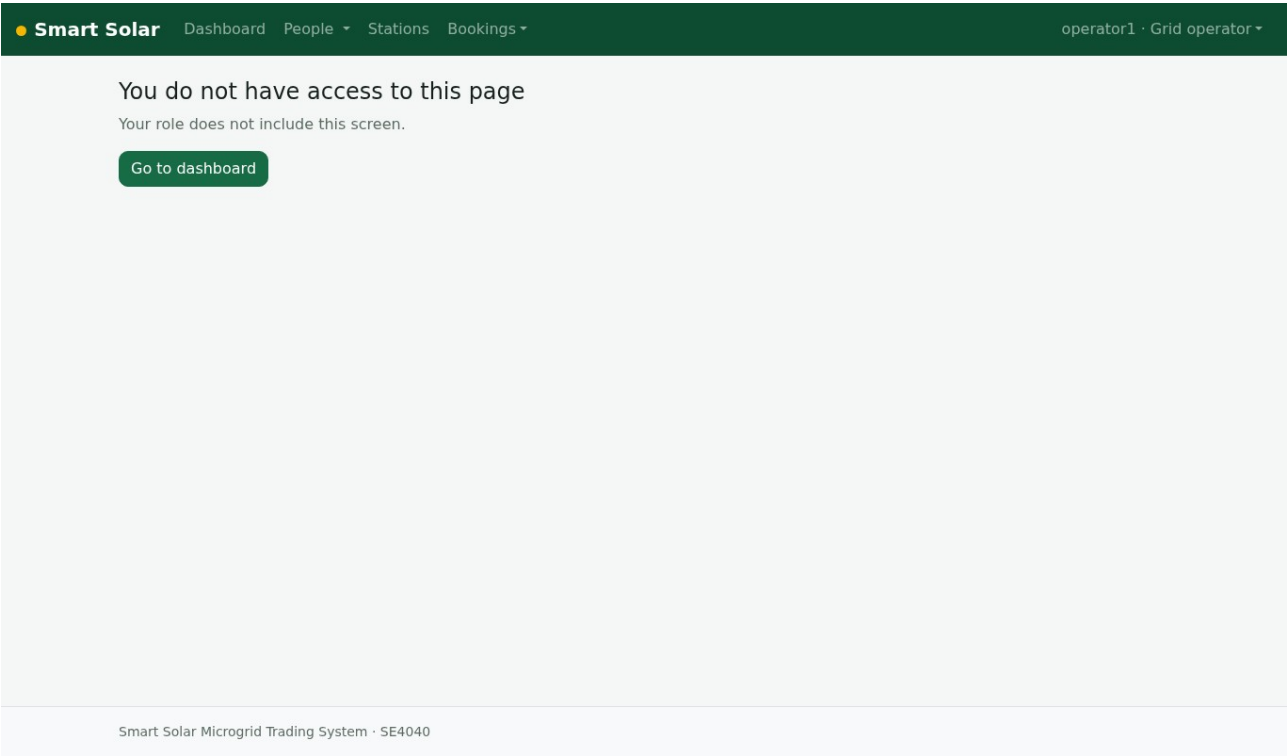


Figure 36. Role check: Grid Operator cannot open staff users (M1)

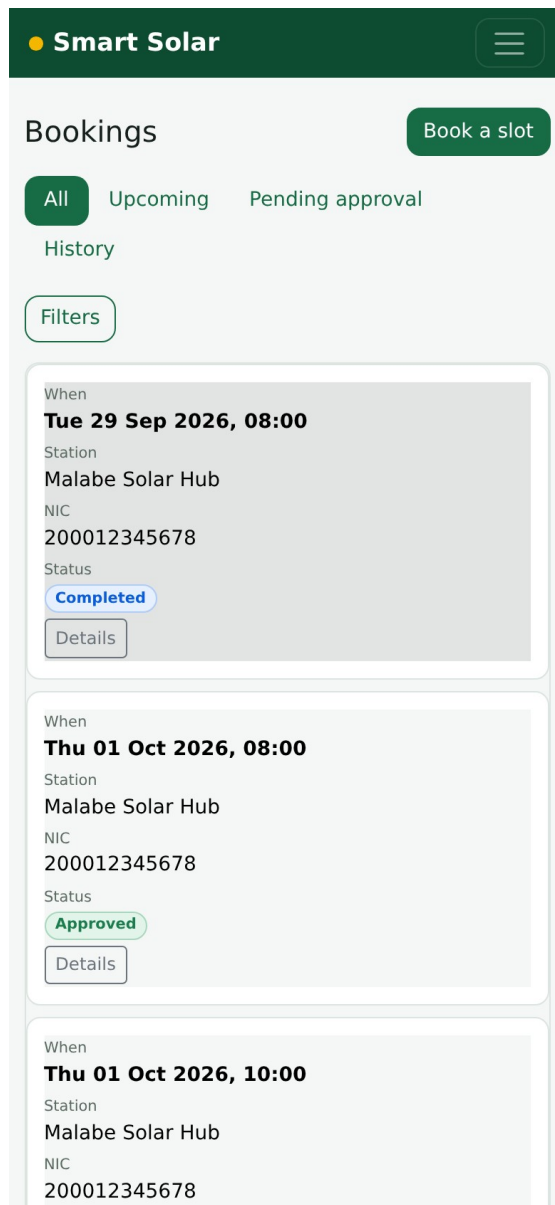


Figure 37. Responsive layout at phone width: booking monitor (M4)

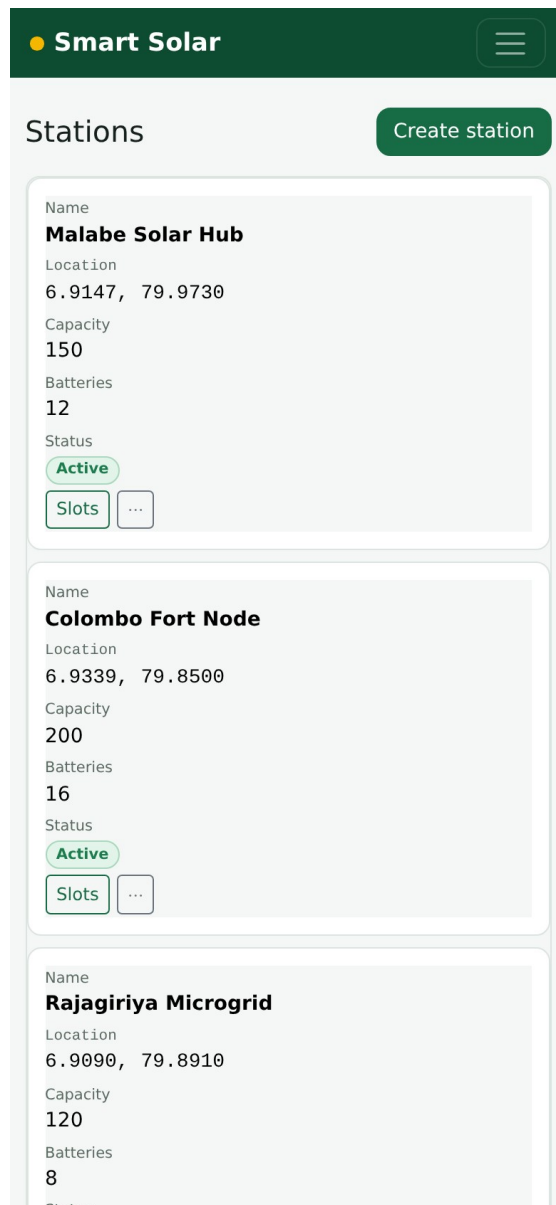


Figure 38. Responsive layout at phone width: stations (M2)

7.2 Android application

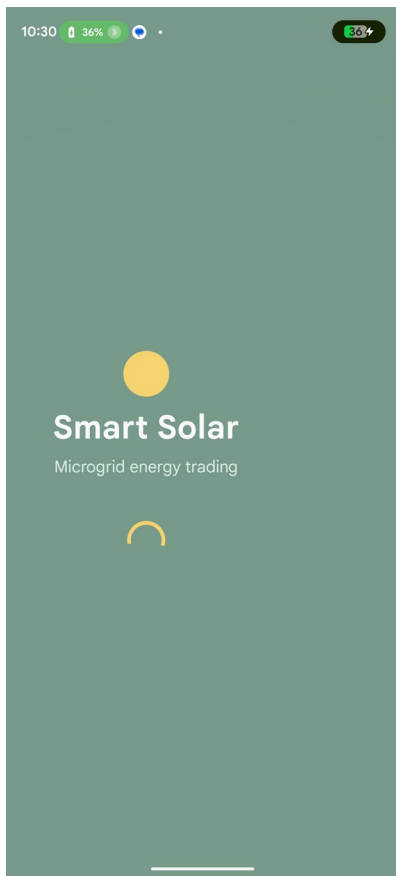


Figure 39. Splash screen (M1)

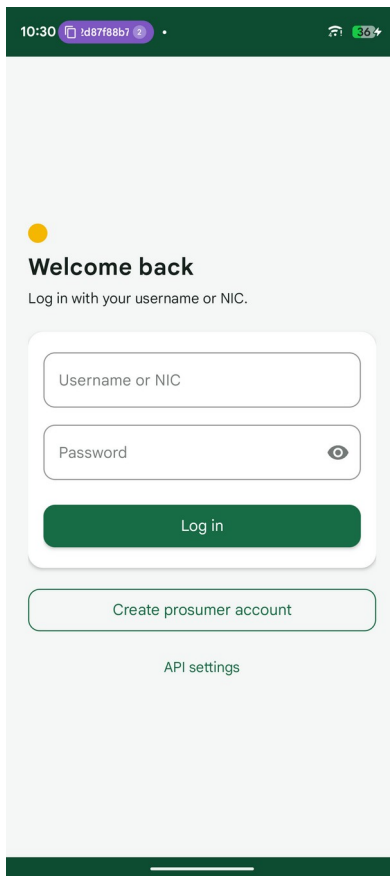


Figure 40. Log in with username or NIC (M1)

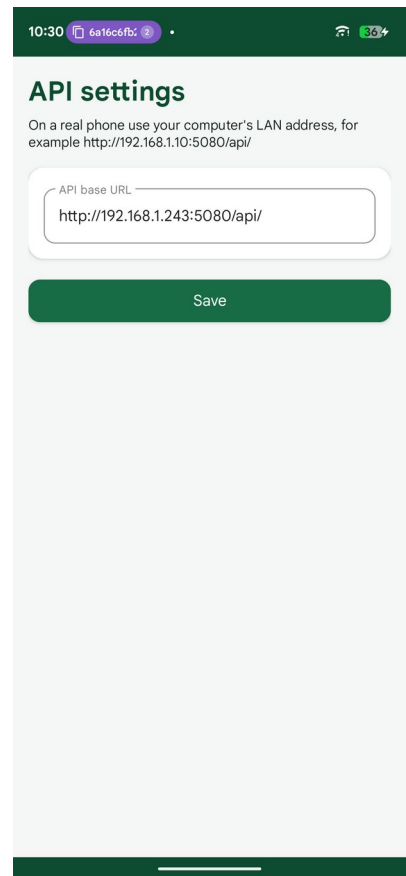


Figure 41. API settings: server address on the LAN (M1)

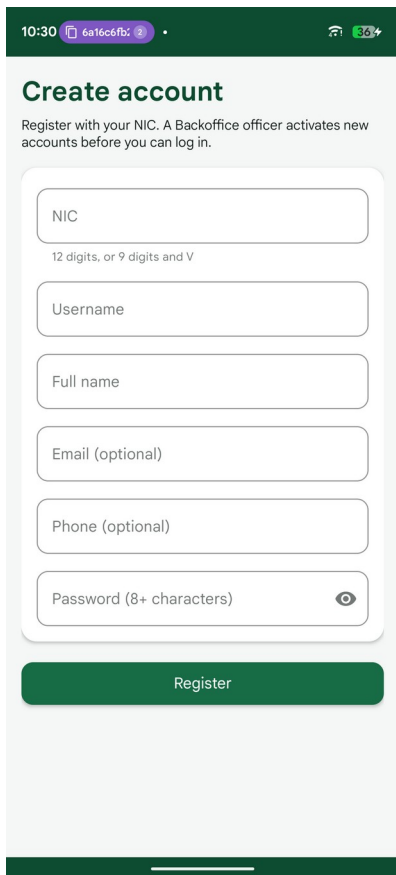


Figure 42. Prosumer registration with NIC (M1)

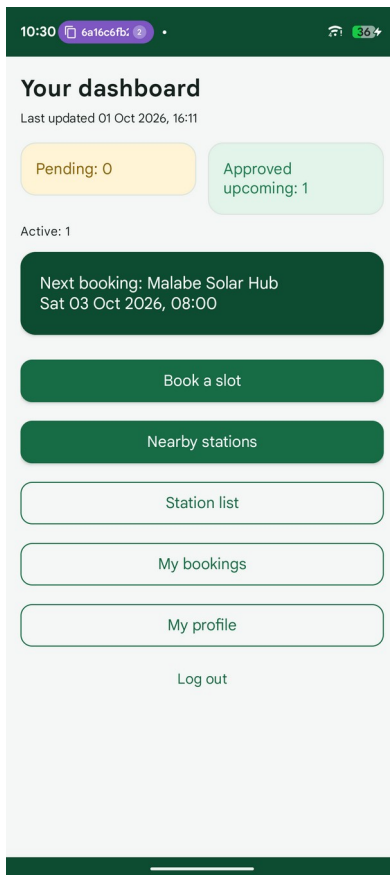


Figure 43. Prosumer dashboard with live counts (M4)

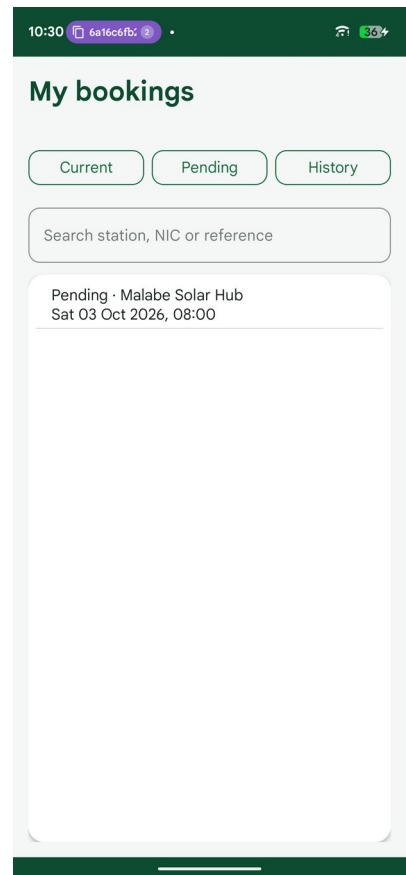


Figure 44. My bookings: current (M4)

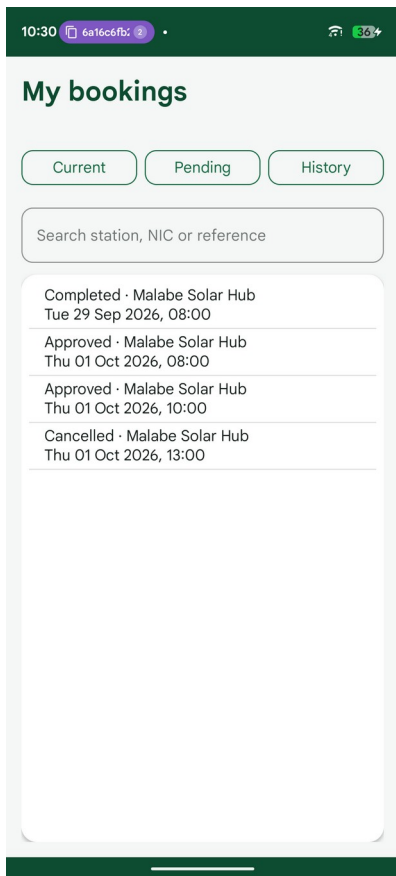


Figure 45. My bookings: history (M4)

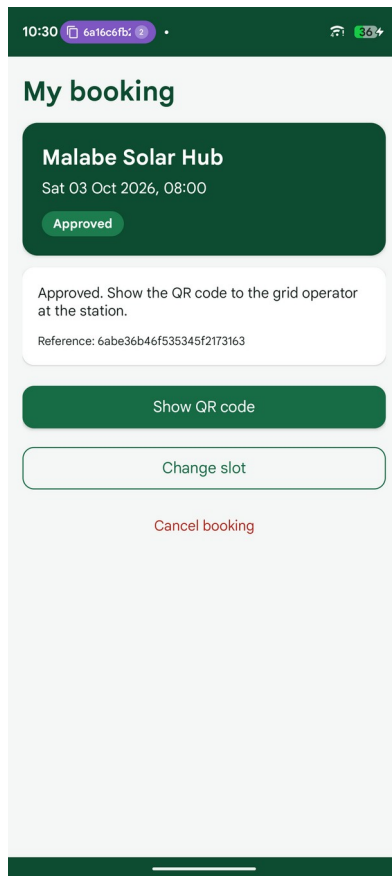


Figure 46. Booking detail with change, cancel and QR (M3)

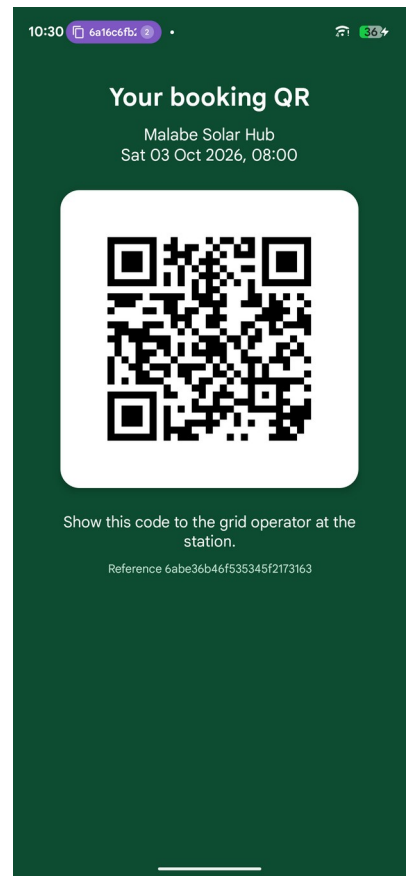


Figure 47. QR code for an approved booking (M3)

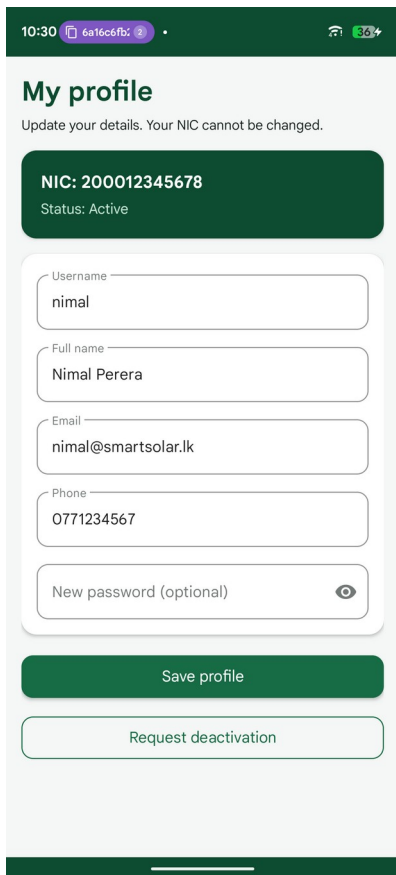


Figure 48. Profile (NIC cannot be changed) (M1)

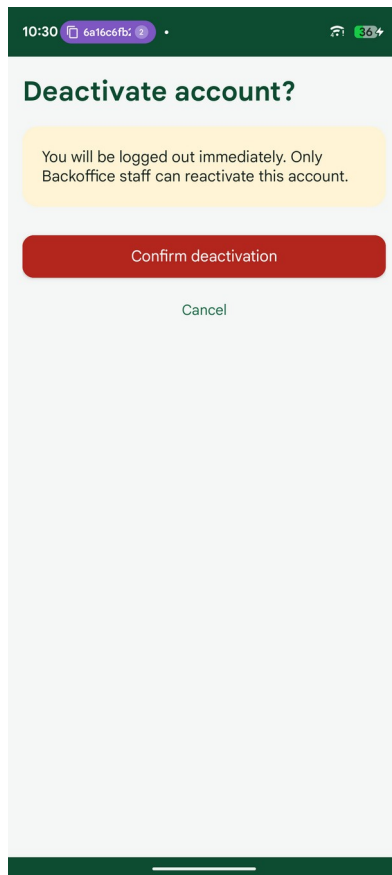


Figure 49. Request account deactivation (M1)

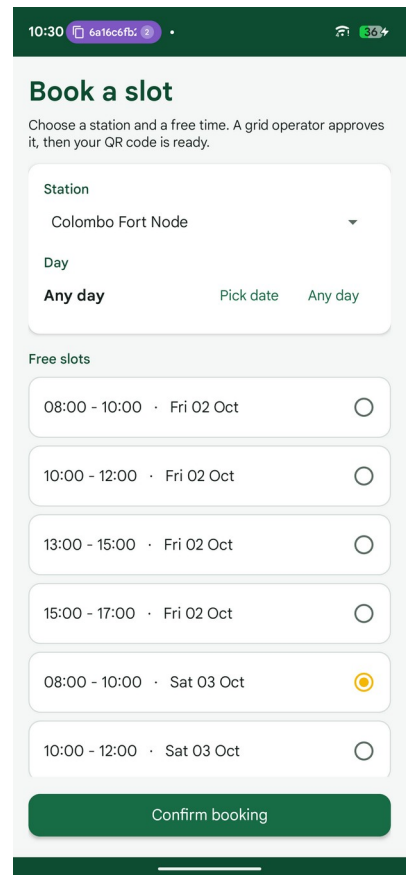


Figure 50. Book a slot: choose station and free slot (M3)

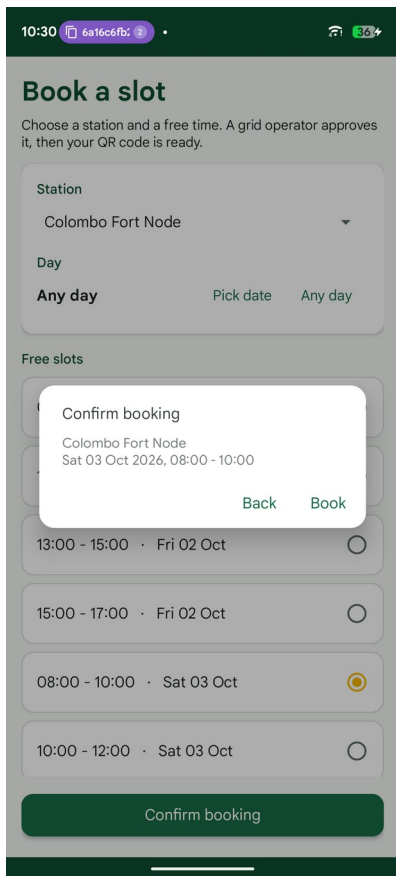


Figure 51. Confirm the booking (M3)

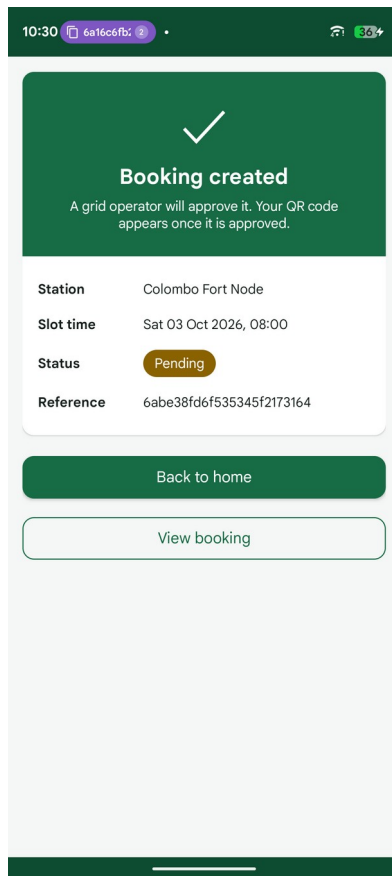


Figure 52. Summary after booking (M3)

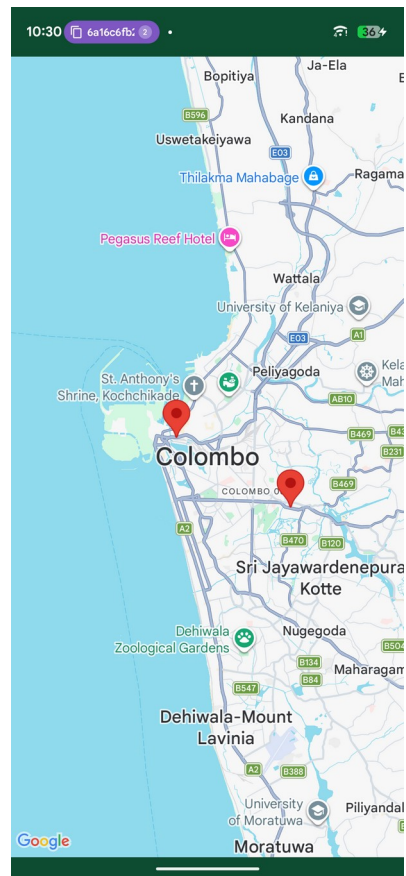


Figure 53. Nearby stations on Google Maps (M2)

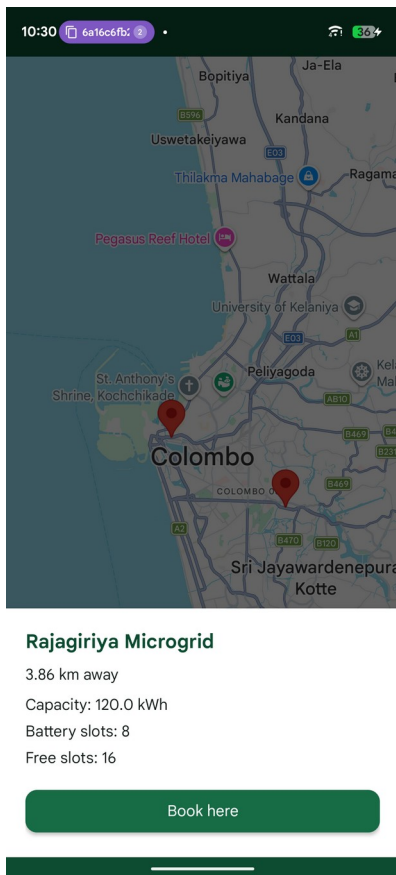


Figure 54. Station details with Book here (M2)

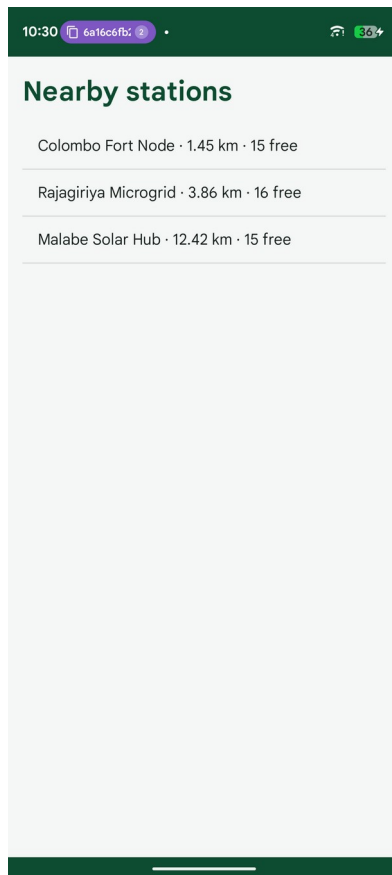


Figure 55. Nearby stations list (M2)

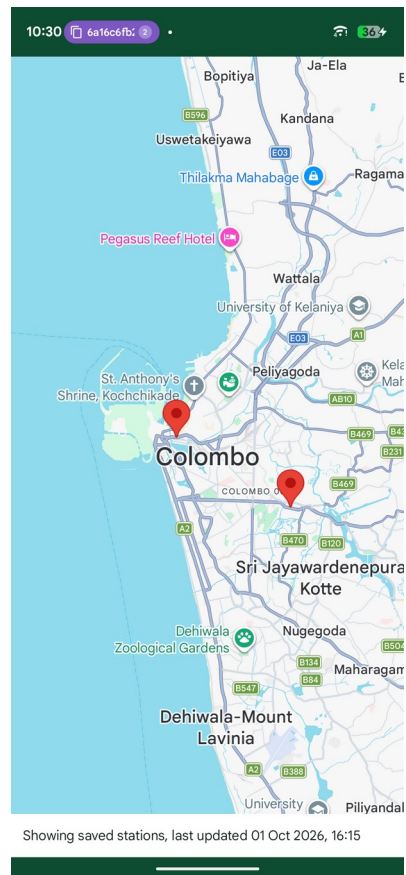


Figure 56. Offline: stations from the SQLite cache (M2)

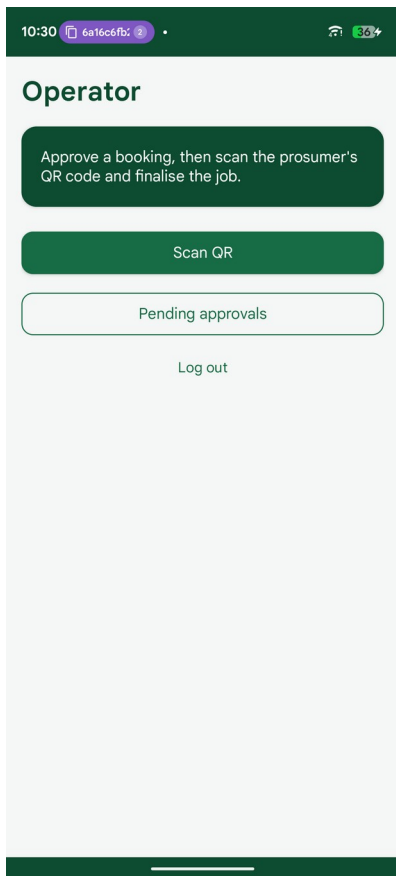


Figure 57. Grid Operator home (M4)

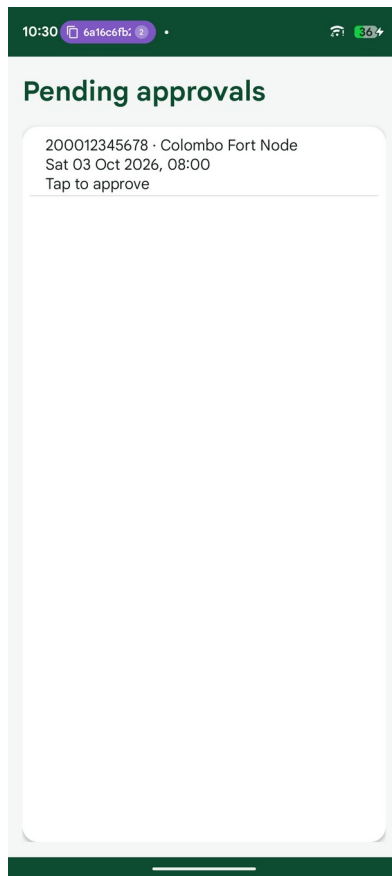


Figure 58. Pending approvals queue (M4)

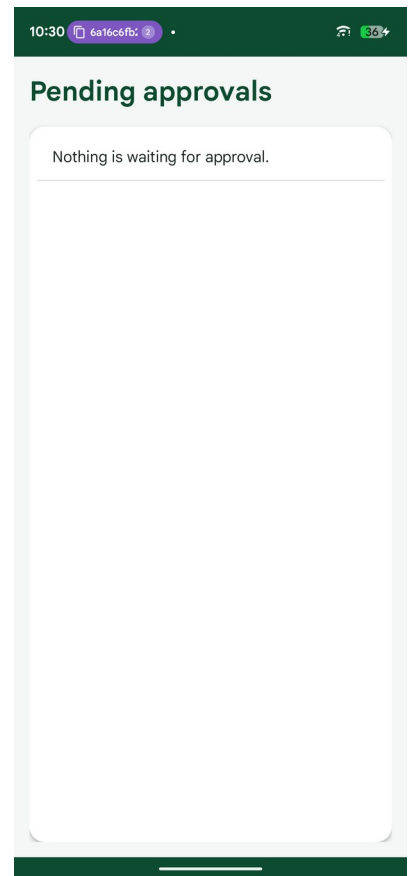


Figure 59. Queue after approving (QR issued to the prosumer) (M4)

8. Deployment on Windows IIS

The web service and the web application are hosted on a Windows 11 PC on the same LAN as the phones. The steps below are the ones used; docs/DEPLOYMENT.md in the repository has the full version.

1. Enable IIS: `Enable-WindowsOptionalFeature -Online -FeatureName IIS-WebServerRole, IIS-WebServer, IIS-CommonHttpFeatures, IIS-ManagementConsole, IIS-StaticContent, IIS-DefaultDocument, IIS-HttpErrors -All`, then restart Windows.
2. Install the .NET 10 Hosting Bundle (ASP.NET Core Module V2), then restart IIS (`net stop w3svc`, `start w3svc`).
3. Install MongoDB Community Server as a Windows service on the same PC (`mongodb://localhost:27017`).
4. Publish both projects framework-dependent: `dotnet publish -c Release -r win-x64 --self-contained false`, into `C:\inetpub\smart-solar-api` and `C:\inetpub\smart-solar-web`.
5. Create `appsettings.Production.json` on the server only, with the Mongo connection string and a random JWT key of at least 32 characters. The key is never committed to Git.
6. Create two application pools, `SmartSolarApi` and `SmartSolarWeb`, with .NET CLR version `No Managed Code` (ANCM hosts .NET, not the CLR).
7. Create the sites: `SmartSolarApi` on port 5080 and `SmartSolarWeb` on port 5081. Grant each pool identity read access to its folder and modify access to its logs folder (icacis).
8. Allow inbound TCP 5080 and 5081 in Windows Firewall, and set the Wi-Fi network profile to Private.
9. Smoke test from another device: `http://<server-ip>:5080/api/health` returns `Healthy`, login returns a token, and the phone app and web app work against the server address.

Item	Value
API site	SmartSolarApi · *:5080 · C:\inetpub\smart-solar-api
Web site	SmartSolarWeb · *:5081 · C:\inetpub\smart-solar-web
App pools	SmartSolarApi, SmartSolarWeb · No Managed Code · Integrated pipeline
Module	AspNetCoreModuleV2 (Hosting Bundle 10.0)
Database	MongoDB service, localhost:27017, database SmartSolar

Table 14. IIS configuration

9. Git repository

Repository: <https://github.com/DilnukDeSilva/solar-microgrid-trading-system>

main holds submission-ready code and dev is the integration branch. Each member worked on their own feature branch and merged into dev through a pull request reviewed by another member, so the history shows who built what.

PR	Title	Author	Merged
#2	Add Mongo-backed Web API with JWT auth for LAN testing	M1	2026-09-20
#3	Add staff Users API and MVC login/users web app	M1	2026-09-21
#4	Add staff Users API, MVC login/users web app, and JWT placeholder guard	M1	2026-09-21
#5	Update README branch table to the new feature branch names	M1	2026-09-26
#6	Feature/reservations	M3	2026-09-26
#7	Add prosumer identity API, web prosumer pages and the Android app shell	M1	2026-09-27
#8	Fix GridOperator 403 on prosumer search and correct author attribution	M1	2026-09-27
#9	Feature/reservations	M3	2026-09-29
#10	Move the web desk to React and Tailwind	M1	2026-09-29
#11	Feature/samu dashboard operator	M4	2026-09-29
#12	Feature/reservations	M3	2026-09-30
#13	M2: stations, slots, nearby map and IIS hosting	M2	2026-09-30
#14	Merge dev into main (conflict-free)	M4	2026-09-30
#16	Remove the unused React web client	M1	2026-09-30
#17	Share one Bootstrap theme and restyle the MVC menu and status labels	M1	2026-09-30
#18	Add a themed button for the empty booking list	M1	2026-09-30
#19	Match the Android screens and popups to the solar theme	M1	2026-09-30

Table 15. *Merged pull requests*

10. Individual contribution

M1 • De Silva R K D H (IT22001252)

Module: Identity & Accounts, Android app shell

- API foundation: MongoDB context, seeding, indexes, JWT authentication, error middleware
- Staff users and prosumer accounts, registration with NIC, activation, deactivation and reactivation
- Web: login with role redirect, staff user and prosumer pages, pending activations with navbar count
- Android: app shell (API client, session, SQLite helper, role routing), login, registration, profile, deactivation, API settings

Commits on dev: 27 · files touched: 186 · lines +8887 / -1342

Commit	Date	Message
ea54d48	2026-09-20	Bind the staff edit username from the route id.
6cd93ba	2026-09-20	Retire the Milestone 0 runbook now that local API development is done.
21ff6fa	2026-09-20	Reject deactivated accounts when the JWT is validated.
ea10a71	2026-09-20	Document LAN/IIS hosting, seed logins and the /health probe for the rest of the team.
112d886	2026-09-20	Block a Backoffice user from changing their own role.
d3f9b15	2026-09-20	Expose health, login and Backoffice users over HTTP with contract error bodies.
4c5469c	2026-09-20	Validate staff email format in the API when an address is supplied.
26ca387	2026-09-20	Add login and user services so auth rules live outside the controllers.
dca55ee	2026-09-20	Reject staff passwords shorter than eight characters.
de0ee36	2026-09-20	Add Mongo repositories so clients never talk to the database.
99e95cd	2026-09-20	Add staff Users API and MVC login/users web app.
1d9f4e1	2026-09-20	Connect MongoDB and seed the four collections on first startup.
f31de80	2026-09-20	Add Mongo document models and public DTOs that match the API contract.
be513a0	2026-09-20	Scaffold the Web API project with config-based Mongo and JWT settings.
ea1b101	2026-09-20	Add project structure, planning docs, API contract and member task files
9ce9494	2026-09-21	Refuse the example JWT placeholder at startup so a copied sample key cannot mint tokens.
c364d17	2026-09-21	Move IIS deployment to Member 2 and document the staff account rules in the API contract.
944f9f6	2026-09-21	Clear finished Member 1 checklist items, leaving IIS support for M2.
faceeff	2026-09-26	Update README branch table to the new feature branch names
950b546	2026-09-27	Fix GridOperator prosumer search returning 403, and correct author headers
3d6f825	2026-09-27	Add prosumer identity API, web prosumer pages and the Android app shell
6dfcfb9	2026-09-29	Move the web desk to React and Tailwind so staff pages call the API directly, and record that stack for every member.
005351e	2026-09-30	Show Open booking monitor as a themed button
c614aab	2026-09-30	Restyle the MVC web menu, status labels, and narrow-screen tables.
e251c4c	2026-09-30	Share one Bootstrap theme across the web app
ceb0fd8	2026-09-30	Remove the unused React web client
a7c880d	2026-10-01	Match the Android screens and popups to the solar theme

Table 16. Commits by De Silva R K D H

M2 • Asath M M (IT22633422)

Module: Microgrid Nodes, Slots & Maps, IIS hosting

- API: stations (create, update, deactivate with the future-reservation check, activate), slots (schedule and battery-overlap rules, operator availability), nearby search with haversine distance
- Web: stations list, create and edit with a weekly schedule, slots page per station
- Android: nearby stations on Google Maps with details sheet and Book here, station list, stations_cache SQLite table for offline use
- Hosting: Windows IIS setup and deployment of the web service and web application; use case diagram

Commits on dev: 16 · files touched: 44 · lines +2261 / -46

Commit	Date	Message
31fe5f3	2026-09-30	android: list nearby stations and open booking from a row
ef08564	2026-09-30	android: nearby stations map with details sheet that opens booking
ccc0a5b	2026-09-30	android: cache nearby stations in SQLite for offline map use
6a963d4	2026-09-30	android: add Google Maps SDK with the key read from local.properties
06f4b2d	2026-09-30	web: show slot time validation messages and station names in banners
d4d0227	2026-09-30	web: leave add-slot times empty until the user picks them
61edb98	2026-09-30	web: manage a station's slots with schedule and booking checks from the API
429ef7c	2026-09-30	web: create and edit stations with a weekly opening schedule
cedd14f	2026-09-30	web: stations list with deactivate and activate showing the API's rule message
a85db9a	2026-09-30	api: require lat and lng on the nearby stations search
6351073	2026-09-30	api: add nearby stations sorted by haversine distance with free slot counts
dee784e	2026-09-30	api: convert offset slot times to UTC before schedule checks
d32d643	2026-09-30	api: let grid operators toggle slot availability unless it is booked
a4cecc2	2026-09-30	api: add slot create, update and delete with schedule and battery overlap rules
9bf8f94	2026-09-30	api: block station deactivation while future reservations exist
571ab68	2026-09-30	Add station list, get and create with field rules in the service.

Table 17. Commits by Asath M M

M3 • Janukshan S (IT22635266)

Module: Energy Reservations & QR Dispatch

- API: reservation create, update, cancel, get and approve with the 7-day and 12-hour rules, atomic slot claim, QR token issue
- Web: reservations list, book on behalf of a prosumer, edit, cancel and summary pages
- Android: book a slot, booking detail with change and cancel, QR screen, summary after each action, my_reservations SQLite table

Commits on dev: 9 · files touched: 105 · lines +5541 / -671

Commit	Date	Message
c8b0b61	2026-09-26	Add reservation update, cancel and approve with 12-hour rule and QR token

Commit	Date	Message
18ca392	2026-09-26	Add reservation create and get endpoints with 7-day and slot rules
4cd1ea3	2026-09-26	Update plan and member task files for the module split
9232a8b	2026-09-27	Add web reservation list with edit, cancel and approve pages
14c94b1	2026-09-27	Add web booking page for staff with summary page
ca6b4db	2026-09-29	Add Android booking screen and summary screen
0a15ec9	2026-09-30	Match all app screens to the shared card style
98d907b	2026-09-30	Add booking detail with change, cancel and QR screen and offline reservation cache
e39fdb6	2026-09-30	Remove machine specific JDK path from gradle properties

Table 18. *Commits by Janukshan S*

M4 • Herath D M S T (IT22639776)

Module: Booking Views, Dashboards & Operator Verification

- API: reservation list with filters, pending queue, prosumer and operations dashboards, QR verification and job completion
- Web: Backoffice and operations dashboards, booking monitor with filters and history
- Android: prosumer dashboard, bookings tabs with search, operator mode (pending queue, scan QR, verify, finalise), booking_history_cache and dashboard_snapshot tables

Commits on dev: 3 • files touched: 66 • lines +2955 / -1137

Commit	Date	Message
0892e6f	2026-09-29	Add dashboards, booking filters and operator QR completion so live counts and a server-checked QR gate job completion.
0e292b8	2026-09-30	Restore the classic Bootstrap navbar web UI while keeping Member 3 reservations and Member 4 dashboards.
c8b8454	2026-09-30	Add Gradle wrapper and pin JDK 17 so the Android app builds without a network timeout.

Table 19. *Commits by Herath D M S T*

11. Challenges

11.1 Team

- Keeping every rule in the web service. Both clients were tempted to check things locally (dates, roles); each rule was moved into a service so the web and Android clients show the same server message.
- Time zones. Slots are stored in UTC but opening hours are Sri Lanka time. Request times are normalised to UTC in the service and converted to local time only when checking the schedule or displaying.
- Double booking. Claiming a slot is a single MongoDB update filtered on isAvailable, so two prosumers cannot take the same slot at the same moment.
- Hosting on IIS over a LAN. Windows marked the Wi-Fi as a Public network and silently blocked inbound connections; IIS needed a restart after the Hosting Bundle; and a headless server slept and installed updates until its power settings were changed.
- Offline use on Android. The QR code, bookings, dashboard and station map had to keep working without signal, so each module cached its data in its own SQLite table and refreshes it when online.

11.2 De Silva R K D H (M1)

[De Silva R K D H: add one genuine paragraph about a challenge you faced in your module and how you solved it.]

11.3 Asath M M (M2)

Two bugs came out of testing my slot rules against the real API rather than from reading the code. Slot times sent with a +05:30 offset were read as local-kind DateTime values and shifted a second time, so valid evening slots were rejected; I fixed it by converting every request time to UTC once, at the start of the service. The nearby search treated missing coordinates as 0,0 and returned an empty list instead of an error; I made lat and lng required. Setting up IIS on a headless Windows laptop took longer than the code: the network profile, a failed IIS install until reboot, and Modern Standby taking the server offline.

11.4 Janukshan S (M3)

[Janukshan S: add one genuine paragraph about a challenge you faced in your module and how you solved it.]

11.5 Herath D M S T (M4)

[Herath D M S T: add one genuine paragraph about a challenge you faced in your module and how you solved it.]

12. Use of AI tools

The team followed AI Assessment Scale Level 2: AI tools were used for planning (ideas, task breakdown, research, design review and testing support), and an AI assistant drafted the diagrams and much of this report from the repository, which the team reviewed and edited. The C# web service, the web application, the Android application and the database were written by the team. Each member's disclosure is below.

De Silva R K D H (M1)

[De Silva R K D H: describe how you used AI tools during planning, and confirm the code in your commits is your own.]

Asath M M (M2)

I used an AI assistant (Claude) to break my module into steps, explain the existing code patterns in the repository, design the business rules and test cases, review my commits and run the API, web and device tests, which found two bugs that I then fixed. It also ran the Windows server setup and IIS deployment commands over SSH and drafted the diagrams and parts of this report, which I reviewed. The C#, Razor and Java code in my commits was written by me.

Janukshan S (M3)

[Janukshan S: describe how you used AI tools during planning, and confirm the code in your commits is your own.]

Herath D M S T (M4)

[Herath D M S T: describe how you used AI tools during planning, and confirm the code in your commits is your own.]

13. References

- [1] Sri Lanka Institute of Information Technology, "SE4040 Enterprise Application Development, Assignment 1: Smart Solar Microgrid Trading System," *Assignment brief*, 2026.
- [2] Microsoft, "ASP.NET Core documentation," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/>
- [3] Microsoft, "Host ASP.NET Core on Windows with IIS," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/iis/>
- [4] Microsoft, "ASP.NET Core Module (ANCM) for IIS," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/host-and-deploy/aspnet-core-module>
- [5] Microsoft, "Configure JWT bearer authentication in ASP.NET Core," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwt-authn>
- [6] Microsoft, "Hash passwords in ASP.NET Core," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/consumer-apis/password-hashing>
- [7] Microsoft, "Bundling and minification in ASP.NET Core," *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/aspnet/core/client-side/bundling-and-minification>
- [8] Swashbuckle.AspNetCore contributors, "Configure and customize SwaggerGen," *GitHub*. [Online]. Available: <https://github.com/domaindrivendev/Swashbuckle.AspNetCore/blob/master/docs/configure-and-customize-swaggergen.md>
- [9] MongoDB, Inc., "MongoDB C# Driver documentation," *MongoDB Docs*. [Online]. Available: <https://www.mongodb.com/docs/drivers/csharp/current/>
- [10] MongoDB, Inc., "C# Driver quick start," *MongoDB Docs*. [Online]. Available: <https://www.mongodb.com/docs/drivers/csharp/current/quick-start/>
- [11] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," *IETF RFC 7519*, May 2015. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc7519>
- [12] Bootstrap team, "Bootstrap 5.3 documentation," *getbootstrap.com*. [Online]. Available: <https://getbootstrap.com/docs/5.3/>
- [13] Android Developers, "SQLiteOpenHelper," *Android API reference*. [Online]. Available: <https://developer.android.com/reference/android/database/sqlite/SQLiteOpenHelper>
- [14] Android Developers, "URLConnection," *Android API reference*. [Online]. Available: <https://developer.android.com/reference/java/net/URLConnection>
- [15] Android Developers, "Request location permissions," *Android Developers*. [Online]. Available: <https://developer.android.com/develop/sensors-and-location/location/permissions>
- [16] Google, "Use API keys with Maps SDK for Android," *Google for Developers*. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk/get-api-key>
- [17] Google, "Add a map to your Android app," *Google for Developers*. [Online]. Available: <https://developers.google.com/maps/documentation/android-sdk/map>
- [18] Google, "Material Components for Android," *GitHub*. [Online]. Available: <https://github.com/material-components/material-components-android>
- [19] JourneyApps, "ZXing Android Embedded," *GitHub*. [Online]. Available: <https://github.com/journeyapps/zxing-android-embedded>
- [20] R. W. Sinnott, "Virtues of the Haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

Appendix A. Source code

All hand-written source files on the dev branch (253 files, 14,305 lines), grouped by project. Third-party libraries (wwwroot/lib), build output and the Gradle wrapper are excluded. The file header in each file names its author.

A.1 Web service (WebService/SmartSolar.Api)

WebService/SmartSolar.Api/Common/AccountRules.cs M1 · 72 lines

```
/*
 * File: AccountRules.cs
 * Description: Shared field checks for staff and prosumer accounts. The API is the only place these run.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */

using System.ComponentModel.DataAnnotations;
using System.Text.RegularExpressions;

namespace SmartSolar.Api.Common;

public static class AccountRules
{
    public const int MinPasswordLength = 8;

    private static readonly Regex PhonePattern = new(@"^(+94\d{9}|0\d{9}|\d{9,12})$", RegexOptions.Compiled);

    // Throws 400 when a required string is blank.
    public static void RequireText(string? value, string message)
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, message);
        }
    }

    // Rejects a non-blank value that is not an email address. Blank is allowed.
    public static void RequireEmailFormat(string? email)
    {
        var trimmed = email?.Trim() ?? string.Empty;
        if (trimmed.Length == 0)
        {
            return;
        }

        if (!new EmailAddressAttribute().IsValid(trimmed))
        {
            throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "Email is not a valid address.");
        }
    }

    // Rejects passwords shorter than the service minimum. Clients are not trusted.
    public static void RequirePasswordLength(string password)
    {
        if (password.Length < MinPasswordLength)
        {
            throw new ApiException(
                StatusCodes.Status400BadRequest,
                ErrorCodes.ValidationError,
                $"Password must be at least {MinPasswordLength} characters.");
        }
    }

    // Accepts blank phone, otherwise requires a Sri Lankan-style or 9–12 digit number.
    public static void RequirePhoneFormat(string? phone)
    {
        var trimmed = phone?.Trim().Replace(" ", string.Empty) ?? string.Empty;
        if (trimmed.Length == 0)
        {
            return;
        }

        if (!PhonePattern.IsMatch(trimmed))
        {
            throw new ApiException(
                StatusCodes.Status400BadRequest,
                ErrorCodes.ValidationError,
                "Phone must be 10 digits starting with 0, or +94 followed by 9 digits.");
        }
    }
}
```

WebService/SmartSolar.Api/Common/ApiException.cs M1 · 22 lines

```
/*
 * File: ApiException.cs
 * Description: Thrown by the service layer so the global handler can return a contract error body.
SE4040 · Smart Solar Microgrid Trading System · 49
```

```
* Author: DE SILVA R K D H (IT22001252)
* Created: 20/09/2026
*/
```

```
namespace SmartSolar.Api.Common;
```

```
public class ApiException : Exception
{
    public int StatusCode { get; }
    public string Code { get; }

    // Captures an HTTP status, machine-readable code and human-readable message.
    public ApiException(int statusCode, string code, string message)
        : base(message)
    {
        StatusCode = statusCode;
        Code = code;
    }
}
```

WebService/SmartSolar.Api/Common/ErrorCodes.cs M1 · 29 lines

```
/*
 * File: ErrorCodes.cs
 * Description: Error `code` values returned in the contract `{ code, message }` body.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
namespace SmartSolar.Api.Common;
```

```
public static class ErrorCodes
{
    public const string ValidationError = "VALIDATION_ERROR";
    public const string Unauthorized = "UNAUTHORIZED";
    public const string Forbidden = "FORBIDDEN";
    public const string NotFound = "NOT_FOUND";
    public const string InvalidCredentials = "INVALID_CREDENTIALS";
    public const string AccountNotActive = "ACCOUNT_NOT_ACTIVE";
    public const string UsernameExists = "USERNAME_EXISTS";
    public const string NicExists = "NIC_EXISTS";
    public const string RuleSevenDays = "RULE_7DAYS";
    public const string RuleTwelveHours = "RULE_12H";
    public const string SlotTaken = "SLOT_TAKEN";
    public const string SlotOverlap = "SLOT_OVERLAP";
    public const string InvalidState = "INVALID_STATE";
    public const string QrInvalid = "QR_INVALID";
    public const string QrAlreadyUsed = "QR_ALREADY_USED";
    public const string StationHasReservations = "STATION_HAS_RESERVATIONS";
    public const string InternalError = "INTERNAL_ERROR";
}
```

WebService/SmartSolar.Api/Common/JsonDefaults.cs M1 · 26 lines

```
/*
 * File: JsonDefaults.cs
 * Description: Shared camelCase JSON options so error bodies match successful responses.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
using System.Text.Json;
using System.Text.Json.Serialization;
```

```
namespace SmartSolar.Api.Common;
```

```
public static class JsonDefaults
{
    public static readonly JsonSerializerOptions Options = Create();

    // Builds the serializer used by middleware and JWT challenge/forbid handlers.
    private static JsonSerializerOptions Create()
    {
        return new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            DefaultIgnoreCondition = JsonIgnoreCondition.Never
        };
    }
}
```

WebService/SmartSolar.Api/Common/NicRules.cs M1 · 41 lines

```
/*
 * File: NicRules.cs
 * Description: Sri Lankan NIC format used for prosumer primary keys. Rules live in the service layer.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
```

```
using System.Text.RegularExpressions;
```

```

namespace SmartSolar.Api.Common;

public static class NicRules
{
    private static readonly Regex OldNic = new(@"^\d{9}[VX]$", RegexOptions.Compiled);
    private static readonly Regex NewNic = new(@"^\d{12}%", RegexOptions.Compiled);

    // Trims and uppercases a NIC so V/X and stored keys match.
    public static string Normalize(string nic)
    {
        return nic.Trim().ToUpperInvariant();
    }

    // Returns true for the old 9-digit+V/X form or the 12-digit form.
    public static bool IsValid(string nic)
    {
        var value = Normalize(nic);
        return OldNic.IsMatch(value) || NewNic.IsMatch(value);
    }

    // Throws VALIDATION_ERROR when the NIC is missing or not a recognised Sri Lankan format.
    public static void RequireValid(string? nic)
    {
        if (string.IsNullOrEmpty(nic) || !IsValid(nic))
        {
            throw new ApiException(
                StatusCodes.Status400BadRequest,
                ErrorCodes.ValidationError,
                "NIC must be 9 digits plus V or X, or 12 digits.");
        }
    }
}

```

WebService/SmartSolar.Api/Common/ReservationStatuses.cs M1 · 16 lines

```

/*
 * File: ReservationStatuses.cs
 * Description: Allowed values for EnergyReservation.status.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

namespace SmartSolar.Api.Common;

public static class ReservationStatuses
{
    public const string Pending = "Pending";
    public const string Approved = "Approved";
    public const string Cancelled = "Cancelled";
    public const string Completed = "Completed";
}

```

WebService/SmartSolar.Api/Common/RoleNames.cs M1 · 15 lines

```

/*
 * File: RoleNames.cs
 * Description: Role strings used by JWT claims and [Authorize(Roles)] attributes.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

namespace SmartSolar.Api.Common;

public static class RoleNames
{
    public const string Backoffice = "Backoffice";
    public const string GridOperator = "GridOperator";
    public const string Prosumer = "Prosumer";
}

```

WebService/SmartSolar.Api/Common/SriLankaClock.cs M4 · 59 lines

```

/*
 * File: SriLankaClock.cs
 * Description: Sri Lanka (UTC+05:30) day boundaries for dashboard "today" and QR scan windows.
 * Author: samudith
 * Created: 29/09/2026
 */

namespace SmartSolar.Api.Common;

public static class SriLankaClock
{
    private static readonly TimeSpan Offset = new(5, 30, 0);
    private static readonly TimeSpan NearWindow = TimeSpan.FromHours(2);

    // Returns the current UTC time used by booking rules.
    public static DateTime UtcNow()
    {

```

```

    return DateTime.UtcNow;
}

// Converts a UTC time to Sri Lanka local time.
public static DateTime ToLocal(DateTime utc)
{
    return DateTime.SpecifyKind(utc, DateTimeKind.Utc).Add(Offset);
}

// Converts a request time to UTC. Unspecified is already UTC and must not be shifted by this machine's zone.
public static DateTime ToUtc(DateTime value)
{
    return value.Kind switch
    {
        DateTimeKind.Utc => value,
        DateTimeKind.Local => value.ToUniversalTime(),
        _ => DateTime.SpecifyKind(value, DateTimeKind.Utc)
    };
}

// Returns the UTC start (inclusive) and end (exclusive) of the Sri Lanka calendar day containing utc.
public static (DateTime StartUtc, DateTime EndUtc) DayRangeUtc(DateTime utc)
{
    var local = DateTime.SpecifyKind(utc, DateTimeKind.Utc).Add(Offset);
    var startLocal = new DateTime(local.Year, local.Month, local.Day, 0, 0, 0, DateTimeKind.Unspecified);
    var startUtc = DateTime.SpecifyKind(startLocal.Subtract(Offset), DateTimeKind.Utc);
    return (startUtc, startUtc.AddDays(1));
}

// True when the slot is on today's Sri Lanka date, or within two hours of now.
public static bool CanScanAt(DateTime scheduledAtUtc, DateTime nowUtc)
{
    var (start, end) = DayRangeUtc(nowUtc);
    if (scheduledAtUtc >= start && scheduledAtUtc < end)
    {
        return true;
    }

    var delta = scheduledAtUtc - nowUtc;
    return delta <= NearWindow && delta >= -NearWindow;
}
}

```

WebService/SmartSolar.Api/Common/StationStatuses.cs M1 · 14 lines

```

/*
 * File: StationStatuses.cs
 * Description: Allowed values for SolarStationInfo.status.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

```

```
namespace SmartSolar.Api.Common;
```

```

public static class StationStatuses
{
    public const string Active = "Active";
    public const string Inactive = "Inactive";
}

```

WebService/SmartSolar.Api/Common/UserStatuses.cs M1 · 15 lines

```

/*
 * File: UserStatuses.cs
 * Description: Allowed values for Users.status, stored as strings in MongoDB.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

```

```
namespace SmartSolar.Api.Common;
```

```

public static class UserStatuses
{
    public const string Pending = "Pending";
    public const string Active = "Active";
    public const string Deactivated = "Deactivated";
}

```

WebService/SmartSolar.Api/Configuration/JwtSettings.cs M1 · 24 lines

```

/*
 * File: JwtSettings.cs
 * Description: JWT issuer/audience/key settings bound from configuration or environment.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

```

```
namespace SmartSolar.Api.Configuration;
```

```
public class JwtSettings
```

```

{
    public const string SectionName = "Jwt";

    // Value in the committed *.example files. AddSmartSolarJwt refuses to use it as a real key.
    public const string PlaceholderKey = "REPLACE_WITH_A_LONG_RANDOM_SECRET_AT_LEAST_32_CHARS";

    public string Issuer { get; set; } = string.Empty;

    public string Audience { get; set; } = string.Empty;

    public int ExpiryMinutes { get; set; } = 60;

    public string Key { get; set; } = string.Empty;
}

```

WebService/SmartSolar.Api/Configuration/MongoDbSettings.cs M1 · 17 lines

```

/*
 * File: MongoDbSettings.cs
 * Description: Mongo connection settings bound from configuration (never hard-coded).
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

namespace SmartSolar.Api.Configuration;

public class MongoDbSettings
{
    public const string SectionName = "MongoDb";

    public string ConnectionString { get; set; } = string.Empty;

    public string DatabaseName { get; set; } = string.Empty;
}

```

WebService/SmartSolar.Api/Controllers/AuthController.cs M1 · 52 lines

```

/*
 * File: AuthController.cs
 * Description: Thin HTTP adapter for login. All credential rules live in AuthService.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/auth")]
[Produces("application/json")]
public class AuthController : ControllerBase
{
    private readonly IAuthService _authService;

    // Injects the auth use-case used by POST /api/auth/login.
    public AuthController(IAuthService authService)
    {
        _authService = authService;
    }

    // Issues a JWT when username/NIC and password match an Active account.
    [HttpPost("login")]
    [AllowAnonymous]
    [ProducesResponseType(typeof(LoginResponseDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status401Unauthorized)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<LoginResponseDto>> Login([FromBody] LoginRequestDto request, CancellationToken cancellationToken)
    {
        var result = await _authService.LoginAsync(request, cancellationToken);
        return Ok(result);
    }

    // Creates a Pending prosumer; no JWT is issued until Backoffice activates the account.
    [HttpPost("register")]
    [AllowAnonymous]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status201Created)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<UserDto>> Register([FromBody] RegisterProsumerRequestDto request, CancellationToken cancellationToken)
    {
        var created = await _authService.RegisterAsync(request, cancellationToken);
        return Created($"api/prosumers/{Uri.EscapeDataString(created.Nic!)}", created);
    }
}

```

WebService/SmartSolar.Api/Controllers/DashboardController.cs M4 · 52 lines

```
/*
 * File: DashboardController.cs
 * Description: Prosumer and operations dashboards. Every count is read from the database.
 * Author: samudith
 * Created: 29/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/dashboard")]
[Produces("application/json")]
public class DashboardController : ControllerBase
{
    private readonly IReservationQueryService _queries;

    // Injects the read service that computes the dashboard figures.
    public DashboardController(IReservationQueryService queries)
    {
        _queries = queries;
    }

    // Returns the signed-in prosumer's pending, approved-future and next booking.
    [HttpGet("me")]
    [Authorize(Roles = RoleNames.Prosumer)]
    [ProducesResponseType(typeof(ProsumerDashboardDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status403Forbidden)]
    public async Task<ActionResult<ProsumerDashboardDto>> Me(CancellationTokens cancellationToken)
    {
        var actor = new ReservationActor(
            User.FindFirst("sub")?.Value ?? string.Empty,
            User.FindFirst("role")?.Value ?? string.Empty,
            User.FindFirst("nic")?.Value);

        return Ok(await _queries.GetProsumerDashboardAsync(actor, cancellationToken));
    }

    // Returns today's bookings, pending approvals, active stations and completions for the web Home.
    [HttpGet("operations")]
    [Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
    [ProducesResponseType(typeof(OperationsDashboardDto), StatusCodes.Status200OK)]
    public async Task<ActionResult<OperationsDashboardDto>> Operations(CancellationTokens cancellationToken)
    {
        return Ok(await _queries.GetOperationsDashboardAsync(cancellationToken));
    }
}
```

WebService/SmartSolar.Api/Controllers/HealthController.cs M1 · 31 lines

```
/*
 * File: HealthController.cs
 * Description: Anonymous health probe used to prove the API is reachable on LAN/IIS.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Controllers;

[ApiController]
[AllowAnonymous]
[Produces("application/json")]
public class HealthController : ControllerBase
{
    // Returns process health and current UTC server time.
    [HttpGet("/health")]
    [HttpGet("/api/health")]
    [ProducesResponseType(typeof(HealthResponseDto), StatusCodes.Status200OK)]
    public ActionResult<HealthResponseDto> Get()
    {
        return Ok(new HealthResponseDto
        {
            Status = "Healthy",
            ServerTime = DateTime.UtcNow
        });
    }
}
```

WebService/SmartSolar.Api/Controllers/MeController.cs M1 · 55 lines

```
/*
 * File: MeController.cs
```

```

* Description: Current prosumer profile endpoints; identity always comes from the JWT.
* Author: DE SILVA R K D H (IT22001252)
* Created: 27/09/2026
*/

```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/me")]
[Authorize(Roles = RoleNames.Prosumer)]
[Produces("application/json")]
public class MeController : ControllerBase
{
    private readonly IProsumerService _prosumers;

    public MeController(IProsumerService prosumers)
    {
        _prosumers = prosumers;
    }

    [HttpGet]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status200OK)]
    public async Task<ActionResult<UserDto>> Get(CancellationToken cancellationToken)
    {
        return Ok(await _prosumers.GetMeAsync(CurrentNic(), cancellationToken));
    }

    [HttpPut]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status200OK)]
    public async Task<ActionResult<UserDto>> Update([FromBody] UpdateProsumerRequestDto request, CancellationToken cancellationToken)
    {
        return Ok(await _prosumers.UpdateMeAsync(CurrentNic(), request, cancellationToken));
    }

    [HttpPost("request-deactivation")]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status200OK)]
    public async Task<ActionResult<UserDto>> RequestDeactivation(CancellationToken cancellationToken)
    {
        return Ok(await _prosumers.DeactivateAsync(CurrentNic(), cancellationToken));
    }

    private string CurrentNic()
    {
        return User.FindFirst("nic")?.Value
            ?? throw new ApiException(StatusCodes.Status401Unauthorized, ErrorCodes.Unauthorized, "The token does not contain a prosumer NIC.");
    }
}

```

WebService/SmartSolar.Api/Controllers/OperatorController.cs M4 · 53 lines

```

/*
* File: OperatorController.cs
* Description: Grid Operator QR verification and job completion. The client only displays the API result.
* Author: samudith
* Created: 29/09/2026
*/

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/reservations")]
[Authorize(Roles = RoleNames.GridOperator)]
[Produces("application/json")]
public class OperatorController : ControllerBase
{
    private readonly IOperatorService _operators;

    // Injects the service that checks and consumes a QR token.
    public OperatorController(IOperatorService operators)
    {
        _operators = operators;
    }

    // Looks up a QR token and returns the booking, or QR_INVALID / QR_ALREADY_USED.
    [HttpPost("verify-qr")]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<ReservationDto>> VerifyQr([FromBody] VerifyQrRequestDto request, CancellationToken cancellationToken)
    {
        return Ok(await _operators.VerifyQrAsync(request.QrToken, cancellationToken));
    }
}

```

```
// Completes an approved booking after the same checks used by verify-qr.
[HttpPost("{id}/complete")]
[ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
[ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
[ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
public async Task<ActionResult<ReservationDto>> Complete(string id, Cancellation_token cancellationToken)
{
    var actor = new ReservationActor(
        User.FindFirst("sub")?.Value ?? string.Empty,
        User.FindFirst("role")?.Value ?? string.Empty,
        User.FindFirst("nic")?.Value);

    return Ok(await _operators.CompleteAsync(id, actor, cancellationToken));
}
}
```

WebService/SmartSolar.Api/Controllers/ProsumersController.cs M1 · 101 lines

```
/*
 * File: ProsumersController.cs
 * Description: Staff HTTP endpoints for searching and administering prosumer accounts.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
```

```
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOS;
using SmartSolar.Api.Services;
```

```
namespace SmartSolar.Api.Controllers;
```

```
[ApiController]
[Route("api/prosumers")]
[Produces("application/json")]
public class ProsumersController : ControllerBase
{
    private readonly IProsumerService _prosumers;

    public ProsumersController(IProsumerService prosumers)
    {
        _prosumers = prosumers;
    }

    [HttpGet]
    [Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
    public async Task<ActionResult<IReadOnlyList<UserDto>>> Search(
        [FromQuery] string? status,
        [FromQuery(Name = "q")] string? query,
        Cancellation_token cancellationToken)
    {
        return Ok(await _prosumers.SearchAsync(status, query, cancellationToken));
    }

    [Authorize(Roles = RoleNames.Backoffice)]
    [HttpGet("pending")]
    public async Task<ActionResult<IReadOnlyList<UserDto>>> Pending(
        Cancellation_token cancellationToken)
    {
        return Ok(await _prosumers.ListPendingAsync(cancellationToken));
    }

    [Authorize(Roles = RoleNames.Backoffice)]
    [HttpGet("{nic}")]
    public async Task<ActionResult<UserDto>> Get(
        string nic,
        Cancellation_token cancellationToken)
    {
        return Ok(await _prosumers.GetAsync(nic, cancellationToken));
    }

    [Authorize(Roles = RoleNames.Backoffice)]
    [HttpPost]
    public async Task<ActionResult<UserDto>> Create(
        [FromBody] CreateProsumerRequestDto request,
        Cancellation_token cancellationToken)
    {
        var created = await _prosumers.CreateAsync(request, cancellationToken);
        return CreatedAtAction(nameof(Get), new { nic = created.Nic }, created);
    }

    [Authorize(Roles = RoleNames.Backoffice)]
    [HttpPut("{nic}")]
    public async Task<ActionResult<UserDto>> Update(
        string nic,
        [FromBody] UpdateProsumerRequestDto request,
        Cancellation_token cancellationToken)
    {
        return Ok(await _prosumers.UpdateAsync(nic, request, cancellationToken));
    }

    [Authorize(Roles = RoleNames.Backoffice)]
```

```

[HttpPost("{nic}/deactivate")]
public async Task<ActionResult<UserDto>> Deactivate(
    string nic,
    CancellationToken cancellationToken)
{
    return Ok(await _prosumers.DeactivateAsync(nic, cancellationToken));
}

[Authorize(Roles = RoleNames.Backoffice)]
[HttpPost("{nic}/reactivate")]
public async Task<ActionResult<UserDto>> Reactivate(
    string nic,
    CancellationToken cancellationToken)
{
    return Ok(await _prosumers.ReactivateAsync(nic, cancellationToken));
}

[Authorize(Roles = RoleNames.Backoffice)]
[HttpPost("{nic}/activate")]
public async Task<ActionResult<UserDto>> Activate(
    string nic,
    CancellationToken cancellationToken)
{
    return Ok(await _prosumers.ActivateAsync(nic, cancellationToken));
}
}

```

WebService/SmartSolar.Api/Controllers/ReservationQueryController.cs M4 · 64 lines

```

/*
 * File: ReservationQueryController.cs
 * Description: Reservation list, history filters and the operator approval queue.
 * Author: samudith
 * Created: 29/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/reservations")]
[Authorize(Roles = RoleNames.Prosumer + ", " + RoleNames.Backoffice + ", " + RoleNames.GridOperator)]
[Produces("application/json")]
public class ReservationQueryController : ControllerBase
{
    private readonly IReservationQueryService _queries;

    // Injects the read service that applies prosumer scoping.
    public ReservationQueryController(IReservationQueryService queries)
    {
        _queries = queries;
    }

    // Lists bookings. Filters combine. A prosumer's NIC comes from the token, not the query.
    [HttpGet]
    [ProducesResponseType(typeof(ReservationPageDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    public async Task<ActionResult<ReservationPageDto>> Search(
        [FromQuery] string? status,
        [FromQuery] DateTime? from,
        [FromQuery] DateTime? to,
        [FromQuery] string? stationId,
        [FromQuery] string? q,
        [FromQuery] string? nic,
        [FromQuery] int page = 1,
        CancellationToken cancellationToken = default)
    {
        return Ok(await _queries.SearchAsync(CurrentActor(), status, from, to, stationId, q, nic, page, cancellationToken));
    }

    // Returns future Pending bookings for Backoffice and Grid Operators.
    [HttpGet("pending")]
    [Authorize(Roles = RoleNames.Backoffice + ", " + RoleNames.GridOperator)]
    [ProducesResponseType(typeof(ReservationPageDto), StatusCodes.Status200OK)]
    public async Task<ActionResult<ReservationPageDto>> Pending([FromQuery] int page = 1, CancellationToken cancellationToken = default)
    {
        return Ok(await _queries.GetPendingQueueAsync(page, cancellationToken));
    }

    // Builds the current user from the JWT claims.
    private ReservationActor CurrentActor()
    {
        return new ReservationActor(
            User.FindFirst("sub")?.Value ?? string.Empty,
            User.FindFirst("role")?.Value ?? string.Empty,
            User.FindFirst("nic")?.Value);
    }
}

```

```

/*
 * File: ReservationsController.cs
 * Description: Reservation endpoints for web and mobile. Only reads the request and calls ReservationService.
 * Author: Janukshan S (IT22635266)
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.ModelBinding;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/reservations")]
[Authorize(Roles = RoleNames.Prosumer + "," + RoleNames.Backoffice + "," + RoleNames.GridOperator)]
[Produces("application/json")]
public class ReservationsController : ControllerBase
{
    private readonly IReservationService _reservations;

    // Injects the service that holds the reservation rules.
    public ReservationsController(IReservationService reservations)
    {
        _reservations = reservations;
    }

    // Lists the stations that are taking bookings, for the booking pickers.
    [HttpGet("bookable-stations")]
    [ProducesResponseType(typeof(IReadOnlyList<BookingStationDto>), StatusCodes.Status200OK)]
    public async Task<ActionResult<IReadOnlyList<BookingStationDto>>> GetBookableStations(CancellationToken cancellationToken)
    {
        return Ok(await _reservations.GetBookableStationsAsync(cancellationToken));
    }

    // Lists free slots of a station inside the 7-day window. `date` is a Sri Lanka date (yyyy-MM-dd).
    [HttpGet("available-slots")]
    [ProducesResponseType(typeof(IReadOnlyList<AvailableSlotDto>), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    public async Task<ActionResult<IReadOnlyList<AvailableSlotDto>>> GetAvailableSlots(
        [FromQuery] string stationId,
        [FromQuery] DateOnly? date,
        CancellationToken cancellationToken)
    {
        return Ok(await _reservations.GetAvailableSlotsAsync(stationId, date, cancellationToken));
    }

    // Creates a booking. Staff send prosumerNic to book on behalf of a prosumer.
    [HttpPost]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status201Created)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<ReservationDto>> Create([FromBody] CreateReservationRequestDto request, CancellationToken cancellationToken)
    {
        var created = await _reservations.CreateAsync(request, CurrentActor(), cancellationToken);
        return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
    }

    // Returns one booking. Prosumers get 403 for someone else's booking.
    [HttpGet("{id}")]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status403Forbidden)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    public async Task<ActionResult<ReservationDto>> GetById(string id, CancellationToken cancellationToken)
    {
        return Ok(await _reservations.GetByIdAsync(id, CurrentActor(), cancellationToken));
    }

    // Moves a booking to another slot. An approved booking goes back to Pending.
    [HttpPut("{id}")]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status403Forbidden)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<ReservationDto>> Update(string id, [FromBody] UpdateReservationRequestDto request, CancellationToken cancellationToken)
    {
        return Ok(await _reservations.UpdateAsync(id, request, CurrentActor(), cancellationToken));
    }

    // Cancels a booking. The body with a reason is optional.
    [HttpPost("{id}/cancel")]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status403Forbidden)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<ReservationDto>> Cancel(
        string id,
        [FromBody(EmptyBodyBehavior = EmptyBodyBehavior.Allow)] CancelReservationRequestDto? request,
        CancellationToken cancellationToken)
    {

```

```

        return Ok(await _reservations.CancelAsync(id, request, CurrentActor(), cancellationToken));
    }

    // Approves a pending booking and returns it with its QR token. Staff only.
    [HttpPost("{id}/approve")]
    [Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
    [ProducesResponseType(typeof(ReservationDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status403Forbidden)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<ReservationDto>> Approve(string id, CancellationToken cancellationToken)
    {
        return Ok(await _reservations.ApproveAsync(id, CurrentActor(), cancellationToken));
    }

    // Builds the current user from the JWT claims.
    private ReservationActor CurrentActor()
    {
        return new ReservationActor(
            User.FindFirst("sub")?.Value ?? string.Empty,
            User.FindFirst("role")?.Value ?? string.Empty,
            User.FindFirst("nic")?.Value);
    }
}

```

WebService/SmartSolar.Api/Controllers/SlotsController.cs M2 · 79 lines

```

/*
 * File: SlotsController.cs
 * Description: HTTP endpoints for listing, creating, updating, deleting and opening or closing slots. Rules stay in SlotService.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOS;
using SmartSolar.Api.Models;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Produces("application/json")]
public class SlotsController : ControllerBase
{
    private readonly ISlotService _slots;

    // Injects the slot service that owns the schedule and battery rules.
    public SlotsController(ISlotService slots)
    {
        _slots = slots;
    }

    // Lists a station's slots for any logged-in role.
    [HttpGet("api/stations/{stationId}/slots")]
    [Authorize]
    public async Task<ActionResult<IReadOnlyList<Slot>>> List(string stationId, CancellationToken cancellationToken)
    {
        return Ok(await _slots.ListByStationAsync(stationId, cancellationToken));
    }

    // Creates a slot. Backoffice and GridOperator.
    [HttpPost("api/stations/{stationId}/slots")]
    [Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
    public async Task<ActionResult<Slot>> Create(
        string stationId,
        [FromBody] SaveSlotRequestDto request,
        CancellationToken cancellationToken)
    {
        var created = await _slots.CreateAsync(stationId, request, cancellationToken);
        return CreatedAtAction(nameof(List), new { stationId = created.StationId }, created);
    }

    // Updates a slot. Backoffice and GridOperator.
    [HttpPut("api/slots/{id}")]
    [Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
    public async Task<ActionResult<Slot>> Update(
        string id,
        [FromBody] SaveSlotRequestDto request,
        CancellationToken cancellationToken)
    {
        return Ok(await _slots.UpdateAsync(id, request, cancellationToken));
    }

    // Deletes a slot. Backoffice and GridOperator.
    [HttpDelete("api/slots/{id}")]
    [Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
    public async Task<IActionResult> Delete(string id, CancellationToken cancellationToken)
    {
        await _slots.DeleteAsync(id, cancellationToken);
        return NoContent();
    }
}

```

```

// Opens or closes a free slot. GridOperator only.
[HttpPut("api/slots/{id}/availability")]
[Authorize(Roles = RoleNames.GridOperator)]
public async Task<ActionResult<Slot>> SetAvailability(
    string id,
    [FromBody] SlotAvailabilityRequestDto request,
    CancellationToken cancellationToken)
{
    return Ok(await _slots.SetAvailabilityAsync(id, request, cancellationToken));
}
}

```

WebService/SmartSolar.Api/Controllers/StationsController.cs M2 · 95 lines

```

/*
 * File: StationsController.cs
 * Description: HTTP endpoints for listing, reading, updating, activating and finding nearby stations. Rules stay in StationService.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOS;
using SmartSolar.Api.Models;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/stations")]
[Produces("application/json")]
public class StationsController : ControllerBase
{
    private readonly IStationService _stations;

    // Injects the station service that owns the node rules.
    public StationsController(IStationService stations)
    {
        _stations = stations;
    }

    // Lists stations for any logged-in role.
    [HttpGet]
    [Authorize]
    public async Task<ActionResult<IReadOnlyList<Station>>> List(CancellationToken cancellationToken)
    {
        return Ok(await _stations.ListAsync(cancellationToken));
    }

    // Returns Active stations near a GPS point for any logged-in role.
    [HttpGet("nearby")]
    [Authorize]
    public async Task<ActionResult<IReadOnlyList<NearbyStationDto>>> Nearby(
        [FromQuery] double? lat,
        [FromQuery] double? lng,
        [FromQuery] double radiusKm = 10,
        CancellationToken cancellationToken = default)
    {
        return Ok(await _stations.NearbyAsync(lat, lng, radiusKm, cancellationToken));
    }

    // Returns one station for any logged-in role.
    [HttpGet("{id}")]
    [Authorize]
    public async Task<ActionResult<Station>> Get(string id, CancellationToken cancellationToken)
    {
        return Ok(await _stations.GetAsync(id, cancellationToken));
    }

    // Creates a station. Backoffice only.
    [HttpPost]
    [Authorize(Roles = RoleNames.Backoffice)]
    public async Task<ActionResult<Station>> Create(
        [FromBody] SaveStationRequestDto request,
        CancellationToken cancellationToken)
    {
        var created = await _stations.CreateAsync(request, cancellationToken);
        return CreatedAtAction(nameof(Get), new { id = created.Id }, created);
    }

    // Updates a station. Backoffice only.
    [HttpPut("{id}")]
    [Authorize(Roles = RoleNames.Backoffice)]
    public async Task<ActionResult<Station>> Update(
        string id,
        [FromBody] SaveStationRequestDto request,
        CancellationToken cancellationToken)
    {
        return Ok(await _stations.UpdateAsync(id, request, cancellationToken));
    }

    // Deactivates a station. Backoffice only.
    [HttpPost("{id}/deactivate")]

```

```

[Authorize(Roles = RoleNames.Backoffice)]
public async Task<ActionResult<Station>> Deactivate(string id, CancellationToken cancellationToken)
{
    return Ok(await _stations.DeactivateAsync(id, cancellationToken));
}

// Activates a station. Backoffice only.
[HttpPost("{id}/activate")]
[Authorize(Roles = RoleNames.Backoffice)]
public async Task<ActionResult<Station>> Activate(string id, CancellationToken cancellationToken)
{
    return Ok(await _stations.ActivateAsync(id, cancellationToken));
}
}

```

WebService/SmartSolar.Api/Controllers/UsersController.cs M1 · 81 lines

```

/*
 * File: UsersController.cs
 * Description: Backoffice-only staff account endpoints. Thin HTTP adapter over UserService.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Services;

namespace SmartSolar.Api.Controllers;

[ApiController]
[Route("api/users")]
[Authorize(Roles = RoleNames.Backoffice)]
[Produces("application/json")]
public class UsersController : ControllerBase
{
    private readonly IUserService _users;

    // Injects the user service that maps entities to public DTOs.
    public UsersController(IUserService users)
    {
        _users = users;
    }

    // Lists staff accounts. Other roles receive 403.
    [HttpGet]
    [ProducesResponseType(typeof(IReadOnlyList<UserDto>), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status401Unauthorized)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status403Forbidden)]
    public async Task<ActionResult<IReadOnlyList<UserDto>>> GetAll(CancellationToken cancellationToken)
    {
        return Ok(await _users.ListStaffAsync(cancellationToken));
    }

    // Returns one staff user by id.
    [HttpGet("{id}")]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    public async Task<ActionResult<UserDto>> GetById(string id, CancellationToken cancellationToken)
    {
        return Ok(await _users.GetStaffByIdAsync(id, cancellationToken));
    }

    // Creates a Backoffice or GridOperator account.
    [HttpPost]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status201Created)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status409Conflict)]
    public async Task<ActionResult<UserDto>> Create([FromBody] CreateStaffRequestDto request, CancellationToken cancellationToken)
    {
        var created = await _users.CreateStaffAsync(request, cancellationToken);
        return CreatedAtAction(nameof(GetById), new { id = created.Id }, created);
    }

    // Updates a staff account.
    [HttpPut("{id}")]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    public async Task<ActionResult<UserDto>> Update(string id, [FromBody] UpdateStaffRequestDto request, CancellationToken cancellationToken)
    {
        var actorId = User.FindFirst("sub")?.Value;
        return Ok(await _users.UpdateStaffAsync(id, request, actorId, cancellationToken));
    }

    // Deactivates a staff account so they can no longer log in.
    [HttpPost("{id}/deactivate")]
    [ProducesResponseType(typeof(UserDto), StatusCodes.Status200OK)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status400BadRequest)]
    [ProducesResponseType(typeof(ApiErrorDto), StatusCodes.Status404NotFound)]
    public async Task<ActionResult<UserDto>> Deactivate(string id, CancellationToken cancellationToken)
    {
        var actorId = User.FindFirst("sub")?.Value;

```

```

        return Ok(await _users.DeactivateStaffAsync(id, actorId, cancellationToken));
    }
}

```

WebService/SmartSolar.Api/DTOs/ApiErrorDto.cs M1 · 26 lines

```

/*
 * File: ApiErrorDto.cs
 * Description: Error body for every non-2xx response: { code, message }.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

namespace SmartSolar.Api.DTOs;

public class ApiErrorDto
{
    public string Code { get; set; } = string.Empty;
    public string Message { get; set; } = string.Empty;

    // Parameterless constructor required for JSON serialization.
    public ApiErrorDto()
    {
    }

    // Builds the contract error object used by middleware and JWT events.
    public ApiErrorDto(string code, string message)
    {
        Code = code;
        Message = message;
    }
}

```

WebService/SmartSolar.Api/DTOs/AvailableSlotDto.cs M3 · 29 lines

```

/*
 * File: AvailableSlotDto.cs
 * Description: A free slot that can be booked right now (inside the 7-day window).
 * Author: Janukshan S (IT22635266)
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.DTOs;

public class AvailableSlotDto
{
    public string Id { get; set; } = string.Empty;
    public string StationId { get; set; } = string.Empty;
    public DateTime StartTime { get; set; }
    public DateTime EndTime { get; set; }

    // Maps a slot document to the picker shape.
    public static AvailableSlotDto From(Slot slot)
    {
        return new AvailableSlotDto
        {
            Id = slot.Id,
            StationId = slot.StationId,
            StartTime = slot.StartTime,
            EndTime = slot.EndTime
        };
    }
}

```

WebService/SmartSolar.Api/DTOs/BookingStationDto.cs M3 · 13 lines

```

/*
 * File: BookingStationDto.cs
 * Description: A station that is currently taking bookings, used by the booking pickers.
 * Author: Janukshan S (IT22635266)
 */

namespace SmartSolar.Api.DTOs;

public class BookingStationDto
{
    public string Id { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
}

```

WebService/SmartSolar.Api/DTOs/CancelReservationRequestDto.cs M3 · 12 lines

```

/*
 * File: CancelReservationRequestDto.cs
 * Description: Optional body for POST /api/reservations/{id}/cancel.
 * Author: Janukshan S (IT22635266)
 */

```

```
namespace SmartSolar.Api.DTOs;

public class CancelReservationRequestDto
{
    public string? Reason { get; set; }
}
```

WebService/SmartSolar.Api/DTOs/CreateProsumerRequestDto.cs M1 · 23 lines

```
/*
 * File: CreateProsumerRequestDto.cs
 * Description: Body for POST /api/prosumers. Backoffice creates an Active prosumer.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
```

```
namespace SmartSolar.Api.DTOs;

public class CreateProsumerRequestDto
{
    public string Nic { get; set; } = string.Empty;

    public string Username { get; set; } = string.Empty;

    public string Password { get; set; } = string.Empty;

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Phone { get; set; } = string.Empty;
}
```

WebService/SmartSolar.Api/DTOs/CreateReservationRequestDto.cs M3 · 16 lines

```
/*
 * File: CreateReservationRequestDto.cs
 * Description: Body for POST /api/reservations. ProsumerNic is only read for staff bookings.
 * Author: Janukshan S (IT22635266)
 */
```

```
namespace SmartSolar.Api.DTOs;

public class CreateReservationRequestDto
{
    public string StationId { get; set; } = string.Empty;

    public string SlotId { get; set; } = string.Empty;

    public string? ProsumerNic { get; set; }
}
```

WebService/SmartSolar.Api/DTOs/CreateStaffRequestDto.cs M1 · 23 lines

```
/*
 * File: CreateStaffRequestDto.cs
 * Description: Body for POST /api/users. Staff roles only; password is hashed in the service.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
namespace SmartSolar.Api.DTOs;

public class CreateStaffRequestDto
{
    public string Username { get; set; } = string.Empty;

    public string Password { get; set; } = string.Empty;

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Phone { get; set; } = string.Empty;

    public string Role { get; set; } = string.Empty;
}
```

WebService/SmartSolar.Api/DTOs/HealthResponseDto.cs M1 · 14 lines

```
/*
 * File: HealthResponseDto.cs
```

```
* Description: Payload for GET /health used as a LAN smoke-test probe.
* Author: DE SILVA R K D H (IT22001252)
* Created: 20/09/2026
*/
```

```
namespace SmartSolar.Api.DTOs;
```

```
public class HealthResponseDto
{
    public string Status { get; set; } = "Healthy";
    public DateTime ServerTime { get; set; }
}
```

WebService/SmartSolar.Api.DTOs/LoginRequestDto.cs M1 · 15 lines

```
/*
 * File: LoginRequestDto.cs
 * Description: Login body: username or NIC plus password, matching API-CONTRACT.md.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
namespace SmartSolar.Api.DTOs;
```

```
public class LoginRequestDto
{
    public string? Username { get; set; }
    public string? Nic { get; set; }
    public string Password { get; set; } = string.Empty;
}
```

WebService/SmartSolar.Api.DTOs/LoginResponseDto.cs M1 · 15 lines

```
/*
 * File: LoginResponseDto.cs
 * Description: Successful login payload: JWT, expiry and the public user object.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
namespace SmartSolar.Api.DTOs;
```

```
public class LoginResponseDto
{
    public string Token { get; set; } = string.Empty;
    public DateTime ExpiresAt { get; set; }
    public UserDto User { get; set; } = new();
}
```

WebService/SmartSolar.Api.DTOs/NearbyStationDto.cs M2 · 27 lines

```
/*
 * File: NearbyStationDto.cs
 * Description: One Active station inside a search radius, with distance and free slots.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
```

```
namespace SmartSolar.Api.DTOs;
```

```
public class NearbyStationDto
{
    public string Id { get; set; } = string.Empty;

    public string Name { get; set; } = string.Empty;

    public double Latitude { get; set; }

    public double Longitude { get; set; }

    public double CapacityKwh { get; set; }

    public int BatterySlotsTotal { get; set; }

    public double DistanceKm { get; set; }

    public int FreeSlots { get; set; }
}
```

WebService/SmartSolar.Api.DTOs/OperationsDashboardDto.cs M4 · 17 lines

```
/*
 * File: OperationsDashboardDto.cs
 * Description: Live counts for the staff Home page from GET /api/dashboard/operations.
 * Author: samudith
 * Created: 29/09/2026
 */
```

```

*/

namespace SmartSolar.Api.DTOs;

public class OperationsDashboardDto
{
    public int TodaysBookingCount { get; set; }
    public int PendingApprovals { get; set; }
    public int ActiveStations { get; set; }
    public int CompletedToday { get; set; }
    public IReadOnlyList<ReservationDto> TodaysBookings { get; set; } = Array.Empty<ReservationDto>();
}

```

WebService/SmartSolar.Api/DTOs/ProsumerDashboardDto.cs M4 · 16 lines

```

/*
 * File: ProsumerDashboardDto.cs
 * Description: Live counts and the next booking for GET /api/dashboard/me.
 * Author: samudith
 * Created: 29/09/2026
 */

```

```

namespace SmartSolar.Api.DTOs;

public class ProsumerDashboardDto
{
    public int PendingCount { get; set; }
    public int ApprovedFutureCount { get; set; }
    public int ActiveCount { get; set; }
    public ReservationDto? NextReservation { get; set; }
}

```

WebService/SmartSolar.Api/DTOs/RegisterProsumerRequestDto.cs M1 · 23 lines

```

/*
 * File: RegisterProsumerRequestDto.cs
 * Description: Body for POST /api/auth/register. Creates a Pending prosumer.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */

```

```

namespace SmartSolar.Api.DTOs;

public class RegisterProsumerRequestDto
{
    public string Nic { get; set; } = string.Empty;

    public string Username { get; set; } = string.Empty;

    public string Password { get; set; } = string.Empty;

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Phone { get; set; } = string.Empty;
}

```

WebService/SmartSolar.Api/DTOs/ReservationDto.cs M3 · 45 lines

```

/*
 * File: ReservationDto.cs
 * Description: Reservation shape returned to both clients, as defined in the API contract.
 * Author: Janukshan S (IT22635266)
 */

```

```

using SmartSolar.Api.Models;

namespace SmartSolar.Api.DTOs;

public class ReservationDto
{
    public string Id { get; set; } = string.Empty;
    public string ProsumerNic { get; set; } = string.Empty;
    public string StationId { get; set; } = string.Empty;
    public string StationName { get; set; } = string.Empty;
    public string SlotId { get; set; } = string.Empty;
    public DateTime ScheduledAt { get; set; }
    public string Status { get; set; } = string.Empty;
    public string? QrToken { get; set; }
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
    public DateTime? CompletedAt { get; set; }
    public string? CompletedBy { get; set; }

    // Maps a reservation document to the contract shape.
    public static ReservationDto From(Reservation reservation)
    {

```

```

return new ReservationDto
{
    Id = reservation.Id,
    ProsumerNic = reservation.ProsumerNic,
    StationId = reservation.StationId,
    StationName = reservation.StationName,
    SlotId = reservation.SlotId,
    ScheduledAt = reservation.ScheduledAt,
    Status = reservation.Status,
    QrToken = reservation.QrToken,
    CreatedAt = reservation.CreatedAt,
    UpdatedAt = reservation.UpdatedAt,
    CompletedAt = reservation.CompletedAt,
    CompletedBy = reservation.CompletedBy
};
}
}

```

WebService/SmartSolar.Api/DTOs/ReservationPageDto.cs M4 · 16 lines

```

/*
 * File: ReservationPageDto.cs
 * Description: One page of reservations for the booking monitor and the operator queue.
 * Author: samudith
 * Created: 29/09/2026
 */

namespace SmartSolar.Api.DTOs;

public class ReservationPageDto
{
    public IReadOnlyList<ReservationDto> Items { get; set; } = Array.Empty<ReservationDto>();
    public int Page { get; set; }
    public int PageSize { get; set; }
    public long TotalCount { get; set; }
}

```

WebService/SmartSolar.Api/DTOs/SaveSlotRequestDto.cs M2 · 15 lines

```

/*
 * File: SaveSlotRequestDto.cs
 * Description: Body for creating or updating a slot. Times are UTC.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

namespace SmartSolar.Api.DTOs;

public class SaveSlotRequestDto
{
    public DateTime StartTime { get; set; }

    public DateTime EndTime { get; set; }
}

```

WebService/SmartSolar.Api/DTOs/SaveStationRequestDto.cs M2 · 25 lines

```

/*
 * File: SaveStationRequestDto.cs
 * Description: Body for creating or updating a station. Validation lives in StationService.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.DTOs;

public class SaveStationRequestDto
{
    public string Name { get; set; } = string.Empty;

    public double Latitude { get; set; }

    public double Longitude { get; set; }

    public double CapacityKwh { get; set; }

    public int BatterySlotsTotal { get; set; }

    public List<StationScheduleEntry> Schedule { get; set; } = new();
}

```

WebService/SmartSolar.Api/DTOs/SlotAvailabilityRequestDto.cs M2 · 13 lines

```

/*
 * File: SlotAvailabilityRequestDto.cs

```

```

* Description: Body for PUT /api/slots/{id}/availability. GridOperator sets the booking flag.
* Author: Mohamed Asath (IT22633422)
* Created: 30/09/2026
*/

```

```
namespace SmartSolar.Api.DTOs;
```

```

public class SlotAvailabilityRequestDto
{
    public bool IsAvailable { get; set; }
}

```

WebService/SmartSolar.Api.DTOs/UpdateProsumerRequestDto.cs M1 · 21 lines

```

/*
* File: UpdateProsumerRequestDto.cs
* Description: Body for PUT /api/prosumers/{nic} and PUT /api/me. NIC and role are never taken from this body.
* Author: DE SILVA R K D H (IT22001252)
* Created: 27/09/2026
*/

```

```
namespace SmartSolar.Api.DTOs;
```

```

public class UpdateProsumerRequestDto
{
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Phone { get; set; } = string.Empty;

    public string? Password { get; set; }

    public string? Username { get; set; }
}

```

WebService/SmartSolar.Api.DTOs/UpdateReservationRequestDto.cs M3 · 14 lines

```

/*
* File: UpdateReservationRequestDto.cs
* Description: Body for PUT /api/reservations/{id}. StationId is only needed when moving to another station.
* Author: Janukshan S (IT22635266)
*/

```

```
namespace SmartSolar.Api.DTOs;
```

```

public class UpdateReservationRequestDto
{
    public string? StationId { get; set; }

    public string SlotId { get; set; } = string.Empty;
}

```

WebService/SmartSolar.Api.DTOs/UpdateStaffRequestDto.cs M1 · 21 lines

```

/*
* File: UpdateStaffRequestDto.cs
* Description: Body for PUT /api/users/{id}. Password is optional; omit it to keep the hash.
* Author: DE SILVA R K D H (IT22001252)
* Created: 20/09/2026
*/

```

```
namespace SmartSolar.Api.DTOs;
```

```

public class UpdateStaffRequestDto
{
    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Phone { get; set; } = string.Empty;

    public string Role { get; set; } = string.Empty;

    public string? Password { get; set; }
}

```

WebService/SmartSolar.Api.DTOs/UserDto.cs M1 · 40 lines

```

/*
* File: UserDto.cs
* Description: Public user shape. Password hashes are never copied onto this DTO.
* Author: DE SILVA R K D H (IT22001252)
* Created: 20/09/2026
*/

```

```

using SmartSolar.Api.Models;

namespace SmartSolar.Api.DTOS;

public class UserDto
{
    public string Id { get; set; } = string.Empty;
    public string? Nic { get; set; }
    public string Username { get; set; } = string.Empty;
    public string FullName { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Phone { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = string.Empty;
    public DateTime CreatedAt { get; set; }

    // Maps a MongoDB user document to the public contract shape.
    public static UserDto From(User user)
    {
        return new UserDto
        {
            Id = user.Id,
            Nic = user.Nic,
            Username = user.Username,
            FullName = user.FullName,
            Email = user.Email,
            Phone = user.Phone,
            Role = user.Role,
            Status = user.Status,
            CreatedAt = user.CreatedAt
        };
    }
}

```

WebService/SmartSolar.Api/DTOS/VerifyQrRequestDto.cs M4 · 13 lines

```

/*
 * File: VerifyQrRequestDto.cs
 * Description: Body for POST /api/reservations/verify-qr. The QR contains only this opaque token.
 * Author: samudith
 * Created: 29/09/2026
 */

namespace SmartSolar.Api.DTOS;

public class VerifyQrRequestDto
{
    public string QrToken { get; set; } = string.Empty;
}

```

WebService/SmartSolar.Api/Data/DatabaseSeeder.cs M1 · 299 lines

```

/*
 * File: DatabaseSeeder.cs
 * Description: Idempotent seed of the contract sample users, stations, slots and reservations.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using System.Security.Cryptography;
using Microsoft.AspNetCore.Identity;
using MongoDB.Driver;
using SmartSolar.Api.Common;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Data;

public class DatabaseSeeder
{
    public const string ActiveProsumerNic = "200012345678";
    public const string PendingProsumerNic = "199912345678";
    public const string MalabeStationId = "stn-malabe";
    public const string ColomboStationId = "stn-colombo-fort";
    public const string RajagiriyaStationId = "stn-rajagiriya";

    private readonly MongoContext _context;
    private readonly ILogger<DatabaseSeeder> _logger;
    private readonly PasswordHasher<User> _passwordHasher = new();

    // Injects Mongo access used to insert sample documents when the database is empty.
    public DatabaseSeeder(MongoContext context, ILogger<DatabaseSeeder> logger)
    {
        _context = context;
        _logger = logger;
    }

    // Seeds contract sample data once. Re-running is a no-op when Users already has documents.
    public async Task SeedAsync(CancellationToken cancellationToken = default)
    {
        var userCount = await _context.Users.CountDocumentsAsync(FilterDefinition<User>.Empty, cancellationToken: cancellationToken);
        if (userCount > 0)

```

```

{
    _logger.LogInformation("Seed skipped: Users already contains {Count} document(s).", userCount);
    return;
}

var createdAt = DateTime.UtcNow;
var users = BuildUsers(createdAt);
var stations = BuildStations();
var slots = BuildSlots();
var reservations = BuildReservations(slots, createdAt);

foreach (var occupiedSlotId in reservations
    .Where(r => r.Status is ReservationStatuses.Pending or ReservationStatuses.Approved or ReservationStatuses.Completed)
    .Select(r => r.SlotId))
{
    var slot = slots.First(s => s.Id == occupiedSlotId);
    slot.IsAvailable = false;
}

await _context.Users.InsertManyAsync(users, cancellationToken: cancellationToken);
await _context.Stations.InsertManyAsync(stations, cancellationToken: cancellationToken);
await _context.Slots.InsertManyAsync(slots, cancellationToken: cancellationToken);
await _context.Reservations.InsertManyAsync(reservations, cancellationToken: cancellationToken);

_logger.LogInformation(
    "Seed complete: {Users} users, {Stations} stations, {Slots} slots, {Reservations} reservations.",
    users.Count,
    stations.Count,
    slots.Count,
    reservations.Count);
}

// Builds the four contract users with hashed passwords (never store plaintext).
private List<User> BuildUsers(DateTime createdAt)
{
    var admin = NewUser("admin", null, "admin", "System Administrator", "admin@smartsolar.lk", "0770000001", RoleNames.Backoffice, UserStatuses.Active,
        createdAt);
    var operatorUser = NewUser("operator1", null, "operator1", "Grid Operator One", "operator1@smartsolar.lk", "0770000002", RoleNames.GridOperator,
        UserStatuses.Active, createdAt);
    var activeProsumer = NewUser(ActiveProsumerNic, ActiveProsumerNic, "nimal", "Nimal Perera", "nimal@smartsolar.lk", "0771234567", RoleNames.Prosumer,
        UserStatuses.Active, createdAt);
    var pendingProsumer = NewUser(PendingProsumerNic, PendingProsumerNic, "saman", "Saman Silva", "saman@smartsolar.lk", "0777654321",
        RoleNames.Prosumer, UserStatuses.Pending, createdAt);

    HashPassword(admin, "Admin@123");
    HashPassword(operatorUser, "Oper@123");
    HashPassword(activeProsumer, "Solar@123");
    HashPassword(pendingProsumer, "Solar@123");

    return new List<User> { admin, operatorUser, activeProsumer, pendingProsumer };
}

// Creates a user document. Prosumer id is the NIC as required by the plan.
private static User NewUser(
    string id,
    string? nic,
    string username,
    string fullName,
    string email,
    string phone,
    string role,
    string status,
    DateTime createdAt)
{
    return new User
    {
        Id = id,
        Nic = nic,
        Username = username,
        FullName = fullName,
        Email = email,
        Phone = phone,
        Role = role,
        Status = status,
        CreatedAt = createdAt
    };
}

// Hashes a seed password with ASP.NET Core Identity PasswordHasher.
private void HashPassword(User user, string password)
{
    user.PasswordHash = _passwordHasher.HashPassword(user, password);
}

// Creates three Colombo/Malabe stations with a simple daily opening schedule.
private static List<Station> BuildStations()
{
    var week = new[] { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday" }
        .Select(day => new StationScheduleEntry { DayOfWeek = day, OpenTime = "08:00", CloseTime = "18:00" })
        .ToList();

    return new List<Station>
    {
        new()
        {
            Id = MalabeStationId,
            Name = "Malabe Solar Hub",

```

```

        Latitude = 6.9147,
        Longitude = 79.9730,
        CapacityKwh = 150,
        BatterySlotsTotal = 12,
        Schedule = week,
        Status = StationStatuses.Active
    },
    new()
    {
        Id = ColomboStationId,
        Name = "Colombo Fort Node",
        Latitude = 6.9339,
        Longitude = 79.8500,
        CapacityKwh = 200,
        BatterySlotsTotal = 16,
        Schedule = week,
        Status = StationStatuses.Active
    },
    new()
    {
        Id = RajagiriyaStationId,
        Name = "Rajagiriya Microgrid",
        Latitude = 6.9090,
        Longitude = 79.8910,
        CapacityKwh = 120,
        BatterySlotsTotal = 8,
        Schedule = week,
        Status = StationStatuses.Active
    }
};

// Builds 4 slots per station for the next 5 local days, plus one historical slot for a completed booking.
private static List<Slot> BuildSlots()
{
    var tz = ColomboTimeZone();
    var localNow = TimeZoneInfo.ConvertTimeFromUtc(DateTime.UtcNow, tz);
    var startDate = localNow.Date.AddDays(1);
    var windows = new (int StartHour, int EndHour)[]
    {
        (8, 10),
        (10, 12),
        (13, 15),
        (15, 17)
    };

    var stationIds = new[] { MalabeStationId, ColomboStationId, RajagiriyaStationId };
    var slots = new List<Slot>();

    for (var day = 0; day < 5; day++)
    {
        var date = startDate.AddDays(day);
        foreach (var stationId in stationIds)
        {
            for (var i = 0; i < windows.Length; i++)
            {
                var (startHour, endHour) = windows[i];
                var startLocal = DateTime.SpecifyKind(date.AddHours(startHour), DateTimeKind.Unspecified);
                var endLocal = DateTime.SpecifyKind(date.AddHours(endHour), DateTimeKind.Unspecified);
                slots.Add(new Slot
                {
                    Id = $"slot-{stationId}-{date:yyyyMMdd}-{i + 1}",
                    StationId = stationId,
                    StartTime = TimeZoneInfo.ConvertTimeToUtc(startLocal, tz),
                    EndTime = TimeZoneInfo.ConvertTimeToUtc(endLocal, tz),
                    IsAvailable = true
                });
            }
        }
    }

    var yesterday = localNow.Date.AddDays(-1);
    var pastStart = DateTime.SpecifyKind(yesterday.AddHours(8), DateTimeKind.Unspecified);
    var pastEnd = DateTime.SpecifyKind(yesterday.AddHours(10), DateTimeKind.Unspecified);
    slots.Add(new Slot
    {
        Id = $"slot-{MalabeStationId}-{yesterday:yyyyMMdd}-past",
        StationId = MalabeStationId,
        StartTime = TimeZoneInfo.ConvertTimeToUtc(pastStart, tz),
        EndTime = TimeZoneInfo.ConvertTimeToUtc(pastEnd, tz),
        IsAvailable = false
    });

    return slots;
}

// Seeds one reservation in each status, including a completed booking in the past.
private static List<Reservation> BuildReservations(IReadOnlyList<Slot> slots, DateTime createdAt)
{
    var malabeTomorrow = slots
        .Where(s => s.StationId == MalabeStationId && s.Id.Contains("-past") == false)
        .OrderBy(s => s.StartTime)
        .Take(3)
        .ToList();

    var pastSlot = slots.First(s => s.Id.EndsWith("-past", StringComparison.Ordinal));
    var qrToken = Convert.ToHexString(RandomNumberGenerator.GetBytes(16)).ToLowerInvariant();

    return new List<Reservation>

```

```

    {
        new()
        {
            Id = "res-pending-1",
            ProsumerNic = ActiveProsumerNic,
            StationId = MalabeStationId,
            StationName = "Malabe Solar Hub",
            SlotId = malabeTomorrow[0].Id,
            ScheduledAt = malabeTomorrow[0].StartTime,
            Status = ReservationStatuses.Pending,
            CreatedAt = createdAt,
            UpdatedAt = createdAt
        },
        new()
        {
            Id = "res-approved-1",
            ProsumerNic = ActiveProsumerNic,
            StationId = MalabeStationId,
            StationName = "Malabe Solar Hub",
            SlotId = malabeTomorrow[1].Id,
            ScheduledAt = malabeTomorrow[1].StartTime,
            Status = ReservationStatuses.Approved,
            QrToken = qrToken,
            CreatedAt = createdAt,
            UpdatedAt = createdAt
        },
        new()
        {
            Id = "res-cancelled-1",
            ProsumerNic = ActiveProsumerNic,
            StationId = MalabeStationId,
            StationName = "Malabe Solar Hub",
            SlotId = malabeTomorrow[2].Id,
            ScheduledAt = malabeTomorrow[2].StartTime,
            Status = ReservationStatuses.Cancelled,
            CreatedAt = createdAt.AddDays(-2),
            UpdatedAt = createdAt
        },
        new()
        {
            Id = "res-completed-past-1",
            ProsumerNic = ActiveProsumerNic,
            StationId = MalabeStationId,
            StationName = "Malabe Solar Hub",
            SlotId = pastSlot.Id,
            ScheduledAt = pastSlot.StartTime,
            Status = ReservationStatuses.Completed,
            CreatedAt = pastSlot.StartTime.AddDays(-1),
            UpdatedAt = pastSlot.EndTime,
            CompletedAt = pastSlot.EndTime,
            CompletedBy = "operator1"
        }
    };
}

// Resolves Sri Lanka time on both macOS (IANA) and Windows (system name).
private static TimeZoneInfo ColomboTimeZone()
{
    try
    {
        return TimeZoneInfo.FindSystemTimeZoneById("Asia/Colombo");
    }
    catch (TimeZoneNotFoundException)
    {
        return TimeZoneInfo.FindSystemTimeZoneById("Sri Lanka Standard Time");
    }
}
}

```

WebService/SmartSolar.Api/Data/MongoContext.cs M1 · 53 lines

```

/*
 * File: MongoContext.cs
 * Description: Shared MongoDB database and the four assignment collections.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 *
 * Adapted from the MongoDB C# Driver quick start:
 * https://www.mongodb.com/docs/drivers/csharp/current/quick-start/
 */

```

```

using Microsoft.Extensions.Options;
using MongoDB.Driver;
using SmartSolar.Api.Configuration;
using SmartSolar.Api.Models;

```

```
namespace SmartSolar.Api.Data;
```

```

public class MongoContext
{
    public IMongoDatabase Database { get; }

    public IMongoCollection<User> Users { get; }

    public IMongoCollection<Station> Stations { get; }

    public IMongoCollection<Slot> Slots { get; }
}

```

```

public IMongoCollection<Reservation> Reservations { get; }

// Builds a MongoClient from configuration and exposes typed collections.
public MongoContext(IOptions<MongoDbSettings> options)
{
    MongoConventions.Register();

    var settings = options.Value;
    if (string.IsNullOrEmpty(settings.ConnectionString))
    {
        throw new InvalidOperationException("MongoDb:ConnectionString is missing. Set it in configuration or the MongoDb_ConnectionString environment variable.");
    }

    if (string.IsNullOrEmpty(settings.DatabaseName))
    {
        throw new InvalidOperationException("MongoDb:DatabaseName is missing.");
    }

    var client = new MongoClient(settings.ConnectionString);
    Database = client.GetDatabase(settings.DatabaseName);
    Users = Database.GetCollection<User>("Users");
    Stations = Database.GetCollection<Station>("SolarStationInfo");
    Slots = Database.GetCollection<Slot>("EnergyBookingSlots");
    Reservations = Database.GetCollection<Reservation>("EnergyReservation");
}
}

```

WebService/SmartSolar.Api/Data/MongoConventions.cs M1 · 35 lines

```

/*
 * File: MongoConventions.cs
 * Description: Registers camelCase + ignore-extra-elements conventions for the C# driver.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 *
 * Adapted from the MongoDB C# Driver documentation:
 * https://www.mongodb.com/docs/drivers/csharp/current/
 */

using MongoDB.Bson.Serialization.Conventions;

namespace SmartSolar.Api.Data;

public static class MongoConventions
{
    private static int _registered;

    // Registers driver conventions once so documents match the API camelCase contract.
    public static void Register()
    {
        if (Interlocked.Exchange(ref _registered, 1) == 1)
        {
            return;
        }

        var pack = new ConventionPack
        {
            new CamelCaseElementNameConvention(),
            new IgnoreExtraElementsConvention(true)
        };

        ConventionRegistry.Register("smart-solar", pack, _ => true);
    }
}

```

WebService/SmartSolar.Api/Data/MongoIndexInitializer.cs M1 · 82 lines

```

/*
 * File: MongoIndexInitializer.cs
 * Description: Creates unique NIC/username indexes and reservation lookup indexes on startup.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using MongoDB.Driver;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Data;

public class MongoIndexInitializer
{
    private readonly MongoContext _context;
    private readonly ILogger<MongoIndexInitializer> _logger;

    // Injects the shared Mongo context used to create indexes.
    public MongoIndexInitializer(MongoContext context, ILogger<MongoIndexInitializer> logger)
    {
        _context = context;
    }
}

```

```

        _logger = logger;
    }

    // Ensures the four collections exist and have the indexes required by the contract.
    public async Task EnsureIndexesAsync(CancellationToken cancellationToken = default)
    {
        var nicIndex = new CreateIndexModel<User>(
            Builders<User>.IndexKeys.Ascending(user => user.Nic),
            new CreateIndexOptions
            {
                Unique = true,
                Sparse = true,
                Name = "ux_users_nic"
            });

        var usernameIndex = new CreateIndexModel<User>(
            Builders<User>.IndexKeys.Ascending(user => user.Username),
            new CreateIndexOptions
            {
                Unique = true,
                Name = "ux_users_username"
            });

        await _context.Users.Indexes.CreateManyAsync(new[] { nicIndex, usernameIndex }, cancellationToken);

        await _context.Reservations.Indexes.CreateManyAsync(
            new[]
            {
                new CreateIndexModel<Reservation>(
                    Builders<Reservation>.IndexKeys.Ascending(r => r.ProsumerNic),
                    new CreateIndexOptions { Name = "ix_reservations_prosumerNic" }),
                new CreateIndexModel<Reservation>(
                    Builders<Reservation>.IndexKeys.Ascending(r => r.ScheduledAt),
                    new CreateIndexOptions { Name = "ix_reservations_scheduledAt" }),
                new CreateIndexModel<Reservation>(
                    Builders<Reservation>.IndexKeys.Ascending(r => r.Status),
                    new CreateIndexOptions { Name = "ix_reservations_status" }),
                new CreateIndexModel<Reservation>(
                    Builders<Reservation>.IndexKeys.Ascending(r => r.ProsumerNic).Ascending(r => r.ScheduledAt),
                    new CreateIndexOptions { Name = "ix_reservations_prosumerNic_scheduledAt" }),
                new CreateIndexModel<Reservation>(
                    Builders<Reservation>.IndexKeys.Ascending(r => r.Status).Ascending(r => r.ScheduledAt),
                    new CreateIndexOptions { Name = "ix_reservations_status_scheduledAt" }),
                new CreateIndexModel<Reservation>(
                    Builders<Reservation>.IndexKeys.Ascending(r => r.QrToken),
                    new CreateIndexOptions { Name = "ux_reservations_qrToken", Unique = true, Sparse = true })
            },
            cancellationToken);

        // Touch the remaining collections so Compass shows all four after first boot.
        await _context.Stations.Indexes.CreateOneAsync(
            new CreateIndexModel<Station>(Builders<Station>.IndexKeys.Ascending(s => s.Status), new CreateIndexOptions { Name = "ix_stations_status" }),
            cancellationToken: cancellationToken);

        await _context.Slots.Indexes.CreateOneAsync(
            new CreateIndexModel<Slot>(Builders<Slot>.IndexKeys.Ascending(s => s.StationId), new CreateIndexOptions { Name = "ix_slots_stationId" }),
            cancellationToken: cancellationToken);

        _logger.LogInformation("MongoDB indexes ensured for Users, SolarStationInfo, EnergyBookingSlots and EnergyReservation.");
    }
}

```

WebService/SmartSolar.Api/Extensions/ServiceCollectionExtensions.cs M1 · 216 lines

```

/*
 * File: ServiceCollectionExtensions.cs
 * Description: Registers Mongo, repositories, services, JWT, Swagger and CORS.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 *
 * JWT bearer events adapted from Microsoft docs:
 * https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwt-authn
 * Swagger bearer button adapted from Swashbuckle v10 docs:
 * https://github.com/domaindrivendev/Swashbuckle.AspNetCore/blob/master/docs/configure-and-customize-swaggergen.md
 */

```

```

using System.Text;
using System.Text.Json;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.AspNetCore.Mvc;
using Microsoft.IdentityModel.Tokens;
using Microsoft.OpenApi;
using SmartSolar.Api.Common;
using SmartSolar.Api.Configuration;
using SmartSolar.Api.Data;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Repositories;
using SmartSolar.Api.Services;

```

```

namespace SmartSolar.Api.Extensions;

```

```

public static class ServiceCollectionExtensions
{
    public const string CorsPolicyName = "ClientApps";
}

```

```

// Registers every application service behind the FAT Web API.
public static IServiceCollection AddSmartSolarServices(this IServiceCollection services, IConfiguration configuration)
{
    services.Configure<MongoDbSettings>(configuration.GetSection(MongoDbSettings.SectionName));
    services.Configure<JwtSettings>(configuration.GetSection(JwtSettings.SectionName));

    services.AddSingleton<MongoContext>();
    services.AddSingleton<MongoIndexInitializer>();
    services.AddSingleton<DatabaseSeeder>();

    services.AddSingleton<IUserRepository, UserRepository>();
    services.AddSingleton<IStationRepository, StationRepository>();
    services.AddSingleton<ISlotRepository, SlotRepository>();
    services.AddSingleton<IReservationRepository, ReservationRepository>();

    services.AddSingleton<IPasswordService, PasswordService>();
    services.AddSingleton<IJwtTokenService, JwtTokenService>();
    services.AddScoped<IAuthService, AuthService>();
    services.AddScoped<IUserService, UserService>();
    services.AddScoped<IProsumerService>();
    services.AddScoped<IProsumerService>(provider => provider.GetRequiredService<ProsumerService>());
    services.AddScoped<IStationService, StationService>();
    services.AddScoped<ISlotService, SlotService>();
    services.AddScoped<IReservationService, ReservationService>();
    services.AddSingleton<IReservationQueryRepository, ReservationQueryRepository>();
    services.AddScoped<IReservationQueryService, ReservationQueryService>();
    services.AddScoped<IOperatorService, OperatorService>();
    services.AddSingleton<IActiveAccountGuard, ActiveAccountGuard>();

    services.AddControllers()
        .AddJsonOptions(options =>
        {
            options.JsonSerializerOptions.PropertyNamingPolicy = JsonNamingPolicy.CamelCase;
        })
        .ConfigureApiBehaviorOptions(options =>
        {
            options.InvalidModelStateResponseFactory = context =>
            {
                var message = context.ModelState.Values
                    .SelectMany(value => value.Errors)
                    .Select(error => error.ErrorMessage)
                    .FirstOrDefault(text => !string.IsNullOrEmpty(text))
                    ?? "Validation failed.";

                return new BadRequestObjectResult(new ApiErrorDto(ErrorCodes.ValidationError, message));
            };
        });

    return services;
}

// Configures JWT bearer auth so [Authorize(Roles=...)] reads the `role` claim.
public static IServiceCollection AddSmartSolarJwt(this IServiceCollection services, IConfiguration configuration)
{
    var jwt = configuration.GetSection(JwtSettings.SectionName).Get<JwtSettings>()
        ?? throw new InvalidOperationException("Jwt configuration section is missing.");

    if (string.IsNullOrEmpty(jwt.Key) || jwt.Key.Length < 32)
    {
        throw new InvalidOperationException("Jwt:Key must be at least 32 characters. Set Jwt__Key, user-secrets, or appsettings.Development.json.");
    }

    if (string.Equals(jwt.Key, JwtSettings.PlaceholderKey, StringComparison.Ordinal))
    {
        throw new InvalidOperationException(
            "Jwt:Key is still the example placeholder. Set a random secret of at least 32 characters via Jwt__Key, user-secrets, or appsettings.Production.json.");
    }

    services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
        .AddJwtBearer(options =>
        {
            // Keep claim names as `sub`, `role`, `nic` (API contract). Default inbound mapping
            // would rename `role` and break [Authorize(Roles)].
            options.MapInboundClaims = false;
            options.TokenValidationParameters = new TokenValidationParameters
            {
                ValidateIssuer = true,
                ValidateAudience = true,
                ValidateIssuerSigningKey = true,
                ValidateLifetime = true,
                ValidIssuer = jwt.Issuer,
                ValidAudience = jwt.Audience,
                IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwt.Key)),
                ClockSkew = TimeSpan.FromMinutes(1),
                RoleClaimType = "role",
                NameClaimType = "sub"
            };

            options.Events = new JwtBearerEvents
            {
                // Re-check Users.status so a deactivated account cannot keep using an old JWT.
                OnTokenValidated = async context =>
                {
                    var userId = context.Principal?.FindFirst("sub")?.Value;
                    var guard = context.HttpContext.RequestServices.GetRequiredService<IActiveAccountGuard>();
                    if (!await guard.IsActiveAsync(userId, context.HttpContext.RequestAborted))
                    {

```

```

        context.Fail("Account is not active.");
    }
},
OnChallenge = async context =>
{
    context.HandleResponse();
    if (context.Response.HasStarted)
    {
        return;
    }

    context.Response.StatusCode = StatusCodes.Status401Unauthorized;
    context.Response.ContentType = "application/json";
    var body = JsonSerializer.Serialize(
        new ApiErrorDto(ErrorCodes.Unauthorized, "Authentication is required or the token is invalid."),
        JsonSerializer.Default.Options);
    await context.Response.WriteAsync(body);
},
OnForbidden = async context =>
{
    if (context.Response.HasStarted)
    {
        return;
    }

    context.Response.StatusCode = StatusCodes.Status403Forbidden;
    context.Response.ContentType = "application/json";
    var body = JsonSerializer.Serialize(
        new ApiErrorDto(ErrorCodes.Forbidden, "You do not have permission to access this resource."),
        JsonSerializer.Default.Options);
    await context.Response.WriteAsync(body);
}
};
});

services.AddAuthorization();
return services;
}

// Enables Swagger UI with an Authorize button that sends Bearer tokens.
public static IServiceCollection AddSmartSolarSwagger(this IServiceCollection services)
{
    const string schemeId = "Bearer";

    services.AddEndpointsApiExplorer();
    services.AddSwaggerGen(options =>
    {
        options.SwaggerDoc("v1", new OpenApiInfo
        {
            Title = "Smart Solar Microgrid API",
            Version = "v1",
            Description = "FAT service for the SE4040 EAD assignment. All business rules live here."
        });

        options.AddSecurityDefinition(schemeId, new OpenApiSecurityScheme
        {
            Type = SecuritySchemeType.Http,
            Scheme = "bearer",
            BearerFormat = "JWT",
            Description = "Paste the JWT from POST /api/auth/login. Swagger adds the Bearer prefix."
        });

        options.AddSecurityRequirement(document => new OpenApiSecurityRequirement
        {
            [new OpenApiSecuritySchemeReference(schemeId, document)] = []
        });
    });

    return services;
}

// Allows the MVC web app on LAN. Mobile apps do not use CORS.
public static IServiceCollection AddSmartSolarCors(this IServiceCollection services)
{
    services.AddCors(options =>
    {
        options.AddPolicy(CorsPolicyName, policy =>
        {
            policy.AllowAnyHeader()
                .AllowAnyMethod()
                .AllowAnyOrigin();
        });
    });

    return services;
}
}

```

WebService/SmartSolar.Api/Middleware/ExceptionHandlerMiddleware.cs M1 · 61 lines

```

/*
 * File: ExceptionHandlingMiddleware.cs
 * Description: Converts unhandled exceptions into the contract `{ code, message }` JSON body.
 * Author: DE SILVA R K D H (IT22001252)

```

```

* Created: 20/09/2026
*/

using System.Text.Json;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Middleware;

public class ExceptionHandlingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<ExceptionHandlingMiddleware> _logger;

    // Captures the next middleware and the logger used for unexpected failures.
    public ExceptionHandlingMiddleware(RequestDelegate next, ILogger<ExceptionHandlingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    // Runs the pipeline and maps exceptions to HTTP status + contract error JSON.
    public async Task InvokeAsync(HttpContext context)
    {
        try
        {
            await _next(context);
        }
        catch (ApiException ex)
        {
            await WriteErrorAsync(context, ex.StatusCode, ex.Code, ex.Message);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled exception while processing {Path}", context.Request.Path);
            await WriteErrorAsync(
                context,
                StatusCodes.Status500InternalServerError,
                ErrorCodes.InternalError,
                "An unexpected error occurred.");
        }
    }

    // Writes a JSON error body only if the response has not already started.
    private static async Task WriteErrorAsync(HttpContext context, int statusCode, string code, string message)
    {
        if (context.Response.HasStarted)
        {
            return;
        }

        context.Response.StatusCode = statusCode;
        context.Response.ContentType = "application/json";
        var payload = JsonSerializer.Serialize(new ApiErrorDto(code, message), JsonDefaults.Options);
        await context.Response.WriteAsync(payload);
    }
}

```

WebService/SmartSolar.Api/Models/Reservation.cs M1 · 67 lines

```

/*
* File: Reservation.cs
* Description: MongoDB document for EnergyReservation, including the operator QR token.
* Author: DE SILVA R K D H (IT22001252)
* Created: 20/09/2026
*/

```

```

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SmartSolar.Api.Models;

public class Reservation
{
    [BsonId]
    [BsonRepresentation(BsonType.String)]
    public string Id { get; set; } = string.Empty;

    public string ProsumerNic { get; set; } = string.Empty;

    public string StationId { get; set; } = string.Empty;

    public string StationName { get; set; } = string.Empty;

    public string SlotId { get; set; } = string.Empty;

    [BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
    public DateTime ScheduledAt { get; set; }

    public string Status { get; set; } = string.Empty;

    [BsonIgnoreIfNull]

```

```

public string? QrToken { get; set; }

[BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
public DateTime CreatedAt { get; set; }

[BsonIgnoreIfNull]
public string? CreatedBy { get; set; }

[BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
public DateTime UpdatedAt { get; set; }

[BsonIgnoreIfNull]
[BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
public DateTime? CompletedAt { get; set; }

[BsonIgnoreIfNull]
public string? CompletedBy { get; set; }

[BsonIgnoreIfNull]
[BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
public DateTime? ApprovedAt { get; set; }

[BsonIgnoreIfNull]
public string? ApprovedBy { get; set; }

[BsonIgnoreIfNull]
[BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
public DateTime? CancelledAt { get; set; }

[BsonIgnoreIfNull]
public string? CancelledBy { get; set; }

[BsonIgnoreIfNull]
public string? CancelReason { get; set; }
}

```

WebService/SmartSolar.Api/Models/Slot.cs M1 · 28 lines

```

/*
 * File: Slot.cs
 * Description: MongoDB document for EnergyBookingSlots. Times are stored as UTC.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SmartSolar.Api.Models;

public class Slot
{
    [BsonId]
    [BsonRepresentation(BsonType.String)]
    public string Id { get; set; } = string.Empty;

    public string StationId { get; set; } = string.Empty;

    [BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
    public DateTime StartTime { get; set; }

    [BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
    public DateTime EndTime { get; set; }

    public bool IsAvailable { get; set; } = true;
}

```

WebService/SmartSolar.Api/Models/Station.cs M1 · 41 lines

```

/*
 * File: Station.cs
 * Description: MongoDB document for SolarStationInfo, including weekly opening hours.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SmartSolar.Api.Models;

public class Station
{
    [BsonId]
    [BsonRepresentation(BsonType.String)]
    public string Id { get; set; } = string.Empty;
}

```

```

    public string Name { get; set; } = string.Empty;

    public double Latitude { get; set; }

    public double Longitude { get; set; }

    public double CapacityKwh { get; set; }

    public int BatterySlotsTotal { get; set; }

    public List<StationScheduleEntry> Schedule { get; set; } = new();

    public string Status { get; set; } = string.Empty;
}

public class StationScheduleEntry
{
    public string DayOfWeek { get; set; } = string.Empty;

    public string OpenTime { get; set; } = string.Empty;

    public string CloseTime { get; set; } = string.Empty;
}

```

WebService/SmartSolar.Api/Models/User.cs M1 · 38 lines

```

/*
 * File: User.cs
 * Description: MongoDB document for the Users collection. Prosumer `_id` is the NIC.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using MongoDB.Bson;
using MongoDB.Bson.Serialization.Attributes;

namespace SmartSolar.Api.Models;

public class User
{
    [BsonId]
    [BsonRepresentation(BsonType.String)]
    public string Id { get; set; } = string.Empty;

    [BsonIgnoreIfNull]
    public string? Nic { get; set; }

    public string Username { get; set; } = string.Empty;

    public string PasswordHash { get; set; } = string.Empty;

    public string Role { get; set; } = string.Empty;

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Phone { get; set; } = string.Empty;

    public string Status { get; set; } = string.Empty;

    [BsonDateTimeOptions(Kind = DateTimeKind.Utc)]
    public DateTime CreatedAt { get; set; }
}

```

WebService/SmartSolar.Api/Program.cs M1 · 49 lines

```

/*
 * File: Program.cs
 * Description: Composition root for the FAT Web API: pipeline, Mongo, JWT, Swagger and seed.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.Data;
using SmartSolar.Api.Extensions;
using SmartSolar.Api.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddSmartSolarServices(builder.Configuration);
builder.Services.AddSmartSolarJwt(builder.Configuration);
builder.Services.AddSmartSolarSwagger();
builder.Services.AddSmartSolarCors();

```

```

var app = builder.Build();

await EnsureDatabaseAsync(app);

app.UseMiddleware<ExceptionHandlerMiddleware>();

app.UseSwagger();
app.UseSwaggerUI(options =>
{
    options.SwaggerEndpoint("/swagger/v1/swagger.json", "Smart Solar API v1");
    options.RoutePrefix = "swagger";
});

// LAN/IIS hosting is HTTP. Redirecting to HTTPS would break phones and Swagger on the lab Wi-Fi.

app.UseCors(ServiceCollectionExtensions.CorsPolicyName);
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();

// Creates indexes and loads seed data once before the first HTTP request.
static async Task EnsureDatabaseAsync(WebApplication app)
{
    using var scope = app.Services.CreateScope();
    var indexes = scope.ServiceProvider.GetRequiredService<MongoIndexInitializer>();
    var seeder = scope.ServiceProvider.GetRequiredService<DatabaseSeeder>();
    await indexes.EnsureIndexesAsync();
    await seeder.SeedAsync();
}

```

WebService/SmartSolar.Api/Repositories/IReservationQueryRepository.cs M4 · 55 lines

```

/*
 * File: IReservationQueryRepository.cs
 * Description: Read-only reservation queries for lists, dashboards and QR lookup.
 * Author: samudith
 * Created: 29/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public interface IReservationQueryRepository
{
    // Returns one page of reservations matching every supplied filter, sorted by scheduledAt.
    Task<IReadOnlyList<Reservation> Items, long TotalCount> SearchAsync(
        string? prosumerNic,
        string? status,
        string? stationId,
        DateTime? fromUtc,
        DateTime? toUtc,
        string? query,
        int skip,
        int take,
        CancellationToken cancellationToken = default);

    // Counts reservations for one prosumer, optionally by status and a scheduledAt lower bound.
    Task<long> CountForProsumerAsync(
        string prosumerNic,
        string? status,
        DateTime? scheduledAfterUtc,
        bool activeOnly,
        CancellationToken cancellationToken = default);

    // Returns the soonest future Pending or Approved reservation for one prosumer.
    Task<Reservation?> FindNextUpcomingAsync(string prosumerNic, DateTime nowUtc, CancellationToken cancellationToken = default);

    // Counts reservations whose scheduledAt falls in [startUtc, endUtc).
    Task<long> CountScheduledBetweenAsync(DateTime startUtc, DateTime endUtc, CancellationToken cancellationToken = default);

    // Counts Pending reservations that have not started yet.
    Task<long> CountPendingFutureAsync(DateTime nowUtc, CancellationToken cancellationToken = default);

    // Counts reservations completed during [startUtc, endUtc).
    Task<long> CountCompletedBetweenAsync(DateTime startUtc, DateTime endUtc, CancellationToken cancellationToken = default);

    // Lists reservations scheduled during [startUtc, endUtc), earliest first.
    Task<IReadOnlyList<Reservation>> ListScheduledBetweenAsync(
        DateTime startUtc,
        DateTime endUtc,
        int limit,
        CancellationToken cancellationToken = default);

    // Loads the reservation that carries this QR token, including one already completed.
    Task<Reservation?> FindByQrTokenAsync(string qrToken, CancellationToken cancellationToken = default);
}

```

WebService/SmartSolar.Api/Repositories/IReservationRepository.cs M1 · 28 lines

```
/*
 * File: IReservationRepository.cs
 * Description: Data-access contract for EnergyReservation. Ready for Members 2 and 4 to extend.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public interface IReservationRepository
{
    // Returns every reservation document.
    Task<IReadOnlyList<Reservation>> GetAllAsync(CancellationToken cancellationToken = default);

    // Loads one reservation by id.
    Task<Reservation?> FindByIdAsync(string id, CancellationToken cancellationToken = default);

    // Finds reservations matching the optional filters, ordered by slot time.
    Task<IReadOnlyList<Reservation>> SearchAsync(string? nic, string? status, DateTime? from, DateTime? to, string? text, CancellationToken cancellationToken = default);

    // Inserts a new reservation.
    Task InsertAsync(Reservation reservation, CancellationToken cancellationToken = default);

    // Saves a changed reservation only if nobody else changed it since it was loaded.
    Task<bool> TryReplaceAsync(Reservation reservation, DateTime loadedUpdatedAt, CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Repositories/ISlotRepository.cs M1 · 40 lines

```
/*
 * File: ISlotRepository.cs
 * Description: Data-access contract for EnergyBookingSlots. Ready for Member 2 to extend.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public interface ISlotRepository
{
    // Returns every slot belonging to a station.
    Task<IReadOnlyList<Slot>> GetByStationAsync(string stationId, CancellationToken cancellationToken = default);

    // Loads one slot by id.
    Task<Slot?> FindByIdAsync(string id, CancellationToken cancellationToken = default);

    // Returns free slots of a station that start after `from` and no later than `to`.
    Task<IReadOnlyList<Slot>> GetFreeAsync(string stationId, DateTime from, DateTime to, CancellationToken cancellationToken = default);

    // Marks a free slot as taken. Returns false if someone else already has it.
    Task<bool> TryClaimAsync(string id, CancellationToken cancellationToken = default);

    // Marks a slot as free again.
    Task ReleaseAsync(string id, CancellationToken cancellationToken = default);

    // Inserts a new slot document.
    Task InsertAsync(Slot slot, CancellationToken cancellationToken = default);

    // Replaces an existing slot document by id.
    Task ReplaceAsync(Slot slot, CancellationToken cancellationToken = default);

    // Deletes a slot document by id.
    Task DeleteAsync(string id, CancellationToken cancellationToken = default);

    // True when a Pending or Approved reservation still holds this slot.
    Task<bool> HasActiveReservationAsync(string slotId, CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Repositories/IStationRepository.cs M1 · 28 lines

```
/*
 * File: IStationRepository.cs
 * Description: Data-access contract for SolarStationInfo. Ready for Member 2 to extend.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;
```

```

public interface IStationRepository
{
    // Returns every station document.
    Task<IReadOnlyList<Station>> GetAllAsync(CancellationToken cancellationToken = default);

    // Loads one station by id.
    Task<Station?> FindByIdAsync(string id, CancellationToken cancellationToken = default);

    // Inserts a new station document.
    Task InsertAsync(Station station, CancellationToken cancellationToken = default);

    // Replaces an existing station document by id.
    Task ReplaceAsync(Station station, CancellationToken cancellationToken = default);

    // True when a future Pending or Approved reservation still points at this station.
    Task<bool> HasActiveFutureReservationsAsync(string stationId, DateTime nowUtc, CancellationToken cancellationToken = default);
}

```

WebService/SmartSolar.Api/Repositories/IUserRepository.cs M1 · 37 lines

```

/*
 * File: IUserRepository.cs
 * Description: Data-access contract for the Users collection. Other members must reuse this.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public interface IUserRepository
{
    // Returns how many user documents exist.
    Task<long> CountAsync(CancellationToken cancellationToken = default);

    // Loads a user by document id (staff username or prosumer NIC).
    Task<User?> FindByIdAsync(string id, CancellationToken cancellationToken = default);

    // Resolves a login value against either username or NIC.
    Task<User?> FindByUsernameOrNicAsync(string login, CancellationToken cancellationToken = default);

    // Returns every user document.
    Task<IReadOnlyList<User>> GetAllAsync(CancellationToken cancellationToken = default);

    // Returns Backoffice and GridOperator accounts for the staff Users API.
    Task<IReadOnlyList<User>> GetStaffAsync(CancellationToken cancellationToken = default);

    // Returns prosumer accounts, optionally filtered by status and a free-text search.
    Task<IReadOnlyList<User>> SearchProsumersAsync(string? status, string? query, CancellationToken cancellationToken = default);

    // Inserts a new user document.
    Task InsertAsync(User user, CancellationToken cancellationToken = default);

    // Replaces an existing user document by id.
    Task ReplaceAsync(User user, CancellationToken cancellationToken = default);
}

```

WebService/SmartSolar.Api/Repositories/ReservationQueryRepository.cs M4 · 183 lines

```

/*
 * File: ReservationQueryRepository.cs
 * Description: MongoDB queries for reservation lists, dashboard counts and QR lookup.
 * Author: samudith
 * Created: 29/09/2026
 */

using System.Text.RegularExpressions;
using MongoDB.Bson;
using MongoDB.Driver;
using SmartSolar.Api.Common;
using SmartSolar.Api.Data;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public class ReservationQueryRepository : IReservationQueryRepository
{
    private readonly MongoContext _context;

    // Stores the shared Mongo context used by read queries.
    public ReservationQueryRepository(MongoContext context)
    {
        _context = context;
    }

    // Returns one page of reservations matching every supplied filter, sorted by scheduledAt.

```

```

public async Task<IReadOnlyList<Reservation> Items, long TotalCount> SearchAsync(
    string? prosumerNic,
    string? status,
    string? stationId,
    DateTime? fromUtc,
    DateTime? toUtc,
    string? query,
    int skip,
    int take,
    CancellationToken cancellationToken = default)
{
    var filter = BuildFilter(prosumerNic, status, stationId, fromUtc, toUtc, query, activeOnly: false, scheduledAfterUtc: null);
    var total = await _context.Reservations.CountDocumentsAsync(filter, cancellationToken: cancellationToken);
    var items = await _context.Reservations
        .Find(filter)
        .SortBy(reservation => reservation.ScheduledAt)
        .Skip(skip)
        .Limit(take)
        .ToListAsync(cancellationToken);

    return (items, total);
}

// Counts reservations for one prosumer, optionally by status and a scheduledAt lower bound.
public Task<long> CountForProsumerAsync(
    string prosumerNic,
    string? status,
    DateTime? scheduledAfterUtc,
    bool activeOnly,
    CancellationToken cancellationToken = default)
{
    var filter = BuildFilter(prosumerNic, status, null, null, null, null, activeOnly, scheduledAfterUtc);
    return _context.Reservations.CountDocumentsAsync(filter, cancellationToken: cancellationToken);
}

// Returns the soonest future Pending or Approved reservation for one prosumer.
public Task<Reservation?> FindNextUpcomingAsync(string prosumerNic, DateTime nowUtc, CancellationToken cancellationToken = default)
{
    var filter = BuildFilter(prosumerNic, null, null, null, null, null, activeOnly: true, scheduledAfterUtc: nowUtc);
    return _context.Reservations
        .Find(filter)
        .SortBy(reservation => reservation.ScheduledAt)
        .FirstOrDefaultAsync(cancellationToken)!;
}

// Counts reservations whose scheduledAt falls in [startUtc, endUtc).
public Task<long> CountScheduledBetweenAsync(DateTime startUtc, DateTime endUtc, CancellationToken cancellationToken = default)
{
    var filter = Builders<Reservation>.Filter.Gte(reservation => reservation.ScheduledAt, startUtc)
        & Builders<Reservation>.Filter.Lt(reservation => reservation.ScheduledAt, endUtc);
    return _context.Reservations.CountDocumentsAsync(filter, cancellationToken: cancellationToken);
}

// Counts Pending reservations that have not started yet.
public Task<long> CountPendingFutureAsync(DateTime nowUtc, CancellationToken cancellationToken = default)
{
    var filter = Builders<Reservation>.Filter.Eq(reservation => reservation.Status, ReservationStatuses.Pending)
        & Builders<Reservation>.Filter.Gt(reservation => reservation.ScheduledAt, nowUtc);
    return _context.Reservations.CountDocumentsAsync(filter, cancellationToken: cancellationToken);
}

// Counts reservations completed during [startUtc, endUtc).
public Task<long> CountCompletedBetweenAsync(DateTime startUtc, DateTime endUtc, CancellationToken cancellationToken = default)
{
    var filter = Builders<Reservation>.Filter.Eq(reservation => reservation.Status, ReservationStatuses.Completed)
        & Builders<Reservation>.Filter.Gte(reservation => reservation.CompletedAt, startUtc)
        & Builders<Reservation>.Filter.Lt(reservation => reservation.CompletedAt, endUtc);
    return _context.Reservations.CountDocumentsAsync(filter, cancellationToken: cancellationToken);
}

// Lists reservations scheduled during [startUtc, endUtc), earliest first.
public async Task<IReadOnlyList<Reservation>> ListScheduledBetweenAsync(
    DateTime startUtc,
    DateTime endUtc,
    int limit,
    CancellationToken cancellationToken = default)
{
    var filter = Builders<Reservation>.Filter.Gte(reservation => reservation.ScheduledAt, startUtc)
        & Builders<Reservation>.Filter.Lt(reservation => reservation.ScheduledAt, endUtc);
    return await _context.Reservations
        .Find(filter)
        .SortBy(reservation => reservation.ScheduledAt)
        .Limit(limit)
        .ToListAsync(cancellationToken);
}

// Loads the reservation that carries this QR token, including one already completed.
public Task<Reservation?> FindByQrTokenAsync(string qrToken, CancellationToken cancellationToken = default)
{
    return _context.Reservations
        .Find(reservation => reservation.QrToken == qrToken)
        .FirstOrDefaultAsync(cancellationToken)!;
}

// Combines equality, range and keyword filters. A blank value is ignored.
private static FilterDefinition<Reservation> BuildFilter(
    string? prosumerNic,
    string? status,

```

```

        string? stationId,
        DateTime? fromUtc,
        DateTime? toUtc,
        string? query,
        bool activeOnly,
        DateTime? scheduledAfterUtc)
    {
        var filter = Builders<Reservation>.Filter.Empty;

        if (!string.IsNullOrEmpty(prosumerNic))
        {
            filter &= Builders<Reservation>.Filter.Eq(reservation => reservation.ProsumerNic, prosumerNic.Trim());
        }

        if (!string.IsNullOrEmpty(status))
        {
            filter &= Builders<Reservation>.Filter.Eq(reservation => reservation.Status, status.Trim());
        }

        if (!string.IsNullOrEmpty(stationId))
        {
            filter &= Builders<Reservation>.Filter.Eq(reservation => reservation.StationId, stationId.Trim());
        }

        if (fromUtc is not null)
        {
            filter &= Builders<Reservation>.Filter.Gte(reservation => reservation.ScheduledAt, fromUtc.Value);
        }

        if (toUtc is not null)
        {
            filter &= Builders<Reservation>.Filter.Lte(reservation => reservation.ScheduledAt, toUtc.Value);
        }

        if (scheduledAfterUtc is not null)
        {
            filter &= Builders<Reservation>.Filter.Gt(reservation => reservation.ScheduledAt, scheduledAfterUtc.Value);
        }

        if (activeOnly)
        {
            filter &= Builders<Reservation>.Filter.In(
                reservation => reservation.Status,
                new[] { ReservationStatuses.Pending, ReservationStatuses.Approved });
        }

        if (!string.IsNullOrEmpty(query))
        {
            var pattern = new BsonRegularExpression(Regex.Escape(query.Trim()), "i");
            filter &= Builders<Reservation>.Filter.Or(
                Builders<Reservation>.Filter.Regex(reservation => reservation.StationName, pattern),
                Builders<Reservation>.Filter.Regex(reservation => reservation.ProsumerNic, pattern),
                Builders<Reservation>.Filter.Regex(reservation => reservation.Id, pattern));
        }

        return filter;
    }
}

```

WebService/SmartSolar.Api/Repositories/ReservationRepository.cs M1 · 93 lines

```

/*
 * File: ReservationRepository.cs
 * Description: MongoDB access for EnergyReservation.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using System.Text.RegularExpressions;
using MongoDB.Bson;
using MongoDB.Driver;
using SmartSolar.Api.Data;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public class ReservationRepository : IReservationRepository
{
    private readonly MongoContext _context;

    // Stores the shared Mongo context used by reservation queries.
    public ReservationRepository(MongoContext context)
    {
        _context = context;
    }

    // Returns every reservation document.
    public async Task<IReadOnlyList<Reservation>> GetAllAsync(CancellationToken cancellationToken = default)
    {
        return await _context.Reservations.Find(FilterDefinition<Reservation>.Empty).ToListAsync(cancellationToken);
    }

    // Loads one reservation by id.

```

```

public async Task<Reservation?> FindByIdAsync(string id, CancellationToken cancellationToken = default)
{
    return await _context.Reservations.Find(reservation => reservation.Id == id).FirstOrDefaultAsync(cancellationToken);
}

// Finds reservations matching the optional filters, ordered by slot time.
public async Task<IReadOnlyList<Reservation>> SearchAsync(string? nic, string? status, DateTime? from, DateTime? to, string? text, CancellationToken cancellationToken = default)
{
    var f = Builders<Reservation>.Filter;
    var filter = f.Empty;

    if (!string.IsNullOrEmpty(nic))
    {
        filter &= f.Eq(r => r.ProsumerNic, nic);
    }

    if (!string.IsNullOrEmpty(status))
    {
        filter &= f.Eq(r => r.Status, status);
    }

    if (from is not null)
    {
        filter &= f.Gte(r => r.ScheduledAt, from.Value);
    }

    if (to is not null)
    {
        filter &= f.Lte(r => r.ScheduledAt, to.Value);
    }

    if (!string.IsNullOrEmpty(text))
    {
        // Escape the user's text so it is matched as plain text, not as a regex.
        var pattern = new BsonRegularExpression(Regex.Escape(text), "i");
        filter &= f.Or(
            f.Regex(r => r.StationName, pattern),
            f.Regex(r => r.ProsumerNic, pattern),
            f.Regex(r => r.Id, pattern));
    }

    return await _context.Reservations.Find(filter).SortBy(r => r.ScheduledAt).ToListAsync(cancellationToken);
}

// Inserts a new reservation.
public async Task InsertAsync(Reservation reservation, CancellationToken cancellationToken = default)
{
    await _context.Reservations.InsertOneAsync(reservation, cancellationToken: cancellationToken);
}

// Saves a changed reservation only if nobody else changed it since it was loaded.
public async Task<bool> TryReplaceAsync(Reservation reservation, DateTime loadedUpdatedAt, CancellationToken cancellationToken = default)
{
    var result = await _context.Reservations.ReplaceOneAsync(
        r => r.Id == reservation.Id && r.UpdatedAt == loadedUpdatedAt,
        reservation,
        cancellationToken: cancellationToken);

    return result.ModifiedCount == 1;
}
}

```

WebService/SmartSolar.Api/Repositories/SlotRepository.cs M1 · 95 lines

```

/*
 * File: SlotRepository.cs
 * Description: MongoDB access for EnergyBookingSlots.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using MongoDB.Driver;
using SmartSolar.Api.Common;
using SmartSolar.Api.Data;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public class SlotRepository : ISlotRepository
{
    private readonly MongoContext _context;

    // Stores the shared Mongo context used by slot queries.
    public SlotRepository(MongoContext context)
    {
        _context = context;
    }

    // Returns every slot belonging to a station.
    public async Task<IReadOnlyList<Slot>> GetByStationAsync(string stationId, CancellationToken cancellationToken = default)
    {
        return await _context.Slots.Find(slot => slot.StationId == stationId).ToListAsync(cancellationToken);
    }
}

```

```

}

// Loads one slot by id.
public async Task<Slot?> FindByIdAsync(string id, CancellationToken cancellationToken = default)
{
    return await _context.Slots.Find(slot => slot.Id == id).FirstOrDefaultAsync(cancellationToken);
}

// Returns free slots of a station that start after `from` and no later than `to`.
public async Task<IReadOnlyList<Slot>> GetFreeAsync(string stationId, DateTime from, DateTime to, CancellationToken cancellationToken = default)
{
    return await _context.Slots
        .Find(slot => slot.StationId == stationId && slot.IsAvailable && slot.StartTime > from && slot.StartTime <= to)
        .SortBy(slot => slot.StartTime)
        .ToListAsync(cancellationToken);
}

// Marks a free slot as taken. Returns false if someone else already has it.
public async Task<bool> TryClaimAsync(string id, CancellationToken cancellationToken = default)
{
    // The filter and the update run as one operation in MongoDB, so only one request can win.
    var result = await _context.Slots.UpdateOneAsync(
        slot => slot.Id == id && slot.IsAvailable,
        Builders<Slot>.Update.Set(slot => slot.IsAvailable, false),
        cancellationToken: cancellationToken);

    return result.ModifiedCount == 1;
}

// Marks a slot as free again.
public async Task ReleaseAsync(string id, CancellationToken cancellationToken = default)
{
    await _context.Slots.UpdateOneAsync(
        slot => slot.Id == id,
        Builders<Slot>.Update.Set(slot => slot.IsAvailable, true),
        cancellationToken: cancellationToken);
}

// Inserts a new slot document.
public Task InsertAsync(Slot slot, CancellationToken cancellationToken = default)
{
    return _context.Slots.InsertOneAsync(slot, cancellationToken: cancellationToken);
}

// Replaces an existing slot document by id.
public Task ReplaceAsync(Slot slot, CancellationToken cancellationToken = default)
{
    return _context.Slots.ReplaceOneAsync(existing => existing.Id == slot.Id, slot, cancellationToken: cancellationToken);
}

// Deletes a slot document by id.
public Task DeleteAsync(string id, CancellationToken cancellationToken = default)
{
    return _context.Slots.DeleteOneAsync(slot => slot.Id == id, cancellationToken);
}

// True when a Pending or Approved reservation still holds this slot.
public Task<bool> HasActiveReservationAsync(string slotId, CancellationToken cancellationToken = default)
{
    var filter = Builders<Reservation>.Filter.Eq(reservation => reservation.SlotId, slotId)
        & Builders<Reservation>.Filter.In(
            reservation => reservation.Status,
            new[] { ReservationStatuses.Pending, ReservationStatuses.Approved });

    return _context.Reservations.Find(filter).AnyAsync(cancellationToken);
}
}

```

WebService/SmartSolar.Api/Repositories/StationRepository.cs M1 - 60 lines

```

/*
 * File: StationRepository.cs
 * Description: MongoDB access for SolarStationInfo.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using MongoDB.Driver;
using SmartSolar.Api.Common;
using SmartSolar.Api.Data;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public class StationRepository : IStationRepository
{
    private readonly MongoContext _context;

    // Stores the shared Mongo context used by station queries.
    public StationRepository(MongoContext context)
    {
        _context = context;
    }
}

```

```

// Returns every station document.
public async Task<IReadOnlyList<Station>> GetAllAsync(CancellationToken cancellationToken = default)
{
    return await _context.Stations.Find(FilterDefinition<Station>.Empty).ToListAsync(cancellationToken);
}

// Loads one station by id.
public async Task<Station?> FindByIdAsync(string id, CancellationToken cancellationToken = default)
{
    return await _context.Stations.Find(station => station.Id == id).FirstOrDefaultAsync(cancellationToken);
}

// Inserts a new station document.
public Task InsertAsync(Station station, CancellationToken cancellationToken = default)
{
    return _context.Stations.InsertOneAsync(station, cancellationToken: cancellationToken);
}

// Replaces an existing station document by id.
public Task ReplaceAsync(Station station, CancellationToken cancellationToken = default)
{
    return _context.Stations.ReplaceOneAsync(existing => existing.Id == station.Id, station, cancellationToken: cancellationToken);
}

// True when a future Pending or Approved reservation still points at this station.
public Task<bool> HasActiveFutureReservationsAsync(string stationId, DateTime nowUtc, CancellationToken cancellationToken = default)
{
    var filter = Builders<Reservation>.Filter.Eq(reservation => reservation.StationId, stationId)
        & Builders<Reservation>.Filter.Gt(reservation => reservation.ScheduledAt, nowUtc)
        & Builders<Reservation>.Filter.In(
            reservation => reservation.Status,
            new[] { ReservationStatuses.Pending, ReservationStatuses.Approved });

    return _context.Reservations.Find(filter).AnyAsync(cancellationToken);
}
}

```

WebService/SmartSolar.Api/Repositories/UserRepository.cs M1 · 102 lines

```

/*
 * File: UserRepository.cs
 * Description: MongoDB access for Users. No business rules live here.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using System.Text.RegularExpressions;
using MongoDB.Bson;
using MongoDB.Driver;
using SmartSolar.Api.Common;
using SmartSolar.Api.Data;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Repositories;

public class UserRepository : IUserRepository
{
    private readonly MongoContext _context;

    // Stores the shared Mongo context used by every user query.
    public UserRepository(MongoContext context)
    {
        _context = context;
    }

    // Returns how many user documents exist (used by the seeder guard).
    public Task<long> CountAsync(CancellationToken cancellationToken = default)
    {
        return _context.Users.CountDocumentsAsync(FilterDefinition<User>.Empty, cancellationToken: cancellationToken);
    }

    // Loads a user by document id (staff username or prosumer NIC).
    public async Task<User?> FindByIdAsync(string id, CancellationToken cancellationToken = default)
    {
        return await _context.Users.Find(user => user.Id == id).FirstOrDefaultAsync(cancellationToken);
    }

    // Resolves a login value against either username or NIC.
    public async Task<User?> FindByUsernameOrNicAsync(string login, CancellationToken cancellationToken = default)
    {
        var filter = Builders<User>.Filter.Or(
            Builders<User>.Filter.Eq(user => user.Username, login),
            Builders<User>.Filter.Eq(user => user.Nic, login));

        return await _context.Users.Find(filter).FirstOrDefaultAsync(cancellationToken);
    }

    // Returns every user document for Backoffice listing.
    public async Task<IReadOnlyList<User>> GetAllAsync(CancellationToken cancellationToken = default)
    {
        return await _context.Users.Find(FilterDefinition<User>.Empty).ToListAsync(cancellationToken);
    }
}

```

```

// Returns staff accounts only (Backoffice and GridOperator).
public async Task<IReadOnlyList<User>> GetStaffAsync(CancellationToken cancellationToken = default)
{
    var filter = Builders<User>.Filter.In(user => user.Role, new[] { RoleNames.Backoffice, RoleNames.GridOperator });
    return await _context.Users.Find(filter).ToListAsync(cancellationToken);
}

// Returns Prosumer documents matching optional status and NIC/name/phone/username text.
public async Task<IReadOnlyList<User>> SearchProsumersAsync(string? status, string? query, CancellationToken cancellationToken = default)
{
    var filters = new List<FilterDefinition<User>>
    {
        Builders<User>.Filter.Eq(user => user.Role, RoleNames.Prosumer)
    };

    if (!string.IsNullOrEmpty(status))
    {
        filters.Add(Builders<User>.Filter.Eq(user => user.Status, status.Trim()));
    }

    if (!string.IsNullOrEmpty(query))
    {
        var escaped = Regex.Escape(query.Trim());
        var pattern = new BsonRegularExpression(escaped, "i");
        filters.Add(Builders<User>.Filter.Or(
            Builders<User>.Filter.Regex(user => user.Nic, pattern),
            Builders<User>.Filter.Regex(user => user.Username, pattern),
            Builders<User>.Filter.Regex(user => user.FullName, pattern),
            Builders<User>.Filter.Regex(user => user.Phone, pattern)));
    }

    return await _context.Users.Find(Builders<User>.Filter.And(filters))
        .SortBy(user => user.FullName)
        .ToListAsync(cancellationToken);
}

// Inserts a new user document.
public Task InsertAsync(User user, CancellationToken cancellationToken = default)
{
    return _context.Users.InsertOneAsync(user, cancellationToken: cancellationToken);
}

// Replaces an existing user document by id.
public Task ReplaceAsync(User user, CancellationToken cancellationToken = default)
{
    return _context.Users.ReplaceOneAsync(existing => existing.Id == user.Id, user, cancellationToken: cancellationToken);
}
}

```

WebService/SmartSolar.Api/Services/ActiveAccountGuard.cs M1 · 34 lines

```

/*
 * File: ActiveAccountGuard.cs
 * Description: Reads Users by id so JWT validation can reject deactivated or missing accounts.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 21/09/2026
 */

using SmartSolar.Api.Common;
using SmartSolar.Api.Repositories;

namespace SmartSolar.Api.Services;

public class ActiveAccountGuard : IActiveAccountGuard
{
    private readonly IUserRepository _users;

    // Injects the shared user repository used on every authenticated request.
    public ActiveAccountGuard(IUserRepository users)
    {
        _users = users;
    }

    // Returns true only when the user exists and Status is Active.
    public async Task<bool> IsActiveAsync(string? userId, CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrEmpty(userId))
        {
            return false;
        }

        var user = await _users.FindByIdAsync(userId, cancellationToken);
        return user is not null && user.Status == UserStatuses.Active;
    }
}

```

WebService/SmartSolar.Api/Services/AuthService.cs M1 · 86 lines

```

/*
 * File: AuthService.cs

```

```

* Description: Validates credentials, rejects inactive accounts, and issues a JWT.
* Author: DE SILVA R K D H (IT22001252)
* Created: 20/09/2026
*/

```

```

using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Repositories;

```

```

namespace SmartSolar.Api.Services;

```

```

public class AuthService : IAuthService
{
    private readonly IUserRepository _users;
    private readonly IPasswordService _passwords;
    private readonly IJwtTokenService _tokens;
    private readonly ProsumerService _prosumers;

    // Wires the repository and crypto helpers used during login.
    public AuthService(IUserRepository users, IPasswordService passwords, IJwtTokenService tokens, ProsumerService prosumers)
    {
        _users = users;
        _passwords = passwords;
        _tokens = tokens;
        _prosumers = prosumers;
    }

    // Registers a Pending account. Activation is deliberately a separate Backoffice action.
    public Task<UserDto> RegisterAsync(RegisterProsumerRequestDto request, CancellationToken cancellationToken = default)
    {
        return _prosumers.CreateAccountAsync(
            request.Nic,
            request.Username,
            request.Password,
            request.FullName,
            request.Email,
            request.Phone,
            UserStatuses.Pending,
            cancellationToken);
    }

    // Authenticates by username or NIC and returns a token plus the public user.
    public async Task<LoginResponseDto> LoginAsync(LoginRequestDto request, CancellationToken cancellationToken = default)
    {
        if (string.IsNullOrEmpty(request.Password))
        {
            throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "Password is required.");
        }

        var login = FirstNonEmpty(request.Username, request.Nic);
        if (string.IsNullOrEmpty(login))
        {
            throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "Username or NIC is required.");
        }

        var user = await _users.FindByUsernameOrNicAsync(login.Trim(), cancellationToken);
        if (user is null || !_passwords.Verify(user, request.Password))
        {
            throw new ApiException(StatusCodes.Status401Unauthorized, ErrorCodes.InvalidCredentials, "Invalid username or password.");
        }

        if (user.Status is not UserStatuses.Active)
        {
            throw new ApiException(
                StatusCodes.Status409Conflict,
                ErrorCodes.AccountNotActive,
                "Account is not active. Pending accounts must be activated by Backoffice.");
        }

        var (token, expiresAt) = _tokens.CreateToken(user);
        return new LoginResponseDto
        {
            Token = token,
            ExpiresAt = expiresAt,
            User = UserDto.From(user)
        };
    }

    // Picks the first non-blank login identifier from the request.
    private static string? FirstNonEmpty(params string?[] values)
    {
        return values.FirstOrDefault(value => !string.IsNullOrEmpty(value));
    }
}

```

WebService/SmartSolar.Api/Services/IActiveAccountGuard.cs M1 · 14 lines

```

/*
* File: IActiveAccountGuard.cs
* Description: Looks up the current user status during JWT validation. Deactivated tokens must fail.
* Author: DE SILVA R K D H (IT22001252)
* Created: 21/09/2026
*/

```

```
namespace SmartSolar.Api.Services;

public interface IActiveAccountGuard
{
    // Returns true only when the user exists and Status is Active.
    Task<bool> IsActiveAsync(string? userId, CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Services/IAuthService.cs M1 · 19 lines

```
/*
 * File: IAuthService.cs
 * Description: Login use-case contract used by AuthController.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Services;

public interface IAuthService
{
    // Authenticates by username or NIC and returns a JWT plus the public user.
    Task<LoginResponseDto> LoginAsync(LoginRequestDto request, CancellationToken cancellationToken = default);

    // Creates a Pending prosumer account that Backoffice must activate before login.
    Task<UserDto> RegisterAsync(RegisterProsumerRequestDto request, CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Services/IJwtTokenService.cs M1 · 16 lines

```
/*
 * File: IJwtTokenService.cs
 * Description: Issues signed JWTs with the contract claims: sub, role, nic.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.Services;

public interface IJwtTokenService
{
    // Creates a signed token containing sub, role and nic (prosumers only).
    (string Token, DateTime ExpiresAt) CreateToken(User user);
}
```

WebService/SmartSolar.Api/Services/IOperatorService.cs M4 · 19 lines

```
/*
 * File: IOperatorService.cs
 * Description: Grid-operator QR verification and job completion.
 * Author: samudith
 * Created: 29/09/2026
 */

using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Services;

public interface IOperatorService
{
    // Checks a QR token and returns the booking when it can be completed today.
    Task<ReservationDto> VerifyQrAsync(string? qrToken, CancellationToken cancellationToken = default);

    // Marks an approved, verified booking Completed and keeps the token so a reuse is rejected.
    Task<ReservationDto> CompleteAsync(string id, ReservationActor actor, CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Services/IPasswordService.cs M1 · 19 lines

```
/*
 * File: IPasswordService.cs
 * Description: Password hashing/verification contract. Never store or log plaintext passwords.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.Models;

namespace SmartSolar.Api.Services;

public interface IPasswordService
{
}
```

```
// Creates a salted hash of the given plaintext password.
string Hash(User user, string password);

// Returns true when the plaintext matches the stored hash.
bool Verify(User user, string password);
}
```

WebService/SmartSolar.Api/Services/IProsumerService.cs M1 · 24 lines

```
/*
 * File: IProsumerService.cs
 * Description: Prosumer profile and Backoffice administration use-cases.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */

using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Services;

public interface IProsumerService
{
    Task<IReadOnlyList<UserDto>> SearchAsync(string? status, string? query, CancellationToken cancellationToken = default);
    Task<IReadOnlyList<UserDto>> ListPendingAsync(CancellationToken cancellationToken = default);
    Task<UserDto> GetAsync(string nic, CancellationToken cancellationToken = default);
    Task<UserDto> GetMeAsync(string nic, CancellationToken cancellationToken = default);
    Task<UserDto> CreateAsync(CreateProsumerRequestDto request, CancellationToken cancellationToken = default);
    Task<UserDto> UpdateAsync(string nic, UpdateProsumerRequestDto request, CancellationToken cancellationToken = default);
    Task<UserDto> UpdateMeAsync(string nic, UpdateProsumerRequestDto request, CancellationToken cancellationToken = default);
    Task<UserDto> DeactivateAsync(string nic, CancellationToken cancellationToken = default);
    Task<UserDto> ReactivateAsync(string nic, CancellationToken cancellationToken = default);
    Task<UserDto> ActivateAsync(string nic, CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Services/IReservationQueryService.cs M4 · 34 lines

```
/*
 * File: IReservationQueryService.cs
 * Description: List, history and dashboard reads. Prosumer scope is applied here from the JWT.
 * Author: samudith
 * Created: 29/09/2026
 */

using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Services;

public interface IReservationQueryService
{
    // Lists reservations. A prosumer only ever sees their own NIC, taken from the token.
    Task<ReservationPageDto> SearchAsync(
        ReservationActor actor,
        string? status,
        DateTime? from,
        DateTime? to,
        string? stationId,
        string? query,
        string? nic,
        int page,
        CancellationToken cancellationToken = default);

    // Returns the operator approval queue: future Pending bookings.
    Task<ReservationPageDto> GetPendingQueueAsync(int page, CancellationToken cancellationToken = default);

    // Returns the signed-in prosumer's live dashboard counts and next booking.
    Task<ProsumerDashboardDto> GetProsumerDashboardAsync(ReservationActor actor, CancellationToken cancellationToken = default);

    // Returns today's operational counts for the staff Home page.
    Task<OperationsDashboardDto> GetOperationsDashboardAsync(CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Services/IReservationService.cs M3 · 33 lines

```
/*
 * File: IReservationService.cs
 * Description: Reservation use cases. All booking rules for web and mobile run behind this interface.
 * Author: Janukshan S (IT22635266)
 */

using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Services;

public interface IReservationService
{
    // Returns the stations that are taking bookings.
    Task<IReadOnlyList<BookingStationDto>> GetBookableStationsAsync(CancellationToken cancellationToken = default);

    // Returns free slots of a station inside the booking window, optionally for one Sri Lanka date.
}
```

```

Task<IReadOnlyList<AvailableSlotDto>> GetAvailableSlotsAsync(string stationId, DateOnly? date, Cancellation_token cancellationToken = default);

// Books a free slot for a prosumer, or for the given NIC when staff book on their behalf.
Task<ReservationDto> CreateAsync(CreateReservationRequestDto request, ReservationActor actor, Cancellation_token cancellationToken = default);

// Returns one reservation. Prosumers can only read their own.
Task<ReservationDto> GetByIdAsync(string id, ReservationActor actor, Cancellation_token cancellationToken = default);

// Moves a booking to another free slot. Needs 12 hours notice.
Task<ReservationDto> UpdateAsync(string id, UpdateReservationRequestDto request, ReservationActor actor, Cancellation_token cancellationToken = default);

// Cancels a booking and frees its slot. Needs 12 hours notice.
Task<ReservationDto> CancelAsync(string id, CancelReservationRequestDto? request, ReservationActor actor, Cancellation_token cancellationToken = default);

// Approves a pending booking and gives it a QR token.
Task<ReservationDto> ApproveAsync(string id, ReservationActor actor, Cancellation_token cancellationToken = default);
}

```

WebService/SmartSolar.Api/Services/ISlotService.cs M2 · 29 lines

```

/*
 * File: ISlotService.cs
 * Description: Slot use-cases. Schedule, battery overlap, booking locks and availability live here.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using SmartSolar.Api.DTOS;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Services;

public interface ISlotService
{
    // Returns one station's slots, earliest start first.
    Task<IReadOnlyList<Slot>> ListByStationAsync(string stationId, Cancellation_token cancellationToken = default);

    // Creates an available slot after the schedule and battery rules pass.
    Task<Slot> CreateAsync(string stationId, SaveSlotRequestDto request, Cancellation_token cancellationToken = default);

    // Moves a free slot. A slot with an active reservation cannot be moved.
    Task<Slot> UpdateAsync(string slotId, SaveSlotRequestDto request, Cancellation_token cancellationToken = default);

    // Deletes a free slot. A slot with an active reservation cannot be deleted.
    Task DeleteAsync(string slotId, Cancellation_token cancellationToken = default);

    // Sets the booking flag. A slot with an active reservation cannot be changed either way.
    Task<Slot> SetAvailabilityAsync(string slotId, SlotAvailabilityRequestDto request, Cancellation_token cancellationToken = default);
}

```

WebService/SmartSolar.Api/Services/IStationService.cs M2 · 35 lines

```

/*
 * File: IStationService.cs
 * Description: Station use-cases. All node rules, including deactivation and nearby search, live here.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using SmartSolar.Api.DTOS;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Services;

public interface IStationService
{
    // Returns every station document.
    Task<IReadOnlyList<Station>> ListAsync(Cancellation_token cancellationToken = default);

    // Returns one station or throws 404 when the id is unknown.
    Task<Station> GetAsync(string id, Cancellation_token cancellationToken = default);

    // Creates an Active station after the field rules pass.
    Task<Station> CreateAsync(SaveStationRequestDto request, Cancellation_token cancellationToken = default);

    // Updates station fields. Id and status stay unchanged.
    Task<Station> UpdateAsync(string id, SaveStationRequestDto request, Cancellation_token cancellationToken = default);

    // Sets a station Inactive, or 409 when a future Pending or Approved reservation exists.
    Task<Station> DeactivateAsync(string id, Cancellation_token cancellationToken = default);

    // Sets a station Active again.
    Task<Station> ActivateAsync(string id, Cancellation_token cancellationToken = default);

    // Returns Active stations within radiusKm, nearest first, with free slots in the next 7 days.
    Task<IReadOnlyList<NearbyStationDto>> NearbyAsync(double? lat, double? lng, double radiusKm, Cancellation_token cancellationToken = default);
}

```

WebService/SmartSolar.Api/Services/IUserService.cs M1 · 28 lines

```
/*
 * File: IUserService.cs
 * Description: Staff user use-cases for Backoffice. Prosumer admin endpoints belong to Member 2.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Api.DTOs;

namespace SmartSolar.Api.Services;

public interface IUserService
{
    // Returns staff accounts without password hashes.
    Task<IReadOnlyList<UserDto>> ListStaffAsync(CancellationToken cancellationToken = default);

    // Returns one staff user or throws 404 when the id is missing or is not staff.
    Task<UserDto> GetStaffByIdAsync(string id, CancellationToken cancellationToken = default);

    // Creates a Backoffice or GridOperator account.
    Task<UserDto> CreateStaffAsync(CreateStaffRequestDto request, CancellationToken cancellationToken = default);

    // Updates a staff account. Password is optional. Callers pass actorId to block changing their own role.
    Task<UserDto> UpdateStaffAsync(string id, UpdateStaffRequestDto request, string? actorId, CancellationToken cancellationToken = default);

    // Sets a staff account to Deactivated. Callers pass the current user id to block self-deactivation.
    Task<UserDto> DeactivateStaffAsync(string id, string? actorId, CancellationToken cancellationToken = default);
}
```

WebService/SmartSolar.Api/Services/JwtTokenService.cs M1 · 58 lines

```
/*
 * File: JwtTokenService.cs
 * Description: Builds HMAC-signed JWTs from configuration (issuer, audience, signing key).
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 *
 * JWT structure and bearer authentication follow Microsoft docs:
 * https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwt-authn
 */

using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;
using Microsoft.Extensions.Options;
using Microsoft.IdentityModel.Tokens;
using SmartSolar.Api.Configuration;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Services;

public class JwtTokenService : IJwtTokenService
{
    private readonly JwtSettings _settings;

    // Loads JWT settings that must come from configuration, not source code.
    public JwtTokenService(IOptions<JwtSettings> options)
    {
        _settings = options.Value;
    }

    // Creates a signed token containing sub, role and nic (prosumers only).
    public (string Token, DateTime ExpiresAt) CreateToken(User user)
    {
        var expiresAt = DateTime.UtcNow.AddMinutes(_settings.ExpiryMinutes);
        var claims = new List<Claim>
        {
            new("sub", user.Id),
            new("role", user.Role)
        };

        if (!string.IsNullOrEmpty(user.Nic))
        {
            claims.Add(new Claim("nic", user.Nic));
        }

        var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
        var credentials = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
        var jwt = new JwtSecurityToken(
            issuer: _settings.Issuer,
            audience: _settings.Audience,
            claims: claims,
            expires: expiresAt,
            signingCredentials: credentials);

        var token = new JwtSecurityTokenHandler().WriteToken(jwt);
        return (token, expiresAt);
    }
}
```

```
}  
}
```

WebService/SmartSolar.Api/Services/OperatorService.cs M4 · 105 lines

```
/*  
 * File: OperatorService.cs  
 * Description: Server-side QR checks. The token is opaque, single-use, and only valid near the booking time.  
 * Author: samudith  
 * Created: 29/09/2026  
 */  
  
using SmartSolar.Api.Common;  
using SmartSolar.Api.DTOs;  
using SmartSolar.Api.Repositories;  
  
namespace SmartSolar.Api.Services;  
  
public class OperatorService : IOperatorService  
{  
    private readonly IReservationQueryRepository _queries;  
    private readonly IReservationRepository _reservations;  
  
    // Injects the QR lookup and the reservation writer used to mark a job completed.  
    public OperatorService(IReservationQueryRepository queries, IReservationRepository reservations)  
    {  
        _queries = queries;  
        _reservations = reservations;  
    }  
  
    // Checks a QR token and returns the booking when it can be completed today.  
    public async Task<ReservationDto> VerifyQrAsync(string? qrToken, CancellationToken cancellationToken = default)  
    {  
        var reservation = await FindTokenAsync(qrToken, cancellationToken);  
        EnsureScannable(reservation, SriLankaClock.UtcNow());  
        return ReservationDto.From(reservation);  
    }  
  
    // Marks an approved booking Completed. The token is kept so a second scan returns QR_ALREADY_USED.  
    public async Task<ReservationDto> CompleteAsync(string id, ReservationActor actor, CancellationToken cancellationToken = default)  
    {  
        var reservation = await _reservations.FindByIdAsync(id, cancellationToken);  
        if (reservation is null)  
        {  
            throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Reservation was not found.");  
        }  
  
        if (reservation.Status == ReservationStatuses.Completed)  
        {  
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.QrAlreadyUsed, "This QR code has already been used.");  
        }  
  
        if (reservation.Status != ReservationStatuses.Approved || string.IsNullOrEmpty(reservation.QrToken))  
        {  
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.InvalidState, "Verify the QR code before completing this booking.");  
        }  
  
        var now = SriLankaClock.UtcNow();  
        EnsureScannable(reservation, now);  
  
        var loadedUpdatedAt = reservation.UpdatedAt;  
        reservation.Status = ReservationStatuses.Completed;  
        reservation.CompletedAt = now;  
        reservation.CompletedBy = actor.UserId;  
        reservation.UpdatedAt = now;  
  
        if (!await _reservations.TryReplaceAsync(reservation, loadedUpdatedAt, cancellationToken))  
        {  
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.InvalidState, "This reservation was just changed by someone else. Please reload and try again.");  
        }  
  
        return ReservationDto.From(reservation);  
    }  
  
    // Loads a token or rejects a blank or unknown code as QR_INVALID.  
    private async Task<Models.Reservation> FindTokenAsync(string? qrToken, CancellationToken cancellationToken)  
    {  
        if (string.IsNullOrEmpty(qrToken))  
        {  
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.QrInvalid, "The QR code is not valid.");  
        }  
  
        var reservation = await _queries.FindByQrTokenAsync(qrToken.Trim(), cancellationToken);  
        if (reservation is null)  
        {  
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.QrInvalid, "The QR code is not valid.");  
        }  
  
        return reservation;  
    }  
}
```

```
// Rejects a used, unapproved, or out-of-window booking. A completed token is QR_ALREADY_USED.
private static void EnsureScannable(Models.Reservation reservation, DateTime nowUtc)
{
    if (reservation.Status == ReservationStatuses.Completed)
    {
        throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.QrAlreadyUsed, "This QR code has already been used.");
    }

    if (reservation.Status != ReservationStatuses.Approved)
    {
        throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.QrInvalid, "This QR code is not valid for the booking's current status.");
    }

    if (!SriLankaClock.CanScanAt(reservation.ScheduledAt, nowUtc))
    {
        throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.QrInvalid, "This QR code can only be used on the day of the booking.");
    }
}
}
```

WebService/SmartSolar.Api/Services/PasswordService.cs M1 · 32 lines

```
/*
 * File: PasswordService.cs
 * Description: Hashes and verifies passwords with ASP.NET Core Identity PasswordHasher.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 *
 * Uses Microsoft.AspNetCore.Identity.PasswordHasher as described in:
 * https://learn.microsoft.com/en-us/aspnet/core/security/data-protection/consumer-apis/password-hashing
 */

using Microsoft.AspNetCore.Identity;
using SmartSolar.Api.Models;

namespace SmartSolar.Api.Services;

public class PasswordService : IPasswordService
{
    private readonly PasswordHasher<User> _hasher = new();

    // Creates a salted hash of the given plaintext password.
    public string Hash(User user, string password)
    {
        return _hasher.HashPassword(user, password);
    }

    // Returns true when the plaintext matches the stored hash.
    public bool Verify(User user, string password)
    {
        var result = _hasher.VerifyHashedPassword(user, user.PasswordHash, password);
        return result is PasswordVerificationResult.Success or PasswordVerificationResult.SuccessRehashNeeded;
    }
}
```

WebService/SmartSolar.Api/Services/ProsumerService.cs M1 · 242 lines

```
/*
 * File: ProsumerService.cs
 * Description: Business rules for registration, profiles and staff-managed prosumer accounts.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */

using MongoDB.Driver;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Models;
using SmartSolar.Api.Repositories;

namespace SmartSolar.Api.Services;

public class ProsumerService : IProsumerService
{
    private readonly IUserRepository _users;
    private readonly IPasswordService _passwords;

    public ProsumerService(IUserRepository users, IPasswordService passwords)
    {
        _users = users;
        _passwords = passwords;
    }

    public async Task<IReadOnlyList<UserDto>> SearchAsync(string? status, string? query, CancellationToken cancellationToken = default)
    {
        ValidateStatusFilter(status);
        var users = await _users.SearchProsumersAsync(status, query, cancellationToken);
        return users.Select(UserDto.From).ToList();
    }

    public Task<IReadOnlyList<UserDto>> ListPendingAsync(CancellationToken cancellationToken = default)
```

```

    {
        return SearchAsync(UserStatuses.Pending, null, cancellationToken);
    }

    public async Task<UserDto> GetAsync(string nic, CancellationToken cancellationToken = default)
    {
        return UserDto.From(await LoadProsumerAsync(nic, cancellationToken));
    }

    public Task<UserDto> GetMeAsync(string nic, CancellationToken cancellationToken = default)
    {
        return GetAsync(nic, cancellationToken);
    }

    public Task<UserDto> CreateAsync(CreateProsumerRequestDto request, CancellationToken cancellationToken = default)
    {
        return CreateAccountAsync(
            request.Nic,
            request.Username,
            request.Password,
            request.FullName,
            request.Email,
            request.Phone,
            UserStatuses.Active,
            cancellationToken);
    }

    public Task<UserDto> UpdateAsync(string nic, UpdateProsumerRequestDto request, CancellationToken cancellationToken = default)
    {
        return UpdateAccountAsync(nic, request, cancellationToken);
    }

    public Task<UserDto> UpdateMeAsync(string nic, UpdateProsumerRequestDto request, CancellationToken cancellationToken = default)
    {
        return UpdateAccountAsync(nic, request, cancellationToken);
    }

    public Task<UserDto> DeactivateAsync(string nic, CancellationToken cancellationToken = default)
    {
        return SetStatusAsync(nic, UserStatuses.Deactivated, requiredCurrentStatus: null, cancellationToken);
    }

    public Task<UserDto> ReactivateAsync(string nic, CancellationToken cancellationToken = default)
    {
        return SetStatusAsync(nic, UserStatuses.Active, UserStatuses.Deactivated, cancellationToken);
    }

    public Task<UserDto> ActivateAsync(string nic, CancellationToken cancellationToken = default)
    {
        return SetStatusAsync(nic, UserStatuses.Active, UserStatuses.Pending, cancellationToken);
    }

    internal async Task<UserDto> CreateAccountAsync(
        string? nic,
        string? username,
        string? password,
        string? fullName,
        string? email,
        string? phone,
        string status,
        CancellationToken cancellationToken)
    {
        NicRules.RequireValid(nic);
        AccountRules.RequireText(username, "Username is required.");
        AccountRules.RequireText(password, "Password is required.");
        AccountRules.RequirePasswordLength(password!);
        AccountRules.RequireText(fullName, "Full name is required.");
        AccountRules.RequireEmailFormat(email);
        AccountRules.RequirePhoneFormat(phone);

        var normalizedNic = NicRules.Normalize(nic!);
        var normalizedUsername = username!.Trim();

        if (await _users.FindByIdAsync(normalizedNic, cancellationToken) is not null)
        {
            throw NicExists();
        }

        var existingLogin = await _users.FindByUsernameOrNicAsync(normalizedUsername, cancellationToken);
        if (existingLogin is not null)
        {
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.UsernameExists, "That username is already taken.");
        }

        var user = new User
        {
            Id = normalizedNic,
            Nic = normalizedNic,
            Username = normalizedUsername,
            FullName = fullName!.Trim(),
            Email = email?.Trim() ?? string.Empty,
            Phone = NormalizePhone(phone),
            Role = RoleNames.Prosumer,
            Status = status,
            CreatedAt = DateTime.UtcNow
        };
    }

```

```

user.PasswordHash = _passwords.Hash(user, password!);

try
{
    await _users.InsertAsync(user, cancellationTokens);
}
catch (MongoWriteException ex) when (ex.WriteError?.Category == ServerErrorCategory.DuplicateKey)
{
    // A concurrent request may win after the pre-check. Re-read to return a stable contract code.
    if (await _users.FindByIdAsync(normalizedNic, cancellationTokens) is not null)
    {
        throw NicExists();
    }

    throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.UsernameExists, "That username is already taken.");
}

return UserDto.From(user);
}

private async Task<UserDto> UpdateAccountAsync(string nic, UpdateProsumerRequestDto request, CancellationToken cancellationTokens)
{
    var user = await LoadProsumerAsync(nic, cancellationTokens);
    AccountRules.RequireText(request.FullName, "Full name is required.");
    AccountRules.RequireEmailFormat(request.Email);
    AccountRules.RequirePhoneFormat(request.Phone);

    if (!string.IsNullOrEmpty(request.Username))
    {
        var username = request.Username.Trim();
        var existing = await _users.FindByUsernameOrNicAsync(username, cancellationTokens);
        if (existing is not null && !string.Equals(existing.Id, user.Id, StringComparison.Ordinal))
        {
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.UsernameExists, "That username is already taken.");
        }

        user.Username = username;
    }

    user.FullName = request.FullName.Trim();
    user.Email = request.Email?.Trim() ?? string.Empty;
    user.Phone = NormalizePhone(request.Phone);

    if (!string.IsNullOrEmpty(request.Password))
    {
        AccountRules.RequirePasswordLength(request.Password);
        user.PasswordHash = _passwords.Hash(user, request.Password);
    }

    try
    {
        await _users.ReplaceAsync(user, cancellationTokens);
    }
    catch (MongoWriteException ex) when (ex.WriteError?.Category == ServerErrorCategory.DuplicateKey)
    {
        throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.UsernameExists, "That username is already taken.");
    }

    return UserDto.From(user);
}

private async Task<UserDto> SetStatusAsync(
    string nic,
    string nextStatus,
    string? requiredCurrentStatus,
    CancellationToken cancellationTokens)
{
    var user = await LoadProsumerAsync(nic, cancellationTokens);
    if (requiredCurrentStatus is not null && user.Status != requiredCurrentStatus)
    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.InvalidState,
            $"Only {requiredCurrentStatus} prosumers can be changed to {nextStatus}.");
    }

    user.Status = nextStatus;
    await _users.ReplaceAsync(user, cancellationTokens);
    return UserDto.From(user);
}

private async Task<User> LoadProsumerAsync(string nic, CancellationToken cancellationTokens)
{
    var normalizedNic = NicRules.Normalize(nic);
    var user = await _users.FindByIdAsync(normalizedNic, cancellationTokens);
    if (user is null || user.Role != RoleNames.Prosumer)
    {
        throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Prosumer was not found.");
    }

    return user;
}

private static void ValidateStatusFilter(string? status)
{

```

```

        if (!string.IsNullOrEmpty(status)
            && status is not UserStatuses.Pending and not UserStatuses.Active and not UserStatuses.Deactivated)
        {
            throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "Status is not valid.");
        }
    }

    private static string NormalizePhone(string? phone)
    {
        return phone?.Trim().Replace(" ", string.Empty) ?? string.Empty;
    }

    private static ApiException NicExists()
    {
        return new ApiException(StatusCodes.Status409Conflict, ErrorCodes.NicExists, "A prosumer with that NIC already exists.");
    }
}

```

WebService/SmartSolar.Api/Services/ReservationActor.cs M3 · 14 lines

```

/*
 * File: ReservationActor.cs
 * Description: The logged-in user making a reservation request, read from the JWT claims.
 * Author: Janukshan S (IT22635266)
 */

using SmartSolar.Api.Common;

namespace SmartSolar.Api.Services;

public record ReservationActor(string UserId, string Role, string? Nic)
{
    public bool IsProsumer => Role == RoleNames.Prosumer;
}

```

WebService/SmartSolar.Api/Services/ReservationQueryService.cs M4 · 197 lines

```

/*
 * File: ReservationQueryService.cs
 * Description: Booking lists and dashboards. Counts are computed from Mongo, never hard-coded.
 * Author: samudith
 * Created: 29/09/2026
 */

using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Repositories;

namespace SmartSolar.Api.Services;

public class ReservationQueryService : IReservationQueryService
{
    public const int PageSize = 20;

    private static readonly HashSet<string> AllowedStatuses = new(StringComparer.Ordinal)
    {
        ReservationStatuses.Pending,
        ReservationStatuses.Approved,
        ReservationStatuses.Cancelled,
        ReservationStatuses.Completed
    };

    private readonly IReservationQueryRepository _reservations;
    private readonly IStationRepository _stations;

    // Injects the read repository and the station repository used for the active-station count.
    public ReservationQueryService(IReservationQueryRepository reservations, IStationRepository stations)
    {
        _reservations = reservations;
        _stations = stations;
    }

    // Lists reservations. A prosumer only ever sees their own NIC, taken from the token.
    public async Task<ReservationPageDto> SearchAsync(
        ReservationActor actor,
        string? status,
        DateTime? from,
        DateTime? to,
        string? stationId,
        string? query,
        string? nic,
        int page,
        CancellationToken cancellationToken = default)
    {
        status = NormalizeStatus(status);
        page = page < 1 ? 1 : page;
        var scopedNic = ScopeNic(actor, nic);

        if (actor.IsProsumer && string.IsNullOrEmpty(scopedNic))
        {
            return EmptyPage(page);
        }
    }
}

```

```

    }

    var (items, total) = await _reservations.SearchAsync(
        scopedNic,
        status,
        stationId,
        AsUtc(from),
        AsUtc(to),
        query,
        (page - 1) * PageSize,
        PageSize,
        cancellationTokens);

    return ToPage(items.Select(ReservationDto.From).ToList(), page, total);
}

// Returns the operator approval queue: future Pending bookings.
public async Task<ReservationPageDto> GetPendingQueueAsync(int page, CancellationToken cancellationToken = default)
{
    page = page < 1 ? 1 : page;
    var now = SriLankaClock.UtcNow();
    var (items, total) = await _reservations.SearchAsync(
        null,
        ReservationStatuses.Pending,
        null,
        now,
        null,
        null,
        (page - 1) * PageSize,
        PageSize,
        cancellationTokens);

    return ToPage(items.Select(ReservationDto.From).ToList(), page, total);
}

// Returns the signed-in prosumer's live dashboard counts and next booking.
public async Task<ProsumerDashboardDto> GetProsumerDashboardAsync(ReservationActor actor, CancellationToken cancellationToken = default)
{
    if (!actor.IsProsumer || string.IsNullOrWhiteSpace(actor.Nic))
    {
        throw new ApiException(StatusCodes.Status403Forbidden, ErrorCodes.Forbidden, "Only a prosumer can open this dashboard.");
    }

    var now = SriLankaClock.UtcNow();
    var nic = actor.Nic;
    var pending = await _reservations.CountForProsumerAsync(nic, ReservationStatuses.Pending, null, false, cancellationTokens);
    var approvedFuture = await _reservations.CountForProsumerAsync(nic, ReservationStatuses.Approved, now, false, cancellationTokens);
    var active = await _reservations.CountForProsumerAsync(nic, null, now, true, cancellationTokens);
    var next = await _reservations.FindNextUpcomingAsync(nic, now, cancellationTokens);

    return new ProsumerDashboardDto
    {
        PendingCount = ToCount(pending),
        ApprovedFutureCount = ToCount(approvedFuture),
        ActiveCount = ToCount(active),
        NextReservation = next is null ? null : ReservationDto.From(next)
    };
}

// Returns today's operational counts for the staff Home page.
public async Task<OperationsDashboardDto> GetOperationsDashboardAsync(CancellationToken cancellationToken = default)
{
    var now = SriLankaClock.UtcNow();
    var (start, end) = SriLankaClock.DayRangeUtc(now);
    var stations = await _stations.GetAllAsync(cancellationTokens);

    var todaysCount = await _reservations.CountScheduledBetweenAsync(start, end, cancellationTokens);
    var pending = await _reservations.CountPendingFutureAsync(now, cancellationTokens);
    var completed = await _reservations.CountCompletedBetweenAsync(start, end, cancellationTokens);
    var todays = await _reservations.ListScheduledBetweenAsync(start, end, PageSize, cancellationTokens);

    return new OperationsDashboardDto
    {
        TodaysBookingCount = ToCount(todaysCount),
        PendingApprovals = ToCount(pending),
        ActiveStations = stations.Count(station => station.Status == StationStatuses.Active),
        CompletedToday = ToCount(completed),
        TodaysBookings = todays.Select(ReservationDto.From).ToList()
    };
}

// Prosumers are locked to the NIC in the token. Staff may filter by the query NIC.
private static string? ScopeNic(ReservationActor actor, string? requestedNic)
{
    if (actor.IsProsumer)
    {
        return actor.Nic;
    }

    return string.IsNullOrWhiteSpace(requestedNic) ? null : requestedNic.Trim();
}

// Rejects a status that is not one of the four contract values.
private static string? NormalizeStatus(string? status)
{
    if (string.IsNullOrWhiteSpace(status))

```

```

    {
        return null;
    }

    var trimmed = status.Trim();
    if (!AllowedStatuses.Contains(trimmed))
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.Validation.Error, "Status must be Pending, Approved, Cancelled or Completed.");
    }

    return trimmed;
}

// Treats an unspecified timestamp as UTC so date filters do not shift with the server locale.
private static DateTime? AsUtc(DateTime? value)
{
    if (value is null)
    {
        return null;
    }

    return value.Value.Kind switch
    {
        DateTimeKind.Utc => value.Value,
        DateTimeKind.Local => value.Value.ToUniversalTime(),
        _ => DateTime.SpecifyKind(value.Value, DateTimeKind.Utc)
    };
}

// Builds an empty page when a prosumer token has no NIC.
private static ReservationPageDto EmptyPage(int page)
{
    return new ReservationPageDto { Items = Array.Empty<ReservationDto>(), Page = page, PageSize = PageSize, TotalCount = 0 };
}

// Wraps a result list in the page envelope.
private static ReservationPageDto ToPage(IReadOnlyList<ReservationDto> items, int page, long total)
{
    return new ReservationPageDto { Items = items, Page = page, PageSize = PageSize, TotalCount = total };
}

// Narrows a Mongo count into the int field on the dashboard DTOs.
private static int ToCount(long value)
{
    return value > int.MaxValue ? int.MaxValue : (int)value;
}
}

```

WebService/SmartSolar.Api/Services/ReservationService.cs M3 - 414 lines

```

/*
 * File: ReservationService.cs
 * Description: Reservation business rules: 7-day window, 12-hour notice, slot availability, status changes and QR approval.
 * Author: Janukshan S (IT22635266)
 */

using System.Buffers.Text;
using System.Security.Cryptography;
using MongoDB.Bson;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Models;
using SmartSolar.Api.Repositories;

namespace SmartSolar.Api.Services;

public class ReservationService : IReservationService
{
    public const int BookingWindowDays = 7;
    public const int ChangeNoticeHours = 12;

    // Sri Lanka has no daylight saving, so local time is always UTC+05:30.
    private static readonly TimeSpan SriLankaOffset = new(5, 30, 0);

    private readonly IReservationRepository _reservations;
    private readonly ISlotRepository _slots;
    private readonly IStationRepository _stations;
    private readonly IUserRepository _users;

    // Injects the repositories used by the reservation rules.
    public ReservationService(
        IReservationRepository reservations,
        ISlotRepository slots,
        IStationRepository stations,
        IUserRepository users)
    {
        _reservations = reservations;
        _slots = slots;
        _stations = stations;
        _users = users;
    }

    // Returns the stations that are taking bookings.

```

```

public async Task<IReadOnlyList<BookingStationDto>> GetBookableStationsAsync(CancellationToken cancellationToken = default)
{
    var stations = await _stations.GetAllAsync(cancellationToken);
    return stations
        .Where(station => station.Status == StationStatuses.Active)
        .OrderBy(station => station.Name)
        .Select(station => new BookingStationDto { Id = station.Id, Name = station.Name })
        .ToList();
}

// Returns free slots of a station inside the booking window, optionally for one Sri Lanka date.
public async Task<IReadOnlyList<AvailableSlotDto>> GetAvailableSlotsAsync(string stationId, DateOnly? date, CancellationToken cancellationToken = default)
{
    var station = await LoadStationAsync(stationId, cancellationToken);
    EnsureStationActive(station);

    var now = DateTime.UtcNow;
    var from = now;
    var to = now.AddDays(BookingWindowDays);

    if (date is not null)
    {
        // The client picks a local date, so convert that day's start and end to UTC.
        var dayStart = date.Value.ToDateTime(TimeOnly.MinValue, DateTimeKind.Utc) - SriLankaOffset;
        from = dayStart > from ? dayStart : from;
        to = dayStart.AddDays(1) < to ? dayStart.AddDays(1) : to;
    }

    if (from >= to)
    {
        return Array.Empty<AvailableSlotDto>();
    }

    var slots = await _slots.GetFreeAsync(station.Id, from, to, cancellationToken);
    return slots.Select(AvailableSlotDto.From).ToList();
}

// Books a free slot for a prosumer, or for the given NIC when staff book on their behalf.
public async Task<ReservationDto> CreateAsync(CreateReservationRequestDto request, ReservationActor actor, CancellationToken cancellationToken = default)
{
    RequireText(request.StationId, "Station is required.");
    RequireText(request.SlotId, "Slot is required.");

    // A prosumer always books for themselves, so their NIC comes from the token and not the body.
    var nic = actor.IsProsumer ? actor.Nic : request.ProsumerNic?.Trim();
    if (actor.IsProsumer && string.IsNullOrEmpty(nic))
    {
        throw new ApiException(StatusCodes.Status403Forbidden, ErrorCodes.Forbidden, "Your account has no NIC linked to it.");
    }

    RequireText(nic, "Prosumer NIC is required when booking on behalf of a prosumer.");

    var now = DateTime.UtcNow;
    var prosumer = await LoadProsumerAsync(nic!, cancellationToken);
    var (station, slot) = await LoadBookableSlotAsync(request.StationId.Trim(), request.SlotId.Trim(), now, cancellationToken);
    EnsureProsumerActive(prosumer);

    await ClaimSlotAsync(slot.Id, cancellationToken);

    var reservation = new Reservation
    {
        Id = ObjectId.GenerateNewId().ToString(),
        ProsumerNic = prosumer.Id,
        StationId = station.Id,
        StationName = station.Name,
        SlotId = slot.Id,
        ScheduledAt = slot.StartTime,
        Status = ReservationStatuses.Pending,
        CreatedAt = now,
        UpdatedAt = now,
        CreatedBy = actor.IsProsumer ? null : actor.UserId
    };

    try
    {
        await _reservations.InsertAsync(reservation, cancellationToken);
    }
    catch
    {
        // The slot was claimed but the booking was not saved, so give the slot back.
        await _slots.ReleaseAsync(slot.Id, CancellationToken.None);
        throw;
    }

    return ReservationDto.From(reservation);
}

// Returns one reservation. Prosumers can only read their own.
public async Task<ReservationDto> GetByIdAsync(string id, ReservationActor actor, CancellationToken cancellationToken = default)
{
    var reservation = await LoadReservationAsync(id, cancellationToken);
    EnsureCanAccess(reservation, actor);
    return ReservationDto.From(reservation);
}

```

```

// Moves a booking to another free slot. Needs 12 hours notice.
public async Task<ReservationDto> UpdateAsync(string id, UpdateReservationRequestDto request, ReservationActor actor, CancellationToken cancellationToken =
default)
{
    RequireText(request.SlotId, "Slot is required.");

    var reservation = await LoadReservationAsync(id, cancellationToken);
    EnsureCanAccess(reservation, actor);
    EnsureChangeable(reservation);

    var now = DateTime.UtcNow;
    EnsureTwelveHoursNotice(reservation, now);

    var stationId = string.IsNullOrEmpty(request.StationId) ? reservation.StationId : request.StationId.Trim();
    var slotId = request.SlotId.Trim();
    if (slotId == reservation.SlotId)
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "Please choose a different slot from the one already booked.");
    }

    var (station, slot) = await LoadBookableSlotAsync(stationId, slotId, now, cancellationToken);
    await ClaimSlotAsync(slot.Id, cancellationToken);

    var oldSlotId = reservation.SlotId;
    var loadedUpdatedAt = reservation.UpdatedAt;

    reservation.StationId = station.Id;
    reservation.StationName = station.Name;
    reservation.SlotId = slot.Id;
    reservation.ScheduledAt = slot.StartTime;
    reservation.UpdatedAt = now;

    // A changed booking needs to be approved again, and the old QR must stop working.
    reservation.Status = ReservationStatuses.Pending;
    reservation.QrToken = null;
    reservation.ApprovedAt = null;
    reservation.ApprovedBy = null;

    var saved = false;
    try
    {
        saved = await _reservations.TryReplaceAsync(reservation, loadedUpdatedAt, cancellationToken);
    }
    finally
    {
        if (!saved)
        {
            await _slots.ReleaseAsync(slot.Id, CancellationToken.None);
        }
    }

    if (!saved)
    {
        throw ChangedBySomeoneElse();
    }

    await _slots.ReleaseAsync(oldSlotId, CancellationToken.None);
    return ReservationDto.From(reservation);
}

// Cancels a booking and frees its slot. Needs 12 hours notice.
public async Task<ReservationDto> CancelAsync(string id, CancelReservationRequestDto? request, ReservationActor actor, CancellationToken cancellationToken =
default)
{
    var reservation = await LoadReservationAsync(id, cancellationToken);
    EnsureCanAccess(reservation, actor);
    EnsureChangeable(reservation);

    var now = DateTime.UtcNow;
    EnsureTwelveHoursNotice(reservation, now);

    var reason = request?.Reason?.Trim();
    var loadedUpdatedAt = reservation.UpdatedAt;
    reservation.Status = ReservationStatuses.Cancelled;
    reservation.QrToken = null;
    reservation.CancelledAt = now;
    reservation.CancelledBy = actor.UserId;
    reservation.CancelReason = string.IsNullOrEmpty(reason) ? null : reason;
    reservation.UpdatedAt = now;

    if (!await _reservations.TryReplaceAsync(reservation, loadedUpdatedAt, cancellationToken))
    {
        throw ChangedBySomeoneElse();
    }

    await _slots.ReleaseAsync(reservation.SlotId, CancellationToken.None);
    return ReservationDto.From(reservation);
}

// Approves a pending booking and gives it a QR token.
public async Task<ReservationDto> ApproveAsync(string id, ReservationActor actor, CancellationToken cancellationToken = default)
{
    var reservation = await LoadReservationAsync(id, cancellationToken);
    if (reservation.Status != ReservationStatuses.Pending)

```

```

    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.InvalidState,
            $"Only pending reservations can be approved. This one is {reservation.Status}.");
    }

    var now = DateTime.UtcNow;
    if (reservation.ScheduledAt <= now)
    {
        throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.InvalidState, "This reservation's slot has already started, so it cannot be approved.");
    }

    var loadedUpdatedAt = reservation.UpdatedAt;
    reservation.Status = ReservationStatuses.Approved;
    reservation.QrToken = NewQrToken();
    reservation.ApprovedAt = now;
    reservation.ApprovedBy = actor.UserId;
    reservation.UpdatedAt = now;

    if (!await _reservations.TryReplaceAsync(reservation, loadedUpdatedAt, cancellationToken))
    {
        throw ChangedBySomeoneElse();
    }

    return ReservationDto.From(reservation);
}

// Loads a reservation or throws 404.
private async Task<Reservation> LoadReservationAsync(string id, CancellationToken cancellationToken)
{
    var reservation = await _reservations.FindByIdAsync(id, cancellationToken);
    if (reservation is null)
    {
        throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Reservation was not found.");
    }

    return reservation;
}

// Loads a prosumer by NIC or throws 404.
private async Task<User> LoadProsumerAsync(string nic, CancellationToken cancellationToken)
{
    var user = await _users.FindByIdAsync(nic, cancellationToken);
    if (user is null || user.Role != RoleNames.Prosumer)
    {
        throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "No prosumer was found with that NIC.");
    }

    return user;
}

// Loads a station or throws 404.
private async Task<Station> LoadStationAsync(string stationId, CancellationToken cancellationToken)
{
    var station = await _stations.FindByIdAsync(stationId, cancellationToken);
    if (station is null)
    {
        throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Station was not found.");
    }

    return station;
}

// Loads a slot and checks it belongs to the chosen station.
private async Task<Slot> LoadSlotAsync(string slotId, string stationId, CancellationToken cancellationToken)
{
    var slot = await _slots.FindByIdAsync(slotId, cancellationToken);
    if (slot is null)
    {
        throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Slot was not found.");
    }

    if (slot.StationId != stationId)
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "The selected slot does not belong to this station.");
    }

    return slot;
}

// Loads the station and slot for a new booking and checks both can be booked.
private async Task<(Station Station, Slot Slot)> LoadBookableSlotAsync(string stationId, string slotId, DateTime now, CancellationToken cancellationToken)
{
    var station = await LoadStationAsync(stationId, cancellationToken);
    var slot = await LoadSlotAsync(slotId, station.Id, cancellationToken);
    EnsureWithinBookingWindow(slot.StartTime, now);
    EnsureStationActive(station);
    return (station, slot);
}

// Takes the slot, or throws SLOT_TAKEN if another booking got it first.
private async Task ClaimSlotAsync(string slotId, CancellationToken cancellationToken)
{
    if (!await _slots.TryClaimAsync(slotId, cancellationToken))

```

```

    {
        throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.SlotTaken, "This slot has already been booked. Please pick another one.");
    }
}

// Completed and cancelled bookings are final.
private static void EnsureChangeable(Reservation reservation)
{
    if (reservation.Status is not (ReservationStatuses.Pending or ReservationStatuses.Approved))
    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.InvalidState,
            $"This reservation is {reservation.Status} and can no longer be changed.");
    }
}

// Updates and cancellations need at least 12 hours before the booked slot starts.
private static void EnsureTwelveHoursNotice(Reservation reservation, DateTime now)
{
    if (reservation.ScheduledAt - now < TimeSpan.FromHours(ChangeNoticeHours))
    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.RuleTwelveHours,
            $"Reservations can only be changed or cancelled at least {ChangeNoticeHours} hours before the slot starts.");
    }
}

// Error for when another request changed the reservation first.
private static ApiException ChangedBySomeoneElse()
{
    return new ApiException(StatusCodes.Status409Conflict, ErrorCodes.InvalidState, "This reservation was just changed by someone else. Please reload and try again.");
}

// Makes a random, URL-safe token for the QR code. It holds no personal data.
private static string NewQrToken()
{
    return Base64Url.EncodeToString(RandomNumberGenerator.GetBytes(32));
}

// Slot must start in the future and no later than 7 days from now.
private static void EnsureWithinBookingWindow(DateTime slotStart, DateTime now)
{
    if (slotStart <= now || slotStart > now.AddDays(BookingWindowDays))
    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.RuleSevenDays,
            $"Reservations can only be made for a future slot within the next {BookingWindowDays} days.");
    }
}

// Only Active prosumers can hold bookings.
private static void EnsureProsumerActive(User prosumer)
{
    if (prosumer.Status != UserStatuses.Active)
    {
        throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.AccountNotActive, "This prosumer account is not active.");
    }
}

// Deactivated stations cannot take new bookings.
private static void EnsureStationActive(Station station)
{
    if (station.Status != StationStatuses.Active)
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "This station is not accepting bookings.");
    }
}

// A prosumer may only touch their own reservations. Staff can see all.
private static void EnsureCanAccess(Reservation reservation, ReservationActor actor)
{
    if (actor.IsProsumer && reservation.ProsumerNic != actor.Nic)
    {
        throw new ApiException(StatusCodes.Status403Forbidden, ErrorCodes.Forbidden, "You can only access your own reservations.");
    }
}

// Throws 400 when a required value is blank.
private static void RequireText(string? value, string message)
{
    if (string.IsNullOrEmpty(value))
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, message);
    }
}
}

```

WebService/SmartSolar.Api/Services/SlotService.cs M2 · 204 lines

/*
 * File: SlotService.cs
 * Description: Business rules for creating, moving, deleting and opening or closing battery slots.

```
* Author: Mohamed Asath (IT22633422)
* Created: 30/09/2026
*/
```

```
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Models;
using SmartSolar.Api.Repositories;
```

```
namespace SmartSolar.Api.Services;
```

```
public class SlotService : ISlotService
{
    private readonly ISlotRepository _slots;
    private readonly IStationRepository _stations;

    // Stores the slot and station repositories. Slot rules need the station schedule and battery count.
    public SlotService(ISlotRepository slots, IStationRepository stations)
    {
        _slots = slots;
        _stations = stations;
    }

    // Returns one station's slots, earliest start first.
    public async Task<IReadOnlyList<Slot>> ListByStationAsync(string stationId, CancellationToken cancellationToken = default)
    {
        await LoadStationAsync(stationId, cancellationToken);
        var slots = await _slots.GetByStationAsync(stationId, cancellationToken);
        return slots.OrderBy(slot => slot.StartTime).ToList();
    }

    // Creates an available slot after the schedule and battery rules pass.
    public async Task<Slot> CreateAsync(string stationId, SaveSlotRequestDto request, CancellationToken cancellationToken = default)
    {
        var start = SriLankaClock.UtcNow(request.StartTime);
        var end = SriLankaClock.UtcNow(request.EndTime);
        var station = await LoadStationAsync(stationId, cancellationToken);
        if (station.Status != StationStatuses.Active)
        {
            throw new ApiException(
                StatusCodes.Status409Conflict,
                ErrorCodes.InvalidState,
                "Slots can only be added to an Active station.");
        }

        var existing = await _slots.GetByStationAsync(station.Id, cancellationToken);
        RequireValidSlot(station, start, end, existing, ignoreSlotId: null);

        var slot = new Slot
        {
            Id = Guid.NewGuid().ToString(),
            StationId = station.Id,
            StartTime = start,
            EndTime = end,
            IsAvailable = true
        };

        await _slots.InsertAsync(slot, cancellationToken);
        return slot;
    }

    // Moves a free slot. A slot with an active reservation cannot be moved.
    public async Task<Slot> UpdateAsync(string slotId, SaveSlotRequestDto request, CancellationToken cancellationToken = default)
    {
        var start = SriLankaClock.UtcNow(request.StartTime);
        var end = SriLankaClock.UtcNow(request.EndTime);
        var slot = await LoadSlotAsync(slotId, cancellationToken);
        await RejectIfBookedAsync(slot.Id, cancellationToken);

        var station = await LoadStationAsync(slot.StationId, cancellationToken);
        var existing = await _slots.GetByStationAsync(slot.StationId, cancellationToken);
        RequireValidSlot(station, start, end, existing, slot.Id);

        slot.StartTime = start;
        slot.EndTime = end;
        await _slots.ReplaceAsync(slot, cancellationToken);
        return slot;
    }

    // Deletes a free slot. A slot with an active reservation cannot be deleted.
    public async Task DeleteAsync(string slotId, CancellationToken cancellationToken = default)
    {
        var slot = await LoadSlotAsync(slotId, cancellationToken);
        await RejectIfBookedAsync(slot.Id, cancellationToken);
        await _slots.DeleteAsync(slot.Id, cancellationToken);
    }

    // Sets the booking flag. A slot with an active reservation cannot be changed either way.
    public async Task<Slot> SetAvailabilityAsync(string slotId, SlotAvailabilityRequestDto request, CancellationToken cancellationToken = default)
    {
        var slot = await LoadSlotAsync(slotId, cancellationToken);
        await RejectIfBookedAsync(slot.Id, cancellationToken);
        slot.IsAvailable = request.IsAvailable;
        await _slots.ReplaceAsync(slot, cancellationToken);
        return slot;
    }
}
```

```

}

// Rejects a slot that is in the past, outside the station schedule, or over the battery count.
private static void RequireValidSlot(
    Station station,
    DateTime start,
    DateTime end,
    IReadOnlyList<Slot> existingSlots,
    string? ignoreSlotId)
{
    if (start >= end)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "Start time must be before end time.");
    }

    if (start <= DateTime.UtcNow)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "Start time must be in the future.");
    }

    var localStart = SriLankaClock.ToLocal(start);
    var localEnd = SriLankaClock.ToLocal(end);
    if (localStart.Date != localEnd.Date)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "A slot must start and end on the same day.");
    }

    var entry = (station.Schedule ?? []).FirstOrDefault(item =>
        string.Equals(item.DayOfWeek, localStart.DayOfWeek.ToString(), StringComparison.OrdinalIgnoreCase));
    if (entry is null)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "The station is closed on that day.");
    }

    var open = TimeOnly.Parse(entry.OpenTime);
    var close = TimeOnly.Parse(entry.CloseTime);
    if (TimeOnly.FromDateTime(localStart) < open || TimeOnly.FromDateTime(localEnd) > close)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "The slot must fall inside the station schedule.");
    }

    var overlapCount = existingSlots.Count(slot =>
        slot.Id != ignoreSlotId
        && slot.StartTime < end
        && start < slot.EndTime);

    if (overlapCount >= station.BatterySlotsTotal)
    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.SlotOverlap,
            "That time overlaps more slots than this station's battery count allows.");
    }
}

// Rejects a change when a Pending or Approved reservation still holds this slot.
private async Task RejectIfBookedAsync(string slotId, CancellationToken cancellationToken)
{
    if (await _slots.HasActiveReservationAsync(slotId, cancellationToken))
    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.SlotTaken,
            "This slot has an active reservation.");
    }
}

// Loads a station or throws 404.
private async Task<Station> LoadStationAsync(string stationId, CancellationToken cancellationToken)
{
    var station = await _stations.FindByIdAsync(stationId, cancellationToken);
    if (station is null)
    {
        throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Station was not found.");
    }

    return station;
}

// Loads a slot or throws 404.
private async Task<Slot> LoadSlotAsync(string slotId, CancellationToken cancellationToken)
{

```

```

        var slot = await _slots.FindByIdAsync(slotId, cancellationToken);
        if (slot is null)
        {
            throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Slot was not found.");
        }

        return slot;
    }
}

```

WebService/SmartSolar.Api/Services/StationService.cs M2 · 258 lines

```

/*
 * File: StationService.cs
 * Description: Business rules for listing, reading, updating, activating and finding nearby stations.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Models;
using SmartSolar.Api.Repositories;

namespace SmartSolar.Api.Services;

public class StationService : IStationService
{
    private readonly IStationRepository _stations;
    private readonly ISlotRepository _slots;

    // Stores the station and slot repositories. Nearby search counts free slots.
    public StationService(IStationRepository stations, ISlotRepository slots)
    {
        _stations = stations;
        _slots = slots;
    }

    // Returns every station document.
    public Task<IReadOnlyList<Station>> ListAsync(CancellationToken cancellationToken = default)
    {
        return _stations.GetAllAsync(cancellationToken);
    }

    // Returns one station or throws 404 when the id is unknown.
    public async Task<Station> GetAsync(string id, CancellationToken cancellationToken = default)
    {
        var station = await _stations.FindByIdAsync(id, cancellationToken);
        if (station is null)
        {
            throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Station was not found.");
        }

        return station;
    }

    // Creates an Active station after the field rules pass.
    public async Task<Station> CreateAsync(SaveStationRequestDto request, CancellationToken cancellationToken = default)
    {
        var schedule = NormalizeSchedule(request.Schedule);
        RequireValid(request.Name, request.Latitude, request.Longitude, request.CapacityKwh, request.BatterySlotsTotal, schedule);

        var station = new Station
        {
            Id = Guid.NewGuid().ToString(),
            Name = request.Name.Trim(),
            Latitude = request.Latitude,
            Longitude = request.Longitude,
            CapacityKwh = request.CapacityKwh,
            BatterySlotsTotal = request.BatterySlotsTotal,
            Schedule = schedule,
            Status = StationStatuses.Active
        };

        await _stations.InsertAsync(station, cancellationToken);
        return station;
    }

    // Updates station fields. Id and status stay unchanged.
    public async Task<Station> UpdateAsync(string id, SaveStationRequestDto request, CancellationToken cancellationToken = default)
    {
        var station = await GetAsync(id, cancellationToken);
        var schedule = NormalizeSchedule(request.Schedule);
        RequireValid(request.Name, request.Latitude, request.Longitude, request.CapacityKwh, request.BatterySlotsTotal, schedule);

        station.Name = request.Name.Trim();
        station.Latitude = request.Latitude;
        station.Longitude = request.Longitude;
        station.CapacityKwh = request.CapacityKwh;
        station.BatterySlotsTotal = request.BatterySlotsTotal;
        station.Schedule = schedule;

        await _stations.ReplaceAsync(station, cancellationToken);
    }
}

```

```

    return station;
}

// Sets a station Inactive, or 409 when a future Pending or Approved reservation exists.
public async Task<Station> DeactivateAsync(string id, CancellationToken cancellationToken = default)
{
    var station = await GetAsync(id, cancellationToken);
    if (await _stations.HasActiveFutureReservationsAsync(id, DateTime.UtcNow, cancellationToken))
    {
        throw new ApiException(
            StatusCodes.Status409Conflict,
            ErrorCodes.StationHasReservations,
            "This station has future reservations and cannot be deactivated.");
    }

    station.Status = StationStatuses.Inactive;
    await _stations.ReplaceAsync(station, cancellationToken);
    return station;
}

// Sets a station Active again.
public async Task<Station> ActivateAsync(string id, CancellationToken cancellationToken = default)
{
    var station = await GetAsync(id, cancellationToken);
    station.Status = StationStatuses.Active;
    await _stations.ReplaceAsync(station, cancellationToken);
    return station;
}

// Returns Active stations within radiusKm, nearest first, with free slots in the next 7 days.
public async Task<IReadOnlyList<NearbyStationDto>> NearbyAsync(
    double? lat,
    double? lng,
    double radiusKm,
    CancellationToken cancellationToken = default)
{
    if (lat is null || lng is null)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "lat and lng are required.");
    }

    RequireCoordinates(lat.Value, lng.Value);
    if (!double.IsFinite(radiusKm) || radiusKm <= 0 || radiusKm > 100)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "Radius must be greater than 0 and at most 100 km.");
    }

    var now = DateTime.UtcNow;
    var stations = await _stations.GetAllAsync(cancellationToken);
    var withinRadius = stations
        .Where(station => station.Status == StationStatuses.Active)
        .Select(station => (Station: station, Distance: DistanceKm(lat.Value, lng.Value, station.Latitude, station.Longitude)))
        .Where(item => item.Distance <= radiusKm)
        .OrderBy(item => item.Distance)
        .ToList();

    var nearby = new List<NearbyStationDto>(withinRadius.Count);
    foreach (var (station, distance) in withinRadius)
    {
        var free = await _slots.GetFreeAsync(station.Id, now, now.AddDays(7), cancellationToken);
        nearby.Add(new NearbyStationDto
        {
            Id = station.Id,
            Name = station.Name,
            Latitude = station.Latitude,
            Longitude = station.Longitude,
            CapacityKwh = station.CapacityKwh,
            BatterySlotsTotal = station.BatterySlotsTotal,
            DistanceKm = Math.Round(distance, 2),
            FreeSlots = free.Count
        });
    }

    return nearby;
}

// Great-circle distance in kilometres between two GPS points.
private static double DistanceKm(double lat1, double lng1, double lat2, double lng2)
{
    const double earthRadiusKm = 6371;
    var phi1 = DegreesToRadians(lat1);
    var phi2 = DegreesToRadians(lat2);
    var deltaPhi = DegreesToRadians(lat2 - lat1);
    var deltaLambda = DegreesToRadians(lng2 - lng1);

    var a = Math.Sin(deltaPhi / 2) * Math.Sin(deltaPhi / 2)
        + Math.Cos(phi1) * Math.Cos(phi2) * Math.Sin(deltaLambda / 2) * Math.Sin(deltaLambda / 2);
    return 2 * earthRadiusKm * Math.Atan2(Math.Sqrt(a), Math.Sqrt(1 - a));
}

// Converts degrees to radians for the haversine formula.

```

```

private static double DegreesToRadians(double degrees)
{
    return degrees * Math.PI / 180;
}

// Rejects a latitude or longitude outside the GPS ranges.
private static void RequireCoordinates(double latitude, double longitude)
{
    if (!double.IsFinite(latitude) || latitude < -90 || latitude > 90
        || !double.IsFinite(longitude) || longitude < -180 || longitude > 180)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "Latitude must be between -90 and 90, and longitude between -180 and 180.");
    }
}

// Rejects a station body that breaks a MEMBER-2 field rule.
private static void RequireValid(
    string? name,
    double latitude,
    double longitude,
    double capacityKwh,
    int batterySlotsTotal,
    IReadOnlyList<StationScheduleEntry> schedule)
{
    if (string.IsNullOrEmpty(name))
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "Name is required.");
    }

    RequireCoordinates(latitude, longitude);

    if (!double.IsFinite(capacityKwh) || capacityKwh <= 0 || batterySlotsTotal < 1)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "Capacity must be greater than 0 and battery slots must be at least 1.");
    }

    foreach (var entry in schedule)
    {
        if (!TimeOnly.TryParse(entry.OpenTime, out var open)
            || !TimeOnly.TryParse(entry.CloseTime, out var close)
            || open >= close)
        {
            throw new ApiException(
                StatusCodes.Status400BadRequest,
                ErrorCodes.ValidationError,
                "Each schedule entry must open before it closes.");
        }
    }
}

// Copies schedule rows so a null list or a later edit of the request cannot change the stored document.
private static List<StationScheduleEntry> NormalizeSchedule(List<StationScheduleEntry>? schedule)
{
    if (schedule is null)
    {
        return new List<StationScheduleEntry>();
    }

    if (schedule.Any(entry => entry is null))
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "Each schedule entry must open before it closes.");
    }

    return schedule.Select(entry => new StationScheduleEntry
    {
        DayOfWeek = entry.DayOfWeek?.Trim() ?? string.Empty,
        OpenTime = entry.OpenTime?.Trim() ?? string.Empty,
        CloseTime = entry.CloseTime?.Trim() ?? string.Empty
    }).ToList();
}

```

WebService/SmartSolar.Api/Services/UserService.cs M1 · 196 lines

```

/*
 * File: UserService.cs
 * Description: Staff account rules for Backoffice. Password hashes never leave this layer.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

```

```

using System.ComponentModel.DataAnnotations;
using MongoDB.Driver;
using SmartSolar.Api.Common;
using SmartSolar.Api.DTOs;
using SmartSolar.Api.Models;
using SmartSolar.Api.Repositories;

```

```

namespace SmartSolar.Api.Services;

public class UserService : IUserService
{
    public const int MinPasswordLength = 8;

    private readonly IUserRepository _users;
    private readonly IPasswordService _passwords;

    // Injects persistence and hashing used by staff account commands.
    public UserService(IUserRepository users, IPasswordService passwords)
    {
        _users = users;
        _passwords = passwords;
    }

    // Returns staff accounts without password hashes.
    public async Task<IReadOnlyList<UserDto>> ListStaffAsync(CancellationToken cancellationToken = default)
    {
        var users = await _users.GetStaffAsync(cancellationToken);
        return users.Select(UserDto.From).ToList();
    }

    // Returns one staff user or 404 when the id does not belong to staff.
    public async Task<UserDto> GetStaffByIdAsync(string id, CancellationToken cancellationToken = default)
    {
        return UserDto.From(await LoadStaffAsync(id, cancellationToken));
    }

    // Creates an Active staff account after checking username uniqueness and role.
    public async Task<UserDto> CreateStaffAsync(CreateStaffRequestDto request, CancellationToken cancellationToken = default)
    {
        ValidateStaffRole(request.Role);
        RequireText(request.Username, "Username is required.");
        RequireText(request.Password, "Password is required.");
        RequirePasswordLength(request.Password);
        RequireText(request.FullName, "Full name is required.");
        RequireEmailFormat(request.Email);

        var username = request.Username.Trim();
        var existing = await _users.FindByUsernameOrNicAsync(username, cancellationToken);
        if (existing is not null)
        {
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.UsernameExists, "That username is already taken.");
        }

        var user = new User
        {
            Id = username,
            Username = username,
            FullName = request.FullName.Trim(),
            Email = request.Email?.Trim() ?? string.Empty,
            Phone = request.Phone?.Trim() ?? string.Empty,
            Role = request.Role.Trim(),
            Status = UserStatuses.Active,
            CreatedAt = DateTime.UtcNow
        };
        user.PasswordHash = _passwords.Hash(user, request.Password);

        try
        {
            await _users.InsertAsync(user, cancellationToken);
        }
        catch (MongoWriteException ex) when (ex.WriteError?.Category == ServerErrorCategory.DuplicateKey)
        {
            throw new ApiException(StatusCodes.Status409Conflict, ErrorCodes.UsernameExists, "That username is already taken.");
        }

        return UserDto.From(user);
    }

    // Updates contact fields and optionally role or password.
    public async Task<UserDto> UpdateStaffAsync(string id, UpdateStaffRequestDto request, string? actorId, CancellationToken cancellationToken = default)
    {
        var user = await LoadStaffAsync(id, cancellationToken);
        ValidateStaffRole(request.Role);
        RequireText(request.FullName, "Full name is required.");
        RequireEmailFormat(request.Email);

        var nextRole = request.Role.Trim();
        if (!string.IsNullOrEmpty(actorId)
            && string.Equals(actorId, user.Id, StringComparison.Ordinal)
            && !string.Equals(user.Role, nextRole, StringComparison.Ordinal))
        {
            throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "You cannot change your own role.");
        }

        user.FullName = request.FullName.Trim();
        user.Email = request.Email?.Trim() ?? string.Empty;
        user.Phone = request.Phone?.Trim() ?? string.Empty;
        user.Role = nextRole;

        if (!string.IsNullOrEmpty(request.Password))
        {

```

```

        RequirePasswordLength(request.Password);
        user.PasswordHash = _passwords.Hash(user, request.Password);
    }

    await _users.ReplaceAsync(user, cancellationToken);
    return UserDto.From(user);
}

// Marks a staff account Deactivated so they can no longer log in.
public async Task<UserDto> DeactivateStaffAsync(string id, string? actorId, CancellationToken cancellationToken = default)
{
    var user = await LoadStaffAsync(id, cancellationToken);
    if (!string.IsNullOrEmpty(actorId) && string.Equals(actorId, user.Id, StringComparison.Ordinal))
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "You cannot deactivate your own account.");
    }

    user.Status = UserStatuses.Deactivated;
    await _users.ReplaceAsync(user, cancellationToken);
    return UserDto.From(user);
}

// Loads a staff user or throws 404 for missing/prosumer ids.
private async Task<User> LoadStaffAsync(string id, CancellationToken cancellationToken)
{
    var user = await _users.FindByIdAsync(id, cancellationToken);
    if (user is null || !IsStaff(user.Role))
    {
        throw new ApiException(StatusCodes.Status404NotFound, ErrorCodes.NotFound, "Staff user was not found.");
    }

    return user;
}

// Rejects roles that this endpoint must not create (prosumers are Member 2/3).
private static void ValidateStaffRole(string? role)
{
    if (string.IsNullOrEmpty(role) || !IsStaff(role))
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            "Role must be Backoffice or GridOperator.");
    }
}

// Returns true when the role is a web staff role.
private static bool IsStaff(string role)
{
    return role is RoleNames.Backoffice or RoleNames.GridOperator;
}

// Throws 400 when a required string is blank.
private static void RequireText(string? value, string message)
{
    if (string.IsNullOrEmpty(value))
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, message);
    }
}

// Rejects a non-blank value that is not an email address. Blank is allowed.
private static void RequireEmailFormat(string? email)
{
    var trimmed = email?.Trim() ?? string.Empty;
    if (trimmed.Length == 0)
    {
        return;
    }

    if (!new EmailAddressAttribute().IsValid(trimmed))
    {
        throw new ApiException(StatusCodes.Status400BadRequest, ErrorCodes.ValidationError, "Email is not a valid address.");
    }
}

// Rejects passwords shorter than the service minimum. The web form is not trusted.
private static void RequirePasswordLength(string password)
{
    if (password.Length < MinPasswordLength)
    {
        throw new ApiException(
            StatusCodes.Status400BadRequest,
            ErrorCodes.ValidationError,
            $"Password must be at least {MinPasswordLength} characters.");
    }
}
}

```

WebService/SmartSolar.Api/appsettings.json M1 · 20 lines

```

{
  "MongoDb": {
    "ConnectionString": "mongodb://localhost:27017",
    "DatabaseName": "SmartSolar"
  }
}

```

```

},
"Jwt": {
  "Issuer": "SmartSolar.Api",
  "Audience": "SmartSolar.Clients",
  "ExpiryMinutes": 480,
  "Key": ""
},
"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning",
    "SmartSolar": "Information"
  }
},
"AllowedHosts": "*"
}

```

WebService/SmartSolar.Api/web.config M1 · 25 lines

```

<?xml version="1.0" encoding="utf-8"?>
<!--
IIS hosting for the FAT service.
App pool must be "No Managed Code". stdout logs go to ./logs for 502.5 diagnosis.
Do not put Jwt:Key or Mongo passwords in this file (it is committed). Set them in
IIS environment variables or a machine-local appsettings.Production.json.
-->
<configuration>
  <location path="." inheritInChildApplications="false">
    <system.webServer>
      <handlers>
        <add name="aspNetCore" path="*" verb="*" modules="AspNetCoreModuleV2" resourceType="Unspecified" />
      </handlers>
      <aspNetCore processPath="dotnet"
        arguments=".\SmartSolar.Api.dll"
        stdoutLogEnabled="true"
        stdoutLogFile=".\logs\stdout"
        hostingModel="OutOfProcess">
        <environmentVariables>
          <environmentVariable name="ASPNETCORE_ENVIRONMENT" value="Production" />
        </environmentVariables>
      </aspNetCore>
    </system.webServer>
  </location>
</configuration>

```

A.2 Web application (WebApp/SmartSolar.Web)

WebApp/SmartSolar.Web/Api/ApiClient.Dashboards.cs M4 · 66 lines

```

/*
 * File: ApiClient.Dashboards.cs
 * Description: Booking list, dashboard, approval and QR calls. Kept separate to avoid merge conflicts.
 * Author: samudith
 * Created: 29/09/2026
 */

using SmartSolar.Web.Models;

namespace SmartSolar.Web.Api;

public partial class ApiClient
{
  // Calls GET /reservations with the combined filters.
  public Task<ReservationPageDto> SearchReservationsAsync(
    string? status,
    DateTime? fromUtc,
    DateTime? toUtc,
    string? stationId,
    string? query,
    string? nic,
    int page,
    CancellationToken cancellationToken = default)
  {
    var url = "reservations?page=" + (page < 1 ? 1 : page);
    url += Append("status", status);
    url += Append("stationId", stationId);
    url += Append("q", query);
    url += Append("nic", nic);
    if (fromUtc is not null)
    {
      url += "&from=" + Uri.EscapeDataString(fromUtc.Value.ToUniversalTime().ToString("o"));
    }

    if (toUtc is not null)
    {
      url += "&to=" + Uri.EscapeDataString(toUtc.Value.ToUniversalTime().ToString("o"));
    }

    return SendAsync<ReservationPageDto>(HttpMethod.Get, url, null, true, cancellationToken);
  }

  // Calls GET /dashboard/operations.
  public Task<OperationsDashboardDto> GetOperationsDashboardAsync(CancellationToken cancellationToken = default)

```

```

{
    return SendAsync<OperationsDashboardDto>(HttpMethod.Get, "dashboard/operations", null, true, cancellationTokens);
}

// Calls POST /reservations/verify-qr.
public Task<ReservationDto> VerifyQrAsync(string qrToken, CancellationToken cancellationToken = default)
{
    return SendAsync<ReservationDto>(HttpMethod.Post, "reservations/verify-qr", new { qrToken }, true, cancellationTokens);
}

// Calls POST /reservations/{id}/complete.
public Task<ReservationDto> CompleteReservationAsync(string id, CancellationToken cancellationToken = default)
{
    return SendAsync<ReservationDto>(HttpMethod.Post, $"reservations/{Uri.EscapeDataString(id)}/complete", null, true, cancellationTokens);
}

// Adds one query parameter when the value is not blank.
private static string Append(string name, string? value)
{
    return string.IsNullOrEmpty(value) ? string.Empty : "&" + name + "=" + Uri.EscapeDataString(value.Trim());
}
}

```

WebApp/SmartSolar.Web/Api/ApiClient.Prosumers.cs M1 · 47 lines

/* File: ApiClient.Prosumers.cs | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
using SmartSolar.Web.Models;

```

namespace SmartSolar.Web.Api;

public partial class ApiClient
{
    public async Task<IReadOnlyList<UserDto>> GetProsumersAsync(string? status, string? query, CancellationToken cancellationToken = default)
    {
        var parameters = new List<string>();
        if (!string.IsNullOrEmpty(status)) parameters.Add("status=" + Uri.EscapeDataString(status));
        if (!string.IsNullOrEmpty(query)) parameters.Add("q=" + Uri.EscapeDataString(query));
        var suffix = parameters.Count == 0 ? string.Empty : "?" + string.Join("&", parameters);
        return await SendAsync<List<UserDto>>(HttpMethod.Get, "prosumers" + suffix, null, true, cancellationToken);
    }

    public async Task<IReadOnlyList<UserDto>> GetPendingProsumersAsync(CancellationToken cancellationToken = default)
    {
        return await SendAsync<List<UserDto>>(HttpMethod.Get, "prosumers/pending", null, true, cancellationToken);
    }

    public Task<UserDto> GetProsumerAsync(string nic, CancellationToken cancellationToken = default) =>
        SendAsync<UserDto>(HttpMethod.Get, $"prosumers/{Uri.EscapeDataString(nic)}", null, true, cancellationToken);

    public Task<UserDto> CreateProsumerAsync(ProsumerForm form, CancellationToken cancellationToken = default) =>
        SendAsync<UserDto>(HttpMethod.Post, "prosumers", new
        {
            form.Nic, form.Username, form.Password, form.FullName, form.Email, form.Phone
        }, true, cancellationToken);

    public Task<UserDto> UpdateProsumerAsync(string nic, ProsumerForm form, CancellationToken cancellationToken = default)
    {
        object body = string.IsNullOrEmpty(form.Password)
            ? new { form.Username, form.FullName, form.Email, form.Phone }
            : new { form.Username, form.Password, form.FullName, form.Email, form.Phone };
        return SendAsync<UserDto>(HttpMethod.Put, $"prosumers/{Uri.EscapeDataString(nic)}", body, true, cancellationToken);
    }

    public Task<UserDto> DeactivateProsumerAsync(string nic, CancellationToken cancellationToken = default) =>
        SendAsync<UserDto>(HttpMethod.Post, $"prosumers/{Uri.EscapeDataString(nic)}/deactivate", null, true, cancellationToken);

    public Task<UserDto> ReactivateProsumerAsync(string nic, CancellationToken cancellationToken = default) =>
        SendAsync<UserDto>(HttpMethod.Post, $"prosumers/{Uri.EscapeDataString(nic)}/reactivate", null, true, cancellationToken);

    public Task<UserDto> ActivateProsumerAsync(string nic, CancellationToken cancellationToken = default) =>
        SendAsync<UserDto>(HttpMethod.Post, $"prosumers/{Uri.EscapeDataString(nic)}/activate", null, true, cancellationToken);
}

```

WebApp/SmartSolar.Web/Api/ApiClient.Reservations.cs M3 · 73 lines

/*
* File: ApiClient.Reservations.cs
* Description: Reservation calls on the shared ApiClient. Kept in its own file to avoid merge conflicts.
* Author: Janukshan S (IT22635266)
*/

```

using SmartSolar.Web.Models;

namespace SmartSolar.Web.Api;

public partial class ApiClient
{
    // Calls GET /reservations/bookable-stations.
    public async Task<IReadOnlyList<BookingStationDto>> GetBookableStationsAsync(CancellationToken cancellationToken = default)
    {

```

```

        return await SendAsync<List<BookingStationDto>>(HttpMethod.Get, "reservations/bookable-stations", null, true, cancellationToken);
    }

    // Calls GET /reservations/available-slots for one station, optionally for one date.
    public async Task<IReadOnlyList<AvailableSlotDto>> GetAvailableSlotsAsync(string stationId, DateOnly? date, CancellationToken cancellationToken = default)
    {
        var url = $"reservations/available-slots?stationId={Uri.EscapeDataString(stationId)}";
        if (date is not null)
        {
            url += "&date={date.Value:yyyy-MM-dd}";
        }

        return await SendAsync<List<AvailableSlotDto>>(HttpMethod.Get, url, null, true, cancellationToken);
    }

    // Calls POST /reservations to book on behalf of a prosumer.
    public Task<ReservationDto> CreateReservationAsync(string prosumerNic, string stationId, string slotId, CancellationToken cancellationToken = default)
    {
        return SendAsync<ReservationDto>(HttpMethod.Post, "reservations", new { prosumerNic, stationId, slotId }, true, cancellationToken);
    }

    // Calls GET /reservations/{id}.
    public Task<ReservationDto> GetReservationAsync(string id, CancellationToken cancellationToken = default)
    {
        return SendAsync<ReservationDto>(HttpMethod.Get, $"reservations/{Uri.EscapeDataString(id)}", null, true, cancellationToken);
    }

    // Calls GET /reservations with the list filters. Reads the paged envelope from Member 4's endpoint.
    public async Task<IReadOnlyList<ReservationDto>> SearchReservationsAsync(string? status, string? nic, string? q, DateTime? from, CancellationToken cancellationToken = default)
    {
        var query = new List<string> { "page=1" };
        if (!string.IsNullOrEmpty(status)) query.Add($"status={Uri.EscapeDataString(status)}");
        if (!string.IsNullOrEmpty(nic)) query.Add($"nic={Uri.EscapeDataString(nic.Trim())}");
        if (!string.IsNullOrEmpty(q)) query.Add($"q={Uri.EscapeDataString(q.Trim())}");
        if (from is not null) query.Add($"from={Uri.EscapeDataString(from.Value.ToUniversalTime().ToString("o"))}");

        var page = await SendAsync<ReservationPageDto>(HttpMethod.Get, "reservations?" + string.Join("&", query), null, true, cancellationToken);
        return page.Items;
    }

    // Calls PUT /reservations/{id} to move a booking to another slot.
    public Task<ReservationDto> UpdateReservationAsync(string id, string stationId, string slotId, CancellationToken cancellationToken = default)
    {
        return SendAsync<ReservationDto>(HttpMethod.Put, $"reservations/{Uri.EscapeDataString(id)}", new { stationId, slotId }, true, cancellationToken);
    }

    // Calls POST /reservations/{id}/cancel.
    public Task<ReservationDto> CancelReservationAsync(string id, string? reason, CancellationToken cancellationToken = default)
    {
        return SendAsync<ReservationDto>(HttpMethod.Post, $"reservations/{Uri.EscapeDataString(id)}/cancel", new { reason }, true, cancellationToken);
    }

    // Calls POST /reservations/{id}/approve.
    public Task<ReservationDto> ApproveReservationAsync(string id, CancellationToken cancellationToken = default)
    {
        return SendAsync<ReservationDto>(HttpMethod.Post, $"reservations/{Uri.EscapeDataString(id)}/approve", null, true, cancellationToken);
    }
}

```

WebApp/SmartSolar.Web/Api/ApiClient.Stations.cs M2 · 80 lines

```

/*
 * File: ApiClient.Stations.cs
 * Description: Station calls for the MVC pages. The JWT is attached; rules stay on the API.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using SmartSolar.Web.Models;

namespace SmartSolar.Web.Api;

public partial class ApiClient
{
    // Calls GET /stations.
    public async Task<IReadOnlyList<StationDto>> GetStationsAsync(CancellationToken cancellationToken = default)
    {
        return await SendAsync<List<StationDto>>(HttpMethod.Get, "stations", null, true, cancellationToken);
    }

    // Calls GET /stations/{id}.
    public Task<StationDto> GetStationAsync(string id, CancellationToken cancellationToken = default) =>
        SendAsync<StationDto>(HttpMethod.Get, $"stations/{Uri.EscapeDataString(id)}", null, true, cancellationToken);

    // Calls POST /stations/{id}/deactivate.
    public Task<StationDto> DeactivateStationAsync(string id, CancellationToken cancellationToken = default) =>
        SendAsync<StationDto>(HttpMethod.Post, $"stations/{Uri.EscapeDataString(id)}/deactivate", null, true, cancellationToken);

    // Calls POST /stations/{id}/activate.
    public Task<StationDto> ActivateStationAsync(string id, CancellationToken cancellationToken = default) =>
        SendAsync<StationDto>(HttpMethod.Post, $"stations/{Uri.EscapeDataString(id)}/activate", null, true, cancellationToken);
}

```

```

// Calls POST /stations. Closed days are omitted from the schedule.
public Task<StationDto> CreateStationAsync(StationForm form, CancellationToken cancellationToken = default) =>
    SendAsync<StationDto>(HttpMethod.Post, "stations", StationBody(form), true, cancellationToken);

// Calls PUT /stations/{id}. Closed days are omitted from the schedule.
public Task<StationDto> UpdateStationAsync(string id, StationForm form, CancellationToken cancellationToken = default) =>
    SendAsync<StationDto>(HttpMethod.Put, $"stations/{Uri.EscapeDataString(id)}", StationBody(form), true, cancellationToken);

// Calls GET /stations/{id}/slots.
public async Task<IReadOnlyList<SlotDto>> GetSlotsAsync(string stationId, CancellationToken cancellationToken = default)
{
    return await SendAsync<List<SlotDto>>(HttpMethod.Get, $"stations/{Uri.EscapeDataString(stationId)}/slots", null, true, cancellationToken);
}

// Calls POST /stations/{id}/slots. Local form times are sent as +05:30.
public Task<SlotDto> CreateSlotAsync(string stationId, SlotForm form, CancellationToken cancellationToken = default)
{
    var offset = TimeSpan.FromHours(5.5);
    return SendAsync<SlotDto>({
        HttpMethod.Post,
        $"stations/{Uri.EscapeDataString(stationId)}/slots",
        new
        {
            StartTime = new DateTimeOffset(form.Start!.Value, offset),
            EndTime = new DateTimeOffset(form.End!.Value, offset)
        },
        true,
        cancellationToken);
}

// Calls DELETE /slots/{id}. A successful delete has an empty body.
public Task DeleteSlotAsync(string id, CancellationToken cancellationToken = default) =>
    SendAsync<object>(HttpMethod.Delete, $"slots/{Uri.EscapeDataString(id)}", null, true, cancellationToken, allowEmptyBody: true);

// Calls PUT /slots/{id}/availability.
public Task<SlotDto> SetSlotAvailabilityAsync(string id, bool isAvailable, CancellationToken cancellationToken = default) =>
    SendAsync<SlotDto>(HttpMethod.Put, $"slots/{Uri.EscapeDataString(id)}/availability", new { isAvailable }, true, cancellationToken);

// Builds the station JSON. Only days marked open are included.
private static object StationBody(StationForm form) => new
{
    form.Name,
    form.Latitude,
    form.Longitude,
    form.CapacityKwh,
    form.BatterySlotsTotal,
    Schedule = form.Days.Where(day => day.Open).Select(day => new { day.DayOfWeek, day.OpenTime, day.CloseTime })
};
}

```

WebApp/SmartSolar.Web/Api/ApiClient.cs M1 · 175 lines

```

/*
 * File: ApiClient.cs
 * Description: Shared HTTP client for every MVC page. Attach the session JWT; do not encode rules here.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using System.Net.Http.Headers;
using System.Text.Json;
using SmartSolar.Web.Models;
using SmartSolar.Web.Services;

namespace SmartSolar.Web.Api;

public partial class ApiClient
{
    private static readonly JsonSerializerOptions JsonOptions = new()
    {
        PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
        PropertyNameCaseInsensitive = true
    };

    private readonly HttpClient _http;
    private readonly IWebSession _session;

    // Injects the typed HttpClient (base URL from config) and the session that holds the JWT.
    public ApiClient(HttpClient http, IWebSession session)
    {
        _http = http;
        _session = session;
    }

    // Calls POST /auth/login. No bearer token is sent.
    public Task<LoginResponseDto> LoginAsync(string username, string password, CancellationToken cancellationToken = default)
    {
        return SendAsync<LoginResponseDto>({
            HttpMethod.Post,
            "auth/login",
            new { username, password },
            withBearer: false,
            cancellationToken);
    }

```

```

}

// Calls GET /users (Backoffice staff list).
public async Task<IReadOnlyList<UserDto>> GetStaffAsync(CancellationToken cancellationToken = default)
{
    return await SendAsync<List<UserDto>>(HttpMethod.Get, "users", null, true, cancellationToken);
}

// Calls GET /users/{id}.
public Task<UserDto> GetStaffByIdAsync(string id, CancellationToken cancellationToken = default)
{
    return SendAsync<UserDto>(HttpMethod.Get, $"users/{Uri.EscapeDataString(id)}", null, true, cancellationToken);
}

// Calls POST /users to create a staff account.
public Task<UserDto> CreateStaffAsync(StaffUserForm form, CancellationToken cancellationToken = default)
{
    return SendAsync<UserDto>(
        HttpMethod.Post,
        "users",
        new
        {
            form.Username,
            form.Password,
            form.FullName,
            form.Email,
            form.Phone,
            form.Role
        },
        true,
        cancellationToken);
}

// Calls PUT /users/{id}. Password is omitted when left blank so the hash is unchanged.
public Task<UserDto> UpdateStaffAsync(string id, StaffUserForm form, CancellationToken cancellationToken = default)
{
    object body = string.IsNullOrEmpty(form.Password)
        ? new { form.FullName, form.Email, form.Phone, form.Role }
        : new { form.FullName, form.Email, form.Phone, form.Role, form.Password };

    return SendAsync<UserDto>(HttpMethod.Put, $"users/{Uri.EscapeDataString(id)}", body, true, cancellationToken);
}

// Calls POST /users/{id}/deactivate.
public Task<UserDto> DeactivateStaffAsync(string id, CancellationToken cancellationToken = default)
{
    return SendAsync<UserDto>(HttpMethod.Post, $"users/{Uri.EscapeDataString(id)}/deactivate", null, true, cancellationToken);
}

// Sends JSON to the API and maps non-2xx bodies onto ApiClientException.
private async Task<T> SendAsync<T>(
    HttpMethod method,
    string relativeUrl,
    object? body,
    bool withBearer,
    CancellationToken cancellationToken,
    bool allowEmptyBody = false)
{
    using var request = new HttpRequestMessage(method, relativeUrl);

    if (withBearer)
    {
        {
            var token = _session.GetJwt();
            if (string.IsNullOrEmpty(token))
            {
                throw new ApiClientException(StatusCodes.Status401Unauthorized, "UNAUTHORIZED", "You are not signed in.");
            }
        }

        request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);
    }

    if (body is not null)
    {
        request.Content = JsonContent.Create(body, options: JsonOptions);
    }

    HttpResponseMessage response;
    try
    {
        response = await _http.SendAsync(request, cancellationToken);
    }
    catch (HttpRequestException)
    {
        throw new ApiClientException(
            StatusCodes.Status503ServiceUnavailable,
            "API_UNAVAILABLE",
            "Cannot reach the API. Start SmartSolar.Api on port 5080 and try again.");
    }

    using (response)
    {
        var json = await response.Content.ReadAsStringAsync(cancellationToken);
        if (!response.IsSuccessStatusCode)
        {
            var error = TryReadError(json);
            throw new ApiClientException(

```

```

        (int)response.StatusCode,
        error?.Code ?? "ERROR",
        string.IsNullOrEmpty(error?.Message) ? "The API request failed." : error.Message);
    }

    if (allowEmptyBody && string.IsNullOrEmpty(json))
    {
        return default!;
    }

    var result = JsonSerializer.Deserialize<T>(json, JsonOptions);
    if (result is null)
    {
        throw new ApiClientException(StatusCodes.Status502BadGateway, "INVALID_RESPONSE", "The API returned an empty body.");
    }

    return result;
}

// Parses the contract error JSON when present; returns null if the body is not JSON.
private static ApiErrorDto? TryReadError(string json)
{
    if (string.IsNullOrEmpty(json))
    {
        return null;
    }

    try
    {
        return JsonSerializer.Deserialize<ApiErrorDto>(json, JsonOptions);
    }
    catch (JsonException)
    {
        return null;
    }
}
}

```

WebApp/SmartSolar.Web/Api/ApiClientException.cs M1 · 26 lines

```

/*
 * File: ApiClientException.cs
 * Description: Wraps a non-2xx API response so controllers can display the server message.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

namespace SmartSolar.Web.Api;

public class ApiClientException : Exception
{
    public int StatusCode { get; }

    public string Code { get; }

    public string ApiMessage { get; }

    // Stores the HTTP status and contract error fields from the API.
    public ApiClientException(int statusCode, string code, string message)
        : base(message)
    {
        StatusCode = statusCode;
        Code = code;
        ApiMessage = message;
    }
}

```

WebApp/SmartSolar.Web/Common/RoleNames.cs M1 · 15 lines

```

/*
 * File: RoleNames.cs
 * Description: Role strings that match the API contract. Used only for navbar and redirects.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

namespace SmartSolar.Web.Common;

public static class RoleNames
{
    public const string Backoffice = "Backoffice";
    public const string GridOperator = "GridOperator";
    public const string Prosumer = "Prosumer";
}

```

WebApp/SmartSolar.Web/Common/SriLankaTime.cs M3 · 24 lines

```

/*
 * File: SriLankaTime.cs

```

```

* Description: Shows the API's UTC times in Sri Lanka time (UTC+05:30). Display only.
* Author: Janukshan S (IT22635266)
*/

```

```
namespace SmartSolar.Web.Common;
```

```
public static class SriLankaTime
{
    private static readonly TimeSpan Offset = new(5, 30, 0);

    // Converts a UTC time from the API to Sri Lanka local time.
    public static DateTime ToSriLanka(this DateTime utc)
    {
        return DateTime.SpecifyKind(utc.ToUniversalTime() + Offset, DateTimeKind.Unspecified);
    }

    // Formats a UTC time as e.g. "Mon 28 Sep 2026, 08:00".
    public static string ToSriLankaText(this DateTime utc)
    {
        return utc.ToSriLanka().ToString("ddd dd MMM yyyy, HH:mm");
    }
}

```

WebApp/SmartSolar.Web/Controllers/AccountController.cs M1 · 126 lines

```

/*
* File: AccountController.cs
* Description: Login, logout and 403 pages. Credentials are checked only by the API.
* Author: DE SILVA R K D H (IT22001252)
* Created: 20/09/2026
*/

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;
using SmartSolar.Web.Services;

namespace SmartSolar.Web.Controllers;

public class AccountController : Controller
{
    private readonly ApiClient _api;
    private readonly IWebSession _session;

    // Injects the shared API client and the cookie/session helper.
    public AccountController(ApiClient api, IWebSession session)
    {
        _api = api;
        _session = session;
    }

    // Shows the login form. Already-signed-in users go to their role home.
    [HttpGet]
    [AllowAnonymous]
    public IActionResult Login(bool timedOut = false, string? returnUrl = null)
    {
        if (User.Identity?.IsAuthenticated == true)
        {
            return RedirectToRoleHome();
        }

        if (timedOut)
        {
            ModelState.AddModelError(string.Empty, "Your session expired. Please sign in again.");
        }

        ViewBag.ReturnUrl = returnUrl;
        return View(new LoginViewModel());
    }

    // Posts username and password to POST /api/auth/login, then stores the JWT in session.
    [HttpPost]
    [AllowAnonymous]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> Login(LoginViewModel model, string? returnUrl = null, CancellationToken cancellationToken = default)
    {
        if (!ModelState.IsValid)
        {
            return View(model);
        }

        try
        {
            var result = await _api.LoginAsync(model.Username.Trim(), model.Password, cancellationToken);
            await _session.SignInAsync(result);
            return RedirectAfterLogin(result.User.Role, returnUrl);
        }
        catch (ApiClientException ex)
        {
            ModelState.AddModelError(string.Empty, ex.ApiMessage);
            return View(model);
        }
    }
}

```

```

}

// Clears the JWT session and the auth cookie.
[Authorize]
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Logout()
{
    await _session.SignOutAsync();
    return RedirectToAction("Login");
}

// Shown when the signed-in role cannot open the requested page.
[Authorize]
[HttpGet]
public IActionResult AccessDenied()
{
    return View();
}

// Honours a local returnUrl, otherwise sends the user to the role dashboard.
private IActionResult RedirectAfterLogin(string role, string? returnUrl)
{
    if (!string.IsNullOrEmpty(returnUrl) && Url.IsLocalUrl(returnUrl))
    {
        return Redirect(returnUrl);
    }

    if (string.Equals(role, RoleNames.Backoffice, StringComparison.Ordinal))
    {
        return RedirectToAction("Index", "Admin");
    }

    if (string.Equals(role, RoleNames.GridOperator, StringComparison.Ordinal))
    {
        return RedirectToAction("Index", "Operations");
    }

    return RedirectToAction(nameof(AccessDenied));
}

// Sends Backoffice to Admin and GridOperator to Operations when already signed in.
private IActionResult RedirectToRoleHome()
{
    if (User.IsInRole(RoleNames.Backoffice))
    {
        return RedirectToAction("Index", "Admin");
    }

    if (User.IsInRole(RoleNames.GridOperator))
    {
        return RedirectToAction("Index", "Operations");
    }

    return RedirectToAction(nameof(AccessDenied));
}
}

```

WebApp/SmartSolar.Web/Controllers/AdminController.cs M1 · 59 lines

```

/*
 * File: AdminController.cs
 * Description: Backoffice dashboard. Live operations counts plus account desk links.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

[Authorize(Roles = RoleNames.Backoffice)]
public class AdminController : Controller
{
    private readonly ApiClient _api;

    // Injects the API client used for operations and account counts.
    public AdminController(ApiClient api)
    {
        _api = api;
    }

    // Loads operations figures and account counts for the Backoffice home.
    [HttpGet]
    public async Task<IActionResult> Index(CancellationToken cancellationToken)
    {
        var model = new AdminDashboardViewModel();
        try
        {

```

```

        model.Operations = await _api.GetOperationsDashboardAsync(cancellationToken);
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        model.LoadError = ex.ApiMessage;
        model.Operations = new OperationsDashboardDto();
    }

    try
    {
        var staff = await _api.GetStaffAsync(cancellationToken);
        var prosumers = await _api.GetProsumersAsync(null, null, cancellationToken);
        var pending = await _api.GetPendingProsumersAsync(cancellationToken);
        model.StaffCount = staff.Count;
        model.ProsumerCount = prosumers.Count;
        model.ActiveProsumerCount = prosumers.Count(user => user.Status == "Active");
        model.PendingCount = pending.Count;
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        model.LoadError = string.IsNullOrEmpty(model.LoadError) ? ex.ApiMessage : model.LoadError + " " + ex.ApiMessage;
    }

    return View(model);
}
}

```

WebApp/SmartSolar.Web/Controllers/BookingsController.cs M4 · 162 lines

```

/*
 * File: BookingsController.cs
 * Description: Booking monitor, approval and QR verification pages. Rules stay in the API.
 * Author: samudith
 * Created: 29/09/2026
 */

```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

```

```

namespace SmartSolar.Web.Controllers;

```

```

[Authorize(Roles = RoleNames.Backoffice + " , " + RoleNames.GridOperator)]

```

```

public class BookingsController : Controller
{
    private static readonly TimeSpan SriLankaOffset = new(5, 30, 0);
    private readonly ApiClient _api;

```

```

    // Injects the shared API client.
    public BookingsController(ApiClient api)
    {
        _api = api;
    }

```

```

    // Shows the filterable booking table. Tabs add upcoming, pending or history bounds.

```

```

    [HttpGet]
    public async Task<IActionResult> Index(
        string? tab,
        string? status,
        string? stationId,
        string? nic,
        string? q,
        DateOnly? from,
        DateOnly? to,
        int page = 1,
        CancellationToken cancellationToken = default)
    {
        tab = string.IsNullOrEmpty(tab) ? "all" : tab;
        if (tab == "pending")
        {
            status = "Pending";
        }
    }

```

```

        var fromUtc = StartOfSriLankaDay(from);
        var toUtc = EndOfSriLankaDay(to);
        if (tab == "upcoming" && fromUtc is null)
        {
            fromUtc = DateTime.UtcNow;
        }
    }

```

```

        if (tab == "history" && toUtc is null)
        {
            toUtc = DateTime.UtcNow;
        }
    }

```

```

    var model = new BookingMonitorViewModel
    {
        Tab = tab,
        Status = status,
        StationId = stationId,
        Nic = nic,
        Query = q,
        From = from,
    }

```

```

        To = to,
        Page = page < 1 ? 1 : page
    };

    try
    {
        model.Stations = await _api.GetStationsAsync(cancellationTokens);
        model.Result = await _api.SearchReservationsAsync(status, fromUtc, toUtc, stationId, q, nic, model.Page, cancellationTokens);
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
    }

    return View(model);
}

// Approves a pending booking through the API, then returns to the monitor.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Approve(string id, string? tab, CancellationToken cancellationTokens)
{
    try
    {
        await _api.ApproveReservationAsync(id, cancellationTokens);
        TempData["Success"] = "Booking approved. The prosumer can show the QR code.";
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
    }

    return RedirectToAction(nameof(Index), new { tab = string.IsNullOrEmpty(tab) ? "pending" : tab });
}

// Checks a pasted QR token and shows the booking the operator is about to finish.
[HttpPost]
[Authorize(Roles = RoleNames.GridOperator)]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Verify(string qrToken, CancellationToken cancellationTokens)
{
    try
    {
        var reservation = await _api.VerifyQrAsync(qrToken, cancellationTokens);
        return View("Verified", reservation);
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }
}

// Marks the verified booking completed.
[HttpPost]
[Authorize(Roles = RoleNames.GridOperator)]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Complete(string id, CancellationToken cancellationTokens)
{
    try
    {
        var reservation = await _api.CompleteReservationAsync(id, cancellationTokens);
        TempData["Success"] = "Booking marked completed.";
        return View("Verified", reservation);
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }
}

// Converts a Sri Lanka calendar date to the UTC instant at local midnight.
private static DateTime? StartOfSriLankaDay(DateOnly? date)
{
    if (date is null)
    {
        return null;
    }

    var local = date.Value.ToDateTime(TimeOnly.MinValue);
    return DateTime.SpecifyKind(local - SriLankaOffset, DateTimeKind.Utc);
}

// Converts a Sri Lanka calendar date to the UTC instant at local end of day.
private static DateTime? EndOfSriLankaDay(DateOnly? date)
{
    if (date is null)
    {
        return null;
    }

    var local = date.Value.ToDateTime(new TimeOnly(23, 59, 59));
    return DateTime.SpecifyKind(local - SriLankaOffset, DateTimeKind.Utc);
}
}

```

WebApp/SmartSolar.Web/Controllers/HomeController.cs M1 · 53 lines

```
/*
 * File: HomeController.cs
 * Description: Public landing page and generic error page. Logged-in staff are sent to their role home.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using System.Diagnostics;
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

public class HomeController : Controller
{
    // Shows the public landing page, or redirects signed-in staff to their dashboard.
    [AllowAnonymous]
    public IActionResult Index()
    {
        if (User.Identity?.IsAuthenticated == true)
        {
            return RedirectToRoleHome();
        }

        return View();
    }

    // Shows the unhandled-error page.
    [AllowAnonymous]
    [ResponseCache(Duration = 0, Location = ResponseCacheLocation.None, NoStore = true)]
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId = Activity.Current?.Id ?? HttpContext.TraceIdentifier });
    }

    // Sends Backoffice to Admin and GridOperator to Operations.
    private IActionResult RedirectToRoleHome()
    {
        if (User.IsInRole(RoleNames.Backoffice))
        {
            return RedirectToAction("Index", "Admin");
        }

        if (User.IsInRole(RoleNames.GridOperator))
        {
            return RedirectToAction("Index", "Operations");
        }

        return RedirectToAction("AccessDenied", "Account");
    }
}
```

WebApp/SmartSolar.Web/Controllers/OperationsController.cs M1 · 41 lines

```
/*
 * File: OperationsController.cs
 * Description: GridOperator home. Member 2 will hang node and reservation pages off this layout.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

[Authorize(Roles = RoleNames.GridOperator)]
public class OperationsController : Controller
{
    private readonly ApiClient _api;

    // Injects the shared API client used for the live Home figures.
    public OperationsController(ApiClient api)
    {
        _api = api;
    }

    // Shows the Grid Operator home with live counts. No operational rules are evaluated here.
    [HttpGet]
    public async Task<IActionResult> Index(CancellationToken cancellationToken)
    {
        try
        {
            return View(await _api.GetOperationsDashboardAsync(cancellationToken));
        }
        catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    }
}
```

```

    {
        TempData["Error"] = ex.ApiMessage;
        return View(new OperationsDashboardDto());
    }
}

```

WebApp/SmartSolar.Web/Controllers/PendingActivationsController.cs M1 · 37 lines

```

/* File: PendingActivationsController.cs | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

[Authorize(Roles = RoleNames.Backoffice)]
public class PendingActivationsController : Controller
{
    private readonly ApiClient _api;
    public PendingActivationsController(ApiClient api) { _api = api; }

    [HttpGet]
    public async Task<IActionResult> Index(CancellationToken cancellationToken)
    {
        try { return View(await _api.GetPendingProsumersAsync(cancellationToken)); }
        catch (ApiClientException ex) { TempData["Error"] = ex.ApiMessage; return View(Array.Empty<UserDto>()); }
    }

    [HttpPost, ValidateAntiForgeryToken]
    public Task<IActionResult> Activate(string id, CancellationToken cancellationToken) =>
        Run(() => _api.ActivateProsumerAsync(id, cancellationToken), $"Activated {id}.");

    [HttpPost, ValidateAntiForgeryToken]
    public Task<IActionResult> Reject(string id, CancellationToken cancellationToken) =>
        Run(() => _api.DeactivateProsumerAsync(id, cancellationToken), $"Rejected {id}.");

    private async Task<IActionResult> Run(Func<Task<UserDto>> command, string success)
    {
        try { await command(); TempData["Success"] = success; }
        catch (ApiClientException ex) { TempData["Error"] = ex.ApiMessage; }
        return RedirectToAction(nameof(Index));
    }
}

```

WebApp/SmartSolar.Web/Controllers/ProsumersController.cs M1 · 80 lines

```

/*
 * File: ProsumersController.cs
 * Description: Staff MVC pages for prosumer search and Backoffice account management.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

[Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
public class ProsumersController : Controller
{
    private readonly ApiClient _api;
    public ProsumersController(ApiClient api) { _api = api; }

    [HttpGet]
    public async Task<IActionResult> Index(string? status, string? q, CancellationToken cancellationToken)
    {
        ViewBag.Status = status;
        ViewBag.Query = q;
        try { return View(await _api.GetProsumersAsync(status, q, cancellationToken)); }
        catch (ApiClientException ex) { TempData["Error"] = ex.ApiMessage; return View(Array.Empty<UserDto>()); }
    }

    [HttpGet, Authorize(Roles = RoleNames.Backoffice)]
    public IActionResult Create() => View(new ProsumerForm());

    [HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
    public async Task<IActionResult> Create(ProsumerForm form, CancellationToken cancellationToken)
    {
        if (string.IsNullOrEmpty(form.Password)) ModelState.AddModelError(nameof(form.Password), "Password is required.");
        if (!ModelState.IsValid) return View(form);
        try {
            await _api.CreateProsumerAsync(form, cancellationToken);
            TempData["Success"] = $"Created active prosumer {form.Nic}.";
            return RedirectToAction(nameof(Index));
        } catch (ApiClientException ex) { ModelState.AddModelError(string.Empty, ex.ApiMessage); return View(form); }
    }
}

```

```

[HttpGet, Authorize(Roles = RoleNames.Backoffice)]
public async Task<IActionResult> Edit(string id, CancellationToken cancellationToken)
{
    try {
        var user = await _api.GetProsumerAsync(id, cancellationToken);
        return View(new ProsumerForm { Nic = user.Nic ?? user.Id, Username = user.Username, FullName = user.FullName, Email = user.Email, Phone = user.Phone,
IsEdit = true });
    } catch (ApiClientException ex) { TempData["Error"] = ex.ApiMessage; return RedirectToAction(nameof(Index)); }
}

[HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
public async Task<IActionResult> Edit(string id, ProsumerForm form, CancellationToken cancellationToken)
{
    form.Nic = id; form.IsEdit = true; ModelState.Remove(nameof(ProsumerForm.Nic));
    if (!ModelState.IsValid) return View(form);
    try {
        await _api.UpdateProsumerAsync(id, form, cancellationToken);
        TempData["Success"] = $"Updated {id}.";
        return RedirectToAction(nameof(Index));
    } catch (ApiClientException ex) { ModelState.AddModelError(string.Empty, ex.ApiMessage); return View(form); }
}

[HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
public Task<IActionResult> Deactivate(string id, CancellationToken cancellationToken) =>
    RunStatusAction(() => _api.DeactivateProsumerAsync(id, cancellationToken), $"Deactivated {id}.");

[HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
public Task<IActionResult> Reactivate(string id, CancellationToken cancellationToken) =>
    RunStatusAction(() => _api.ReactivateProsumerAsync(id, cancellationToken), $"Reactivated {id}.");

private async Task<IActionResult> RunStatusAction(Func<Task<UserDto>> command, string success)
{
    try { await command(); TempData["Success"] = success; }
    catch (ApiClientException ex) { TempData["Error"] = ex.ApiMessage; }
    return RedirectToAction(nameof(Index));
}
}

```

WebApp/SmartSolar.Web/Controllers/ReservationsController.cs M3 · 257 lines

```

/*
 * File: ReservationsController.cs
 * Description: Reservation pages for Backoffice and Grid Operators. Calls the API and shows its answer; no booking rules here.
 * Author: Janukshan S (IT22635266)
 */

```

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

```

```

namespace SmartSolar.Web.Controllers;

```

```

[Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]

```

```

public class ReservationsController : Controller
{
    private readonly ApiClient _api;

    // Injects the shared API client.
    public ReservationsController(ApiClient api)
    {
        _api = api;
    }

```

```

    // Lists bookings. Shows upcoming ones unless "include past" is ticked.
    [HttpGet]
    public async Task<IActionResult> Index(ReservationIndexViewModel model, CancellationToken cancellationToken)
    {
        try
        {
            DateTime? from = model.IncludePast ? null : DateTime.UtcNow;
            model.Items = await _api.SearchReservationsAsync(model.Status, model.Nic, model.Q, from, cancellationToken);
        }
        catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
        {
            TempData["Error"] = ex.ApiMessage;
        }

        return View(model);
    }

```

```

    // Shows the booking form. Once a station is picked, it also lists that station's free slots.
    [HttpGet]
    public async Task<IActionResult> Create(ReservationCreateViewModel model, CancellationToken cancellationToken)
    {
        ModelState.Clear();
        await LoadCreateChoicesAsync(model, cancellationToken);
        return View(model);
    }

```

```

    // Books the chosen slot for the prosumer, then shows the summary page.
    [HttpPost]

```

```

[ActionName("Create")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CreateConfirmed(ReservationCreateViewModel model, CancellationToken cancellationToken)
{
    if (string.IsNullOrEmpty(model.SlotId))
    {
        ModelState.AddModelError(string.Empty, "Please choose a slot.");
    }
    else
    {
        try
        {
            var created = await _api.CreateReservationAsync(
                model.ProsumerNic ?? string.Empty,
                model.StationId ?? string.Empty,
                model.SlotId,
                cancellationToken);

            return RedirectToAction(nameof(Summary), new { id = created.Id, done = "created" });
        }
        catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
        {
            ModelState.AddModelError(string.Empty, ex.ApiMessage);
        }
    }

    await LoadCreateChoicesAsync(model, cancellationToken);
    return View(model);
}

// Shows the current booking and a picker for the new slot.
[HttpGet]
public async Task<IActionResult> Edit(ReservationEditViewModel model, CancellationToken cancellationToken)
{
    ModelState.Clear();
    try
    {
        model.Current = await _api.GetReservationAsync(model.Id, cancellationToken);
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }

    model.StationId ??= model.Current.StationId;
    await LoadEditChoicesAsync(model, cancellationToken);
    return View(model);
}

// Moves the booking to the chosen slot, then shows the summary page.
[HttpPost]
[ActionName("Edit")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> EditConfirmed(ReservationEditViewModel model, CancellationToken cancellationToken)
{
    if (string.IsNullOrEmpty(model.SlotId))
    {
        ModelState.AddModelError(string.Empty, "Please choose a new slot.");
    }
    else
    {
        try
        {
            var updated = await _api.UpdateReservationAsync(model.Id, model.StationId ?? string.Empty, model.SlotId, cancellationToken);
            return RedirectToAction(nameof(Summary), new { id = updated.Id, done = "updated" });
        }
        catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
        {
            ModelState.AddModelError(string.Empty, ex.ApiMessage);
        }
    }

    try
    {
        model.Current = await _api.GetReservationAsync(model.Id, cancellationToken);
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }

    await LoadEditChoicesAsync(model, cancellationToken);
    return View(model);
}

// Asks staff to confirm the cancellation.
[HttpGet]
public async Task<IActionResult> Cancel(string id, CancellationToken cancellationToken)
{
    try
    {
        var current = await _api.GetReservationAsync(id, cancellationToken);
        return View(new ReservationCancelViewModel { Id = id, Current = current });
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
    }
}

```

```

        return RedirectToAction(nameof(Index));
    }
}

// Cancels the booking. The API decides if there is still enough notice.
[HttpPost]
[ActionName("Cancel")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> CancelConfirmed(ReservationCancelViewModel model, CancellationToken cancellationToken)
{
    try
    {
        var cancelled = await _api.CancelReservationAsync(model.Id, model.Reason, cancellationToken);
        return RedirectToAction(nameof(Summary), new { id = cancelled.Id, done = "cancelled" });
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        ModelState.AddModelError(string.Empty, ex.ApiMessage);
    }

    try
    {
        model.Current = await _api.GetReservationAsync(model.Id, cancellationToken);
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }

    return View(model);
}

// Approves a pending booking, then shows the summary page with its QR token.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Approve(string id, CancellationToken cancellationToken)
{
    try
    {
        var approved = await _api.ApproveReservationAsync(id, cancellationToken);
        return RedirectToAction(nameof(Summary), new { id = approved.Id, done = "approved" });
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }
}

// Shows the result of the last action on a booking.
[HttpGet]
public async Task<IActionResult> Summary(string id, string? done, CancellationToken cancellationToken)
{
    try
    {
        var reservation = await _api.GetReservationAsync(id, cancellationToken);
        return View(new ReservationSummaryViewModel { Reservation = reservation, Done = done ?? string.Empty });
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }
}

// Fills the station list and free slots on the create page.
private async Task LoadCreateChoicesAsync(ReservationCreateViewModel model, CancellationToken cancellationToken)
{
    (model.Stations, model.Slots, model.SlotsSearched) = await LoadChoicesAsync(model.StationId, model.Date, cancellationToken);
}

// Fills the station list and free slots on the edit page.
private async Task LoadEditChoicesAsync(ReservationEditViewModel model, CancellationToken cancellationToken)
{
    (model.Stations, model.Slots, model.SlotsSearched) = await LoadChoicesAsync(model.StationId, model.Date, cancellationToken);
}

// Gets the bookable stations, and the free slots when a station has been picked.
private async Task<(IReadOnlyList<BookingStationDto>, IReadOnlyList<AvailableSlotDto>, bool)> LoadChoicesAsync(
    string? stationId,
    DateOnly? date,
    CancellationToken cancellationToken)
{
    IReadOnlyList<BookingStationDto> stations = [];
    IReadOnlyList<AvailableSlotDto> slots = [];
    var searched = false;

    try
    {
        stations = await _api.GetBookableStationsAsync(cancellationToken);
        if (!string.IsNullOrEmpty(stationId))
        {
            slots = await _api.GetAvailableSlotsAsync(stationId, date, cancellationToken);
            searched = true;
        }
    }
    catch (ApiClientException ex) when (ex.StatusCode != StatusCodes.Status401Unauthorized)

```

```

        {
            ModelState.AddModelError(string.Empty, ex.ApiMessage);
        }

        return (stations, slots, searched);
    }
}

```

WebApp/SmartSolar.Web/Controllers/SlotsController.cs M2 · 105 lines

```

/*
 * File: SlotsController.cs
 * Description: Staff MVC page for one station's slots. Schedule and booking rules stay on the API.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

[Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
public class SlotsController : Controller
{
    private readonly ApiClient _api;

    // Injects the API client that attaches the session JWT.
    public SlotsController(ApiClient api)
    {
        _api = api;
    }

    // Shows one station and its slots.
    [HttpGet]
    public async Task<IActionResult> Index(string stationId, CancellationToken cancellationToken)
    {
        try
        {
            return View(await LoadPageAsync(stationId, new SlotForm(), cancellationToken));
        }
        catch (ApiClientException ex)
        {
            TempData["Error"] = ex.ApiMessage;
            return RedirectToAction("Index", "Stations");
        }
    }

    // Adds a slot. The API's validation message is shown on the form.
    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Create(string stationId, SlotForm form, CancellationToken cancellationToken)
    {
        if (!ModelState.IsValid)
        {
            return View("Index", await LoadPageAsync(stationId, form, cancellationToken));
        }

        try
        {
            await _api.CreateSlotAsync(stationId, form, cancellationToken);
            TempData["Success"] = "Slot added.";
            return RedirectToAction(nameof(Index), new { stationId });
        }
        catch (ApiClientException ex)
        {
            ModelState.AddModelError(string.Empty, ex.ApiMessage);
            return View("Index", await LoadPageAsync(stationId, form, cancellationToken));
        }
    }

    // Deletes a slot. A booked slot's 409 is shown as a banner.
    [HttpPost, ValidateAntiForgeryToken]
    public async Task<IActionResult> Delete(string id, string stationId, CancellationToken cancellationToken)
    {
        try
        {
            await _api.DeleteSlotAsync(id, cancellationToken);
            TempData["Success"] = "Slot deleted.";
        }
        catch (ApiClientException ex)
        {
            TempData["Error"] = ex.ApiMessage;
        }

        return RedirectToAction(nameof(Index), new { stationId });
    }

    // Opens or closes a free slot. GridOperator only.
    [HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.GridOperator)]
    public async Task<IActionResult> Availability(string id, string stationId, bool isAvailable, CancellationToken cancellationToken)
    {
        try

```

```

        {
            await _api.SetSlotAvailabilityAsync(id, isAvailable, cancellationToken);
            TempData["Success"] = isAvailable ? "Slot opened." : "Slot closed.";
        }
        catch (ApiClientException ex)
        {
            TempData["Error"] = ex.ApiMessage;
        }

        return RedirectToAction(nameof(Index), new { stationId });
    }

    // Loads the station and its slots for the page.
    private async Task<SlotsPage> LoadPageAsync(string stationId, SlotForm form, CancellationToken cancellationToken)
    {
        var station = await _api.GetStationAsync(stationId, cancellationToken);
        var slots = await _api.GetSlotsAsync(stationId, cancellationToken);
        return new SlotsPage { Station = station, Slots = slots, Form = form };
    }
}

```

WebApp/SmartSolar.Web/Controllers/StationsController.cs M2 · 131 lines

```

/*
 * File: StationsController.cs
 * Description: Staff MVC pages for listing, creating and editing stations. Rules stay on the API.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

[Authorize(Roles = RoleNames.Backoffice + "," + RoleNames.GridOperator)]
public class StationsController : Controller
{
    private readonly ApiClient _api;

    // Injects the API client that attaches the session JWT.
    public StationsController(ApiClient api)
    {
        _api = api;
    }

    // Lists stations. Operators can view; they cannot change status.
    [HttpGet]
    public async Task<IActionResult> Index(CancellationToken cancellationToken)
    {
        try
        {
            return View(await _api.GetStationsAsync(cancellationToken));
        }
        catch (ApiClientException ex)
        {
            TempData["Error"] = ex.ApiMessage;
            return View(Array.Empty<StationDto>());
        }
    }

    // Shows an empty station form. Backoffice only.
    [HttpGet, Authorize(Roles = RoleNames.Backoffice)]
    public IActionResult Create() => View(new StationForm());

    // Creates a station. The API's validation message is shown on the form.
    [HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
    public async Task<IActionResult> Create(StationForm form, CancellationToken cancellationToken)
    {
        if (!ModelState.IsValid)
        {
            return View(form);
        }

        try
        {
            await _api.CreateStationAsync(form, cancellationToken);
            TempData["Success"] = $"Created station {form.Name}.";
            return RedirectToAction(nameof(Index));
        }
        catch (ApiClientException ex)
        {
            ModelState.AddModelError(string.Empty, ex.ApiMessage);
            return View(form);
        }
    }

    // Loads a station into the edit form. Backoffice only.
    [HttpGet, Authorize(Roles = RoleNames.Backoffice)]
    public async Task<IActionResult> Edit(string id, CancellationToken cancellationToken)
    {
        try

```

```

    {
        var station = await _api.GetStationAsync(id, cancellationTokens);
        return View(StationForm.From(station));
    }
    catch (ApiClientException ex)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }
}

// Saves station edits. The API's validation message is shown on the form.
[HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
public async Task<IActionResult> Edit(string id, StationForm form, CancellationToken cancellationTokens)
{
    if (!ModelState.IsValid)
    {
        return View(form);
    }

    try
    {
        await _api.UpdateStationAsync(id, form, cancellationTokens);
        TempData["Success"] = $"Updated {form.Name}.";
        return RedirectToAction(nameof(Index));
    }
    catch (ApiClientException ex)
    {
        ModelState.AddModelError(string.Empty, ex.ApiMessage);
        return View(form);
    }
}

// Deactivates a station. The API's 409 is shown as a banner.
[HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
public Task<IActionResult> Deactivate(string id, CancellationToken cancellationTokens) =>
    RunStatusAction(() => _api.DeactivateStationAsync(id, cancellationTokens), "Deactivated");

// Activates a station again.
[HttpPost, ValidateAntiForgeryToken, Authorize(Roles = RoleNames.Backoffice)]
public Task<IActionResult> Activate(string id, CancellationToken cancellationTokens) =>
    RunStatusAction(() => _api.ActivateStationAsync(id, cancellationTokens), "Activated");

// Runs a status change and always returns to the list, with the API message on failure.
private async Task<IActionResult> RunStatusAction(Func<Task<StationDto>> command, string verb)
{
    try
    {
        var station = await command();
        TempData["Success"] = $" {verb} {station.Name}.";
    }
    catch (ApiClientException ex)
    {
        TempData["Error"] = ex.ApiMessage;
    }

    return RedirectToAction(nameof(Index));
}
}

```

WebApp/SmartSolar.Web/Controllers/UsersController.cs M1 · 170 lines

```

/*
 * File: UsersController.cs
 * Description: Backoffice user-management pages. Controllers only call ApiClient and display results.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

```

```

using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;
using SmartSolar.Web.Common;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Controllers;

[Authorize(Roles = RoleNames.Backoffice)]
public class UsersController : Controller
{
    private readonly ApiClient _api;

    // Injects the shared API client used by every staff-user page.
    public UsersController(ApiClient api)
    {
        _api = api;
    }

    // Lists staff accounts from GET /api/users.
    [HttpGet]
    public async Task<IActionResult> Index(CancellationToken cancellationTokens)
    {
        try
        {
            var users = await _api.GetStaffAsync(cancellationTokens);

```

```

        return View(users);
    }
    catch (ApiClientException ex)
    {
        TempData["Error"] = ex.ApiMessage;
        return View(Array.Empty<UserDto>());
    }
}

// Shows the create form.
[HttpGet]
public IActionResult Create()
{
    return View(new StaffUserForm { Role = RoleNames.GridOperator });
}

// Posts a new staff account to POST /api/users.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Create(StaffUserForm form, CancellationToken cancellationToken)
{
    if (string.IsNullOrEmpty(form.Password))
    {
        ModelState.AddModelError(nameof(form.Password), "Password is required.");
    }

    if (!ModelState.IsValid)
    {
        return View(form);
    }

    try
    {
        await _api.CreateStaffAsync(form, cancellationToken);
        TempData["Success"] = $"Created staff account {form.Username}.";
        return RedirectToAction(nameof(Index));
    }
    catch (ApiClientException ex)
    {
        ModelState.AddModelError(string.Empty, ex.ApiMessage);
        return View(form);
    }
}

// Loads one staff account into the edit form.
[HttpGet]
public async Task<IActionResult> Edit(string id, CancellationToken cancellationToken)
{
    try
    {
        var user = await _api.GetStaffByIdAsync(id, cancellationToken);
        return View(ToForm(user, isEdit: true));
    }
    catch (ApiClientException ex)
    {
        TempData["Error"] = ex.ApiMessage;
        return RedirectToAction(nameof(Index));
    }
}

// Posts updates to PUT /api/users/{id}. Username is taken from the route id, not the form.
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(string id, StaffUserForm form, CancellationToken cancellationToken)
{
    form.IsEdit = true;
    form.Id = id;
    form.Username = id;
    ModelState.Remove(nameof(StaffUserForm.Username));
    if (!ModelState.IsValid)
    {
        return View(form);
    }

    try
    {
        await _api.UpdateStaffAsync(id, form, cancellationToken);
        TempData["Success"] = $"Updated {form.Username}.";
        return RedirectToAction(nameof(Index));
    }
    catch (ApiClientException ex)
    {
        ModelState.AddModelError(string.Empty, ex.ApiMessage);
        return View(form);
    }
}

// Confirms deactivation before calling the API.
[HttpGet]
public async Task<IActionResult> Deactivate(string id, CancellationToken cancellationToken)
{
    try
    {
        var user = await _api.GetStaffByIdAsync(id, cancellationToken);
        return View(user);
    }
    catch (ApiClientException ex)
    {
        TempData["Error"] = ex.ApiMessage;
    }
}

```

```

        return RedirectToAction(nameof(Index));
    }
}

// Posts deactivation to POST /api/users/{id}/deactivate.
[HttpPost]
[ActionName("Deactivate")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeactivateConfirmed(string id, CancellationToken cancellationToken)
{
    try
    {
        var user = await _api.DeactivateStaffAsync(id, cancellationToken);
        TempData["Success"] = $"{user.Username} is now Deactivated.";
    }
    catch (ApiClientException ex)
    {
        TempData["Error"] = ex.ApiMessage;
    }

    return RedirectToAction(nameof(Index));
}

// Copies API user fields onto the edit form. Mapping only; no rules.
private static StaffUserForm ToForm(UserDto user, bool isEdit)
{
    return new StaffUserForm
    {
        Id = user.Id,
        Username = user.Username,
        FullName = user.FullName,
        Email = user.Email,
        Phone = user.Phone,
        Role = user.Role,
        IsEdit = isEdit
    };
}
}

```

WebApp/SmartSolar.Web/Filters/ApiUnauthorizedFilter.cs M1 · 45 lines

```

/*
 * File: ApiUnauthorizedFilter.cs
 * Description: Signs the browser out when the API returns 401 so expired JWTs cannot linger.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Mvc.Filters;
using SmartSolar.Web.Api;
using SmartSolar.Web.Services;

namespace SmartSolar.Web.Filters;

public class ApiUnauthorizedFilter : IAsyncExceptionFilter
{
    private readonly IWebSession _session;

    // Injects the session helper used to clear the cookie after a 401.
    public ApiUnauthorizedFilter(IWebSession session)
    {
        _session = session;
    }

    // Redirects to login when a page call fails with 401. Login itself handles its own errors.
    public async Task OnExceptionAsync(ExceptionContext context)
    {
        if (context.Exception is not ApiClientException ex || ex.StatusCode != StatusCodes.Status401Unauthorized)
        {
            return;
        }

        var controller = context.RouteData.Values["controller"]?.ToString();
        var action = context.RouteData.Values["action"]?.ToString();
        if (string.Equals(controller, "Account", StringComparison.OrdinalIgnoreCase)
            && string.Equals(action, "Login", StringComparison.OrdinalIgnoreCase))
        {
            return;
        }

        await _session.SignOutAsync();
        context.Result = new RedirectToActionResult("Login", "Account", new { timedOut = true });
        context.ExceptionHandled = true;
    }
}

```

WebApp/SmartSolar.Web/Models/AdminDashboardViewModel.cs M1 · 23 lines

```

/*
 * File: AdminDashboardViewModel.cs
 * Description: Counts shown on the Backoffice home. Numbers come from the API; this model holds display data only.
 * Author: DE SILVA R K D H (IT22001252)

```

```
* Created: 29/09/2026
*/
```

```
namespace SmartSolar.Web.Models;
```

```
public class AdminDashboardViewModel
{
    public OperationsDashboardDto Operations { get; set; } = new();

    public int StaffCount { get; set; }

    public int ProsumerCount { get; set; }

    public int ActiveProsumerCount { get; set; }

    public int PendingCount { get; set; }

    public string? LoadError { get; set; }
}
```

WebApp/SmartSolar.Web/Models/ApiErrorDto.cs M1 · 15 lines

```
/*
 * File: ApiErrorDto.cs
 * Description: Contract error body { code, message } parsed from non-2xx API responses.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class ApiErrorDto
{
    public string Code { get; set; } = string.Empty;

    public string Message { get; set; } = string.Empty;
}
```

WebApp/SmartSolar.Web/Models/AvailableSlotDto.cs M3 · 15 lines

```
/*
 * File: AvailableSlotDto.cs
 * Description: A free slot returned by the API for the booking picker.
 * Author: Janukshan S (IT22635266)
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class AvailableSlotDto
{
    public string Id { get; set; } = string.Empty;
    public string StationId { get; set; } = string.Empty;
    public DateTime StartTime { get; set; }
    public DateTime EndTime { get; set; }
}
```

WebApp/SmartSolar.Web/Models/BookingMonitorViewModel.cs M4 · 23 lines

```
/*
 * File: BookingMonitorViewModel.cs
 * Description: Filter form and the current page of the booking monitor.
 * Author: samudith
 * Created: 29/09/2026
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class BookingMonitorViewModel
{
    public string Tab { get; set; } = "all";
    public string? Status { get; set; }
    public string? StationId { get; set; }
    public string? Nic { get; set; }
    public string? Query { get; set; }
    public DateOnly? From { get; set; }
    public DateOnly? To { get; set; }
    public int Page { get; set; } = 1;
    public ReservationPageDto Result { get; set; } = new();

    public IReadOnlyList<StationDto> Stations { get; set; } = [];
}
```

WebApp/SmartSolar.Web/Models/BookingStationDto.cs M3 · 13 lines

```
/*
 * File: BookingStationDto.cs
```

```
/*
 * Description: A station that is taking bookings, as returned by the API.
 * Author: Janukshan S (IT22635266)
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class BookingStationDto
{
    public string Id { get; set; } = string.Empty;
    public string Name { get; set; } = string.Empty;
}
```

WebApp/SmartSolar.Web/Models/ErrorViewModel.cs M1 · 15 lines

```
/*
 * File: ErrorViewModel.cs
 * Description: Request id shown on the generic error page.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class ErrorViewModel
{
    public string? RequestId { get; set; }

    public bool ShowRequestId => !string.IsNullOrEmpty(RequestId);
}
```

WebApp/SmartSolar.Web/Models/LoginResponseDto.cs M1 · 17 lines

```
/*
 * File: LoginResponseDto.cs
 * Description: Successful POST /auth/login payload from the API.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class LoginResponseDto
{
    public string Token { get; set; } = string.Empty;

    public DateTime ExpiresAt { get; set; }

    public UserDto User { get; set; } = new();
}
```

WebApp/SmartSolar.Web/Models/LoginViewModel.cs M1 · 21 lines

```
/*
 * File: LoginViewModel.cs
 * Description: Login form fields posted to AccountController. The API validates credentials.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */
```

```
using System.ComponentModel.DataAnnotations;
```

```
namespace SmartSolar.Web.Models;
```

```
public class LoginViewModel
{
    [Required]
    [Display(Name = "Username or NIC")]
    public string Username { get; set; } = string.Empty;

    [Required]
    [DataType(DataType.Password)]
    public string Password { get; set; } = string.Empty;
}
```

WebApp/SmartSolar.Web/Models/OperationsDashboardDto.cs M4 · 17 lines

```
/*
 * File: OperationsDashboardDto.cs
 * Description: Staff Home figures from GET /dashboard/operations.
 * Author: samudith
 * Created: 29/09/2026
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class OperationsDashboardDto
```

```

{
    public int TodaysBookingCount { get; set; }
    public int PendingApprovals { get; set; }
    public int ActiveStations { get; set; }
    public int CompletedToday { get; set; }
    public List<ReservationDto> TodaysBookings { get; set; } = new();
}

```

WebApp/SmartSolar.Web/Models/ProsumerForm.cs M1 · 27 lines

```

/* File: ProsumerForm.cs | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
using System.ComponentModel.DataAnnotations;

```

```

namespace SmartSolar.Web.Models;

public class ProsumerForm
{
    [Required, Display(Name = "NIC")]
    public string Nic { get; set; } = string.Empty;

    [Required]
    public string Username { get; set; } = string.Empty;

    [MinLength(8), DataType(DataType.Password)]
    public string? Password { get; set; }

    [Required, Display(Name = "Full name")]
    public string FullName { get; set; } = string.Empty;

    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    [Phone]
    public string Phone { get; set; } = string.Empty;

    public bool IsEdit { get; set; }
}

```

WebApp/SmartSolar.Web/Models/ReservationCancelViewModel.cs M3 · 16 lines

```

/*
 * File: ReservationCancelViewModel.cs
 * Description: The booking to cancel and the optional reason typed by staff.
 * Author: Janukshan S (IT22635266)
 */

```

```

namespace SmartSolar.Web.Models;

public class ReservationCancelViewModel
{
    public string Id { get; set; } = string.Empty;

    public ReservationDto? Current { get; set; }

    public string? Reason { get; set; }
}

```

WebApp/SmartSolar.Web/Models/ReservationCreateViewModel.cs M3 · 29 lines

```

/*
 * File: ReservationCreateViewModel.cs
 * Description: Values picked on the "book on behalf of a prosumer" page, plus the lists shown to pick from.
 * Author: Janukshan S (IT22635266)
 */

```

```

using System.ComponentModel.DataAnnotations;

namespace SmartSolar.Web.Models;

public class ReservationCreateViewModel
{
    [Display(Name = "Prosumer NIC")]
    public string? ProsumerNic { get; set; }

    [Display(Name = "Station")]
    public string? StationId { get; set; }

    [Display(Name = "Date")]
    public DateOnly? Date { get; set; }

    public string? SlotId { get; set; }

    public IReadOnlyList<BookingStationDto> Stations { get; set; } = [];

    public IReadOnlyList<AvailableSlotDto> Slots { get; set; } = [];
}

```

```
    public bool SlotsSearched { get; set; }  
}
```

WebApp/SmartSolar.Web/Models/ReservationDto.cs M3 · 23 lines

```
/*  
 * File: ReservationDto.cs  
 * Description: Reservation JSON shape returned by the API.  
 * Author: Janukshan S (IT22635266)  
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class ReservationDto  
{  
    public string Id { get; set; } = string.Empty;  
    public string ProsumerNic { get; set; } = string.Empty;  
    public string StationId { get; set; } = string.Empty;  
    public string StationName { get; set; } = string.Empty;  
    public string SlotId { get; set; } = string.Empty;  
    public DateTime ScheduledAt { get; set; }  
    public string Status { get; set; } = string.Empty;  
    public string? QrToken { get; set; }  
    public DateTime CreatedAt { get; set; }  
    public DateTime UpdatedAt { get; set; }  
    public DateTime? CompletedAt { get; set; }  
    public string? CompletedBy { get; set; }  
}
```

WebApp/SmartSolar.Web/Models/ReservationEditViewModel.cs M3 · 26 lines

```
/*  
 * File: ReservationEditViewModel.cs  
 * Description: The booking being moved, plus the new station, date and slot picked for it.  
 * Author: Janukshan S (IT22635266)  
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class ReservationEditViewModel  
{  
    public string Id { get; set; } = string.Empty;  
  
    public ReservationDto? Current { get; set; }  
  
    public string? StationId { get; set; }  
  
    public DateOnly? Date { get; set; }  
  
    public string? SlotId { get; set; }  
  
    public IReadOnlyList<BookingStationDto> Stations { get; set; } = [];  
  
    public IReadOnlyList<AvailableSlotDto> Slots { get; set; } = [];  
  
    public bool SlotsSearched { get; set; }  
}
```

WebApp/SmartSolar.Web/Models/ReservationIndexViewModel.cs M3 · 20 lines

```
/*  
 * File: ReservationIndexViewModel.cs  
 * Description: Filters and results for the reservations list page.  
 * Author: Janukshan S (IT22635266)  
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class ReservationIndexViewModel  
{  
    public string? Status { get; set; }  
  
    public string? Nic { get; set; }  
  
    public string? Q { get; set; }  
  
    public bool IncludePast { get; set; }  
  
    public IReadOnlyList<ReservationDto> Items { get; set; } = [];  
}
```

WebApp/SmartSolar.Web/Models/ReservationPageDto.cs M4 · 16 lines

```
/*
 * File: ReservationPageDto.cs
 * Description: Paged reservation list returned by GET /reservations.
 * Author: samudith
 * Created: 29/09/2026
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class ReservationPageDto
{
    public List<ReservationDto> Items { get; set; } = new();
    public int Page { get; set; }
    public int PageSize { get; set; }
    public long TotalCount { get; set; }
}
```

WebApp/SmartSolar.Web/Models/ReservationSummaryViewModel.cs M3 · 14 lines

```
/*
 * File: ReservationSummaryViewModel.cs
 * Description: What the summary page shows after a booking is created, updated, cancelled or approved.
 * Author: Janukshan S (IT22635266)
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class ReservationSummaryViewModel
{
    public ReservationDto Reservation { get; set; } = new();

    public string Done { get; set; } = string.Empty;
}
```

WebApp/SmartSolar.Web/Models/SlotDto.cs M2 · 21 lines

```
/*
 * File: SlotDto.cs
 * Description: Slot JSON returned by the API. Times are UTC.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
```

```
namespace SmartSolar.Web.Models;
```

```
public class SlotDto
{
    public string Id { get; set; } = string.Empty;

    public string StationId { get; set; } = string.Empty;

    public DateTime StartTime { get; set; }

    public DateTime EndTime { get; set; }

    public bool IsAvailable { get; set; }
}
```

WebApp/SmartSolar.Web/Models/SlotForm.cs M2 · 28 lines

```
/*
 * File: SlotForm.cs
 * Description: Add-slot form. The times are Sri Lanka local and are sent with a +05:30 offset.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
```

```
using System.ComponentModel.DataAnnotations;
```

```
namespace SmartSolar.Web.Models;
```

```
public class SlotForm
{
    [Required, Display(Name = "Start")]
    public DateTime? Start { get; set; }

    [Required, Display(Name = "End")]
    public DateTime? End { get; set; }
}
```

```
public class SlotsPage
{
    public StationDto Station { get; set; } = new();

    public IReadOnlyList<SlotDto> Slots { get; set; } = new List<SlotDto>();
}
```

```
    public SlotForm Form { get; set; } = new();  
}
```

WebApp/SmartSolar.Web/Models/StaffUserForm.cs M1 · 37 lines

```
/*  
 * File: StaffUserForm.cs  
 * Description: Create/edit form for staff accounts. Required-field checks are UX only; the API enforces rules.  
 * Author: DE SILVA R K D H (IT22001252)  
 * Created: 20/09/2026  
 */  
  
using System.ComponentModel.DataAnnotations;  
  
namespace SmartSolar.Web.Models;  
  
public class StaffUserForm  
{  
    public string? Id { get; set; }  
  
    [Required]  
    [Display(Name = "Username")]  
    public string Username { get; set; } = string.Empty;  
  
    [DataType(DataType.Password)]  
    [Display(Name = "Password")]  
    public string? Password { get; set; }  
  
    [Required]  
    [Display(Name = "Full name")]  
    public string FullName { get; set; } = string.Empty;  
  
    [EmailAddress]  
    public string Email { get; set; } = string.Empty;  
  
    public string Phone { get; set; } = string.Empty;  
  
    [Required]  
    public string Role { get; set; } = string.Empty;  
  
    public bool IsEdit { get; set; }  
}
```

WebApp/SmartSolar.Web/Models/StationDto.cs M2 · 36 lines

```
/*  
 * File: StationDto.cs  
 * Description: Station JSON returned by the API, including the weekly schedule.  
 * Author: Mohamed Asath (IT22633422)  
 * Created: 30/09/2026  
 */  
  
namespace SmartSolar.Web.Models;  
  
public class StationDto  
{  
    public string Id { get; set; } = string.Empty;  
  
    public string Name { get; set; } = string.Empty;  
  
    public double Latitude { get; set; }  
  
    public double Longitude { get; set; }  
  
    public double CapacityKwh { get; set; }  
  
    public int BatterySlotsTotal { get; set; }  
  
    public List<StationScheduleDto> Schedule { get; set; } = new();  
  
    public string Status { get; set; } = string.Empty;  
}  
  
public class StationScheduleDto  
{  
    public string DayOfWeek { get; set; } = string.Empty;  
  
    public string OpenTime { get; set; } = string.Empty;  
  
    public string CloseTime { get; set; } = string.Empty;  
}
```

```

/*
 * File: StationForm.cs
 * Description: Create and edit form for a station. Field rules are enforced by the API.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */

using System.ComponentModel.DataAnnotations;

namespace SmartSolar.Web.Models;

public class StationForm
{
    [Required]
    public string Name { get; set; } = string.Empty;

    [Display(Name = "Latitude")]
    public double Latitude { get; set; }

    [Display(Name = "Longitude")]
    public double Longitude { get; set; }

    [Display(Name = "Capacity (kWh)")]
    public double CapacityKwh { get; set; }

    [Display(Name = "Battery slots")]
    public int BatterySlotsTotal { get; set; }

    public List<ScheduleRow> Days { get; set; } = NewWeek();

    // Builds Monday-to-Sunday rows, closed until the user ticks Open.
    public static List<ScheduleRow> NewWeek()
    {
        return new[] { "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday" }
            .Select(day => new ScheduleRow { DayOfWeek = day })
            .ToList();
    }

    // Copies an API station onto the form and ticks the days that have hours.
    public static StationForm From(StationDto station)
    {
        var form = new StationForm
        {
            Name = station.Name,
            Latitude = station.Latitude,
            Longitude = station.Longitude,
            CapacityKwh = station.CapacityKwh,
            BatterySlotsTotal = station.BatterySlotsTotal
        };

        foreach (var day in form.Days)
        {
            var entry = station.Schedule.FirstOrDefault(item =>
                string.Equals(item.DayOfWeek, day.DayOfWeek, StringComparison.OrdinalIgnoreCase));
            if (entry is null)
            {
                continue;
            }

            day.Open = true;
            day.OpenTime = entry.OpenTime;
            day.CloseTime = entry.CloseTime;
        }

        return form;
    }
}

public class ScheduleRow
{
    public string DayOfWeek { get; set; } = string.Empty;

    public bool Open { get; set; }

    public string OpenTime { get; set; } = "08:00";

    public string CloseTime { get; set; } = "18:00";
}

```

```

/*
 * File: UserDto.cs
 * Description: Public user JSON shape returned by the API. Password hashes are never present.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

namespace SmartSolar.Web.Models;

```

```

public class UserDto
{
    public string Id { get; set; } = string.Empty;

    public string? Nic { get; set; }

    public string Username { get; set; } = string.Empty;

    public string FullName { get; set; } = string.Empty;

    public string Email { get; set; } = string.Empty;

    public string Phone { get; set; } = string.Empty;

    public string Role { get; set; } = string.Empty;

    public string Status { get; set; } = string.Empty;

    public DateTime CreatedAt { get; set; }
}

```

WebApp/SmartSolar.Web/Program.cs M1 · 80 lines

```

/*
 * File: Program.cs
 * Description: Composition root for the MVC web client. No MongoDB and no business rules live here.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using Microsoft.AspNetCore.Authentication.Cookies;
using SmartSolar.Web.Api;
using SmartSolar.Web.Filters;
using SmartSolar.Web.Services;

var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHttpContextAccessor();
builder.Services.AddDistributedMemoryCache();
builder.Services.AddSession(options =>
{
    options.Cookie.Name = "SmartSolar.Web.Session";
    options.Cookie.HttpOnly = true;
    options.Cookie.IsEssential = true;
    options.IdleTimeout = TimeSpan.FromHours(8);
});

builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/AccessDenied";
        options.Cookie.Name = "SmartSolar.Web";
        options.Cookie.HttpOnly = true;
        options.SlidingExpiration = true;
        options.ExpireTimeSpan = TimeSpan.FromHours(8);
    });

builder.Services.AddScoped<IWebSession, WebSession>();
builder.Services.AddScoped<ApiUnauthorizedFilter>();
builder.Services.AddHttpClient<ApiClient>(ConfigureApiClient);

builder.Services.AddControllersWithViews(options =>
{
    options.Filters.Add<ApiUnauthorizedFilter>();
});

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
}

// Serves wwwroot directly. MapStaticAssets returns an empty body when the browser asks for gzip.
app.UseStaticFiles();
app.UseRouting();
app.UseSession();
app.UseAuthentication();
app.UseAuthorization();

app.MapStaticAssets();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}")
    .WithStaticAssets();

app.Run();

```

```
// Points the shared HttpClient at the FAT API base URL from configuration.
static void ConfigureApiClient(IServiceProvider services, HttpClient client)
{
    var baseUrl = services.GetRequiredService<IConfiguration>()["Api:BaseUrl"]
        ?? throw new InvalidOperationException("Api:BaseUrl is missing from configuration.");

    if (!baseUrl.EndsWith("/"))
    {
        baseUrl += "/";
    }

    client.BaseAddress = new Uri(baseUrl);
    client.Timeout = TimeSpan.FromSeconds(30);
}
```

WebApp/SmartSolar.Web/Services/IWebSession.cs M1 · 22 lines

```
/*
 * File: IWebSession.cs
 * Description: Cookie + session helpers that store the JWT after login. No API rules live here.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using SmartSolar.Web.Models;

namespace SmartSolar.Web.Services;

public interface IWebSession
{
    // Writes the JWT to session and signs the auth cookie from the login payload.
    Task SignInAsync(LoginResponseDto login);

    // Clears the JWT session key and the auth cookie.
    Task SignOutAsync();

    // Returns the bearer token used by ApiClient, or null when the user is not signed in.
    string? GetJwt();
}
```

WebApp/SmartSolar.Web/Services/WebSession.cs M1 · 66 lines

```
/*
 * File: WebSession.cs
 * Description: Stores the API JWT in session and mirrors role claims onto the auth cookie.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 20/09/2026
 */

using System.Security.Claims;
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using SmartSolar.Web.Models;

namespace SmartSolar.Web.Services;

public class WebSession : IWebSession
{
    public const string JwtSessionKey = "JwtToken";

    private readonly IHttpContextAccessor _httpContextAccessor;

    // Injects the current HTTP context used to read and write session and cookies.
    public WebSession(IHttpContextAccessor httpContextAccessor)
    {
        _httpContextAccessor = httpContextAccessor;
    }

    // Writes the JWT to session and signs the auth cookie from the login payload.
    public async Task SignInAsync(LoginResponseDto login)
    {
        var context = RequireHttpContext();
        context.Session.SetString(JwtSessionKey, login.Token);

        var claims = new List<Claim>
        {
            new(ClaimTypes.NameIdentifier, login.User.Id),
            new(ClaimTypes.Name, login.User.Username),
            new(ClaimTypes.Role, login.User.Role),
            new("FullName", login.User.FullName)
        };

        var identity = new ClaimsIdentity(claims, CookieAuthenticationDefaults.AuthenticationScheme);
        var principal = new ClaimsPrincipal(identity);
        await context.SignInAsync(CookieAuthenticationDefaults.AuthenticationScheme, principal);
    }

    // Clears the JWT session key and the auth cookie.
    public async Task SignOutAsync()
    {

```

```

    var context = RequireHttpContext();
    context.Session.Remove(JwtSessionKey);
    await context.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);
}

// Returns the bearer token used by ApiClient, or null when the user is not signed in.
public string? GetJwt()
{
    return _httpContextAccessor.HttpContext?.Session.GetString(JwtSessionKey);
}

// Returns the current HTTP context or throws when the accessor is empty.
private HttpContext RequireHttpContext()
{
    return _httpContextAccessor.HttpContext
        ?? throw new InvalidOperationException("HTTP context is not available.");
}
}

```

WebApp/SmartSolar.Web/ViewComponents/PendingActivationBadgeViewComponent.cs M1 · 17 lines

```

/* File: PendingActivationBadgeViewComponent.cs | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
using Microsoft.AspNetCore.Mvc;
using SmartSolar.Web.Api;

```

```

namespace SmartSolar.Web.ViewComponents;

public class PendingActivationBadgeViewComponent : ViewComponent
{
    private readonly ApiClient _api;
    public PendingActivationBadgeViewComponent(ApiClient api) { _api = api; }

    public async Task<IViewComponentResult> InvokeAsync()
    {
        try { return View(await _api.GetPendingProsumersAsync(HttpContext.RequestAborted)); }
        catch (ApiClientException) { return View(Array.Empty<Models.UserDto>()); }
    }
}

```

WebApp/SmartSolar.Web/Views/Account/AccessDenied.cshtml M1 · 7 lines

```

@{
    ViewData["Title"] = "Access denied";
}

<h1 class="h4">You do not have access to this page</h1>
<p class="text-muted">Your role does not include this screen.</p>
<a class="btn btn-primary" asp-controller="Home" asp-action="Index">Go to dashboard</a>

```

WebApp/SmartSolar.Web/Views/Account/Login.cshtml M1 · 38 lines

```

@model LoginViewModel
@{
    ViewData["Title"] = "Log in";
}

<div class="row justify-content-center py-4">
    <div class="col-12">
        <div class="card auth-card shadow-none">
            <div class="card-body p-4">
                <div class="d-flex align-items-center gap-2 mb-3">
                    <span class="sun-mark"></span>
                    <span class="fw-semibold">Smart Solar</span>
                </div>
                <h1 class="h4 mb-2">Staff log in</h1>
                <p class="text-muted mb-4">Backoffice and Grid Operator accounts.</p>
                <form asp-action="Login" method="post">
                    <div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>
                    <input type="hidden" name="returnUrl" value="@ViewBag.ReturnUrl" />
                    <div class="mb-3">
                        <label asp-for="Username" class="form-label"></label>
                        <input asp-for="Username" class="form-control" autocomplete="username" autofocus />
                        <span asp-validation-for="Username" class="text-danger"></span>
                    </div>
                    <div class="mb-3">
                        <label asp-for="Password" class="form-label"></label>
                        <input asp-for="Password" class="form-control" autocomplete="current-password" />
                        <span asp-validation-for="Password" class="text-danger"></span>
                    </div>
                    <button type="submit" class="btn btn-primary w-100">Log in</button>
                </form>
            </div>
        </div>
    </div>
</div>

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}

```

WebApp/SmartSolar.Web/Views/Admin/Index.cshtml M1 · 56 lines

```
@model AdminDashboardViewModel
@{
    ViewData["Title"] = "Dashboard";
}

<h1 class="h3 mb-3">Backoffice dashboard</h1>

@if (!string.IsNullOrEmpty(Model.LoadError))
{
    <div class="alert alert-warning" role="alert">Counts could not be loaded. @Model.LoadError</div>
}

<partial name="_OperationsDashboard" model="Model.Operations" />

@if (Model.PendingCount > 0)
{
    <a class="card tile-attention text-decoration-none text-body mb-4" asp-controller="PendingActivations" asp-action="Index">
        <div class="card-body d-flex justify-content-between align-items-center">
            <div>
                <div class="small text-muted">Awaiting activation</div>
                <div class="tile-number">@Model.PendingCount</div>
            </div>
            <span>Open queue</span>
        </div>
    </a>
}

<div class="row g-3 mb-4">
    <div class="col-md-4">
        <div class="card h-100">
            <div class="card-body">
                <h2 class="h6">Staff users</h2>
                <p class="text-muted">Create, update and deactivate staff accounts.</p>
                <a class="btn btn-primary" asp-controller="Users" asp-action="Index">Manage users</a>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card h-100">
            <div class="card-body">
                <h2 class="h6">Bookings</h2>
                <p class="text-muted">Filter today's work, the approval queue and booking history.</p>
                <a class="btn btn-outline-primary" asp-controller="Bookings" asp-action="Index">Open monitor</a>
            </div>
        </div>
    </div>
    <div class="col-md-4">
        <div class="card h-100">
            <div class="card-body">
                <h2 class="h6">Reservations</h2>
                <p class="text-muted">List, edit, cancel or approve bookings.</p>
                <a class="btn btn-outline-primary" asp-controller="Reservations" asp-action="Index">Open list</a>
            </div>
        </div>
    </div>
</div>
```

WebApp/SmartSolar.Web/Views/Bookings/Index.cshtml M1 · 161 lines

```
@model BookingMonitorViewModel
@{
    ViewData["Title"] = "Bookings";
    var tabs = new (string Key, string Label)[]
    {
        ("all", "All"),
        ("upcoming", "Upcoming"),
        ("pending", "Pending approval"),
        ("history", "History")
    };
    var pageCount = Model.Result.PageSize == 0 ? 1 : (int)Math.Ceiling(Model.Result.TotalCount / (double)Model.Result.PageSize);
    if (pageCount < 1) pageCount = 1;
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <h1 class="h3 mb-0">Bookings</h1>
    <a class="btn btn-primary" asp-controller="Reservations" asp-action="Create">Book a slot</a>
</div>

<ul class="nav nav-pills mb-3">
    @foreach (var item in tabs)
    {
        <li class="nav-item">
            <a class="nav-link @(Model.Tab == item.Key ? "active" : "")"
                asp-action="Index"
                asp-route-tab="@item.Key"
                asp-route-q="@Model.Query"
                asp-route-stationId="@Model.StationId"
                asp-route-nic="@Model.Nic"
                asp-route-from="@Model.From?.ToString("yyyy-MM-dd")"
                asp-route-to="@Model.To?.ToString("yyyy-MM-dd")">@item.Label</a>
        </li>
    }
</ul>
```

```

<button class="btn btn-outline-primary d-md-none mb-3" type="button" data-bs-toggle="collapse" data-bs-target="#bookingFilters">Filters</button>
<div class="collapse" id="bookingFilters">
<form method="get" class="row g-2 align-items-end mb-3">
  <input type="hidden" name="tab" value="@Model.Tab" />
  <div class="col-md-2">
    <label class="form-label" for="from">From</label>
    <input class="form-control" type="date" id="from" name="from" value="@Model.From?.ToString("yyyy-MM-dd")" />
  </div>
  <div class="col-md-2">
    <label class="form-label" for="to">To</label>
    <input class="form-control" type="date" id="to" name="to" value="@Model.To?.ToString("yyyy-MM-dd")" />
  </div>
  <div class="col-md-3">
    <label class="form-label" for="stationId">Station</label>
    <select class="form-select" id="stationId" name="stationId">
      <option value="">All stations</option>
      @foreach (var station in Model.Stations)
      {
        if (Model.StationId == station.Id)
        {
          <option value="@station.Id" selected>@station.Name</option>
        }
        else
        {
          <option value="@station.Id">@station.Name</option>
        }
      }
    </select>
  </div>
  <div class="col-md-2">
    <label class="form-label" for="nic">NIC</label>
    <input class="form-control" id="nic" name="nic" value="@Model.Nic" />
  </div>
  <div class="col-md-2">
    <label class="form-label" for="q">Keyword</label>
    <input class="form-control" id="q" name="q" value="@Model.Query" placeholder="Name or NIC" />
  </div>
  <div class="col-12">
    <button class="btn btn-outline-primary" type="submit">Apply filters</button>
  </div>
</form>
</div>

<div class="card">
  <table class="table table-hover align-middle mb-0 table-stack">
    <thead>
      <tr>
        <th>When</th>
        <th>Station</th>
        <th>NIC</th>
        <th>Status</th>
        <th></th>
      </tr>
    </thead>
    <tbody>
      @if (Model.Result.Items.Count == 0)
      {
        <tr>
          <td colspan="5" class="text-muted">No bookings match these filters.</td>
        </tr>
      }
      @foreach (var booking in Model.Result.Items)
      {
        <tr>
          <td data-label="When">@booking.ScheduledAt.ToSriLankaText()</td>
          <td data-label="Station">@booking.StationName</td>
          <td data-label="NIC">@booking.ProsumerNic</td>
          <td data-label="Status"><partial name="_ReservationStatusBadge" model="booking.Status" /></td>
          <td class="text-end text-nowrap">
            <a class="btn btn-sm btn-outline-secondary" asp-controller="Reservations" asp-action="Summary" asp-route-id="@booking.Id">Details</a>
            @if (booking.Status == "Pending")
            {
              <form asp-action="Approve" method="post" class="d-inline">
                <input type="hidden" name="id" value="@booking.Id" />
                <input type="hidden" name="tab" value="@Model.Tab" />
                <button class="btn btn-sm btn-success" type="submit">Approve</button>
              </form>
            }
          </td>
        </tr>
      }
    </tbody>
  </table>
</div>

@if (pageCount > 1)
{
  <nav aria-label="Booking pages">
    <ul class="pagination">
      @for (var number = 1; number <= pageCount; number++)
      {
        <li class="page-item @(number == Model.Result.Page ? "active" : "")">
          <a class="page-link"
            asp-action="Index"
            asp-route-tab="@Model.Tab"
            asp-route-status="@Model.Status"
            asp-route-stationId="@Model.StationId"
            asp-route-nic="@Model.Nic"
            asp-route-q="@Model.Query"
            asp-route-from="@Model.From?.ToString("yyyy-MM-dd")"
            asp-route-to="@Model.To?.ToString("yyyy-MM-dd")"
            asp-route-page="@number">@number</a>
        </li>
      }
    </ul>
  </nav>
}

```

```

        </li>
    }
</ul>
</nav>
}

@if (User.IsInRole(RoleNames.GridOperator))
{
    <div class="card mt-4">
        <div class="card-body">
            <h2 class="h5">Verify a QR code</h2>
            <p class="text-muted">Paste the token from the prosumer's QR. The API accepts it only for an approved booking scheduled today.</p>
            <form asp-action="Verify" method="post" class="row g-2">
                <div class="col-md-8">
                    <label class="form-label" for="qrToken">QR token</label>
                    <input class="form-control" id="qrToken" name="qrToken" required />
                </div>
                <div class="col-md-4 d-flex align-items-end">
                    <button class="btn btn-primary" type="submit">Verify</button>
                </div>
            </form>
        </div>
    </div>
}

```

WebApp/SmartSolar.Web/Views/Bookings/Verified.cshtml M4 · 38 lines

```

@model ReservationDto
@{
    ViewData["Title"] = Model.Status == "Completed" ? "Booking completed" : "QR verified";
}

<div class="row justify-content-center">
    <div class="col-lg-7">
        <div class="card shadow-sm">
            <div class="card-body">
                <h1 class="h4">@(Model.Status == "Completed" ? "Job completed" : "QR verified")</h1>
                <dl class="row">
                    <dt class="col-sm-4">Reference</dt>
                    <dd class="col-sm-8 font-monospace">@Model.Id</dd>
                    <dt class="col-sm-4">Status</dt>
                    <dd class="col-sm-8"><partial name="_ReservationStatusBadge" model="Model.Status" /></dd>
                    <dt class="col-sm-4">Prosumer NIC</dt>
                    <dd class="col-sm-8">@Model.ProsumerNic</dd>
                    <dt class="col-sm-4">Station</dt>
                    <dd class="col-sm-8">@Model.StationName</dd>
                    <dt class="col-sm-4">Slot time</dt>
                    <dd class="col-sm-8">@Model.ScheduledAt.ToSriLankaText()</dd>
                </dl>
                @if (Model.Status == "Approved")
                {
                    <form asp-action="Complete" method="post">
                        <input type="hidden" name="id" value="@Model.Id" />
                        <button class="btn btn-success" type="submit">Finalise</button>
                        <a class="btn btn-outline-secondary" asp-action="Index">Back</a>
                    </form>
                }
                else
                {
                    <a class="btn btn-outline-secondary" asp-action="Index">Back to bookings</a>
                }
            </div>
        </div>
    </div>
</div>

```

WebApp/SmartSolar.Web/Views/Home/Index.cshtml M1 · 15 lines

```

@{
    ViewData["Title"] = "Home";
}

<div class="row justify-content-center py-4">
    <div class="col-lg-8">
        <div class="card auth-card shadow-sm">
            <div class="card-body p-4 p-md-5">
                <h1 class="h3 mb-2">Smart Solar</h1>
                <p class="text-muted mb-4">Manage microgrid stations, prosumers and energy bookings.</p>
                <a class="btn btn-primary" asp-controller="Account" asp-action="Login">Staff log in</a>
            </div>
        </div>
    </div>
</div>

```

WebApp/SmartSolar.Web/Views/Operations/Index.cshtml M4 · 39 lines

```

@model OperationsDashboardDto
@{
    ViewData["Title"] = "Operations";
}

<h1 class="h3 mb-3">Operations home</h1>
<p class="text-muted">Today's bookings, pending approvals, and tools to look up a prosumer or book a slot.</p>

```

```

<partial name="_OperationsDashboard" model="Model" />

<div class="row g-3">
  <div class="col-md-4">
    <div class="card h-100">
      <div class="card-body">
        <h2 class="h5">Booking monitor</h2>
        <p class="text-muted">Filter the queue, verify a QR token, and finalise completed jobs.</p>
        <a class="btn btn-primary" asp-controller="Bookings" asp-action="Index">Open monitor</a>
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <div class="card h-100">
      <div class="card-body">
        <h2 class="h5">Find a prosumer</h2>
        <p class="text-muted">Search by NIC, name, username or phone.</p>
        <a class="btn btn-outline-primary" asp-controller="Prosumers" asp-action="Index">Prosumers</a>
      </div>
    </div>
  </div>
  <div class="col-md-4">
    <div class="card h-100">
      <div class="card-body">
        <h2 class="h5">Book a slot</h2>
        <p class="text-muted">Choose the prosumer, station and a free window.</p>
        <a class="btn btn-outline-primary" asp-controller="Reservations" asp-action="Create">Book a slot</a>
      </div>
    </div>
  </div>
</div>
</div>

```

WebApp/SmartSolar.Web/Views/PendingActivations/Index.cshtml M1 · 36 lines

```

@model IReadOnlyList<UserDto>
@{
    ViewData["Title"] = "Pending activations";
}

<div class="d-flex justify-content-between align-items-center mb-3">
  <div>
    <h1 class="h4 mb-1">Pending activations</h1>
    <p class="text-muted mb-0">Registrations waiting for a Backoffice decision.</p>
  </div>
  <span class="badge badge-status badge-status-pending">@Model.Count waiting</span>
</div>
<div class="card">
<table class="table table-hover align-middle mb-0 table-stack">
  <thead><tr><th>NIC</th><th>Name</th><th>Username</th><th>Contact</th><th class="text-end">Decision</th></tr></thead>
  <tbody>
    @if (Model.Count == 0)
    {
      <tr><td colspan="5" class="text-muted">No accounts are waiting. New phone registrations appear here.</td></tr>
    }
    @foreach (var user in Model)
    {
      <tr>
        <td data-label="NIC">@user.Nic</td>
        <td data-label="Name">@user.FullName</td>
        <td data-label="Username">@user.Username</td>
        <td data-label="Contact">@user.Email<br /><span class="text-muted">@user.Phone</span></td>
        <td class="text-end text-nowrap">
          <form asp-action="Activate" asp-route-id="@user.Nic" method="post" class="d-inline"><button class="btn btn-sm btn-primary">Activate</button></form>
          <form asp-action="Reject" asp-route-id="@user.Nic" method="post" class="d-inline" onsubmit="return confirm('Reject and deactivate this registration?');"><button class="btn btn-sm btn-outline-danger">Reject</button></form>
        </td>
      </tr>
    }
  </tbody>
</table>
</div>

```

WebApp/SmartSolar.Web/Views/Prosumers/Create.cshtml M1 · 9 lines

```

@model ProsumerForm
@{ ViewData["Title"] = "Create prosumer"; }
<div class="d-flex justify-content-between align-items-center mb-3"><h1 class="h3 mb-0">Create prosumer</h1><a asp-action="Index" class="btn btn-outline-secondary">Back</a></div>
<p class="text-muted">Staff-created prosumer accounts are active immediately.</p>
<form method="post" class="card card-body shadow-sm">
  <partial name="_Form" model="Model" />
  <div class="mt-4"><button class="btn btn-primary" type="submit">Create active account</button></div>
</form>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

WebApp/SmartSolar.Web/Views/Prosumers/Edit.cshtml M1 · 8 lines

```

@model ProsumerForm
@{ ViewData["Title"] = "Edit prosumer"; }
<div class="d-flex justify-content-between align-items-center mb-3"><h1 class="h3 mb-0">Edit prosumer</h1><a asp-action="Index" class="btn btn-outline-secondary">Back</a></div>
<form method="post" class="card card-body shadow-sm">
  <partial name="_Form" model="Model" />
  <div class="mt-4"><button class="btn btn-primary" type="submit">Save changes</button></div>

```

```
</form>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }
```

WebApp/SmartSolar.Web/Views/Prosumers/Index.cshtml M1 · 43 lines

```
@model IReadOnlyList<UserDto>
@{ ViewData["Title"] = "Prosumers"; var isBackoffice = User.IsInRole(RoleNames.Backoffice); }
<div class="d-flex justify-content-between align-items-center mb-3">
    <div>
        <h1 class="h4 mb-1">Prosumers</h1>
        <p class="text-muted mb-0">@(isBackoffice ? "Reactivate is available only to Backoffice." : "You can look up accounts. Only Backoffice can reactivate one.")</p>
    </div>
    @if (isBackoffice) { <a asp-action="Create" class="btn btn-primary">Create prosumer</a> }
</div>
<form method="get" class="row g-2 mb-3">
    <div class="col-md-6"><input name="q" value="@ViewBag.Query" class="form-control" placeholder="Search NIC, name, username or phone" /></div>
    <div class="col-md-3">
        <select name="status" class="form-select">
            <option value="">All statuses</option>
            @foreach (var status in new[] { "Pending", "Active", "Deactivated" }) { <option value="@status" selected="@((ViewBag.Status == status))">@status</option> }
        </select>
    </div>
    <div class="col-md-3"><button class="btn btn-outline-primary" type="submit">Search</button> <a asp-action="Index" class="btn btn-link">Clear</a></div>
</form>
<div class="card">
<table class="table table-hover align-middle mb-0 table-stack">
    <thead><tr><th>NIC</th><th>Name</th><th>Username</th><th>Phone</th><th>Status</th>@if (isBackoffice) { <th class="text-end">Actions</th> }</tr></thead>
    <tbody>
        @if (Model.Count == 0) { <tr><td colspan="@((isBackoffice ? 6 : 5))" class="text-center text-muted py-4">No prosumers found.</td></tr> }
        @foreach (var user in Model) {
            <tr>
                <td data-label="NIC">@user.Nic</td><td data-label="Name">@user.FullName</td><td data-label="Username">@user.Username</td><td data-label="Phone">@user.Phone</td>
                <td data-label="Status"><partial name="_AccountStatus" model="user.Status" /></td>
                @if (isBackoffice) {
                    <td class="text-end text-nowrap">
                        <a asp-action="Edit" asp-route-id="@user.Nic" class="btn btn-sm btn-outline-primary">Edit</a>
                        @if (user.Status == "Deactivated") {
                            <form asp-action="Reactivate" asp-route-id="@user.Nic" method="post" class="d-inline"><button class="btn btn-sm btn-outline-success">Reactivate</button></form>
                        } else {
                            <form asp-action="Deactivate" asp-route-id="@user.Nic" method="post" class="d-inline" onsubmit="return confirm('Deactivate this prosumer?');"><button class="btn btn-sm btn-outline-danger">Deactivate</button></form>
                        }
                    </td>
                </tr>
            </tr>
        }
    </tbody>
</table>
</div>
```

WebApp/SmartSolar.Web/Views/Prosumers/_Form.cshtml M1 · 26 lines

```
@model ProsumerForm
<div asp-validation-summary="ModelOnly" class="text-danger"></div>
<div class="row g-3">
    <div class="col-md-6">
        <label asp-for="Nic" class="form-label"></label>
        <input asp-for="Nic" class="form-control" readonly="@Model.IsEdit" />
        <span asp-validation-for="Nic" class="text-danger"></span>
    </div>
    <div class="col-md-6">
        <label asp-for="Username" class="form-label"></label>
        <input asp-for="Username" class="form-control" />
        <span asp-validation-for="Username" class="text-danger"></span>
    </div>
    <div class="col-12">
        <label asp-for="FullName" class="form-label"></label>
        <input asp-for="FullName" class="form-control" />
        <span asp-validation-for="FullName" class="text-danger"></span>
    </div>
    <div class="col-md-6"><label asp-for="Email" class="form-label"></label><input asp-for="Email" class="form-control" /><span asp-validation-for="Email" class="text-danger"></span></div>
    <div class="col-md-6"><label asp-for="Phone" class="form-label"></label><input asp-for="Phone" class="form-control" /><span asp-validation-for="Phone" class="text-danger"></span></div>
    <div class="col-12">
        <label asp-for="Password" class="form-label">@(Model.IsEdit ? "New password (leave blank to keep current)" : "Password")</label>
        <input asp-for="Password" class="form-control" />
        <span asp-validation-for="Password" class="text-danger"></span>
    </div>
</div>
```

WebApp/SmartSolar.Web/Views/Reservations/Cancel.cshtml M3 · 42 lines

```
@model ReservationCancelViewModel
@{
    ViewData["Title"] = "Cancel booking";
    var current = Model.Current!;
}

<div class="col-lg-7">
    <h1 class="h3 mb-3">Cancel this booking?</h1>

    <partial name="_ApiErrors" />
```

```

<div class="alert alert-info">
  Bookings can only be cancelled at least 12 hours before the slot starts. The slot becomes free for others straight away.
</div>

<div class="card shadow-sm mb-3">
  <div class="card-body">
    <dl class="row mb-0">
      <dt class="col-sm-4">Reference</dt>
      <dd class="col-sm-8 font-monospace small">@current.Id</dd>
      <dt class="col-sm-4">Prosumer</dt>
      <dd class="col-sm-8">@current.ProsumerNic</dd>
      <dt class="col-sm-4">Station</dt>
      <dd class="col-sm-8">@current.StationName</dd>
      <dt class="col-sm-4">Slot time</dt>
      <dd class="col-sm-8">@current.ScheduledAt.ToSriLankaText()</dd>
      <dt class="col-sm-4">Status</dt>
      <dd class="col-sm-8"><partial name="_ReservationStatusBadge" model="current.Status" /></dd>
    </dl>
  </div>
</div>

<form asp-action="Cancel" method="post">
  <input type="hidden" asp-for="Id" />
  <div class="mb-3">
    <label asp-for="Reason" class="form-label">Reason (optional)</label>
    <input asp-for="Reason" class="form-control" maxlength="200" placeholder="e.g. Prosumer called to cancel" />
  </div>
  <button type="submit" class="btn btn-danger">Cancel booking</button>
  <a class="btn btn-outline-secondary" asp-action="Index">Keep it</a>
</form>
</div>

```

WebApp/SmartSolar.Web/Views/Reservations/Create.cshtml M3 · 69 lines

```

@model ReservationCreateViewModel
@{
  ViewData["Title"] = "Book a slot";
}

<h1 class="h3 mb-1">Book a slot for a prosumer</h1>
<p class="text-muted mb-4">For prosumers who call or visit. The API checks the booking rules and tells you if something is not allowed.</p>

<partial name="_ApiErrors" />

<div class="row g-4">
  <div class="col-lg-5">
    <div class="card shadow-sm">
      <div class="card-header fw-semibold">1. Prosumer and station</div>
      <div class="card-body">
        <form asp-action="Create" method="get">
          <div class="mb-3">
            <label asp-for="ProsumerNic" class="form-label"></label>
            <input asp-for="ProsumerNic" class="form-control" placeholder="e.g. 200012345678" required />
          </div>
          <div class="mb-3">
            <label asp-for="StationId" class="form-label"></label>
            <select asp-for="StationId" class="form-select" required>
              <option value="">Choose a station</option>
              @foreach (var station in Model.Stations)
              {
                <option value="@station.Id" selected="@((station.Id == Model.StationId))">@station.Name</option>
              }
            </select>
          </div>
          <div class="mb-3">
            <label asp-for="Date" class="form-label"></label>
            <input asp-for="Date" type="date" class="form-control" />
            <div class="form-text">Optional. Leave empty to see every free slot you can book.</div>
          </div>
          <button type="submit" class="btn btn-outline-primary">Show free slots</button>
        </form>
      </div>
    </div>
  </div>
  <div class="col-lg-7">
    <div class="card shadow-sm">
      <div class="card-header fw-semibold">2. Pick a slot and confirm</div>
      <div class="card-body">
        @if (!Model.SlotsSearched)
        {
          <p class="text-muted mb-0">Enter the prosumer's NIC and choose a station to see its free slots.</p>
        }
        else if (Model.Slots.Count == 0)
        {
          <p class="text-muted mb-0">No free slots found. Try another date or station.</p>
        }
        else
        {
          <form asp-action="Create" method="post">
            <input type="hidden" asp-for="ProsumerNic" />
            <input type="hidden" asp-for="StationId" />
            <input type="hidden" asp-for="Date" />

            <partial name="_SlotPicker" model="(Model.Slots, Model.SlotId)" />
          </form>
        }
      </div>
    </div>
  </div>
</div>

```

```

        <button type="submit" class="btn btn-primary">Confirm booking</button>
    </form>
    }
</div>
</div>
</div>
</div>

```

WebApp/SmartSolar.Web/Views/Reservations/Edit.cshtml M3 · 79 lines

```

@model ReservationEditViewModel
@{
    ViewData["Title"] = "Change booking";
    var current = Model.Current!;
}

<h1 class="h3 mb-1">Change booking</h1>
<p class="text-muted mb-4">Changes need at least 12 hours notice before the booked slot. An approved booking goes back to pending and needs approval again.</p>

<partial name="_ApiErrors" />

<div class="row g-4">
    <div class="col-lg-5">
        <div class="card shadow-sm mb-4">
            <div class="card-header fw-semibold">Current booking</div>
            <div class="card-body">
                <dl class="row mb-0">
                    <dt class="col-sm-4">Reference</dt>
                    <dd class="col-sm-8 font-monospace small">@current.Id</dd>
                    <dt class="col-sm-4">Prosumer</dt>
                    <dd class="col-sm-8">@current.ProsumerNic</dd>
                    <dt class="col-sm-4">Station</dt>
                    <dd class="col-sm-8">@current.StationName</dd>
                    <dt class="col-sm-4">Slot time</dt>
                    <dd class="col-sm-8">@current.ScheduledAt.ToSriLankaText()</dd>
                    <dt class="col-sm-4">Status</dt>
                    <dd class="col-sm-8"><partial name="_ReservationStatusBadge" model="current.Status" /></dd>
                </dl>
            </div>
        </div>

        <div class="card shadow-sm">
            <div class="card-header fw-semibold">1. New station and date</div>
            <div class="card-body">
                <form asp-action="Edit" method="get">
                    <input type="hidden" asp-for="Id" />
                    <div class="mb-3">
                        <label asp-for="StationId" class="form-label">Station</label>
                        <select asp-for="StationId" class="form-select" required>
                            @foreach (var station in Model.Stations)
                            {
                                <option value="@station.Id" selected="@((station.Id == Model.StationId))">@station.Name</option>
                            }
                        </select>
                    </div>
                    <div class="mb-3">
                        <label asp-for="Date" class="form-label">Date</label>
                        <input asp-for="Date" type="date" class="form-control" />
                        <div class="form-text">Optional. Leave empty to see every free slot you can book.</div>
                    </div>
                    <button type="submit" class="btn btn-outline-primary">Show free slots</button>
                </form>
            </div>
        </div>
    </div>

    <div class="col-lg-7">
        <div class="card shadow-sm">
            <div class="card-header fw-semibold">2. Pick the new slot</div>
            <div class="card-body">
                @if (Model.Slots.Count == 0)
                {
                    <p class="text-muted mb-0">No free slots found. Try another date or station.</p>
                }
                else
                {
                    <form asp-action="Edit" method="post">
                        <input type="hidden" asp-for="Id" />
                        <input type="hidden" asp-for="StationId" />
                        <input type="hidden" asp-for="Date" />
                        <partial name="_SlotPicker" model="(Model.Slots, Model.SlotId)" />
                        <button type="submit" class="btn btn-primary">Save change</button>
                        <a class="btn btn-outline-secondary" asp-action="Index">Back</a>
                    </form>
                }
            </div>
        </div>
    </div>
</div>
</div>

```

WebApp/SmartSolar.Web/Views/Reservations/Index.cshtml M1 · 90 lines

```

@model ReservationIndexViewModel
@{

```

```

ViewData["Title"] = "Reservations";
string[] statuses = ["Pending", "Approved", "Cancelled", "Completed"];
}

<div class="d-flex justify-content-between align-items-center mb-3">
  <h1 class="h3 mb-0">Reservations</h1>
  <a class="btn btn-primary" asp-action="Create">Book a slot</a>
</div>

<form asp-action="Index" method="get" class="card card-body shadow-sm mb-3">
  <div class="row g-2 align-items-end">
    <div class="col-md-3">
      <label asp-for="Status" class="form-label small">Status</label>
      <select asp-for="Status" class="form-select">
        <option value="">All</option>
        @foreach (var status in statuses)
        {
          <option value="@status" selected="@((status == Model.Status))">@status</option>
        }
      </select>
    </div>
    <div class="col-md-3">
      <label asp-for="Nic" class="form-label small">Prosumer NIC</label>
      <input asp-for="Nic" class="form-control" />
    </div>
    <div class="col-md-3">
      <label asp-for="Q" class="form-label small">Search</label>
      <input asp-for="Q" class="form-control" placeholder="Station or reference" />
    </div>
    <div class="col-md-3 d-flex gap-3 align-items-center">
      <div class="form-check">
        <input asp-for="IncludePast" class="form-check-input" />
        <label asp-for="IncludePast" class="form-check-label">Include past</label>
      </div>
      <button type="submit" class="btn btn-outline-primary ms-auto">Filter</button>
    </div>
  </div>
</form>

<p class="text-muted small mb-2">@Model.Items.Count booking(s)@(Model.IncludePast ? "" : ", upcoming only")</p>

<div class="card">
  <table class="table table-hover align-middle mb-0 table-stack">
    <thead>
      <tr>
        <th>Slot time</th>
        <th>Station</th>
        <th>Prosumer NIC</th>
        <th>Status</th>
        <th>Reference</th>
      </tr>
    </thead>
    <tbody>
      @if (Model.Items.Count == 0)
      {
        <tr>
          <td colspan="6" class="text-muted">No bookings match these filters.</td>
        </tr>
      }
      @foreach (var r in Model.Items)
      {
        var open = r.Status is "Pending" or "Approved";
        <tr>
          <td data-label="When" class="text-nowrap">@r.ScheduledAt.ToSriLankaText()</td>
          <td data-label="Station">@r.StationName</td>
          <td data-label="NIC">@r.ProsumerNic</td>
          <td data-label="Status"><partial name="_ReservationStatusBadge" model="r.Status" /></td>
          <td data-label="Reference" class="font-monospace small text-muted">@r.Id</td>
          <td class="text-end text-nowrap">
            <a class="btn btn-sm btn-outline-secondary" asp-action="Summary" asp-route-id="@r.Id">View</a>
            @if (open)
            {
              <a class="btn btn-sm btn-outline-primary" asp-action="Edit" asp-route-id="@r.Id">Edit</a>
              <a class="btn btn-sm btn-outline-danger" asp-action="Cancel" asp-route-id="@r.Id">Cancel</a>
            }
            @if (r.Status == "Pending")
            {
              <form asp-action="Approve" asp-route-id="@r.Id" method="post" class="d-inline">
                <button type="submit" class="btn btn-sm btn-success">Approve</button>
              </form>
            }
          </td>
        </tr>
      }
    </tbody>
  </table>
</div>

```

WebApp/SmartSolar.Web/Views/Reservations/Summary.cshtml M3 · 67 lines

```

@model ReservationSummaryViewModel
@{
  var r = Model.Reservation;
  var (heading, colour) = Model.Done switch
  {
    "created" => ("Booking created", "success"),
    "updated" => ("Booking updated", "primary"),

```

```

        "cancelled" => ("Booking cancelled", "secondary"),
        "approved" => ("Booking approved", "success"),
        _ => ("Booking details", "dark")
    };
    ViewData["Title"] = heading;
}

<div class="row justify-content-center">
    <div class="col-lg-7">
        <div class="card shadow-sm border-@colour">
            <div class="card-header text-bg-@colour">
                <h1 class="h4 mb-0">@heading</h1>
            </div>
            <div class="card-body">
                <dl class="row mb-0">
                    <dt class="col-sm-4">Reference</dt>
                    <dd class="col-sm-8 font-monospace">@r.Id</dd>

                    <dt class="col-sm-4">Status</dt>
                    <dd class="col-sm-8"><partial name="_ReservationStatusBadge" model="r.Status" /></dd>

                    <dt class="col-sm-4">Prosumer NIC</dt>
                    <dd class="col-sm-8">@r.ProsumerNic</dd>

                    <dt class="col-sm-4">Station</dt>
                    <dd class="col-sm-8">@r.StationName</dd>

                    <dt class="col-sm-4">Slot time</dt>
                    <dd class="col-sm-8">@r.ScheduledAt.ToSriLankaText()</dd>

                    <dt class="col-sm-4">Last updated</dt>
                    <dd class="col-sm-8">@r.UpdatedAt.ToSriLankaText()</dd>

                    @if (!string.IsNullOrEmpty(r.QrToken))
                    {
                        <dt class="col-sm-4">QR token</dt>
                        <dd class="col-sm-8">
                            <code class="text-break">@r.QrToken</code>
                            <div class="form-text">The prosumer shows this as a QR code in the mobile app at the station.</div>
                        </dd>
                    }
                </dl>
            </div>
            <div class="card-footer bg-transparent">
                <a class="btn btn-outline-secondary" asp-action="Index">Back to reservations</a>
                @if (r.Status is "Pending" or "Approved")
                {
                    <a class="btn btn-outline-primary" asp-action="Edit" asp-route-id="@r.Id">Change</a>
                    <a class="btn btn-outline-danger" asp-action="Cancel" asp-route-id="@r.Id">Cancel booking</a>
                }
                @if (r.Status == "Pending")
                {
                    <form asp-action="Approve" asp-route-id="@r.Id" method="post" class="d-inline">
                        <button type="submit" class="btn btn-success">Approve</button>
                    </form>
                }
            </div>
        </div>
    </div>
</div>

```

WebApp/SmartSolar.Web/Views/Shared/Components/PendingActivationBadge/Default.cshtml M1 · 5 lines

```

@model IReadOnlyList<UserDto>
@if (Model.Count > 0)
{
    <span class="badge rounded-pill ms-1" style="background:#F5B700;color:#17211C">@Model.Count</span>
}

```

WebApp/SmartSolar.Web/Views/Shared/Error.cshtml M1 · 12 lines

```

@model ErrorViewModel
@{
    ViewData["Title"] = "Error";
}

<h1 class="h4">Something went wrong</h1>
<p>The page could not be shown.</p>
@if (Model.ShowRequestId)
{
    <p class="text-muted">Reference <code>@Model.RequestId</code></p>
}
<a class="btn btn-primary" asp-controller="Home" asp-action="Index">Go to dashboard</a>

```

WebApp/SmartSolar.Web/Views/Shared/_AccountStatus.cshtml M1 · 11 lines

```

@model string
@{
    var css = (Model ?? "").ToLowerInvariant() switch
    {
        "active" or "approved" or "available" => "badge-status-approved",
        "pending" => "badge-status-pending",
    }
}

```

```

        "completed" => "badge-status-completed",
        _ => "badge-status-muted"
    };
}
<span class="badge badge-status @css">@(string.IsNullOrEmpty(Model) ? "Unknown" : Model)</span>

```

WebApp/SmartSolar.Web/Views/Shared/_Alerts.cshtml M1 · 14 lines

```

@if (TempData["Success"] is string success)
{
    <div class="alert alert-success alert-dismissible fade show notice" role="alert">
        @success
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
}
@if (TempData["Error"] is string error)
{
    <div class="alert alert-danger alert-dismissible fade show notice" role="alert">
        @error
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
}

```

WebApp/SmartSolar.Web/Views/Shared/_ApiErrors.cshtml M3 · 12 lines

```

@{
    var errors = ViewData.ModelState.Values.SelectMany(v => v.Errors).Select(e => e.ErrorMessage).ToList();
}
@if (errors.Count > 0)
{
    <div class="alert alert-danger" role="alert">
        @foreach (var error in errors)
        {
            <div>@error</div>
        }
    </div>
}

```

WebApp/SmartSolar.Web/Views/Shared/_BrandPanel.cshtml M1 · 28 lines

```

<section class="brand-panel">
    <div>
        <div class="brand-kicker">Smart Solar · SE4040</div>
        <h1>The microgrid trading desk.</h1>
        <p class="brand-copy">Backoffice officers run accounts. Grid operators watch bookings. Prosumers stay on the Android app.</p>
        <ul class="role-list">
            <li>
                <span class="brand-mark" aria-hidden="true">
                    <svg viewBox="0 0 32 32" fill="none"><circle cx="16" cy="16" r="5.5" fill="#e39b16"/></svg>
                </span>
                <div>
                    <strong>Backoffice</strong>
                    <span>Staff users, prosumer records, and pending activations.</span>
                </div>
            </li>
            <li>
                <span class="brand-mark" aria-hidden="true">
                    <svg viewBox="0 0 32 32" fill="none"><path d="M8 22V10l8 4 4v12l-8 4-8-4z" stroke="#e39b16" stroke-width="2"/></svg>
                </span>
                <div>
                    <strong>Grid operator</strong>
                    <span>Look up prosumers and book a charging slot on their behalf.</span>
                </div>
            </li>
        </ul>
    </div>
    <p class="mb-0 small" style="color:#cfc3ae; position:relative; z-index:1;">Energy reservations live in the central API. This screen never opens the database.</p>
</section>

```

WebApp/SmartSolar.Web/Views/Shared/_Layout.cshtml M1 · 144 lines

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>@(ViewData["Title"] ?? "Smart Solar") · Smart Solar</title>
    <script type="importmap"></script>
    <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" />
    <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" />
    <link rel="stylesheet" href="~/SmartSolar.Web.styles.css" asp-append-version="true" />
</head>
<body class="d-flex flex-column min-vh-100">
    @{
        var currentController = ViewContext.RouteData.Values["controller"]?.ToString() ?? "";
        var currentAction = ViewContext.RouteData.Values["action"]?.ToString() ?? "";
        string Active(string controller, string? action = null)
        {
            if (!string.Equals(currentController, controller, StringComparison.OrdinalIgnoreCase)) return "";
            if (action != null && !string.Equals(currentAction, action, StringComparison.OrdinalIgnoreCase)) return "";
            return "active";
        }
    }
    <header>

```

```

<nav class="navbar navbar-expand-lg navbar-dark navbar-brand-bg">
  <div class="container-fluid px-3">
    <a class="navbar-brand fw-semibold" asp-controller="Home" asp-action="Index"><span class="sun-mark me-2"></span>Smart Solar</a>
    <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#mainNav"
      aria-controls="mainNav" aria-expanded="false" aria-label="Toggle navigation">
      <span class="navbar-toggler-icon"></span>
    </button>
    <div class="collapse navbar-collapse" id="mainNav">
      <ul class="navbar-nav me-auto">
        @if (User.IsInRole(RoleNames.Backoffice))
        {
          <li class="nav-item">
            <a class="nav-link @Active("Admin")" asp-controller="Admin" asp-action="Index">Dashboard</a>
          </li>
        }
        @if (User.IsInRole(RoleNames.GridOperator))
        {
          <li class="nav-item">
            <a class="nav-link @Active("Operations")" asp-controller="Operations" asp-action="Index">Dashboard</a>
          </li>
        }
        @if (User.IsInRole(RoleNames.Backoffice) || User.IsInRole(RoleNames.GridOperator))
        {
          var peopleActive = Active("Users") != "" || Active("Prosumers") != "" || Active("PendingActivations") != "" ? "active" : "";
          var bookingsActive = Active("Bookings") != "" || Active("Reservations") != "" ? "active" : "";
          <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle @peopleActive" href="#" role="button" data-bs-toggle="dropdown" aria-expanded="false">
              People
              @if (User.IsInRole(RoleNames.Backoffice))
              {
                @await Component.InvokeAsync("PendingActivationBadge")
              }
            </a>
            <ul class="dropdown-menu">
              @if (User.IsInRole(RoleNames.Backoffice))
              {
                <li><a class="dropdown-item" asp-controller="Users" asp-action="Index">Staff users</a></li>
              }
              <li><a class="dropdown-item" asp-controller="Prosumers" asp-action="Index">Prosumers</a></li>
              @if (User.IsInRole(RoleNames.Backoffice))
              {
                <li>
                  <a class="dropdown-item" asp-controller="PendingActivations" asp-action="Index">
                    Pending activations
                    @await Component.InvokeAsync("PendingActivationBadge")
                  </a>
                </li>
              }
            </ul>
          </li>
          var stationsActive = Active("Stations") != "" || Active("Slots") != "" ? "active" : "";
          <li class="nav-item">
            <a class="nav-link @stationsActive" asp-controller="Stations" asp-action="Index">Stations</a>
          </li>
          <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle @bookingsActive" href="#" role="button" data-bs-toggle="dropdown" aria-expanded="false">Bookings</a>
            <ul class="dropdown-menu">
              <li><a class="dropdown-item" asp-controller="Bookings" asp-action="Index">Booking monitor</a></li>
              <li><a class="dropdown-item" asp-controller="Reservations" asp-action="Index">Manage reservations</a></li>
              <li><a class="dropdown-item" asp-controller="Reservations" asp-action="Create">Book for a prosumer</a></li>
            </ul>
          </li>
        }
      </ul>
      <ul class="navbar-nav">
        @if (User.Identity?.IsAuthenticated == true)
        {
          var roleLabel = User.IsInRole(RoleNames.Backoffice) ? "Backoffice" : "Grid operator";
          <li class="nav-item dropdown">
            <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-toggle="dropdown" aria-expanded="false">@User.Identity.Name ·
            @roleLabel</a>
            <ul class="dropdown-menu dropdown-menu-end">
              <li><span class="dropdown-item-text small text-muted">Signed in as @User.Identity.Name</span></li>
              <li>
                <form asp-controller="Account" asp-action="Logout" method="post" class="px-3 py-1">
                  <button type="submit" class="btn btn-link p-0">Log out</button>
                </form>
              </li>
            </ul>
          </li>
        }
        else
        {
          <li class="nav-item">
            <a class="nav-link" asp-controller="Account" asp-action="Login">Log in</a>
          </li>
        }
      </ul>
    </div>
  </div>
</nav>
</header>

<main class="container flex-grow-1 py-4">
  @if (TempData["Success"] is string success)
  {
    <div class="alert alert-success alert-dismissible fade show" role="alert">
      @success
      <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
    </div>
  }
  @if (TempData["Error"] is string error)

```

```

    {
      <div class="alert alert-danger alert-dismissible fade show" role="alert">
        @error
        <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
      </div>
    }
    @RenderBody()
  </main>

  <footer class="border-top py-3 mt-auto bg-light">
    <div class="container small text-muted">
      Smart Solar Microgrid Trading System · SE4040
    </div>
  </footer>
  <script src="~/lib/jquery/dist/jquery.min.js"></script>
  <script src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"></script>
  <script src="~/js/site.js" asp-append-version="true"></script>
  @await RenderSectionAsync("Scripts", required: false)
</body>
</html>

```

WebApp/SmartSolar.Web/Views/Shared/_Layout.cshtml.css M1 · 1 lines

/* Shared colours live in wwwroot/css/site.css. */

WebApp/SmartSolar.Web/Views/Shared/_OperationsDashboard.cshtml M1 · 77 lines

@model OperationsDashboardDto

```

<div class="row g-3 mb-4">
  <div class="col-6 col-lg-3">
    <div class="card h-100 border-0 shadow-sm">
      <div class="card-body">
        <div class="text-muted small">Today's bookings</div>
        <div class="tile-number">@Model.TodaysBookingCount</div>
      </div>
    </div>
  </div>
  <div class="col-6 col-lg-3">
    <a class="card h-100 text-decoration-none text-body @(Model.PendingApprovals > 0 ? "tile-attention" : "")" asp-controller="Bookings" asp-action="Index" asp-route-tab="pending">
      <div class="card-body">
        <div class="small text-muted">Pending approvals</div>
        <div class="tile-number">@Model.PendingApprovals</div>
        @if (Model.PendingApprovals > 0)
        {
          <div class="small">Open queue</div>
        }
      </div>
    </a>
  </div>
  <div class="col-6 col-lg-3">
    <div class="card h-100 border-0 shadow-sm">
      <div class="card-body">
        <div class="text-muted small">Active stations</div>
        <div class="tile-number">@Model.ActiveStations</div>
      </div>
    </div>
  </div>
  <div class="col-6 col-lg-3">
    <div class="card h-100 border-0 shadow-sm">
      <div class="card-body">
        <div class="text-muted small">Completed today</div>
        <div class="tile-number">@Model.CompletedToday</div>
      </div>
    </div>
  </div>
</div>

<div class="d-flex justify-content-between align-items-center mb-2">
  <h2 class="h5 mb-0">Today's bookings</h2>
  <a class="btn btn-sm btn-outline-primary" asp-controller="Bookings" asp-action="Index" asp-route-tab="pending">Approval queue</a>
</div>
<div class="table-responsive mb-4">
  <table class="table table-striped align-middle">
    <thead>
      <tr>
        <th>When</th>
        <th>Station</th>
        <th>NIC</th>
        <th>Status</th>
      </tr>
    </thead>
    <tbody>
      @if (Model.TodaysBookings.Count == 0)
      {
        <tr>
          <td colspan="4">
            <div class="text-muted mb-2">Approved bookings for today appear here.</div>
            <a class="btn btn-sm btn-outline-primary" asp-controller="Bookings" asp-action="Index">Open booking monitor</a>
          </td>
        </tr>
      }
      @foreach (var booking in Model.TodaysBookings)
      {
        <tr>
          <td>@booking.ScheduledAt.ToSriLankaText()</td>

```

```

        <td>@booking.StationName</td>
        <td>@booking.ProsumerNic</td>
        <td><partial name="_ReservationStatusBadge" model="booking.Status" /></td>
    </tr>
}
</tbody>
</table>
</div>

```

WebApp/SmartSolar.Web/Views/Shared/_ReservationStatusBadge.cshtml M1 · 11 lines

```

@model string
@{
    var css = Model switch
    {
        "Pending" => "badge-status-pending",
        "Approved" or "Active" or "Available" => "badge-status-approved",
        "Completed" => "badge-status-completed",
        _ => "badge-status-muted"
    };
}
<span class="badge badge-status @css">@Model</span>

```

WebApp/SmartSolar.Web/Views/Shared/_SlotPicker.cshtml M3 · 17 lines

```

@model (IReadOnlyList<AvailableSlotDto> Slots, string? Selected)

@foreach (var day in Model.Slots.GroupBy(s => s.StartTime.ToSriLanka().Date))
{
    <div class="mb-3">
        <div class="small fw-semibold text-muted mb-2">@day.Key.ToString("dddd, dd MMMM")</div>
        <div class="d-flex flex-wrap gap-2">
            @foreach (var slot in day)
            {
                <input type="radio" class="btn-check" name="SlotId" id="slot-@slot.Id" value="@slot.Id" autocomplete="off" checked="@ (slot.Id == Model.Selected)" required
            />
                <label class="btn btn-outline-success" for="slot-@slot.Id">
                    @slot.StartTime.ToSriLanka().ToString("HH:mm") – @slot.EndTime.ToSriLanka().ToString("HH:mm")
                </label>
            }
        </div>
    </div>
}

```

WebApp/SmartSolar.Web/Views/Shared/_ValidationScriptsPartial.cshtml M1 · 2 lines

```

<script src="~/lib/jquery-validation/dist/jquery.validate.min.js"></script>
<script src="~/lib/jquery-validation-unobtrusive/dist/jquery.validate.unobtrusive.min.js"></script>

```

WebApp/SmartSolar.Web/Views/Slots/Index.cshtml M2 · 74 lines

```

@model SlotsPage
@{
    ViewData["Title"] = Model.Station.Name + " slots";
    var isOperator = User.IsInRole(RoleNames.GridOperator);
}
<div class="d-flex justify-content-between align-items-center mb-3">
    <h1 class="h3 mb-0">@Model.Station.Name</h1>
    <a asp-controller="Stations" asp-action="Index" class="btn btn-outline-secondary">Back to stations</a>
</div>
<p class="text-muted">Times are shown in Sri Lanka time. The API decides whether a slot fits the schedule.</p>
<div class="table-responsive card shadow-sm mb-4">
<table class="table table-hover align-middle mb-0">
    <thead>
        <tr>
            <th>Start</th>
            <th>End</th>
            <th>Status</th>
            <th class="text-end">Actions</th>
        </tr>
    </thead>
    <tbody>
        @if (Model.Slots.Count == 0)
        {
            <tr><td colspan="4" class="text-center text-muted py-4">No slots yet.</td></tr>
        }
        @foreach (var slot in Model.Slots)
        {
            <tr>
                <td>@slot.StartTime.ToSriLankaText()</td>
                <td>@slot.EndTime.ToSriLankaText()</td>
                <td>
                    <span class="badge @(slot.IsAvailable ? "bg-success" : "bg-secondary")">@(slot.IsAvailable ? "Available" : "Booked")</span>
                </td>
                <td class="text-end text-nowrap">
                    @if (isOperator)
                    {
                        <form asp-action="Availability" method="post" class="d-inline">
                            <input type="hidden" name="id" value="@slot.Id" />
                            <input type="hidden" name="stationId" value="@Model.Station.Id" />
                            <input type="hidden" name="isAvailable" value="@(!slot.IsAvailable ? "true" : "false")" />
                            <button class="btn btn-sm btn-outline-secondary" type="submit">@(slot.IsAvailable ? "Close" : "Open")</button>
                        </form>
                    }
                </td>
            </tr>
        }
    </tbody>
</table>

```

```

    }
    <form asp-action="Delete" method="post" class="d-inline" onsubmit="return confirm('Delete this slot?');">
      <input type="hidden" name="id" value="@slot.Id" />
      <input type="hidden" name="stationId" value="@Model.Station.Id" />
      <button class="btn btn-sm btn-outline-danger" type="submit">Delete</button>
    </form>
  </td>
</tr>
}
</tbody>
</table>
</div>
<form asp-action="Create" asp-route-stationId="@Model.Station.Id" method="post" class="card card-body shadow-sm">
  <h2 class="h6">Add a slot</h2>
  <div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>
  <div class="row g-3">
    <div class="col-md-5">
      <label asp-for="Form.Start" class="form-label"></label>
      <input asp-for="Form.Start" class="form-control" type="datetime-local" />
      <span asp-validation-for="Form.Start" class="text-danger"></span>
    </div>
    <div class="col-md-5">
      <label asp-for="Form.End" class="form-label"></label>
      <input asp-for="Form.End" class="form-control" type="datetime-local" />
      <span asp-validation-for="Form.End" class="text-danger"></span>
    </div>
    <div class="col-md-2 d-flex align-items-end">
      <button class="btn btn-primary w-100" type="submit">Add slot</button>
    </div>
  </div>
</form>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

WebApp/SmartSolar.Web/Views/Stations/Create.cshtml M2 · 12 lines

```

@model StationForm
@{ ViewData["Title"] = "Create station"; }
<div class="d-flex justify-content-between align-items-center mb-3">
  <h1 class="h3 mb-0">Create station</h1>
  <a asp-action="Index" class="btn btn-outline-secondary">Back</a>
</div>
<p class="text-muted">New stations are active immediately. The API checks the coordinates, capacity and hours.</p>
<form method="post" class="card card-body shadow-sm">
  <partial name="Form" model="Model" />
  <div class="mt-4"><button class="btn btn-primary" type="submit">Create station</button></div>
</form>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

WebApp/SmartSolar.Web/Views/Stations/Edit.cshtml M2 · 11 lines

```

@model StationForm
@{ ViewData["Title"] = "Edit station"; }
<div class="d-flex justify-content-between align-items-center mb-3">
  <h1 class="h3 mb-0">Edit station</h1>
  <a asp-action="Index" class="btn btn-outline-secondary">Back</a>
</div>
<form method="post" class="card card-body shadow-sm">
  <partial name="Form" model="Model" />
  <div class="mt-4"><button class="btn btn-primary" type="submit">Save changes</button></div>
</form>
@section Scripts { <partial name="_ValidationScriptsPartial" /> }

```

WebApp/SmartSolar.Web/Views/Stations/Index.cshtml M2 · 68 lines

```

@model IReadOnlyList<StationDto>
@{
  ViewData["Title"] = "Stations";
  var isBackoffice = User.IsInRole(RoleNames.Backoffice);
}
<div class="d-flex justify-content-between align-items-center mb-3">
  <h1 class="h3 mb-0">Stations</h1>
  @if (isBackoffice)
  {
    <a asp-action="Create" class="btn btn-primary">Create station</a>
  }
</div>
<div class="card">
<table class="table table-hover align-middle mb-0 table-stack">
  <thead>
    <tr>
      <th>Name</th>
      <th>GPS</th>
      <th>Capacity (kWh)</th>
      <th>Battery slots</th>
      <th>Status</th>
      <th class="text-end">Actions</th>
    </tr>
  </thead>
  <tbody>
    @if (Model.Count == 0)
    {
      <tr><td colspan="6" class="text-center text-muted py-4">No stations found.</td></tr>
    }
    @foreach (var station in Model)
    {
      <tr>

```

```

<td data-label="Name">@station.Name</td>
<td data-label="Location" class="font-monospace">@station.Latitude.ToString("0.0000"), @station.Longitude.ToString("0.0000")</td>
<td data-label="Capacity">@station.CapacityKwh</td>
<td data-label="Batteries">@station.BatterySlotsTotal</td>
<td data-label="Status"><partial name="_AccountStatus" model="station.Status" /></td>
<td class="text-end text-nowrap">
  <a asp-controller="Slots" asp-action="Index" asp-route-stationId="@station.Id" class="btn btn-sm btn-outline-primary">Slots</a>
  @if (isBackoffice)
  {
    <div class="dropdown d-inline">
      <button class="btn btn-sm btn-outline-secondary" type="button" data-bs-toggle="dropdown" aria-label="More actions">...</button>
      <ul class="dropdown-menu dropdown-menu-end">
        <li><a class="dropdown-item" asp-action="Edit" asp-route-id="@station.Id">Edit</a></li>
        <li>
          @if (station.Status == "Active")
          {
            <form asp-action="Deactivate" asp-route-id="@station.Id" method="post" onsubmit="return confirm('Deactivate this station?');">
              <button class="dropdown-item text-danger" type="submit">Deactivate</button>
            </form>
          }
          else
          {
            <form asp-action="Activate" asp-route-id="@station.Id" method="post">
              <button class="dropdown-item" type="submit">Activate</button>
            </form>
          }
        </li>
      </ul>
    </div>
  }
</td>
</tr>
</tbody>
</table>
</div>

```

WebApp/SmartSolar.Web/Views/Stations/_Form.cshtml M2 · 46 lines

```

@model StationForm
<div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>
<div class="row g-3">
  <div class="col-12">
    <label asp-for="Name" class="form-label"></label>
    <input asp-for="Name" class="form-control" />
    <span asp-validation-for="Name" class="text-danger"></span>
  </div>
  <div class="col-md-6">
    <label asp-for="Latitude" class="form-label"></label>
    <input asp-for="Latitude" class="form-control" step="any" />
  </div>
  <div class="col-md-6">
    <label asp-for="Longitude" class="form-label"></label>
    <input asp-for="Longitude" class="form-control" step="any" />
  </div>
  <div class="col-md-6">
    <label asp-for="CapacityKwh" class="form-label"></label>
    <input asp-for="CapacityKwh" class="form-control" step="any" />
  </div>
  <div class="col-md-6">
    <label asp-for="BatterySlotsTotal" class="form-label"></label>
    <input asp-for="BatterySlotsTotal" class="form-control" />
  </div>
</div>
<h2 class="h6 mt-4">Weekly schedule</h2>
<p class="text-muted small">Tick a day to include it. Closed days are not sent.</p>
<div class="table-responsive">
  <table class="table table-sm align-middle">
    <thead><tr><th>Day</th><th>Open</th><th>Opens</th><th>Closes</th></tr></thead>
    <tbody>
      @for (var i = 0; i < Model.Days.Count; i++)
      {
        <tr>
          <td>
            @Model.Days[i].DayOfWeek
            <input type="hidden" asp-for="Days[i].DayOfWeek" />
          </td>
          <td><input asp-for="Days[i].Open" class="form-check-input" /></td>
          <td><input asp-for="Days[i].OpenTime" class="form-control" type="time" /></td>
          <td><input asp-for="Days[i].CloseTime" class="form-control" type="time" /></td>
        </tr>
      }
    </tbody>
  </table>
</div>

```

WebApp/SmartSolar.Web/Views/Users/Create.cshtml M1 · 11 lines

```

@model StaffUserForm
@{
  ViewData["Title"] = "New staff account";
}

<h1 class="h3 mb-3">New staff account</h1>
<partial name="_Form" model="Model" />

@section Scripts {

```

```
    <partial name="_ValidationScriptsPartial" />
}
```

WebApp/SmartSolar.Web/Views/Users/Deactivate.cshtml M1 · 24 lines

```
@model UserDto
@{
    ViewData["Title"] = "Deactivate user";
}

<div class="col-lg-6">
    <h1 class="h3 mb-3">Deactivate @Model.Username?</h1>
    <p>
        This sets status to <strong>Deactivated</strong> in the API. The account will no longer be able to log in.
        You cannot deactivate your own account.
    </p>
    <dl class="row">
        <dt class="col-sm-3">Name</dt>
        <dd class="col-sm-9">@Model.FullName</dd>
        <dt class="col-sm-3">Role</dt>
        <dd class="col-sm-9">@Model.Role</dd>
        <dt class="col-sm-3">Status</dt>
        <dd class="col-sm-9">@Model.Status</dd>
    </dl>
    <form asp-action="Deactivate" asp-route-id="@Model.Id" method="post">
        <button type="submit" class="btn btn-danger">Deactivate</button>
        <a class="btn btn-outline-secondary" asp-action="Index">Cancel</a>
    </form>
</div>
```

WebApp/SmartSolar.Web/Views/Users/Edit.cshtml M1 · 11 lines

```
@model StaffUserForm
@{
    ViewData["Title"] = "Edit staff account";
}

<h1 class="h3 mb-3">Edit @Model.Username</h1>
<partial name="_Form" model="Model" />

@section Scripts {
    <partial name="_ValidationScriptsPartial" />
}
```

WebApp/SmartSolar.Web/Views/Users/Index.cshtml M1 · 52 lines

```
@model IReadOnlyList<UserDto>
@{
    ViewData["Title"] = "Staff users";
}

<div class="d-flex justify-content-between align-items-center mb-3">
    <div>
        <h1 class="h4 mb-1">Staff users</h1>
        <p class="text-muted mb-0">Backoffice and Grid Operator accounts.</p>
    </div>
    <a class="btn btn-primary" asp-action="Create">New staff account</a>
</div>

<div class="card">
    <table class="table table-hover align-middle mb-0 table-stack">
        <thead>
            <tr>
                <th>Username</th>
                <th>Name</th>
                <th>Role</th>
                <th>Status</th>
                <th>Email</th>
                <th></th>
            </tr>
        </thead>
        <tbody>
            @if (Model.Count == 0)
            {
                <tr>
                    <td colspan="6" class="text-muted">No staff accounts yet. New accounts appear here.</td>
                </tr>
            }
            @foreach (var user in Model)
            {
                <tr>
                    <td data-label="Username">@user.Username</td>
                    <td data-label="Name">@user.FullName</td>
                    <td data-label="Role"><partial name="_AccountStatus" model="user.Role" /></td>
                    <td data-label="Status"><partial name="_AccountStatus" model="user.Status" /></td>
                    <td data-label="Email">@user.Email</td>
                    <td class="text-end text-nowrap">
                        <a class="btn btn-sm btn-outline-primary" asp-action="Edit" asp-route-id="@user.Id">Edit</a>
                        @if (user.Status != "Deactivated")
                        {
                            <a class="btn btn-sm btn-outline-danger" asp-action="Deactivate" asp-route-id="@user.Id">Deactivate</a>
                        }
                    </td>
                </tr>
            }
        </tbody>
    </table>
```

```
        </tr>
    }
</tbody>
</table>
</div>
```

WebApp/SmartSolar.Web/Views/Users/_Form.cshtml M1 · 60 lines

```
@model StaffUserForm
@{
    var action = Model.IsEdit ? "Edit" : "Create";
}

<form asp-action="@action" asp-route-id="@Model.Id" method="post" class="col-lg-6">
    <div asp-validation-summary="ModelOnly" class="text-danger mb-3"></div>
    <input type="hidden" asp-for="IsEdit" />
    <input type="hidden" asp-for="Id" />

    <div class="mb-3">
        <label asp-for="Username" class="form-label"></label>
        @if (Model.IsEdit)
        {
            <p class="form-control-plaintext mb-0">@Model.Username</p>
        }
        else
        {
            <input asp-for="Username" class="form-control" autocomplete="username" />
            <span asp-validation-for="Username" class="text-danger"></span>
        }
    </div>

    <div class="mb-3">
        <label asp-for="Password" class="form-label"></label>
        <input asp-for="Password" class="form-control" autocomplete="new-password" />
        <div class="form-text">@(Model.IsEdit ? "Leave blank to keep the current password." : "Required. Hashed by the API, never stored here.")</div>
        <span asp-validation-for="Password" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="FullName" class="form-label"></label>
        <input asp-for="FullName" class="form-control" />
        <span asp-validation-for="FullName" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Email" class="form-label"></label>
        <input asp-for="Email" class="form-control" />
        <span asp-validation-for="Email" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Phone" class="form-label"></label>
        <input asp-for="Phone" class="form-control" />
        <span asp-validation-for="Phone" class="text-danger"></span>
    </div>

    <div class="mb-3">
        <label asp-for="Role" class="form-label"></label>
        <select asp-for="Role" class="form-select">
            <option value="@RoleNames.Backoffice">@RoleNames.Backoffice</option>
            <option value="@RoleNames.GridOperator">@RoleNames.GridOperator</option>
        </select>
        <span asp-validation-for="Role" class="text-danger"></span>
    </div>

    <button type="submit" class="btn btn-primary">Save</button>
    <a class="btn btn-outline-secondary" asp-action="Index">Cancel</a>
</form>
```

WebApp/SmartSolar.Web/Views/_ViewImports.cshtml M1 · 4 lines

```
@using SmartSolar.Web
@using SmartSolar.Web.Common
@using SmartSolar.Web.Models
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

WebApp/SmartSolar.Web/Views/_ViewStart.cshtml M1 · 3 lines

```
@{
    Layout = "_Layout";
}
```

WebApp/SmartSolar.Web/appsettings.json M1 · 12 lines

```
{
  "Api": {
    "BaseUrl": "http://localhost:5080/api/"
  },
  "Logging": {
```

```

"LogLevel": {
  "Default": "Information",
  "Microsoft.AspNetCore": "Warning"
},
"AllowedHosts": "*"
}

```

WebApp/SmartSolar.Web/wwwroot/css/site.css M1 · 147 lines

```

:root {
  --bs-primary: #176b45;
  --bs-primary-rgb: 23, 107, 69;
  --bs-link-color: #176b45;
  --bs-link-hover-color: #0d4c31;
  --bs-body-bg: #f4f7f5;
  --bs-body-color: #17211c;
  --bs-secondary-color: #5b6b62;
  --bs-border-color: #dde5e0;
  --bs-border-radius: 10px;
  --bs-focus-ring-color: rgba(23, 107, 69, 0.35);
  --brand-dark: #0d4c31;
  --sun: #f5b700;
  --ink-2: #5b6b62;
  --danger: #b42318;
}

html { font-size: 14px; }
@media (min-width: 768px) { html { font-size: 16px; } }

body {
  background: var(--bs-body-bg);
  color: var(--bs-body-color);
}

.navbar.navbar-brand-bg {
  background: var(--brand-dark);
}

.navbar .nav-link {
  border-radius: 0;
  margin: 0 0.1rem;
}

.navbar .nav-link.active {
  color: #fff;
  box-shadow: inset 0 -2px 0 var(--sun);
}

.navbar .nav-link:hover { color: #fff; }

.dropdown-menu { border-color: var(--bs-border-color); }

main h1 { font-size: 1.5rem; font-weight: 600; color: var(--brand-dark); }
main h2 { font-weight: 600; }

.card {
  border: 1px solid var(--bs-border-color);
  box-shadow: none;
  border-radius: 10px;
}

.btn-primary {
  --bs-btn-bg: #176b45;
  --bs-btn-border-color: #176b45;
  --bs-btn-hover-bg: #0d4c31;
  --bs-btn-hover-border-color: #0d4c31;
  --bs-btn-active-bg: #0d4c31;
  --bs-btn-active-border-color: #0d4c31;
  --bs-btn-disabled-bg: #176b45;
  --bs-btn-disabled-border-color: #176b45;
}

.btn-outline-primary {
  --bs-btn-color: #176b45;
  --bs-btn-border-color: #176b45;
  --bs-btn-hover-bg: #176b45;
  --bs-btn-hover-border-color: #176b45;
  --bs-btn-active-bg: #0d4c31;
  --bs-btn-active-border-color: #0d4c31;
}

.btn-outline-danger {
  --bs-btn-color: var(--danger);
  --bs-btn-border-color: var(--danger);
  --bs-btn-hover-bg: var(--danger);
  --bs-btn-hover-border-color: var(--danger);
}

.form-control, .form-select { border-radius: 10px; }
.text-body-secondary, .form-text, .text-muted { color: var(--ink-2) !important; }

.auth-card { max-width: 400px; margin-inline: auto; }

```

```

.sun-mark {
  width: 0.7rem;
  height: 0.7rem;
  border-radius: 50%;
  background: var(--sun);
  display: inline-block;
}

.badge-status {
  font-weight: 600;
  border: 1px solid transparent;
}

.badge-status-pending { color: #8a6200; background: #fff4d6; border-color: #ebcb7a; }
.badge-status-approved { color: #1e7d4f; background: #e3f4ea; border-color: #a9d8bc; }
.badge-status-completed { color: #0b5ed7; background: #e7f0ff; border-color: #afcbfa; }
.badge-status-muted { color: #55606a; background: #eef0f2; border-color: #cdd3d8; }

.tile-attention {
  background: #fff4d6;
  border-color: #ebcb7a;
}

.tile-number { font-size: 1.75rem; font-weight: 700; font-variant-numeric: tabular-nums; }

.nav-pills {
  --bs-nav-pills-link-active-color: #fff;
  --bs-nav-pills-link-active-bg: #176b45;
}
.nav-pills .nav-link { color: #176b45; }

@media (min-width: 768px) {
  #bookingFilters.collapse { display: block; height: auto; visibility: visible; }
}

@media (max-width: 767.98px) {
  .table-stack thead { display: none; }
  .table-stack,
  .table-stack tbody { display: block; width: 100%; }
  .table-stack tr {
    display: block;
    border: 1px solid var(--bs-border-color);
    border-radius: 10px;
    padding: 12px;
    margin-bottom: 8px;
    background: #fff;
  }
  .table-stack td {
    display: block;
    border: 0;
    padding: 0.15rem 0;
    text-align: left !important;
    white-space: normal;
  }
  .table-stack td:first-child { font-weight: 600; }
  .table-stack td[data-label]:not([data-label=""]):before {
    content: attr(data-label);
    display: block;
    font-size: 0.75rem;
    font-weight: 500;
    color: var(--ink-2);
  }
}

```

WebApp/SmartSolar.Web/wwwroot/js/site.js M1 · 4 lines

// Please see documentation at <https://learn.microsoft.com/aspnet/core/client-side/bundling-and-minification>
 // for details on configuring this project to bundle and minify static web assets.

// Write your JavaScript code.

A.3 Android application (MobileApp)

MobileApp/app/build.gradle M2 · 40 lines

```

plugins {
  id 'com.android.application'
}

def localProps = new Properties()
def localFile = rootProject.file('local.properties')
if (localFile.exists()) localFile.withInputStream { localProps.load(it) }

android {
  namespace 'lk.smartsolar.mobile'
  compileSdk 35

  defaultConfig {
    applicationId 'lk.smartsolar.mobile'
  }
}

```

```

        minSdk 24
        targetSdk 35
        versionCode 1
        versionName '1.0'
        buildConfigField 'String', 'API_BASE_URL', "'http://10.0.2.2:5080/api'"
        manifestPlaceholders = [MAPS_API_KEY: localProps.getProperty('MAPS_API_KEY', "")]
    }

    buildFeatures {
        buildConfig true
    }

    compileOptions {
        sourceCompatibility JavaVersion.VERSION_17
        targetCompatibility JavaVersion.VERSION_17
    }
}

dependencies {
    implementation 'androidx.appcompat:appcompat:1.7.0'
    implementation 'com.google.android.material:material:1.12.0'
    implementation 'androidx.swiperefreshlayout:swiperefreshlayout:1.1.0'
    implementation 'com.journeyapps:zxing-android-embedded:4.3.0'
    implementation 'com.google.android.gms:play-services-maps:19.0.0'
    implementation 'com.google.android.gms:play-services-location:21.3.0'
}

```

MobileApp/app/src/main/AndroidManifest.xml M2 · 43 lines

```

<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />

    <application
        android:allowBackup="true"
        android:label="Smart Solar"
        android:supportRtl="true"
        android:theme="@style/Theme.SmartSolar"
        android:usesCleartextTraffic="true">
        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="{MAPS_API_KEY}" />
        <activity android:name=".ui.stations.StationsMapActivity" />
        <activity android:name=".ui.stations.StationListActivity" />
        <activity android:name=".ui.operator.QrDetailsActivity" />
        <activity android:name=".ui.operator.PendingQueueActivity" />
        <activity android:name=".ui.bookings.BookingsActivity" />
        <activity android:name=".ui.settings.SettingsActivity" />
        <activity android:name=".ui.reservations.BookSlotActivity" />
        <activity android:name=".ui.reservations.BookingSummaryActivity" />
        <activity android:name=".ui.reservations.BookingDetailActivity" />
        <activity android:name=".ui.reservations.BookingQrActivity" />
        <activity android:name=".ui.auth.LogoutActivity" />
        <activity android:name=".ui.account.DeactivationActivity" />
        <activity android:name=".ui.account.ProfileActivity" />
        <activity android:name=".ui.home.OperatorHomeActivity" />
        <activity android:name=".ui.home.ProsumerHomeActivity" />
        <activity android:name=".ui.auth.RegisterActivity" />
        <activity android:name=".ui.auth.LoginActivity" />
        <activity
            android:name=".ui.auth.SplashActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/local/Booking.java M4 · 29 lines

```

/*
 * File: Booking.java
 * Description: One reservation row used by the bookings list and the offline cache.
 * Author: samudith
 * Created: 29/09/2026
 */
package lk.smartsolar.mobile.data.local;

import org.json.JSONObject;

public class Booking {
    public String id = "";
    public String prosumerNic = "";
    public String stationName = "";
    public String scheduledAt = "";
    public String status = "";
    public long syncedAt;

    // Reads the contract reservation fields used on the bookings screen.
    public static Booking fromJson(JSONObject json) {

```

```

        Booking booking = new Booking();
        booking.id = json.optString("id", "");
        booking.prosumerNic = json.optString("prosumerNic", "");
        booking.stationName = json.optString("stationName", "");
        booking.scheduledAt = json.optString("scheduledAt", "");
        booking.status = json.optString("status", "");
        return booking;
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/local/DashboardSnapshot.java M4 · 33 lines

```

/*
 * File: DashboardSnapshot.java
 * Description: Prosumer dashboard counts, including the copy saved for offline display.
 * Author: samudith
 * Created: 29/09/2026
 */
package lk.smartsolar.mobile.data.local;

import org.json.JSONObject;

public class DashboardSnapshot {
    public int pendingCount;
    public int approvedFutureCount;
    public int activeCount;
    public String nextStation = "";
    public String nextWhen = "";
    public long syncedAt;

    // Reads GET /dashboard/me and stamps the sync time.
    public static DashboardSnapshot fromJson(JSONObject json) {
        DashboardSnapshot snapshot = new DashboardSnapshot();
        snapshot.pendingCount = json.optInt("pendingCount");
        snapshot.approvedFutureCount = json.optInt("approvedFutureCount");
        snapshot.activeCount = json.optInt("activeCount");
        JSONObject next = json.optJSONObject("nextReservation");
        if (next != null) {
            snapshot.nextStation = next.optString("stationName", "");
            snapshot.nextWhen = next.optString("scheduledAt", "");
        }
        snapshot.syncedAt = System.currentTimeMillis();
        return snapshot;
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/local/DbHelper.java M1 · 194 lines

```

/*
 * File: DbHelper.java
 * Description: Single SQLite database for sessions and module-owned offline tables.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.data.local;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;

import java.util.ArrayList;
import java.util.List;

public class DbHelper extends SQLiteOpenHelper {
    private static final String DB_NAME = "smart_solar.db";
    // Every module adds its CREATE statement and increments this version.
    private static final int DB_VERSION = 4;

    public DbHelper(Context context) {
        super(context.getApplicationContext(), DB_NAME, null, DB_VERSION);
    }

    @Override
    public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE session (token TEXT NOT NULL, expires_at INTEGER NOT NULL, role TEXT NOT NULL, user_id TEXT NOT NULL, nic TEXT)");
        db.execSQL("CREATE TABLE user_profile (nic TEXT PRIMARY KEY, username TEXT NOT NULL, full_name TEXT NOT NULL, email TEXT NOT NULL, phone TEXT NOT NULL, status TEXT NOT NULL, synced_at INTEGER NOT NULL)");
        createBookingCache(db);
        createMyReservations(db);
        createStationsCache(db);
    }

    @Override
    public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        // Additive migrations belong here. Never drop other modules' offline data.
        if (oldVersion < 2) {
            createBookingCache(db);
        }
        if (oldVersion < 3) {
            createMyReservations(db);
        }
        if (oldVersion < 4) {

```

```

        createStationsCache(db);
    }
}

// Creates the nearby-stations cache used by the map when the phone is offline.
private void createStationsCache(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE IF NOT EXISTS stations_cache (id TEXT PRIMARY KEY, name TEXT, lat REAL, lng REAL, capacity_kwh REAL, battery_slots INTEGER, free_slots INTEGER, distance_km REAL, synced_at INTEGER NOT NULL)");
}

// Creates the prosumer's own bookings table, so the QR can be opened without signal at the station.
private void createMyReservations(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE IF NOT EXISTS my_reservations (id TEXT PRIMARY KEY, station_name TEXT, slot_start TEXT, status TEXT, qr_token TEXT, synced_at INTEGER NOT NULL)");
}

// Creates the offline booking list and dashboard snapshot tables.
private void createBookingCache(SQLiteDatabase db) {
    db.execSQL("CREATE TABLE IF NOT EXISTS booking_history_cache (id TEXT PRIMARY KEY, prosumer_nic TEXT, station_name TEXT, scheduled_at TEXT, status TEXT, synced_at INTEGER NOT NULL)");
    db.execSQL("CREATE TABLE IF NOT EXISTS dashboard_snapshot (id INTEGER PRIMARY KEY CHECK (id = 1), pending_count INTEGER NOT NULL, approved_future_count INTEGER NOT NULL, active_count INTEGER NOT NULL, next_station TEXT, next_when TEXT, synced_at INTEGER NOT NULL)");
}

public synchronized void saveSession(Session session) {
    SQLiteDatabase db = getWritableDatabase();
    db.beginTransaction();
    try {
        db.delete("session", null, null);
        ContentValues values = new ContentValues();
        values.put("token", session.token);
        values.put("expires_at", session.expiresAt);
        values.put("role", session.role);
        values.put("user_id", session.userId);
        values.put("nic", session.nic);
        db.insertOrThrow("session", null, values);
        db.setTransactionSuccessful();
    } finally {
        db.endTransaction();
    }
}

public synchronized Session readSession() {
    try (Cursor cursor = getReadableDatabase().query("session", null, null, null, null, null, null, "1")) {
        if (!cursor.moveToFirst()) return null;
        return new Session(
            cursor.getString(cursor.getColumnIndexOrThrow("token")),
            cursor.getLong(cursor.getColumnIndexOrThrow("expires_at")),
            cursor.getString(cursor.getColumnIndexOrThrow("role")),
            cursor.getString(cursor.getColumnIndexOrThrow("user_id")),
            cursor.getString(cursor.getColumnIndexOrThrow("nic"))
        );
    }
}

public synchronized void clearSession() {
    getWritableDatabase().delete("session", null, null);
}

public synchronized void saveProfile(UserProfile profile) {
    ContentValues values = new ContentValues();
    values.put("nic", profile.nic);
    values.put("username", profile.username);
    values.put("full_name", profile.fullName);
    values.put("email", profile.email);
    values.put("phone", profile.phone);
    values.put("status", profile.status);
    values.put("synced_at", profile.syncedAt);
    getWritableDatabase().insertWithOnConflict("user_profile", null, values, SQLiteDatabase.CONFLICT_REPLACE);
}

public synchronized UserProfile readProfile(String nic) {
    try (Cursor cursor = getReadableDatabase().query("user_profile", null, "nic = ?", new String[]{nic}, null, null, null, "1")) {
        if (!cursor.moveToFirst()) return null;
        UserProfile profile = new UserProfile();
        profile.nic = cursor.getString(cursor.getColumnIndexOrThrow("nic"));
        profile.username = cursor.getString(cursor.getColumnIndexOrThrow("username"));
        profile.fullName = cursor.getString(cursor.getColumnIndexOrThrow("full_name"));
        profile.email = cursor.getString(cursor.getColumnIndexOrThrow("email"));
        profile.phone = cursor.getString(cursor.getColumnIndexOrThrow("phone"));
        profile.status = cursor.getString(cursor.getColumnIndexOrThrow("status"));
        profile.syncedAt = cursor.getLong(cursor.getColumnIndexOrThrow("synced_at"));
        return profile;
    }
}

// Replaces the cached booking list with the latest API page.
public synchronized void replaceBookings(List<Booking> bookings, long syncedAt) {
    SQLiteDatabase db = getWritableDatabase();
    db.beginTransaction();
    try {
        db.delete("booking_history_cache", null, null);
        for (Booking booking : bookings) {
            ContentValues values = new ContentValues();
            values.put("id", booking.id);
            values.put("prosumer_nic", booking.prosumerNic);
            values.put("station_name", booking.stationName);
            values.put("scheduled_at", booking.scheduledAt);
            values.put("status", booking.status);
        }
    }
}

```

```

        values.put("synced_at", syncedAt);
        db.insertWithOnConflict("booking_history_cache", null, values, SQLiteDatabase.CONFLICT_REPLACE);
    }
    db.setTransactionSuccessful();
} finally {
    db.endTransaction();
}
}

// Returns the last booking list saved on this phone.
public synchronized List<Booking> readBookings() {
    ArrayList<Booking> bookings = new ArrayList<>();
    try (Cursor cursor = getReadableDatabase().query("booking_history_cache", null, null, null, null, null, "scheduled_at ASC")) {
        while (cursor.moveToNext()) {
            Booking booking = new Booking();
            booking.id = cursor.getString(cursor.getColumnIndexOrThrow("id"));
            booking.prosumerNic = cursor.getString(cursor.getColumnIndexOrThrow("prosumer_nic"));
            booking.stationName = cursor.getString(cursor.getColumnIndexOrThrow("station_name"));
            booking.scheduledAt = cursor.getString(cursor.getColumnIndexOrThrow("scheduled_at"));
            booking.status = cursor.getString(cursor.getColumnIndexOrThrow("status"));
            booking.syncedAt = cursor.getLong(cursor.getColumnIndexOrThrow("synced_at"));
            bookings.add(booking);
        }
    }
    return bookings;
}

// Stores the latest prosumer dashboard so it can be shown offline.
public synchronized void saveDashboard(DashboardSnapshot snapshot) {
    ContentValues values = new ContentValues();
    values.put("id", 1);
    values.put("pending_count", snapshot.pendingCount);
    values.put("approved_future_count", snapshot.approvedFutureCount);
    values.put("active_count", snapshot.activeCount);
    values.put("next_station", snapshot.nextStation);
    values.put("next_when", snapshot.nextWhen);
    values.put("synced_at", snapshot.syncedAt);
    getWritableDatabase().insertWithOnConflict("dashboard_snapshot", null, values, SQLiteDatabase.CONFLICT_REPLACE);
}

// Returns the last dashboard snapshot, or null when the phone has never synced.
public synchronized DashboardSnapshot readDashboard() {
    try (Cursor cursor = getReadableDatabase().query("dashboard_snapshot", null, "id = 1", null, null, null, null, "1")) {
        if (!cursor.moveToFirst()) return null;
        DashboardSnapshot snapshot = new DashboardSnapshot();
        snapshot.pendingCount = cursor.getInt(cursor.getColumnIndexOrThrow("pending_count"));
        snapshot.approvedFutureCount = cursor.getInt(cursor.getColumnIndexOrThrow("approved_future_count"));
        snapshot.activeCount = cursor.getInt(cursor.getColumnIndexOrThrow("active_count"));
        snapshot.nextStation = cursor.getString(cursor.getColumnIndexOrThrow("next_station"));
        snapshot.nextWhen = cursor.getString(cursor.getColumnIndexOrThrow("next_when"));
        snapshot.syncedAt = cursor.getLong(cursor.getColumnIndexOrThrow("synced_at"));
        return snapshot;
    }
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/local/Session.java M1 · 27 lines

```

/*
 * File: Session.java
 * Description: Immutable authenticated-session row used by SQLite and role routing.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.data.local;

public class Session {
    public final String token;
    public final long expiresAt;
    public final String role;
    public final String userId;
    public final String nic;

    public Session(String token, long expiresAt, String role, String userId, String nic) {
        this.token = token;
        this.expiresAt = expiresAt;
        this.role = role;
        this.userId = userId;
        this.nic = nic;
    }

    public boolean isUsable() {
        return token != null && !token.isEmpty() && expiresAt > System.currentTimeMillis();
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/local/SessionManager.java M1 · 40 lines

```

/*
 * File: SessionManager.java
 * Description: Persistent login state backed by the shared SQLite database.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.data.local;

```

```

import android.content.Context;

public class SessionManager {
    private static volatile SessionManager instance;
    private final DbHelper db;

    private SessionManager(Context context) {
        db = new DbHelper(context);
    }

    public static SessionManager get(Context context) {
        if (instance == null) {
            synchronized (SessionManager.class) {
                if (instance == null) instance = new SessionManager(context.getApplicationContext());
            }
        }
        return instance;
    }

    public Session current() {
        Session session = db.readSession();
        if (session != null && !session.isUsable()) {
            db.clearSession();
            return null;
        }
        return session;
    }

    public void save(Session session) { db.saveSession(session); }
    public void clear() { db.clearSession(); }
    public DbHelper database() { return db; }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/local/UserProfile.java M1 · 31 lines

```

/*
 * File: UserProfile.java
 * Description: Cached public profile; password data is never stored on the device.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.data.local;

import org.json.JSONObject;

public class UserProfile {
    public String nic = "";
    public String username = "";
    public String fullName = "";
    public String email = "";
    public String phone = "";
    public String status = "";
    public long syncedAt;

    public static UserProfile fromJson(JSONObject json) {
        UserProfile profile = new UserProfile();
        profile.nic = json.optString("nic", "");
        profile.username = json.optString("username", "");
        profile.fullName = json.optString("fullName", "");
        profile.email = json.optString("email", "");
        profile.phone = json.optString("phone", "");
        profile.status = json.optString("status", "");
        profile.syncedAt = System.currentTimeMillis();
        return profile;
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/remote/ApiCallback.java M1 · 6 lines

```

package lk.smartsolar.mobile.data.remote;

public interface ApiCallback<T> {
    void onSuccess(T value);
    void onError(ApiError error);
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/remote/ApiClient.java M1 · 238 lines

```

/*
 * File: ApiClient.java
 * Description: Shared JSON API client with bearer auth, background I/O and contract error parsing.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.data.remote;

import android.content.Context;
import android.net.Uri;
import android.os.Handler;

```

```

import android.os.Looper;

import org.json.JSONArray;
import org.json.JSONObject;

import java.io.BufferedReader;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.time.Instant;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

import lk.smartsolar.mobile.BuildConfig;
import lk.smartsolar.mobile.data.local.Booking;
import lk.smartsolar.mobile.data.local.DashboardSnapshot;
import lk.smartsolar.mobile.data.local.Session;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.data.local.UserProfile;

public class ApiClient {
    private static volatile ApiClient instance;
    private final Context context;
    private final ExecutorService executor = Executors.newFixedThreadPool(3);
    private final Handler main = new Handler(Looper.getMainLooper());

    private ApiClient(Context context) {
        this.context = context.getApplicationContext();
    }

    public static ApiClient get(Context context) {
        if (instance == null) {
            synchronized (ApiClient.class) {
                if (instance == null) instance = new ApiClient(context);
            }
        }
        return instance;
    }

    public void login(String login, String password, ApiCallback<Session> callback) {
        JSONObject body = new JSONObject();
        put(body, "username", login);
        put(body, "password", password);
        request("POST", "auth/login", body, false, json -> {
            JSONObject user = json.optJSONObject("user");
            if (user == null) user = new JSONObject();
            long expiresAt;
            try {
                expiresAt = Instant.parse(json.getString("expiresAt")).toEpochMilli();
            } catch (Exception ignored) {
                expiresAt = System.currentTimeMillis() + 60L * 60L * 1000L;
            }
            Session session = new Session(
                json.optString("token", ""), expiresAt,
                user.optString("role", ""), user.optString("id", ""), user.optString("nic", ""));
            SessionManager.get(context).save(session);
            return session;
        }, callback);
    }

    public void register(String nic, String username, String password, String fullName, String email, String phone, ApiCallback<UserProfile> callback) {
        JSONObject body = profileBody(username, password, fullName, email, phone);
        put(body, "nic", nic);
        request("POST", "auth/register", body, false, UserProfile::fromJson, callback);
    }

    public void getMyProfile(ApiCallback<UserProfile> callback) {
        request("GET", "me", null, true, json -> {
            UserProfile profile = UserProfile.fromJson(json);
            SessionManager.get(context).database().saveProfile(profile);
            return profile;
        }, callback);
    }

    public void updateMyProfile(String username, String password, String fullName, String email, String phone, ApiCallback<UserProfile> callback) {
        JSONObject body = profileBody(username, password, fullName, email, phone);
        request("PUT", "me", body, true, json -> {
            UserProfile profile = UserProfile.fromJson(json);
            SessionManager.get(context).database().saveProfile(profile);
            return profile;
        }, callback);
    }

    public void requestDeactivation(ApiCallback<UserProfile> callback) {
        request("POST", "me/request-deactivation", null, true, UserProfile::fromJson, callback);
    }

    // Calls GET /dashboard/me and stores the counts for offline display.
    public void getMyDashboard(ApiCallback<DashboardSnapshot> callback) {
        request("GET", "dashboard/me", null, true, json -> {
            DashboardSnapshot snapshot = DashboardSnapshot.fromJson(json);

```

```

        SessionManager.get(context).database().saveDashboard(snapshot);
        return snapshot;
    }, callback);
}

// Calls GET /reservations with an already encoded query string and caches the page.
public void listReservations(String query, ApiCallback<List<Booking>> callback) {
    String path = "reservations" + (query == null || query.isEmpty() ? "" : "?" + query);
    request("GET", path, null, true, json -> cacheBookings(json), callback);
}

// Calls GET /reservations/pending for the operator approval queue.
public void listPendingReservations(ApiCallback<List<Booking>> callback) {
    request("GET", "reservations/pending", null, true, json -> cacheBookings(json), callback);
}

// Calls POST /reservations/{id}/approve.
public void approveReservation(String id, ApiCallback<Booking> callback) {
    request("POST", "reservations/" + Uri.encode(id) + "/approve", null, true, Booking::fromJson, callback);
}

// Calls POST /reservations/verify-qr. The body is only the opaque token.
public void verifyQr(String qrToken, ApiCallback<Booking> callback) {
    JSONObject body = new JSONObject();
    put(body, "qrToken", qrToken);
    request("POST", "reservations/verify-qr", body, true, Booking::fromJson, callback);
}

// Calls POST /reservations/{id}/complete.
public void completeReservation(String id, ApiCallback<Booking> callback) {
    request("POST", "reservations/" + Uri.encode(id) + "/complete", null, true, Booking::fromJson, callback);
}

// Reads the page envelope and replaces the on-device booking cache.
private List<Booking> cacheBookings(JSONObject json) throws Exception {
    JSONArray items = json.optJSONArray("items");
    ArrayList<Booking> bookings = new ArrayList<>();
    if (items != null) {
        for (int index = 0; index < items.length(); index++) {
            bookings.add(Booking.fromJson(items.getJSONObject(index)));
        }
    }
    SessionManager.get(context).database().replaceBookings(bookings, System.currentTimeMillis());
    return bookings;
}

private JSONObject profileBody(String username, String password, String fullName, String email, String phone) {
    JSONObject body = new JSONObject();
    put(body, "username", username);
    if (password != null && !password.isEmpty()) put(body, "password", password);
    put(body, "fullName", fullName);
    put(body, "email", email);
    put(body, "phone", phone);
    return body;
}

// Sends an authenticated request whose answer may be a JSON object or a JSON array (used by the reservation screens).
public <T> void send(String method, String path, JSONObject body, TextMapper<T> mapper, ApiCallback<T> callback) {
    execute(method, path, body, true, mapper, callback);
}

private <T> void request(String method, String path, JSONObject body, boolean bearer, JsonMapper<T> mapper, ApiCallback<T> callback) {
    execute(method, path, body, bearer, text -> mapper.map(text.isEmpty() ? new JSONObject() : new JSONObject(text)), callback);
}

private <T> void execute(String method, String path, JSONObject body, boolean bearer, TextMapper<T> mapper, ApiCallback<T> callback) {
    executor.execute() -> {
        HttpURLConnection connection = null;
        try {
            String baseUrl = context.getSharedPreferences("settings", Context.MODE_PRIVATE)
                .getString("api_base_url", BuildConfig.API_BASE_URL);
            if (!baseUrl.endsWith("/")) baseUrl += "/";
            connection = (HttpURLConnection) new URL(baseUrl + path).openConnection();
            connection.setRequestMethod(method);
            connection.setConnectTimeout(15000);
            connection.setReadTimeout(20000);
            connection.setRequestProperty("Accept", "application/json");

            if (bearer) {
                Session session = SessionManager.get(context).current();
                if (session == null) throw new ApiError(401, "UNAUTHORIZED", "Please sign in again.");
                connection.setRequestProperty("Authorization", "Bearer " + session.token);
            }

            if (body != null) {
                connection.setDoOutput(true);
                connection.setRequestProperty("Content-Type", "application/json; charset=utf-8");
                try (OutputStream output = connection.getOutputStream()) {
                    output.write(body.toString().getBytes(StandardCharsets.UTF_8));
                }
            }

            int status = connection.getResponseCode();
            String text = read(status >= 200 && status < 300 ? connection.getInputStream() : connection.getErrorStream());
            if (status < 200 || status >= 300) {
                JSONObject json = text.startsWith("{") ? new JSONObject(text) : new JSONObject();
            }
        } catch (Exception e) {
            callback.onFailure(e);
        }
    }
}

```

```

        if (status == 401) SessionManager.getContext().clear();
        throw new ApiError(status, json.optString("code", "ERROR"), json.optString("message", "The request failed."));
    }

    T value = mapper.map(text);
    main.post() -> callback.onSuccess(value));
} catch (ApiError error) {
    main.post() -> callback.onError(error));
} catch (Exception error) {
    ApiError apiError = new ApiError(0, "NETWORK_ERROR", "Cannot reach the Smart Solar API. Check the API address and connection.");
    main.post() -> callback.onError(apiError));
} finally {
    if (connection != null) connection.disconnect();
}
});
}

private static String read(InputStream stream) throws Exception {
    if (stream == null) return "";
    StringBuilder result = new StringBuilder();
    try (BufferedReader reader = new BufferedReader(new InputStreamReader(stream, StandardCharsets.UTF_8))) {
        String line;
        while ((line = reader.readLine()) != null) result.append(line);
    }
    return result.toString();
}

private static void put(JSONObject object, String key, String value) {
    try { object.put(key, value == null ? "" : value.trim()); } catch (Exception ignored) { }
}

private interface JsonMapper<T> { T map(JSONObject json) throws Exception; }

public interface TextMapper<T> { T map(String body) throws Exception; }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/remote/ApiError.java M1 · 18 lines

```

/*
 * File: ApiError.java
 * Description: API contract error represented as an exception for UI callbacks.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.data.remote;

public class ApiError extends Exception {
    public final int status;
    public final String code;

    public ApiError(int status, String code, String message) {
        super(message);
        this.status = status;
        this.code = code;
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/reservations/AvailableSlot.java M3 · 29 lines

```

/*
 * File: AvailableSlot.java
 * Description: A free slot returned by the API for the slot list.
 * Author: Janukshan S (IT22635266)
 */
package lk.smartsolar.mobile.data.reservations;

import lk.smartsolar.mobile.util.TimeFormat;

public class AvailableSlot {

    public final String id;
    public final String startTime;
    public final String endTime;

    // Keeps the slot id and its UTC start and end times.
    public AvailableSlot(String id, String startTime, String endTime) {
        this.id = id;
        this.startTime = startTime;
        this.endTime = endTime;
    }

    // Text shown on a slot tile, e.g. "08:00 - 10:00 · Mon 28 Sep".
    @Override
    public String toString() {
        return TimeFormat.time(startTime) + " - " + TimeFormat.time(endTime) + " · " + TimeFormat.day(startTime);
    }
}

```

```

/*
 * File: BookingStation.java
 * Description: A station that is taking bookings, shown in the station picker.
 * Author: Janukshan S (IT22635266)
 */

```

```
package lk.smartsolar.mobile.data.reservations;
```

```

public class BookingStation {

    public final String id;
    public final String name;

    // Keeps the station id and display name.
    public BookingStation(String id, String name) {
        this.id = id;
        this.name = name;
    }

    // The spinner shows this text.
    @Override
    public String toString() {
        return name;
    }
}

```

```

/*
 * File: MyReservationStore.java
 * Description: Reads and writes the my_reservations SQLite table. The API is the source of truth;
 *              this is a read-only copy so the booking and its QR can be shown without signal.
 * Author: Janukshan S (IT22635266)
 */

```

```
package lk.smartsolar.mobile.data.reservations;
```

```

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;

```

```

import lk.smartsolar.mobile.data.local.DbHelper;
import lk.smartsolar.mobile.data.local.SessionManager;

```

```

public class MyReservationStore {

    private final DbHelper db;

    // Uses the app's shared SQLite database.
    public MyReservationStore(Context context) {
        db = SessionManager.get(context).database();
    }

    // Inserts or replaces the local copy of a booking with what the API just returned.
    public void save(Reservation reservation) {
        ContentValues values = new ContentValues();
        values.put("id", reservation.id);
        values.put("station_name", reservation.stationName);
        values.put("slot_start", reservation.scheduledAt);
        values.put("status", reservation.status);
        values.put("qr_token", reservation.qrToken);
        values.put("synced_at", System.currentTimeMillis());
        db.getWritableDatabase().insertWithOnConflict("my_reservations", null, values, SQLiteDatabase.CONFLICT_REPLACE);
    }

    // Returns the saved copy of a booking, or null if this phone has never loaded it.
    public Reservation find(String id) {
        try (Cursor cursor = db.getReadableDatabase().query("my_reservations", null, "id = ?", new String[]{id}, null, null, null)) {
            if (!cursor.moveToFirst()) {
                return null;
            }
            Reservation r = new Reservation();
            r.id = cursor.getString(cursor.getColumnIndexOrThrow("id"));
            r.stationName = cursor.getString(cursor.getColumnIndexOrThrow("station_name"));
            r.scheduledAt = cursor.getString(cursor.getColumnIndexOrThrow("slot_start"));
            r.status = cursor.getString(cursor.getColumnIndexOrThrow("status"));
            r.qrToken = cursor.getString(cursor.getColumnIndexOrThrow("qr_token"));
            r.syncedAt = cursor.getLong(cursor.getColumnIndexOrThrow("synced_at"));
            return r;
        }
    }
}

```

```

/*
 * File: Reservation.java
 * Description: A reservation as returned by the API.

```

```

* Author: Janukshan S (IT22635266)
*/

package lk.smartsolar.mobile.data.reservations;

import org.json.JSONObject;

public class Reservation {

    public String id;
    public String prosumerNic;
    public String stationId;
    public String stationName;
    public String slotId;
    public String scheduledAt;
    public String status;
    public String qrToken;
    public long syncedAt;

    // Reads the fields the app uses from the API's JSON.
    public static Reservation fromJson(JSONObject json) {
        Reservation r = new Reservation();
        r.id = json.optString("id");
        r.prosumerNic = json.optString("prosumerNic");
        r.stationId = json.optString("stationId");
        r.stationName = json.optString("stationName");
        r.slotId = json.optString("slotId");
        r.scheduledAt = json.optString("scheduledAt");
        r.status = json.optString("status");
        r.qrToken = json.isNull("qrToken") ? null : json.optString("qrToken", null);
        return r;
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/reservations/ReservationApi.java M3 · 101 lines

```

/*
 * File: ReservationApi.java
 * Description: Reservation calls to the Web API. Turns the JSON answers into objects for the screens
 * and keeps a copy of each returned booking in SQLite.
 * Author: Janukshan S (IT22635266)
 */

package lk.smartsolar.mobile.data.reservations;

import android.content.Context;
import android.net.Uri;

import org.json.JSONArray;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;

import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;

public class ReservationApi {

    private final ApiClient client;
    private final MyReservationStore store;

    // Uses the shared API client, which adds the login token, and the local copy in SQLite.
    public ReservationApi(Context context) {
        client = ApiClient.get(context);
        store = new MyReservationStore(context);
    }

    // GET /reservations/bookable-stations
    public void getBookableStations(ApiCallback<List<BookingStation>> callback) {
        client.send("GET", "reservations/bookable-stations", null, body -> {
            JSONArray array = new JSONArray(body);
            List<BookingStation> stations = new ArrayList<>();
            for (int i = 0; i < array.length(); i++) {
                JSONObject item = array.getJSONObject(i);
                stations.add(new BookingStation(item.getString("id"), item.getString("name")));
            }
            return stations;
        }, callback);
    }

    // GET /reservations/available-slots. date is "yyyy-MM-dd" or null for every day.
    public void getAvailableSlots(String stationId, String date, ApiCallback<List<AvailableSlot>> callback) {
        String path = "reservations/available-slots?stationId=" + Uri.encode(stationId);
        if (date != null) {
            path += "&date=" + date;
        }

        client.send("GET", path, null, body -> {
            JSONArray array = new JSONArray(body);
            List<AvailableSlot> slots = new ArrayList<>();
            for (int i = 0; i < array.length(); i++) {

```

```

        JSONObject item = array.getJSONObject(i);
        slots.add(new AvailableSlot(item.getString("id"), item.getString("startTime"), item.getString("endTime")));
    }
    return slots;
}, callback);
}

// POST /reservations. The API takes the prosumer's NIC from the login token. Returns the raw reservation JSON.
public void create(String stationId, String slotId, ApiCallback<String> callback) {
    JSONObject body = new JSONObject();
    try {
        body.put("stationId", stationId);
        body.put("slotId", slotId);
    } catch (Exception ignored) {
    }
    client.send("POST", "reservations", body, this::keepCopy, callback);
}

// GET /reservations/{id}
public void get(String id, ApiCallback<String> callback) {
    client.send("GET", "reservations/" + Uri.encode(id), null, this::keepCopy, callback);
}

// PUT /reservations/{id}. Moves the booking to another slot; the API checks the 12-hour and 7-day rules.
public void update(String id, String stationId, String slotId, ApiCallback<String> callback) {
    JSONObject body = new JSONObject();
    try {
        body.put("stationId", stationId);
        body.put("slotId", slotId);
    } catch (Exception ignored) {
    }
    client.send("PUT", "reservations/" + Uri.encode(id), body, this::keepCopy, callback);
}

// POST /reservations/{id}/cancel. The API checks the 12-hour rule.
public void cancel(String id, ApiCallback<String> callback) {
    client.send("POST", "reservations/" + Uri.encode(id) + "/cancel", new JSONObject(), this::keepCopy, callback);
}

// Saves every reservation the API returns into SQLite, then passes the JSON on to the screen.
private String keepCopy(String body) throws Exception {
    store.save(Reservation.fromJson(new JSONObject(body)));
    return body;
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/data/stations/NearbyStation.java M2 · 43 lines

```

/*
 * File: NearbyStation.java
 * Description: One station returned by GET /stations/nearby, including distance and free slots.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
package lk.smartsolar.mobile.data.stations;

import org.json.JSONObject;

import java.util.Locale;

public class NearbyStation {

    public String id;
    public String name;
    public double latitude;
    public double longitude;
    public double capacityKwh;
    public int batterySlotsTotal;
    public double distanceKm;
    public int freeSlots;

    // Reads one station object from the nearby-stations JSON.
    public static NearbyStation fromJson(JSONObject json) {
        NearbyStation station = new NearbyStation();
        station.id = json.optString("id");
        station.name = json.optString("name");
        station.latitude = json.optDouble("latitude");
        station.longitude = json.optDouble("longitude");
        station.capacityKwh = json.optDouble("capacityKwh");
        station.batterySlotsTotal = json.optInt("batterySlotsTotal");
        station.distanceKm = json.optDouble("distanceKm");
        station.freeSlots = json.optInt("freeSlots");
        return station;
    }

    // List row: "Malabe Solar Hub · 0.01 km · 18 free".
    @Override
    public String toString() {
        return name + " · " + String.format(Locale.US, "%.2f", distanceKm) + " km · " + freeSlots + " free";
    }
}

```

```

/*
 * File: StationApi.java
 * Description: Nearby-station calls. The map and the list both use this.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
package lk.smartsolar.mobile.data.stations;

import android.content.Context;

import org.json.JSONArray;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.List;

import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;

public class StationApi {

    private final ApiClient client;

    // Uses the shared API client, which adds the login token.
    public StationApi(Context context) {
        client = ApiClient.get(context);
    }

    // GET /stations/nearby. radiusKm is 25 so the map covers Colombo from a phone in Malabe.
    public void getNearby(double lat, double lng, ApiCallback<List<NearbyStation>> callback) {
        String path = "stations/nearby?lat=" + lat + "&lng=" + lng + "&radiusKm=25";
        client.send("GET", path, null, body -> {
            JSONArray array = new JSONArray(body);
            List<NearbyStation> stations = new ArrayList<>();
            for (int i = 0; i < array.length(); i++) {
                JSONObject item = array.getJSONObject(i);
                stations.add(NearbyStation.fromJson(item));
            }
            return stations;
        }, callback);
    }
}

```

```

/*
 * File: StationCacheStore.java
 * Description: Offline copy of nearby stations so the map can open without signal.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
package lk.smartsolar.mobile.data.stations;

import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;

import java.util.ArrayList;
import java.util.List;

import lk.smartsolar.mobile.data.local.DbHelper;
import lk.smartsolar.mobile.data.local.SessionManager;

public class StationCacheStore {

    private final DbHelper db;

    // Uses the app's shared SQLite database.
    public StationCacheStore(Context context) {
        db = SessionManager.get(context).database();
    }

    // Replaces the cached stations with the latest nearby response.
    public void saveAll(List<NearbyStation> stations) {
        SQLiteDatabase database = db.getWritableDatabase();
        database.beginTransaction();
        try {
            long syncedAt = System.currentTimeMillis();
            database.delete("stations_cache", null, null);
            for (NearbyStation station : stations) {
                ContentValues values = new ContentValues();
                values.put("id", station.id);
                values.put("name", station.name);
                values.put("lat", station.latitude);
                values.put("lng", station.longitude);
                values.put("capacity_kwh", station.capacityKwh);
                values.put("battery_slots", station.batterySlotsTotal);
                values.put("free_slots", station.freeSlots);
                values.put("distance_km", station.distanceKm);
            }
        }
    }
}

```

```

        values.put("synced_at", syncedAt);
        database.insert("stations_cache", null, values);
    }
    database.setTransactionSuccessful();
} finally {
    database.endTransaction();
}
}

// Returns the saved stations, nearest first.
public List<NearbyStation> loadAll() {
    ArrayList<NearbyStation> stations = new ArrayList<>();
    try (Cursor cursor = db.getReadableDatabase().query(
        "stations_cache", null, null, null, null, "distance_km ASC")) {
        while (cursor.moveToNext()) {
            NearbyStation station = new NearbyStation();
            station.id = cursor.getString(cursor.getColumnIndexOrThrow("id"));
            station.name = cursor.getString(cursor.getColumnIndexOrThrow("name"));
            station.latitude = cursor.getDouble(cursor.getColumnIndexOrThrow("lat"));
            station.longitude = cursor.getDouble(cursor.getColumnIndexOrThrow("lng"));
            station.capacityKwh = cursor.getDouble(cursor.getColumnIndexOrThrow("capacity_kwh"));
            station.batterySlotsTotal = cursor.getInt(cursor.getColumnIndexOrThrow("battery_slots"));
            station.freeSlots = cursor.getInt(cursor.getColumnIndexOrThrow("free_slots"));
            station.distanceKm = cursor.getDouble(cursor.getColumnIndexOrThrow("distance_km"));
            stations.add(station);
        }
    }
    return stations;
}

// Milliseconds of the last successful nearby save, or 0 when the cache is empty.
public long lastSyncedAt() {
    try (Cursor cursor = db.getReadableDatabase().rawQuery(
        "SELECT MAX(synced_at) FROM stations_cache", null)) {
        if (!cursor.moveToFirst() || cursor.isNull(0)) {
            return 0;
        }
        return cursor.getLong(0);
    }
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/BaseActivity.java M1 · 56 lines

```

/*
 * File: BaseActivity.java
 * Description: Shared loading and API error presentation for every mobile screen.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.ui;

import android.app.Dialog;
import android.os.Bundle;
import android.view.ViewGroup;
import android.widget.ProgressBar;

import androidx.annotation.Nullable;
import androidx.appcompat.app.AppCompatActivity;

import com.google.android.material.dialog.MaterialAlertDialogBuilder;

import lk.smartsolar.mobile.data.remote.ApiError;

public abstract class BaseActivity extends AppCompatActivity {
    private Dialog loading;

    @Override
    protected void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        ProgressBar spinner = new ProgressBar(this);
        int padding = (int) (24 * getResources().getDisplayMetrics().density);
        spinner.setPadding(padding, padding, padding, padding);
        loading = new Dialog(this);
        loading setContentView(spinner, new ViewGroup.LayoutParams(padding * 3, padding * 3));
        loading.setCancelable(false);
    }

    protected void showLoading(boolean show) {
        if (show && !isFinishing()) loading.show(); else loading.dismiss();
    }

    protected void showError(ApiError error) {
        String message;
        if ("ACCOUNT_NOT_ACTIVE".equals(error.code)) {
            message = "Your account is awaiting Backoffice activation.";
        } else if ("NIC_EXISTS".equals(error.code)) {
            message = "An account already exists for that NIC.";
        } else {
            message = error.getMessage();
        }
        new MaterialAlertDialogBuilder(this).setTitle("Smart Solar").setMessage(message).setPositiveButton("OK", null).show();
    }

    @Override

```

```

protected void onDestroy() {
    if (loading != null) loading.dismiss();
    super.onDestroy();
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/account/DeactivationActivity.java M1 · 41 lines

```

/* File: DeactivationActivity.java | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
package lk.smartsolar.mobile.ui.account;

```

```

import android.os.Bundle;

import com.google.android.material.dialog.MaterialAlertDialogBuilder;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.data.local.UserProfile;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.util.RoleRouter;

public class DeactivationActivity extends BaseActivity {
    @Override protected void onCreate(Bundle state) {
        super.onCreate(state); setContentView(R.layout.activity_deactivation);
        findViewById(R.id.cancelDeactivateButton).setOnClickListener(v -> finish());
        findViewById(R.id.confirmDeactivateButton).setOnClickListener(v -> confirm());
    }

    private void confirm() {
        new MaterialAlertDialogBuilder(this).setTitle("Final confirmation")
            .setMessage("Deactivate this account and log out now?")
            .setNegativeButton("Cancel", null)
            .setPositiveButton("Deactivate", (dialog, which) -> deactivate()).show();
    }

    private void deactivate() {
        showLoading(true);
        ApiClient.get(this).requestDeactivation(new ApiCallback<UserProfile>() {
            @Override public void onSuccess(UserProfile value) {
                showLoading(false); SessionManager.get(DeactivationActivity.this).clear();
                RoleRouter.route(DeactivationActivity.this, null);
            }
            @Override public void onError(ApiError error) { showLoading(false); showError(error); }
        });
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/account/ProfileActivity.java M1 · 74 lines

```

/*
 * File: ProfileActivity.java
 * Description: Shows cached profile immediately, refreshes it, and saves edits through the API.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.ui.account;

import android.content.Intent;
import android.os.Bundle;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.Session;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.data.local.UserProfile;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.ui.BaseActivity;

public class ProfileActivity extends BaseActivity {
    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_profile);
        Session session = SessionManager.get(this).current();
        if (session == null || !"Prosumer".equals(session.role)) { finish(); return; }

        UserProfile cached = SessionManager.get(this).database().readProfile(session.nic);
        if (cached != null) bind(cached);
        findViewById(R.id.saveProfileButton).setOnClickListener(v -> save());
        findViewById(R.id.deactivateButton).setOnClickListener(v -> startActivity(new Intent(this, DeactivationActivity.class)));
        refresh(cached == null);
    }

    private void refresh(boolean showSpinner) {
        if (showSpinner) showLoading(true);
        ApiClient.get(this).getMyProfile(new ApiCallback<UserProfile>() {
            @Override public void onSuccess(UserProfile value) { showLoading(false); bind(value); }
        });
    }
}

```

```

        @Override public void onError(ApiError error) { showLoading(false); showError(error); }
    });
}

private void bind(UserProfile profile) {
    ((TextView) findViewById(R.id.profileNic)).setText("NIC: " + profile.nic);
    ((TextView) findViewById(R.id.profileStatus)).setText("Status: " + profile.status);
    ((EditText) findViewById(R.id.profileUsername)).setText(profile.username);
    ((EditText) findViewById(R.id.profileFullName)).setText(profile.fullName);
    ((EditText) findViewById(R.id.profileEmail)).setText(profile.email);
    ((EditText) findViewById(R.id.profilePhone)).setText(profile.phone);
}

private String text(int id) { return ((EditText) findViewById(id)).getText().toString().trim(); }

private void save() {
    String username = text(R.id.profileUsername), fullName = text(R.id.profileFullName);
    if (username.isEmpty() || fullName.isEmpty()) {
        showError(new ApiError(400, "VALIDATION_ERROR", "Username and full name are required."));
        return;
    }
    showLoading(true);
    ApiClient.get(this).updateMyProfile(username, text(R.id.profilePassword), fullName,
        text(R.id.profileEmail), text(R.id.profilePhone), new ApiCallback<UserProfile>() {
        @Override public void onSuccess(UserProfile value) {
            showLoading(false); bind(value);
            ((EditText) findViewById(R.id.profilePassword)).setText("");
            Toast.makeText(ProfileActivity.this, "Profile updated", Toast.LENGTH_SHORT).show();
        }
        @Override public void onError(ApiError error) { showLoading(false); showError(error); }
    });
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/auth/LoginActivity.java M1 · 49 lines

/* File: LoginActivity.java | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
package lk.smartsolar.mobile.ui.auth;

```

import android.content.Intent;
import android.os.Bundle;
import android.widget.EditText;

```

```

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.Session;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.ui.settings.SettingsActivity;
import lk.smartsolar.mobile.util.RoleRouter;

```

```

public class LoginActivity extends BaseActivity {
    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_login);
        findViewById(R.id.loginButton).setOnClickListener(v -> login());
        findViewById(R.id.registerButton).setOnClickListener(v -> startActivity(new Intent(this, RegisterActivity.class)));
        findViewById(R.id.settingsButton).setOnClickListener(v -> startActivity(new Intent(this, SettingsActivity.class)));
    }

    private void login() {
        String identifier = ((EditText) findViewById(R.id.loginIdentifier)).getText().toString().trim();
        String password = ((EditText) findViewById(R.id.loginPassword)).getText().toString();
        if (identifier.isEmpty() || password.isEmpty()) {
            showError(new ApiError(400, "VALIDATION_ERROR", "Enter your username or NIC and password."));
            return;
        }
        showLoading(true);
        ApiClient.get(this).login(identifier, password, new ApiCallback<Session>() {
            @Override public void onSuccess(Session session) {
                showLoading(false);
                // Backoffice is a web-desk role. Saving it would bounce the user back to this screen.
                if ("Backoffice".equals(session.role)) {
                    SessionManager.get(LoginActivity.this).clear();
                    showError(new ApiError(403, "WRONG_CLIENT", "Backoffice accounts use the web application. Log in here as a prosumer or a grid operator."));
                    return;
                }
                RoleRouter.route(LoginActivity.this, session);
            }
        });
        @Override public void onError(ApiError error) { showLoading(false); showError(error); }
    });
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/auth/LogoutActivity.java M1 · 16 lines

/* File: LogoutActivity.java | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
package lk.smartsolar.mobile.ui.auth;

```

import android.os.Bundle;

```

```

import lk.smartsolar.mobile.data.local.SessionManager;

```

```
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.util.RoleRouter;

public class LogoutActivity extends BaseActivity {
    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        SessionManager.get(this).clear();
        RoleRouter.route(this, null);
    }
}
```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/auth/RegisterActivity.java M1 · 44 lines

```
/* File: RegisterActivity.java | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
package lk.smartsolar.mobile.ui.auth;

import android.os.Bundle;
import android.widget.EditText;

import com.google.android.material.dialog.MaterialAlertDialogBuilder;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.UserProfile;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.ui.BaseActivity;

public class RegisterActivity extends BaseActivity {
    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_register);
        findViewById(R.id.createAccountButton).setOnClickListener(v -> register());
    }

    private String text(int id) { return ((EditText) findViewById(id)).getText().toString().trim(); }

    private void register() {
        String nic = text(R.id.registerNic), username = text(R.id.registerUsername), password = text(R.id.registerPassword);
        String fullName = text(R.id.registerFullName), email = text(R.id.registerEmail), phone = text(R.id.registerPhone);
        if (nic.isEmpty() || username.isEmpty() || fullName.isEmpty() || password.length() < 8) {
            showError(new ApiError(400, "VALIDATION_ERROR", "NIC, username, full name and an 8-character password are required."));
            return;
        }
        showLoading(true);
        ApiClient.get(this).register(nic, username, password, fullName, email, phone, new ApiCallback<UserProfile>() {
            @Override public void onSuccess(UserProfile value) {
                showLoading(false);
                new MaterialAlertDialogBuilder(RegisterActivity.this)
                    .setTitle("Registration received")
                    .setMessage("Your account is awaiting Backoffice activation. You can log in after it is activated.")
                    .setPositiveButton("Back to login", (dialog, which) -> finish()).setCancelable(false).show();
            }
            @Override public void onError(ApiError error) { showLoading(false); showError(error); }
        });
    }
}
```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/auth/SplashActivity.java M1 · 20 lines

```
/* File: SplashActivity.java | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
package lk.smartsolar.mobile.ui.auth;

import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.util.RoleRouter;

public class SplashActivity extends BaseActivity {
    @Override protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_splash);
        new Handler(Looper.getMainLooper()).postDelayed(
            () -> RoleRouter.route(this, SessionManager.get(this).current()), 700);
    }
}
```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/bookings/BookingsActivity.java M4 · 114 lines

```
/*
 * File: BookingsActivity.java
 * Description: Current, pending and history tabs for the signed-in prosumer, with a keyword filter.
 * Author: samudith
 * Created: 29/09/2026
 */
package lk.smartsolar.mobile.ui.bookings;
```

```

import android.os.Bundle;
import android.view.inputmethod.EditorInfo;
import android.widget.ArrayAdapter;
import android.widget.EditText;
import android.widget.ListView;
import android.widget.TextView;

import java.time.Instant;
import java.util.ArrayList;
import java.util.List;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.Booking;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.ui.reservations.BookingDetailActivity;
import lk.smartsolar.mobile.util.TimeText;

public class BookingsActivity extends BaseActivity {
    private String tab = "current";
    private ArrayAdapter<String> adapter;
    private final List<Booking> rows = new ArrayList<>();

    // Wires the three tabs and the keyword search, then loads Current.
    @Override
    protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_bookings);
        ((TextView) findViewById(R.id.screenTitle)).setText("My bookings");
        adapter = new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, new ArrayList<>());
        ((ListView) findViewById(R.id.bookingList)).setAdapter(adapter);
        // Tapping a booking opens it with Change slot, Cancel and Show QR.
        ((ListView) findViewById(R.id.bookingList)).setOnItemClickListener((parent, view, position, id) -> {
            if (position < rows.size()) BookingDetailActivity.open(this, rows.get(position).id);
        });
        findViewById(R.id.currentTab).setOnClickListener(v -> { tab = "current"; load(); });
        findViewById(R.id.pendingTab).setOnClickListener(v -> { tab = "pending"; load(); });
        findViewById(R.id.historyTab).setOnClickListener(v -> { tab = "history"; load(); });
        ((EditText) findViewById(R.id.searchBox)).setOnEditorActionListener((view, actionId, event) -> {
            if (actionId == EditorInfo.IME_ACTION_SEARCH) {
                load();
                return true;
            }
            return false;
        });
    }

    // Reloads the tab whenever the screen comes back, so changes made on other screens show up.
    @Override
    protected void onResume() {
        super.onResume();
        load();
    }

    // Asks the API for this tab. On failure it shows the SQLite copy.
    private void load() {
        showLoading(true);
        ApiClient.get(this).listReservations(query(), new ApiCallback<List<Booking>>() {
            @Override
            public void onSuccess(List<Booking> bookings) {
                showLoading(false);
                showRows(bookings, false);
            }

            @Override
            public void onError(ApiError error) {
                showLoading(false);
                List<Booking> cached = SessionManager.get(BookingsActivity.this).database().readBookings();
                showRows(cached, true);
                if (cached.isEmpty()) showError(error);
            }
        });
    }

    // Builds the filter query. The prosumer NIC is applied by the API from the token.
    private String query() {
        StringBuilder query = new StringBuilder("page=1");
        String keyword = ((EditText) findViewById(R.id.searchBox)).getText().toString().trim();
        if (!keyword.isEmpty()) query.append("&q=").append(android.net.Uri.encode(keyword));
        String now = Instant.now().toString();
        if ("current".equals(tab)) query.append("&from=").append(android.net.Uri.encode(now));
        if ("pending".equals(tab)) query.append("&status=Pending");
        if ("history".equals(tab)) query.append("&to=").append(android.net.Uri.encode(now));
        return query.toString();
    }

    // Renders one line per booking and the offline label when the list is cached.
    private void showRows(List<Booking> bookings, boolean offline) {
        TextView updated = findViewById(R.id.updatedLabel);
        long syncedAt = bookings.isEmpty() ? 0 : bookings.get(0).syncedAt;
        updated.setText(offline ? "Offline. " + TimeText.lastUpdated(syncedAt) : "");
        adapter.clear();
        rows.clear();
        rows.addAll(bookings);
    }

```

```

        if (bookings.isEmpty()) {
            adapter.add("No bookings in this tab.");
        }
        for (Booking booking : bookings) {
            adapter.add(booking.status + " - " + booking.stationName + "\n" + TimeText.sriLanka(booking.scheduledAt));
        }
        adapter.notifyDataSetChanged();
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/home/OperatorHomeActivity.java M4 · 93 lines

/* File: OperatorHomeActivity.java | Author: samudith | Created: 29/09/2026 */
package lk.smartsolar.mobile.ui.home;

```

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Bundle;

```

```

import androidx.annotation.NonNull;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;

```

```

import com.journeyapps.barcodescanner.ScanContract;
import com.journeyapps.barcodescanner.ScanOptions;

```

```

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.Booking;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.ui.auth.LogoutActivity;
import lk.smartsolar.mobile.ui.operator.PendingQueueActivity;
import lk.smartsolar.mobile.ui.operator.QrDetailsActivity;

```

```

public class OperatorHomeActivity extends BaseActivity {
    private static final int CAMERA_REQUEST = 41;

```

```

    private final androidx.activity.result.ActivityResultLauncher<ScanOptions> scanner =
        registerForActivityResult(new ScanContract(), result -> {
            if (result.getContents() != null) verify(result.getContents());
        });

```

// Opens the approval queue, the camera scanner and logout.

```

@Override
protected void onCreate(Bundle state) {
    super.onCreate(state);
    setContentView(R.layout.activity_operator_home);
    findViewById(R.id.pendingButton).setOnClickListener(v -> startActivity(new Intent(this, PendingQueueActivity.class)));
    findViewById(R.id.scanButton).setOnClickListener(v -> ensureCamera());
    findViewById(R.id.logoutButton).setOnClickListener(v -> startActivity(new Intent(this, LogoutActivity.class)));
}

```

// Asks for the camera permission before opening the scanner.

```

private void ensureCamera() {
    if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA) == PackageManager.PERMISSION_GRANTED) {
        startScan();
    } else {
        ActivityCompat.requestPermissions(this, new String[]{Manifest.permission.CAMERA}, CAMERA_REQUEST);
    }
}

```

// Starts the embedded ZXing QR scanner. The code it reads is only the booking token.

```

private void startScan() {
    ScanOptions options = new ScanOptions();
    options.setDesiredBarcodeFormats(ScanOptions.QR_CODE);
    options.setPrompt("Scan the booking QR");
    options.setBeepEnabled(true);
    options.setOrientationLocked(false);
    scanner.launch(options);
}

```

// Sends the scanned token to the API and opens the details screen on success.

```

private void verify(String token) {
    showLoading(true);
    ApiClient.get(this).verifyQr(token, new lk.smartsolar.mobile.data.remote.ApiCallback<Booking>() {
        @Override
        public void onSuccess(Booking booking) {
            showLoading(false);
            Intent intent = new Intent(OperatorHomeActivity.this, QrDetailsActivity.class);
            intent.putExtra("id", booking.id);
            intent.putExtra("station", booking.stationName);
            intent.putExtra("nic", booking.prosumerNic);
            intent.putExtra("when", booking.scheduledAt);
            intent.putExtra("status", booking.status);
            startActivity(intent);
        }

        @Override
        public void onError(lk.smartsolar.mobile.data.remote.ApiError error) {
            showLoading(false);
            showError(error);
        }
    });
}

```

```

// Continues the scan after the operator allows the camera.
@Override
public void onRequestPermissionsResult(int requestCode, @NonNull String[] permissions, @NonNull int[] grantResults) {
    super.onRequestPermissionsResult(requestCode, permissions, grantResults);
    if (requestCode == CAMERA_REQUEST && grantResults.length > 0 && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
        startScan();
    }
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/home/ProsumerHomeActivity.java M4 · 89 lines

/* File: ProsumerHomeActivity.java | Author: samudith | Created: 29/09/2026 */

```
package lk.smartsolar.mobile.ui.home;
```

```
import android.content.Intent;
import android.os.Bundle;
import android.widget.TextView;
```

```
import androidx.swiperefreshlayout.widget.SwipeRefreshLayout;
```

```
import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.DashboardSnapshot;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.ui.account.ProfileActivity;
import lk.smartsolar.mobile.ui.auth.LogoutActivity;
import lk.smartsolar.mobile.ui.bookings.BookingsActivity;
import lk.smartsolar.mobile.ui.reservations.BookSlotActivity;
import lk.smartsolar.mobile.ui.stations.StationListActivity;
import lk.smartsolar.mobile.ui.stations.StationsMapActivity;
import lk.smartsolar.mobile.util.TimeText;
```

```
public class ProsumerHomeActivity extends BaseActivity {
    private SwipeRefreshLayout refresh;
```

```
// Loads the live dashboard, falling back to the last SQLite snapshot.
```

```
@Override
protected void onCreate(Bundle state) {
    super.onCreate(state);
    setContentView(R.layout.activity_prosumer_dashboard);
    refresh = findViewById(R.id.refresh);
    refresh.setOnRefreshListener(this::loadDashboard);
    findViewById(R.id.bookSlotButton).setOnClickListener(v -> startActivity(new Intent(this, BookSlotActivity.class)));
    findViewById(R.id.nearbyStationsButton).setOnClickListener(v -> startActivity(new Intent(this, StationsMapActivity.class)));
    findViewById(R.id.stationListButton).setOnClickListener(v -> startActivity(new Intent(this, StationListActivity.class)));
    findViewById(R.id.bookingsButton).setOnClickListener(v -> startActivity(new Intent(this, BookingsActivity.class)));
    findViewById(R.id.profileButton).setOnClickListener(v -> startActivity(new Intent(this, ProfileActivity.class)));
    findViewById(R.id.logoutButton).setOnClickListener(v -> startActivity(new Intent(this, LogoutActivity.class)));
    showSnapshot(SessionManager.get(this).database().readDashboard(), true);
    loadDashboard();
}

```

```
// Requests GET /dashboard/me and paints the counts.
```

```
private void loadDashboard() {
    refresh.setRefreshing(true);
    ApiClient.get(this).getMyDashboard(new lk.smartsolar.mobile.data.remote.ApiCallback<DashboardSnapshot>() {
        @Override
        public void onSuccess(DashboardSnapshot snapshot) {
            refresh.setRefreshing(false);
            showSnapshot(snapshot, false);
        }

        @Override
        public void onError(lk.smartsolar.mobile.data.remote.ApiError error) {
            refresh.setRefreshing(false);
            DashboardSnapshot cached = SessionManager.get(ProsumerHomeActivity.this).database().readDashboard();
            showSnapshot(cached, true);
            if (cached == null) showError(error);
        }
    });
}

```

```
// Writes the counts onto the screen and labels cached data.
```

```
private void showSnapshot(DashboardSnapshot snapshot, boolean offline) {
    TextView updated = findViewById(R.id.updatedLabel);
    TextView pending = findViewById(R.id.pendingCount);
    TextView approved = findViewById(R.id.approvedCount);
    TextView active = findViewById(R.id.activeCount);
    TextView next = findViewById(R.id.nextBooking);
    if (snapshot == null) {
        updated.setText("No saved dashboard yet");
        pending.setText("Pending: —");
        approved.setText("Approved upcoming: —");
        active.setText("Active: —");
        next.setText("Next booking: none");
        return;
    }
}

```

```
updated.setText((offline ? "Offline. " : "") + TimeText.lastUpdated(snapshot.syncedAt));
pending.setText("Pending: " + snapshot.pendingCount);
approved.setText("Approved upcoming: " + snapshot.approvedFutureCount);
active.setText("Active: " + snapshot.activeCount);

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/operator/PendingQueueActivity.java M4 · 89 lines

SE4040 • Smart Solar Microgrid Trading System • 179

```

    }
    });
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/operator/QrDetailsActivity.java M4 · 62 lines

```

/*
 * File: QrDetailsActivity.java
 * Description: Shows a verified booking and sends Finalise to the API.
 * Author: samudith
 * Created: 29/09/2026
 */
package lk.smartsolar.mobile.ui.operator;

import android.os.Bundle;
import android.view.View;
import android.widget.TextView;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.local.Booking;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiClient;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.util.TimeText;

public class QrDetailsActivity extends BaseActivity {
    private String reservationId;

    // Shows the booking returned by verify-qr.
    @Override
    protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_qr_details);
        reservationId = getIntent().getStringExtra("id");
        String status = getIntent().getStringExtra("status");
        ((TextView) findViewById(R.id.resultTitle)).setText("Approved".equals(status) ? "QR verified" : "Booking");
        ((TextView) findViewById(R.id.resultBody)).setText(
            "Station: " + getIntent().getStringExtra("station")
            + "\nNIC: " + getIntent().getStringExtra("nic")
            + "\nWhen: " + TimeText.sriLanka(getIntent().getStringExtra("when"))
            + "\nStatus: " + status);
        View finalise = findViewById(R.id.finaliseButton);
        finalise.setVisibility("Approved".equals(status) ? View.VISIBLE : View.GONE);
        finalise.setOnClickListener(v -> complete());
    }

    // Marks the job completed. A second attempt is rejected by the API.
    private void complete() {
        showLoading(true);
        ApiClient.get(this).completeReservation(reservationId, new ApiCallback<Booking>() {
            @Override
            public void onSuccess(Booking booking) {
                showLoading(false);
                ((TextView) findViewById(R.id.resultTitle)).setText("Job completed");
                ((TextView) findViewById(R.id.resultBody)).setText(
                    booking.stationName + "\n" + booking.prosumerNic + "\nStatus: " + booking.status);
                findViewById(R.id.finaliseButton).setVisibility(View.GONE);
            }
        });

        @Override
        public void onError(ApiError error) {
            showLoading(false);
            showError(error);
        }
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/reservations/BookSlotActivity.java M3 · 255 lines

```

/*
 * File: BookSlotActivity.java
 * Description: Prosumer booking screen: pick a station, an optional date and a free slot, then confirm.
 * Also used to move an existing booking to another slot.
 * The API checks every booking rule; this screen only shows what it returns.
 * Author: Janukshan S (IT22635266)
 */
package lk.smartsolar.mobile.ui.reservations;

import android.app.DatePickerDialog;
import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.Button;
import android.widget.ListView;
import android.widget.ProgressBar;
import android.widget.Spinner;
import android.widget.TextView;

```

```

import com.google.android.material.dialog.MaterialAlertDialogBuilder;

import java.util.ArrayList;
import java.util.Calendar;
import java.util.List;
import java.util.Locale;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.data.reservations.AvailableSlot;
import lk.smartsolar.mobile.data.reservations.BookingStation;
import lk.smartsolar.mobile.data.reservations.Reservation;
import lk.smartsolar.mobile.data.reservations.ReservationApi;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.util.TimeFormat;

public class BookSlotActivity extends BaseActivity {

    // Lets other screens (e.g. the map's "Book here") open this with a station already chosen.
    public static final String EXTRA_STATION_ID = "stationId";
    private static final String EXTRA_RESERVATION_ID = "reservationId";

    // Set when an existing booking is being moved to another slot.
    private String reservationId;

    private ReservationApi api;
    private Spinner stationSpinner;
    private TextView dateText;
    private ListView slotList;
    private TextView emptyText;
    private ProgressBar progress;
    private Button confirmButton;

    private final List<BookingStation> stations = new ArrayList<>();
    private final List<AvailableSlot> slots = new ArrayList<>();
    private ArrayAdapter<AvailableSlot> slotAdapter;
    private String selectedDate;
    private int slotRequest;

    // Opens the same picker to move an existing booking, starting at its current station.
    public static void openForChange(Context context, Reservation reservation) {
        Intent intent = new Intent(context, BookSlotActivity.class);
        intent.putExtra(EXTRA_RESERVATION_ID, reservation.id);
        intent.putExtra(EXTRA_STATION_ID, reservation.stationId);
        context.startActivity(intent);
    }

    // Sets up the pickers and loads the stations.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_book_slot);

        reservationId = getIntent().getStringExtra(EXTRA_RESERVATION_ID);
        if (reservationId != null) {
            ((TextView) findViewById(R.id.screen_title)).setText("Change slot");
            ((Button) findViewById(R.id.confirm_button)).setText("Save change");
        }

        api = new ReservationApi(this);
        stationSpinner = findViewById(R.id.station_spinner);
        dateText = findViewById(R.id.date_text);
        slotList = findViewById(R.id.slot_list);
        emptyText = findViewById(R.id.empty_text);
        progress = findViewById(R.id.progress);
        confirmButton = findViewById(R.id.confirm_button);

        slotAdapter = new ArrayAdapter<>(this, R.layout.item_slot, slots);
        slotList.setAdapter(slotAdapter);
        slotList.setOnItemClickListener((parent, view, position, id) -> confirmButton.setEnabled(true));

        findViewById(R.id.pick_date_button).setOnClickListener(v -> pickDate());
        findViewById(R.id.any_day_button).setOnClickListener(v -> setDate(null));
        confirmButton.setOnClickListener(v -> confirm());

        loadStations();
    }

    // Loads the stations that are taking bookings.
    private void loadStations() {
        showLoading(true);
        api.getBookableStations(new ApiCallback<List<BookingStation>>() {
            @Override
            public void onSuccess(List<BookingStation> value) {
                showLoading(false);
                stations.clear();
                stations.addAll(value);
                showStations();
            }
        });

        @Override
        public void onError(ApiError error) {

```

```

        showLoading(false);
        showError(error);
    }
});
}

// Fills the station spinner and reloads slots whenever the station changes.
private void showStations() {
    ArrayAdapter<BookingStation> adapter = new ArrayAdapter<>(this, android.R.layout.simple_spinner_item, stations);
    adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
    stationSpinner.setAdapter(adapter);

    String wanted = getIntent().getStringExtra(EXTRA_STATION_ID);
    for (int i = 0; i < stations.size(); i++) {
        if (stations.get(i).id.equals(wanted)) {
            stationSpinner.setSelection(i);
        }
    }

    stationSpinner.setOnItemClickListener(new AdapterView.OnItemClickListener() {
        @Override
        public void onItemClick(AdapterView<?> parent, View view, int position, long id) {
            loadSlots();
        }

        @Override
        public void onNothingSelected(AdapterView<?> parent) {
        }
    });
}

// Opens a date picker. The min/max only guide the user; the API still decides what can be booked.
private void pickDate() {
    Calendar today = Calendar.getInstance();
    DatePickerDialog dialog = new DatePickerDialog(this,
        (view, year, month, day) -> setDate(String.format(Locale.US, "%04d-%02d-%02d", year, month + 1, day)),
        today.get(Calendar.YEAR), today.get(Calendar.MONTH), today.get(Calendar.DAY_OF_MONTH));
    dialog.getDatePicker().setMinDate(today.getTimeInMillis());
    today.add(Calendar.DAY_OF_MONTH, 7);
    dialog.getDatePicker().setMaxDate(today.getTimeInMillis());
    dialog.show();
}

// Sets the date filter (null = any day) and reloads the slots.
private void setDate(String date) {
    selectedDate = date;
    dateText.setText(date == null ? "Any day" : date);
    loadSlots();
}

// Loads free slots for the chosen station and date.
private void loadSlots() {
    BookingStation station = (BookingStation) stationSpinner.getSelectedItem();
    if (station == null) {
        return;
    }

    int request = ++slotRequest;
    slots.clear();
    slotAdapter.notifyDataSetChanged();
    slotList.clearChoices();
    confirmButton.setEnabled(false);
    emptyText.setVisibility(View.GONE);
    progress.setVisibility(View.VISIBLE);

    api.getAvailableSlots(station.id, selectedDate, new ApiCallback<List<AvailableSlot>>() {
        @Override
        public void onSuccess(List<AvailableSlot> value) {
            // Ignore answers to an older request if the user changed the station or date meanwhile.
            if (request != slotRequest) {
                return;
            }
            progress.setVisibility(View.GONE);
            slots.addAll(value);
            slotAdapter.notifyDataSetChanged();
            emptyText.setVisibility(value.isEmpty() ? View.VISIBLE : View.GONE);
        }

        @Override
        public void onError(ApiError error) {
            if (request != slotRequest) {
                return;
            }
            progress.setVisibility(View.GONE);
            showError(error);
        }
    });
}

// Asks the user to confirm the chosen slot before booking.
private void confirm() {
    int position = slotList.getCheckedItemPosition();
    BookingStation station = (BookingStation) stationSpinner.getSelectedItem();
    if (position == ListView.INVALID_POSITION || station == null) {
        return;
    }
}

```

```

        AvailableSlot slot = slots.get(position);
        boolean changing = reservationId != null;
        new MaterialAlertDialogBuilder(this)
            .setTitle(changing ? "Move booking to this slot?" : "Confirm booking")
            .setMessage(station.name + "\n" + TimeFormat.full(slot.startTime) + " - " + TimeFormat.time(slot.endTime)
                + (changing ? "\n\nAn approved booking needs to be approved again after a change." : ""))
            .setPositiveButton(changing ? "Move" : "Book", (dialog, which) -> book(station, slot))
            .setNegativeButton("Back", null)
            .show();
    }

    // Sends the new booking, or the change for an existing one. On success the summary screen is shown.
    private void book(BookingStation station, AvailableSlot slot) {
        confirmButton.setEnabled(false);
        showLoading(true);

        boolean changing = reservationId != null;
        ApiCallback<String> callback = new ApiCallback<String>() {
            @Override
            public void onSuccess(String reservationJson) {
                showLoading(false);
                BookingSummaryActivity.open(BookSlotActivity.this, reservationJson,
                    changing ? BookingSummaryActivity.ACTION_UPDATED : BookingSummaryActivity.ACTION_CREATED);
                finish();
            }

            @Override
            public void onError(ApiError error) {
                showLoading(false);
                showError(error);
                // The slot may have been taken meanwhile, so refresh the list.
                loadSlots();
            }
        };

        if (changing) {
            api.update(reservationId, station.id, slot.id, callback);
        } else {
            api.create(station.id, slot.id, callback);
        }
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/reservations/BookingDetailActivity.java M3 · 156 lines

```

/*
 * File: BookingDetailActivity.java
 * Description: One of the prosumer's bookings with Change slot, Cancel and Show QR. Loads from the API and
 *             falls back to the SQLite copy when there is no signal. The API decides whether a change is allowed.
 * Author: Janukshan S (IT22635266)
 */

```

```

package lk.smartsolar.mobile.ui.reservations;

import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.TextView;

import com.google.android.material.dialog.MaterialAlertDialogBuilder;

import org.json.JSONObject;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.data.reservations.MyReservationStore;
import lk.smartsolar.mobile.data.reservations.Reservation;
import lk.smartsolar.mobile.data.reservations.ReservationApi;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.util.TimeFormat;
import lk.smartsolar.mobile.util.TimeText;

public class BookingDetailActivity extends BaseActivity {

    private static final String EXTRA_ID = "reservationId";

    private ReservationApi api;
    private String reservationId;
    private Reservation reservation;

    // Opens the booking screen for one reservation.
    public static void open(Context context, String reservationId) {
        Intent intent = new Intent(context, BookingDetailActivity.class);
        intent.putExtra(EXTRA_ID, reservationId);
        context.startActivity(intent);
    }

    // Shows the saved copy straight away, then refreshes from the API.
    @Override
    protected void onCreate(Bundle savedInstanceState) {

```

```

super.onCreate(savedInstanceState);
setContentView(R.layout.activity_booking_detail);

api = new ReservationApi(this);
reservationId = getIntent().getStringExtra(EXTRA_ID);

findViewById(R.id.qr_button).setOnClickListener(v -> BookingQrActivity.open(this, reservationId));
findViewById(R.id.change_button).setOnClickListener(v -> BookSlotActivity.openForChange(this, reservation));
findViewById(R.id.cancel_button).setOnClickListener(v -> confirmCancel());

Reservation saved = new MyReservationStore(this).find(reservationId);
if (saved != null) {
    show(saved, false);
}
}

// Reloads every time the screen comes back, so a change or cancel made elsewhere is shown.
@Override
protected void onResume() {
    super.onResume();
    load();
}

// Loads the latest copy from the API. Without signal, the saved copy stays on screen.
private void load() {
    showLoading(reservation == null);
    api.get(reservationId, new ApiCallback<String>() {
        @Override
        public void onSuccess(String json) {
            showLoading(false);
            try {
                show(Reservation.fromJson(new JSONObject(json)), false);
            } catch (Exception ignored) {
            }
        }
    });

    @Override
    public void onError(ApiError error) {
        showLoading(false);
        if (reservation != null && error.status == 0) {
            show(reservation, true);
        } else {
            showError(error);
        }
    }
}

// Fills the card and shows the buttons that make sense for the booking's status.
private void show(Reservation r, boolean offline) {
    reservation = r;

    TextView offlineLabel = findViewById(R.id.offline_label);
    offlineLabel.setVisibility(offline ? View.VISIBLE : View.GONE);
    offlineLabel.setText("Offline. Showing the copy saved on this phone. " + TimeText.lastUpdated(r.syncedAt));

    ((TextView) findViewById(R.id.station_name)).setText(r.stationName);
    ((TextView) findViewById(R.id.slot_time)).setText(TimeFormat.full(r.scheduledAt));
    StatusStyle.chip(findViewById(R.id.status_chip), r.status);
    ((TextView) findViewById(R.id.reference_text)).setText("Reference: " + r.id);

    boolean open = "Pending".equals(r.status) || "Approved".equals(r.status);
    boolean hasQr = "Approved".equals(r.status) && r.qrToken != null && !r.qrToken.isEmpty();

    TextView hint = findViewById(R.id.hint_text);
    if ("Pending".equals(r.status)) {
        hint.setText("Waiting for a grid operator to approve it. The QR code appears once it is approved.");
    } else if (hasQr) {
        hint.setText("Approved. Show the QR code to the grid operator at the station.");
    } else {
        hint.setText("This booking is " + r.status + " and can no longer be changed.");
    }

    // Buttons are hidden offline because changes need the API.
    findViewById(R.id.qr_button).setVisibility(hasQr ? View.VISIBLE : View.GONE);
    findViewById(R.id.change_button).setVisibility(open && !offline ? View.VISIBLE : View.GONE);
    findViewById(R.id.cancel_button).setVisibility(open && !offline ? View.VISIBLE : View.GONE);
}

// Asks before cancelling. The 12-hour rule is checked by the API, not here.
private void confirmCancel() {
    new MaterialAlertDialogBuilder(this)
        .setTitle("Cancel this booking?")
        .setMessage(reservation.stationName + "\n" + TimeFormat.full(reservation.scheduledAt)
            + "\n\nBookings can only be cancelled at least 12 hours before the slot starts.")
        .setPositiveButton("Cancel booking", (dialog, which) -> cancel())
        .setNegativeButton("Keep it", null)
        .show();
}

// Sends the cancellation and shows the summary screen.
private void cancel() {
    showLoading(true);
    api.cancel(reservationId, new ApiCallback<String>() {
        @Override

```

```

        public void onSuccess(String json) {
            showLoading(false);
            BookingSummaryActivity.open(BookingDetailActivity.this, json, BookingSummaryActivity.ACTION_CANCELLED);
            finish();
        }

        @Override
        public void onError(ApiError error) {
            showLoading(false);
            showError(error);
        }
    });
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/reservations/BookingQrActivity.java M3 · 70 lines

```

/*
 * File: BookingQrActivity.java
 * Description: Shows the approved booking's qrToken as a QR code for the grid operator to scan.
 *              Reads the SQLite copy, so it also works without signal at the station.
 * Author: Janukshan S (IT22635266)
 */

package lk.smartsolar.mobile.ui.reservations;

import android.content.Context;
import android.content.Intent;
import android.graphics.Bitmap;
import android.os.Bundle;
import android.view.WindowManager;
import android.widget.ImageView;
import android.widget.TextView;

import com.google.zxing.BarcodeFormat;
import com.journeyapps.barcodescanner.BarcodeEncoder;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.reservations.MyReservationStore;
import lk.smartsolar.mobile.data.reservations.Reservation;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.util.TimeFormat;

public class BookingQrActivity extends BaseActivity {

    private static final String EXTRA_ID = "reservationId";

    // Opens the QR screen for one reservation.
    public static void open(Context context, String reservationId) {
        Intent intent = new Intent(context, BookingQrActivity.class);
        intent.putExtra(EXTRA_ID, reservationId);
        context.startActivity(intent);
    }

    // Draws the QR from the saved booking and keeps the screen bright so it scans easily.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_booking_qr);

        getWindow().addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
        WindowManager.LayoutParams params = getWindow().getAttributes();
        params.screenBrightness = 1f;
        getWindow().setAttributes(params);

        Reservation r = new MyReservationStore(this).find(getIntent().getStringExtra(EXTRA_ID));
        TextView message = findViewById(R.id.qr_message);

        if (r == null || r.qrToken == null || r.qrToken.isEmpty() || !"Approved".equals(r.status)) {
            message.setText("The QR code appears once a grid operator approves the booking.");
            return;
        }

        ((TextView) findViewById(R.id.qr_station)).setText(r.stationName + "\n" + TimeFormat.full(r.scheduledAt));
        ((TextView) findViewById(R.id.qr_reference)).setText("Reference " + r.id);
        message.setText("Show this code to the grid operator at the station.");

        // Adapted from the zxing-android-embedded README ("Generate Barcode" example):
        // https://github.com/journeyapps/zxing-android-embedded#generate-barcode
        try {
            Bitmap bitmap = new BarcodeEncoder().encodeBitmap(r.qrToken, BarcodeFormat.QR_CODE, 800, 800);
            ((ImageView) findViewById(R.id.qr_image)).setImageBitmap(bitmap);
        } catch (Exception e) {
            message.setText("The QR code could not be drawn on this phone.");
        }
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/reservations/BookingSummaryActivity.java M3 · 125 lines

```

/*

```

```

* File: BookingSummaryActivity.java
* Description: Summary shown after a booking is created, updated or cancelled: station, slot time, status and reference.
* Author: Janukshan S (IT22635266)
*/

```

```

package lk.smartsolar.mobile.ui.reservations;

import android.content.Context;
import android.content.Intent;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.widget.LinearLayout;
import android.widget.TextView;

import androidx.activity.OnBackPressedCallback;
import androidx.core.content.ContextCompat;

import org.json.JSONException;
import org.json.JSONObject;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.reservations.Reservation;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.ui.home.ProsumerHomeActivity;
import lk.smartsolar.mobile.util.TimeFormat;

public class BookingSummaryActivity extends BaseActivity {

    public static final String ACTION_CREATED = "created";
    public static final String ACTION_UPDATED = "updated";
    public static final String ACTION_CANCELLED = "cancelled";

    private static final String EXTRA_RESERVATION = "reservation";
    private static final String EXTRA_ACTION = "action";

    // Opens the summary for the reservation the API just returned.
    public static void open(Context context, String reservationJson, String action) {
        Intent intent = new Intent(context, BookingSummaryActivity.class);
        intent.putExtra(EXTRA_RESERVATION, reservationJson);
        intent.putExtra(EXTRA_ACTION, action);
        context.startActivity(intent);
    }

    // Fills the header and the detail rows from the reservation.
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_booking_summary);

        Reservation reservation;
        try {
            reservation = Reservation.fromJson(new JSONObject(getIntent().getStringExtra(EXTRA_RESERVATION)));
        } catch (JSONException | NullPointerException e) {
            finish();
            return;
        }

        showHeader(getIntent().getStringExtra(EXTRA_ACTION));

        LinearLayout details = findViewById(R.id.details);
        addRow(details, "Station", reservation.stationName);
        addRow(details, "Slot time", TimeFormat.full(reservation.scheduledAt));
        TextView status = addRow(details, "Status", reservation.status);
        status.setLayoutParams(new LinearLayout.LayoutParams(LinearLayout.LayoutParams.WRAP_CONTENT, LinearLayout.LayoutParams.WRAP_CONTENT));
        StatusStyle.chip(status, reservation.status);
        addRow(details, "Reference", reservation.id);

        findViewById(R.id.home_button).setOnClickListener(v -> goHome());
        findViewById(R.id.view_button).setOnClickListener(v -> {
            BookingDetailActivity.open(this, reservation.id);
            finish();
        });
        getOnBackPressedDispatcher().addCallback(this, new OnBackPressedCallback(true) {
            @Override
            public void handleOnBackPressed() {
                goHome();
            }
        });
    }

    // Sets the icon, title and colour for what just happened.
    private void showHeader(String action) {
        View header = findViewById(R.id.header);
        TextView icon = findViewById(R.id.header_icon);
        TextView title = findViewById(R.id.header_title);
        TextView message = findViewById(R.id.header_message);

        if (ACTION_CANCELLED.equals(action)) {
            icon.setText("X");
            title.setText("Booking cancelled");
            message.setText("The slot has been released.");
            header.setBackgroundColor(ContextCompat.getColor(this, R.color.status_cancelled));
        } else if (ACTION_UPDATED.equals(action)) {

```

```

        icon.setText("🕒");
        title.setText("Booking updated");
        message.setText("Your booking needs to be approved again before the visit.");
        header.setBackgroundColor(ContextCompat.getColor(this, R.color.status_completed));
    } else {
        icon.setText("✓");
        title.setText("Booking created");
        message.setText("A grid operator will approve it. Your QR code appears once it is approved.");
        header.setBackgroundColor(ContextCompat.getColor(this, R.color.solar_green));
    }
}

// Adds a "label: value" row to the details list and returns the value view.
private TextView addRow(LinearLayout parent, String label, String value) {
    View row = LayoutInflater.from(this).inflate(R.layout.item_summary_row, parent, false);
    ((TextView) row.findViewById(R.id.label)).setText(label);
    TextView valueView = row.findViewById(R.id.value);
    valueView.setText(value);
    parent.addView(row);
    return valueView;
}

// Returns to the home screen without stacking another copy of it.
private void goHome() {
    Intent intent = new Intent(this, ProsumerHomeActivity.class);
    intent.addFlags(Intent.FLAG_ACTIVITY_CLEAR_TOP | Intent.FLAG_ACTIVITY_SINGLE_TOP);
    startActivity(intent);
    finish();
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/reservations/StatusStyle.java M3 · 48 lines

```

/*
 * File: StatusStyle.java
 * Description: Colour used for each reservation status (same colours as the web badges), and a rounded status chip.
 * Author: Janukshan S (IT22635266)
 */

package lk.smartsolar.mobile.ui.reservations;

import android.graphics.drawable.GradientDrawable;
import android.widget.TextView;

import androidx.core.content.ContextCompat;

import lk.smartsolar.mobile.R;

public final class StatusStyle {

    private StatusStyle() {
    }

    // Returns the colour resource for a status.
    public static int colour(String status) {
        if ("Pending".equals(status)) {
            return R.color.status_pending;
        }
        if ("Approved".equals(status)) {
            return R.color.status_approved;
        }
        if ("Completed".equals(status)) {
            return R.color.status_completed;
        }
        return R.color.status_cancelled;
    }

    // Turns a TextView into a small coloured pill showing the status.
    public static void chip(TextView view, String status) {
        float density = view.getResources().getDisplayMetrics().density;
        GradientDrawable background = new GradientDrawable();
        background.setColor(ContextCompat.getColor(view.getContext(), colour(status)));
        background.setCornerRadius(50 * density);
        view.setBackground(background);
        view.setTextColor(ContextCompat.getColor(view.getContext(), R.color.white));
        int horizontal = (int) (12 * density);
        int vertical = (int) (4 * density);
        view.setPadding(horizontal, vertical, horizontal, vertical);
        view.setText(status);
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/settings/SettingsActivity.java M1 · 31 lines

```

/* File: SettingsActivity.java | Author: DE SILVA R K D H (IT22001252) | Created: 27/09/2026 */
package lk.smartsolar.mobile.ui.settings;

import android.content.Context;
import android.os.Bundle;
import android.widget.EditText;
import android.widget.Toast;

```

```

import lk.smartsolar.mobile.BuildConfig;
import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.ui.BaseActivity;

public class SettingsActivity extends BaseActivity {
    @Override protected void onCreate(Bundle state) {
        super.onCreate(state); setContentView(R.layout.activity_settings);
        EditText field = findViewById(R.id.apiBaseUrl);
        field.setText(getSharedPreferences("settings", Context.MODE_PRIVATE).getString("api_base_url", BuildConfig.API_BASE_URL));
        findViewById(R.id.saveSettingsButton).setOnClickListener(v -> {
            String url = field.getText().toString().trim();
            if (!url.startsWith("http://") && !url.startsWith("https://")) {
                showError(new ApiError(400, "VALIDATION_ERROR", "Enter a complete http:// or https:// API URL.));
                return;
            }
            if (!url.endsWith("/")) url += "/";
            getSharedPreferences("settings", Context.MODE_PRIVATE).edit().putString("api_base_url", url).apply();
            Toast.makeText(this, "API address saved", Toast.LENGTH_SHORT).show();
            finish();
        });
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/stations/StationListActivity.java M2 · 143 lines

```

/*
 * File: StationListActivity.java
 * Description: Nearby stations as a list. A row opens the same booking screen as the map.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
package lk.smartsolar.mobile.ui.stations;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.view.View;
import android.widget.AdapterView;
import android.widget.AdapterView.OnItemClickListener;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.TextView;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.core.content.ContextCompat;

import com.google.android.gms.location.LocationServices;
import com.google.android.gms.maps.model.LatLng;

import java.util.ArrayList;
import java.util.List;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.data.stations.NearbyStation;
import lk.smartsolar.mobile.data.stations.StationApi;
import lk.smartsolar.mobile.data.stations.StationCacheStore;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.ui.reservations.BookSlotActivity;
import lk.smartsolar.mobile.util.TimeText;

public class StationListActivity extends BaseActivity {

    private static final LatLng COLOMBO = new LatLng(6.9271, 79.8612);

    private StationCacheStore cache;
    private StationApi api;
    private TextView offlineLabel;
    private ArrayAdapter<NearbyStation> adapter;
    private ActivityResultLauncher<String[]> locationPermission;

    // Shows the saved list, then refreshes it from the phone's location.
    @Override
    protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_station_list);
        cache = new StationCacheStore(this);
        api = new StationApi(this);
        offlineLabel = findViewById(R.id.offlineLabel);
        adapter = new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, new ArrayList<>());
        ListView list = findViewById(R.id.stationList);
        list.setAdapter(adapter);
        list.setOnItemClickListener((parent, view, position, id) -> {
            NearbyStation station = adapter.getItem(position);
            if (station == null) return;
            startActivity(new Intent(this, BookSlotActivity.class)
                .putExtra(BookSlotActivity.EXTRA_STATION_ID, station.id));
        });
        show(cache.loadAll(), cache.lastSyncedAt() > 0);
        locationPermission = registerForActivityResult(
            new ActivityResultContracts.RequestMultiplePermissions(),
            result -> {
                boolean granted = Boolean.TRUE.equals(result.get(Manifest.permission.ACCESS_FINE_LOCATION))
            }
        );
    }
}

```

```

        || Boolean.TRUE.equals(result.get(Manifest.permission.ACCESS_COARSE_LOCATION));
        if (granted) {
            useDeviceLocation();
        } else {
            loadAround(COLOMBO);
        }
    });
    if (hasLocationPermission()) {
        useDeviceLocation();
    } else {
        locationPermission.launch(new String[]{
            Manifest.permission.ACCESS_FINE_LOCATION,
            Manifest.permission.ACCESS_COARSE_LOCATION
        });
    }
}

// True when the user already allowed fine or coarse location.
private boolean hasLocationPermission() {
    return ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED
        || ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_COARSE_LOCATION) == PackageManager.PERMISSION_GRANTED;
}

// Loads stations around the last known point, or Colombo when location is missing.
private void useDeviceLocation() {
    if (!hasLocationPermission()) {
        loadAround(COLOMBO);
        return;
    }
    LocationServices.getFusedLocationProviderClient(this).getLastLocation()
        .addOnSuccessListener(this, location -> {
            if (location == null) {
                loadAround(COLOMBO);
            } else {
                loadAround(new LatLng(location.getLatitude(), location.getLongitude()));
            }
        })
        .addOnFailureListener(this, error -> loadAround(COLOMBO));
}

// Asks the API for stations near this point and saves them for the map.
private void loadAround(LatLng point) {
    api.getNearby(point.latitude, point.longitude, new ApiCallback<List<NearbyStation>>() {
        @Override
        public void onSuccess(List<NearbyStation> stations) {
            cache.saveAll(stations);
            show(stations, false);
        }

        @Override
        public void onError(ApiError error) {
            show(cache.loadAll(), true);
        }
    });
}

// Paints the rows. Offline keeps the last saved list and the banner.
private void show(List<NearbyStation> stations, boolean offline) {
    adapter.clear();
    adapter.addAll(stations);
    if (!offline) {
        offlineLabel.setVisibility(View.GONE);
        return;
    }
    long syncedAt = cache.lastSyncedAt();
    if (syncedAt <= 0) {
        offlineLabel.setText("Showing saved stations");
    } else {
        String when = TimeText.lastUpdated(syncedAt);
        offlineLabel.setText("Showing saved stations, " + Character.toLowerCase(when.charAt(0)) + when.substring(1));
    }
    offlineLabel.setVisibility(View.VISIBLE);
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/ui/stations/StationsMapActivity.java M2 · 186 lines

```

/*
 * File: StationsMapActivity.java
 * Description: Nearby stations on a Google Map. Cached pins show first; a tap opens booking.
 * Author: Mohamed Asath (IT22633422)
 * Created: 30/09/2026
 */
package lk.smartsolar.mobile.ui.stations;

import android.Manifest;
import android.content.Intent;
import android.content.pm.PackageManager;
import android.os.Bundle;
import android.view.View;
import android.widget.TextView;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.core.content.ContextCompat;

```

```

import com.google.android.gms.location.LocationServices;
import com.google.android.gms.maps.CameraUpdateFactory;
import com.google.android.gms.maps.GoogleMap;
import com.google.android.gms.maps.OnMapReadyCallback;
import com.google.android.gms.maps.SupportMapFragment;
import com.google.android.gms.maps.model.LatLng;
import com.google.android.gms.maps.model.Marker;
import com.google.android.gms.maps.model.MarkerOptions;
import com.google.android.material.bottomsheet.BottomSheetDialog;

import java.util.List;

import lk.smartsolar.mobile.R;
import lk.smartsolar.mobile.data.remote.ApiCallback;
import lk.smartsolar.mobile.data.remote.ApiError;
import lk.smartsolar.mobile.data.stations.NearbyStation;
import lk.smartsolar.mobile.data.stations.StationApi;
import lk.smartsolar.mobile.data.stations.StationCacheStore;
import lk.smartsolar.mobile.ui.BaseActivity;
import lk.smartsolar.mobile.ui.reservations.BookSlotActivity;
import lk.smartsolar.mobile.util.TimeText;

public class StationsMapActivity extends BaseActivity implements OnMapReadyCallback {

    private static final LatLng COLOMBO = new LatLng(6.9271, 79.8612);

    private GoogleMap map;
    private StationCacheStore cache;
    private StationApi api;
    private TextView offlineLabel;
    private ActivityResultLauncher<String[]> locationPermission;

    // Shows the map, then asks Google Maps to call back when the tiles are ready.
    @Override
    protected void onCreate(Bundle state) {
        super.onCreate(state);
        setContentView(R.layout.activity_stations_map);
        cache = new StationCacheStore(this);
        api = new StationApi(this);
        offlineLabel = findViewById(R.id.offlineLabel);
        locationPermission = registerForActivityResult(
            new ActivityResultContracts.RequestMultiplePermissions(),
            result -> {
                boolean granted = Boolean.TRUE.equals(result.get(Manifest.permission.ACCESS_FINE_LOCATION))
                    || Boolean.TRUE.equals(result.get(Manifest.permission.ACCESS_COARSE_LOCATION));
                if (granted) {
                    useDeviceLocation();
                } else {
                    loadAround(COLOMBO);
                }
            });
        SupportMapFragment fragment = (SupportMapFragment) getSupportFragmentManager().findFragmentById(R.id.map);
        fragment.getMapAsync(this);
    }

    // Draws the saved pins first, then asks for the phone's location.
    @Override
    public void onMapReady(GoogleMap googleMap) {
        map = googleMap;
        map.setOnMarkerClickListener(marker -> {
            Object tag = marker.getTag();
            if (tag instanceof NearbyStation) {
                showStation((NearbyStation) tag);
            }
            return true;
        });
        draw(cache.loadAll());
        if (hasLocationPermission()) {
            useDeviceLocation();
        } else {
            locationPermission.launch(new String[]{
                Manifest.permission.ACCESS_FINE_LOCATION,
                Manifest.permission.ACCESS_COARSE_LOCATION
            });
        }
    }

    // True when the user already allowed fine or coarse location.
    private boolean hasLocationPermission() {
        return ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_FINE_LOCATION) == PackageManager.PERMISSION_GRANTED
            || ContextCompat.checkSelfPermission(this, Manifest.permission.ACCESS_COARSE_LOCATION) == PackageManager.PERMISSION_GRANTED;
    }

    // Turns on the blue dot and loads stations around the last known point.
    private void useDeviceLocation() {
        if (!hasLocationPermission()) {
            loadAround(COLOMBO);
            return;
        }
        try {
            map.setMyLocationEnabled(true);
        } catch (SecurityException ignored) {
            loadAround(COLOMBO);
            return;
        }
        LocationServices.getFusedLocationProviderClient(this).getLastLocation()
            .addOnSuccessListener(this, location -> {
                if (location == null) {

```

```

        loadAround(COLOMBO);
    } else {
        loadAround(new LatLng(location.getLatitude(), location.getLongitude()));
    }
})
.addOnFailureListener(this, error -> loadAround(COLOMBO));
}

// Moves the camera, then replaces the pins when the nearby call succeeds.
private void loadAround(LatLng point) {
    map.moveCamera(CameraUpdateFactory.newLatLngZoom(point, 12));
    api.getNearby(point.latitude, point.longitude, new ApiCallback<List<NearbyStation>>() {
        @Override
        public void onSuccess(List<NearbyStation> stations) {
            cache.saveAll(stations);
            draw(stations);
            offlineLabel.setVisibility(View.GONE);
        }

        @Override
        public void onError(ApiError error) {
            showOffline();
        }
    });
}

// Clears the map and drops one pin per station.
private void draw(List<NearbyStation> stations) {
    map.clear();
    for (NearbyStation station : stations) {
        Marker marker = map.addMarker(new MarkerOptions()
            .position(new LatLng(station.latitude, station.longitude))
            .title(station.name));
        if (marker != null) {
            marker.setTag(station);
        }
    }
}

// Shows the cached-data banner with the time of the last successful save.
private void showOffline() {
    long syncedAt = cache.lastSyncedAt();
    if (syncedAt <= 0) {
        offlineLabel.setText("Showing saved stations");
    } else {
        String when = TimeText.lastUpdated(syncedAt);
        offlineLabel.setText("Showing saved stations, " + Character.toLowerCase(when.charAt(0)) + when.substring(1));
    }
    offlineLabel.setVisibility(View.VISIBLE);
}

// Opens the station details and a button into Janukshan's booking screen.
private void showStation(NearbyStation station) {
    BottomSheetDialog sheet = new BottomSheetDialog(this);
    View view = getLayoutInflater().inflate(R.layout.bottom_sheet_station, null);
    ((TextView) view.findViewById(R.id.stationName)).setText(station.name);
    ((TextView) view.findViewById(R.id.stationDistance)).setText(station.distanceKm + " km away");
    ((TextView) view.findViewById(R.id.stationCapacity)).setText("Capacity: " + station.capacityKwh + " kWh");
    ((TextView) view.findViewById(R.id.stationSlots)).setText("Battery slots: " + station.batterySlotsTotal);
    ((TextView) view.findViewById(R.id.stationFreeSlots)).setText("Free slots: " + station.freeSlots);
    view.findViewById(R.id.bookHereButton).setOnClickListener(v -> {
        sheet.dismiss();
        startActivity(new Intent(this, BookSlotActivity.class)
            .putExtra(BookSlotActivity.EXTRA_STATION_ID, station.id));
    });
    sheet.setContentView(view);
    sheet.show();
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/util/RoleRouter.java M1 · 36 lines

```

/*
 * File: RoleRouter.java
 * Description: Central post-login routing shared by splash and login.
 * Author: DE SILVA R K D H (IT22001252)
 * Created: 27/09/2026
 */
package lk.smartsolar.mobile.util;

import android.app.Activity;
import android.content.Intent;

import lk.smartsolar.mobile.data.local.Session;
import lk.smartsolar.mobile.data.local.SessionManager;
import lk.smartsolar.mobile.ui.auth.LoginActivity;
import lk.smartsolar.mobile.ui.home.OperatorHomeActivity;
import lk.smartsolar.mobile.ui.home.ProsumerHomeActivity;

public final class RoleRouter {
    private RoleRouter() { }

    // Sends each role to its home. Backoffice belongs on the web desk, so that session is cleared.
    public static void route(Activity source, Session session) {
        if (session != null && "Backoffice".equals(session.role)) {
            SessionManager.get(source).clear();

```

```

        session = null;
    }
    Class<?> target;
    if (session == null) target = LoginActivity.class;
    else if ("Prosumer".equals(session.role)) target = ProsumerHomeActivity.class;
    else if ("GridOperator".equals(session.role)) target = OperatorHomeActivity.class;
    else target = LoginActivity.class;
    Intent intent = new Intent(source, target).addFlags(Intent.FLAG_ACTIVITY_NEW_TASK | Intent.FLAG_ACTIVITY_CLEAR_TASK);
    source.startActivity(intent);
    source.finish();
}
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/util/TimeFormat.java M3 · 61 lines

```

/*
 * File: TimeFormat.java
 * Description: Shows the API's UTC times in Sri Lanka time. Display only, no booking rules.
 * Author: Janukshan S (IT2635266)
 */

package lk.smartsolar.mobile.util;

import java.text.ParseException;
import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.Locale;
import java.util.TimeZone;

public final class TimeFormat {

    private static final TimeZone SRI_LANKA = TimeZone.getTimeZone("Asia/Colombo");

    private TimeFormat() {
    }

    // Parses an API time like "2026-09-28T02:30:00Z" or "2026-09-28T02:30:00.123Z". Seconds fractions are dropped.
    public static Date parse(String iso) {
        if (iso == null || iso.length() < 19) {
            return null;
        }
        SimpleDateFormat format = new SimpleDateFormat("yyyy-MM-dd'THH:mm:ss", Locale.US);
        format.setTimeZone(TimeZone.getTimeZone("UTC"));
        try {
            return format.parse(iso.substring(0, 19));
        } catch (ParseException e) {
            return null;
        }
    }

    // e.g. "Mon 28 Sep 2026, 08:00"
    public static String full(String iso) {
        return format(iso, "EEE dd MMM yyyy, HH:mm");
    }

    // e.g. "Mon 28 Sep"
    public static String day(String iso) {
        return format(iso, "EEE dd MMM");
    }

    // e.g. "08:00"
    public static String time(String iso) {
        return format(iso, "HH:mm");
    }

    // Formats a UTC API time in Sri Lanka time with the given pattern.
    private static String format(String iso, String pattern) {
        Date date = parse(iso);
        if (date == null) {
            return "";
        }
        SimpleDateFormat format = new SimpleDateFormat(pattern, Locale.US);
        format.setTimeZone(SRI_LANKA);
        return format.format(date);
    }
}

```

MobileApp/app/src/main/java/lk/smartsolar/mobile/util/TimeText.java M4 · 35 lines

```

/*
 * File: TimeText.java
 * Description: Shows API UTC timestamps in Sri Lanka time. Display only.
 * Author: samudith
 * Created: 29/09/2026
 */
package lk.smartsolar.mobile.util;

import java.time.OffsetDateTime;
import java.time.ZoneId;
import java.time.format.DateTimeFormatter;
import java.util.Date;

```

```

public final class TimeText {
    private static final DateTimeFormatter FORMAT = DateTimeFormatter.ofPattern("EEE dd MMM yyyy, HH:mm");

    private TimeText() {
    }

    // Formats an ISO-8601 UTC string, or returns it unchanged when it cannot be parsed.
    public static String sriLanka(String iso) {
        if (iso == null || iso.isEmpty()) return "";
        try {
            return OffsetDateTime.parse(iso).atZoneSameInstant(ZoneId.of("Asia/Colombo")).format(FORMAT);
        } catch (Exception ignored) {
            return iso;
        }
    }

    // Formats a millisecond timestamp as a local "last updated" label.
    public static String lastUpdated(long millis) {
        if (millis <= 0) return "Not synced yet";
        return "Last updated " + android.text.format.DateFormat.format("dd MMM yyyy, HH:mm", new Date(millis));
    }
}

```

MobileApp/app/src/main/res/drawable/bg_slot.xml M3 · 17 lines

```

<?xml version="1.0" encoding="utf-8"?>
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:state_activated="true">
        <shape android:shape="rectangle">
            <solid android:color="#E3F2EA" />
            <stroke android:width="2dp" android:color="@color/solar_green" />
            <corners android:radius="12dp" />
        </shape>
    </item>
    <item>
        <shape android:shape="rectangle">
            <solid android:color="@color/white" />
            <stroke android:width="1dp" android:color="#D5DDD8" />
            <corners android:radius="12dp" />
        </shape>
    </item>
</selector>

```

MobileApp/app/src/main/res/drawable/sun_dot.xml M1 · 4 lines

```

<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android" android:shape="oval">
    <solid android:color="@color/solar_gold" />
</shape>

```

MobileApp/app/src/main/res/layout/activity_book_slot.xml M3 · 141 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/screen_title"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Book a slot"
        android:textColor="@color/solar_green_dark"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="Choose a station and a free time. A grid operator approves it, then your QR code is ready."
        android:textSize="14sp" />

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        app:cardCornerRadius="12dp"
        app:cardElevation="2dp">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <TextView
                style="@style/TextAppearance.MaterialComponents.Subtitle2"

```

```

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Station"
        android:textColor="@color/solar_green_dark" />

<Spinner
    android:id="@+id/station_spinner"
    android:layout_width="match_parent"
    android:layout_height="48dp" />

<TextView
    style="@style/TextAppearance.MaterialComponents.Subtitle2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Day"
    android:textColor="@color/solar_green_dark" />

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <TextView
        android:id="@+id/date_text"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="Any day"
        android:textColor="@color/black"
        android:textSize="16sp"
        android:textStyle="bold" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/pick_date_button"
        style="@style/Widget.SmartSolar.Button.Text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Pick date" />

    <com.google.android.material.button.MaterialButton
        android:id="@+id/any_day_button"
        style="@style/Widget.SmartSolar.Button.Text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Any day" />

</LinearLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<TextView
    style="@style/TextAppearance.MaterialComponents.Subtitle2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:layout_marginBottom="8dp"
    android:text="Free slots"
    android:textColor="@color/solar_green_dark" />

<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1">

    <ListView
        android:id="@+id/slot_list"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:choiceMode="singleChoice"
        android:divider="@android:color/transparent"
        android:dividerHeight="8dp"
        android:listSelector="@android:color/transparent" />

    <TextView
        android:id="@+id/empty_text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:gravity="center"
        android:text="No free slots.\nTry another day or station."
        android:visibility="gone" />

    <ProgressBar
        android:id="@+id/progress"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:visibility="gone" />
</FrameLayout>

<com.google.android.material.button.MaterialButton
    android:id="@+id/confirm_button"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:layout_marginTop="12dp"

```

```

        android:enabled="false"
        android:text="Confirm booking"
        android:textSize="16sp"
        app:cornerRadius="12dp" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_booking_detail.xml M3 · 133 lines

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background">

```

```

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

```

```

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="My booking"
            android:textColor="@color/solar_green_dark"
            android:textSize="28sp"
            android:textStyle="bold" />

```

```

        <TextView
            android:id="@+id/offline_label"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:background="#FFF4D6"
            android:padding="10dp"
            android:textColor="@color/black"
            android:textSize="13sp"
            android:visibility="gone" />

```

```

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            app:cardBackgroundColor="@color/solar_green_dark"
            app:cardCornerRadius="16dp"
            app:cardElevation="2dp">

```

```

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="20dp">

```

```

                <TextView
                    android:id="@+id/station_name"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:textColor="@color/white"
                    android:textSize="22sp"
                    android:textStyle="bold" />

```

```

                <TextView
                    android:id="@+id/slot_time"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="4dp"
                    android:textColor="@color/white"
                    android:textSize="16sp" />

```

```

                <TextView
                    android:id="@+id/status_chip"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="12dp"
                    android:textSize="13sp"
                    android:textStyle="bold" />
            </LinearLayout>

```

```

        </com.google.android.material.card.MaterialCardView>

```

```

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            app:cardCornerRadius="12dp"
            app:cardElevation="1dp">

```

```

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="16dp">

```

```

                <TextView
                    android:id="@+id/hint_text"

```

```

        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/black"
        android:textSize="15sp" />

        <TextView
            android:id="@+id/reference_text"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="12dp"
            android:textIsSelectable="true"
            android:textSize="12sp" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.button.MaterialButton
    android:id="@+id/qr_button"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:layout_marginTop="16dp"
    android:text="Show QR code"
    android:textSize="16sp"
    android:visibility="gone"
    app:cornerRadius="12dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/change_button"
    style="@style/Widget.SmartSolar.Button.Outlined"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:layout_marginTop="8dp"
    android:text="Change slot"
    android:visibility="gone"
    app:cornerRadius="12dp" />

<com.google.android.material.button.MaterialButton
    android:id="@+id/cancel_button"
    style="@style/Widget.SmartSolar.Button.Text"
    android:layout_width="match_parent"
    android:layout_height="56dp"
    android:layout_marginTop="4dp"
    android:text="Cancel booking"
    android:textColor="#B3261E"
    android:visibility="gone" />
</LinearLayout>
</ScrollView>

```

MobileApp/app/src/main/res/layout/activity_booking_qr.xml M3 · 67 lines

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_green_dark"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center_horizontal"
        android:orientation="vertical"
        android:padding="24dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Your booking QR"
            android:textColor="@color/white"
            android:textSize="26sp"
            android:textStyle="bold" />

        <TextView
            android:id="@+id/qr_station"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="8dp"
            android:gravity="center"
            android:textColor="@color/white"
            android:textSize="17sp" />

        <com.google.android.material.card.MaterialCardView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="24dp"
            app:cardBackgroundColor="@color/white"
            app:cardCornerRadius="20dp"
            app:cardElevation="4dp">

            <ImageView
                android:id="@+id/qr_image"
                android:layout_width="280dp"
                android:layout_height="280dp"
                android:layout_margin="16dp"
                android:contentDescription="Booking QR code" />
        </com.google.android.material.card.MaterialCardView>
    </LinearLayout>
</ScrollView>

```

```

<TextView
    android:id="@+id/qr_message"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    android:gravity="center"
    android:textColor="@color/white"
    android:textSize="16sp" />

<TextView
    android:id="@+id/qr_reference"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:gravity="center"
    android:textColor="#C8E6D5"
    android:textSize="12sp" />
</LinearLayout>
</ScrollView>

```

MobileApp/app/src/main/res/layout/activity_booking_summary.xml M3 · 79 lines

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            app:cardCornerRadius="12dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical">

                <LinearLayout
                    android:id="@+id/header"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:gravity="center"
                    android:orientation="vertical"
                    android:padding="24dp">

                    <TextView
                        android:id="@+id/header_icon"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:textColor="@android:color/white"
                        android:textSize="48sp" />

                    <TextView
                        android:id="@+id/header_title"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:textColor="@android:color/white"
                        android:textSize="22sp"
                        android:textStyle="bold" />

                    <TextView
                        android:id="@+id/header_message"
                        android:layout_width="wrap_content"
                        android:layout_height="wrap_content"
                        android:layout_marginTop="4dp"
                        android:gravity="center"
                        android:textColor="@android:color/white" />
                </LinearLayout>

                <LinearLayout
                    android:id="@+id/details"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:orientation="vertical"
                    android:padding="16dp" />
            </LinearLayout>
        </com.google.android.material.card.MaterialCardView>

        <com.google.android.material.button.MaterialButton
            android:id="@+id/home_button"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="Back to home" />

        <com.google.android.material.button.MaterialButton

```

```

        android:id="@+id/view_button"
        style="@style/Widget.SmartSolar.Button.Outlined"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="View booking" />
    </LinearLayout>
</ScrollView>

```

MobileApp/app/src/main/res/layout/activity_bookings.xml M3 · 90 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/screenTitle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/solar_green_dark"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/updatedLabel"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:textSize="13sp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="12dp"
        android:orientation="horizontal">

        <Button
            android:id="@+id/currentTab"
            style="@style/Widget.SmartSolar.Button.Outlined"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:minHeight="48dp"
            android:text="Current" />

        <Button
            android:id="@+id/pendingTab"
            style="@style/Widget.SmartSolar.Button.Outlined"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:layout_weight="1"
            android:minHeight="48dp"
            android:text="Pending" />

        <Button
            android:id="@+id/historyTab"
            style="@style/Widget.SmartSolar.Button.Outlined"
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_marginStart="8dp"
            android:layout_weight="1"
            android:minHeight="48dp"
            android:text="History" />
    </LinearLayout>

    <com.google.android.material.textfield.TextInputLayout
        style="@style/Widget.SmartSolar.TextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Search station, NIC or reference">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/searchBox"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:imeOptions="actionSearch"
            android:inputType="text"
            android:singleLine="true" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_marginTop="12dp"
        android:layout_weight="1"
        app:cardCornerRadius="16dp"
        app:cardElevation="2dp">

        <ListView

```

```

        android:id="@+id/bookingList"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:padding="8dp" />
    </com.google.android.material.card.MaterialCardView>
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_deactivation.xml M3 · 50 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Deactivate account?"
        android:textColor="@color/solar_green_dark"
        android:textSize="28sp"
        android:textStyle="bold" />

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        app:cardBackgroundColor="#FFF4D6"
        app:cardCornerRadius="16dp"
        app:cardElevation="0dp">

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="20dp"
            android:text="You will be logged out immediately. Only Backoffice staff can reactivate this account."
            android:textColor="@color/black"
            android:textSize="16sp" />
    </com.google.android.material.card.MaterialCardView>

    <Button
        android:id="@+id/confirmDeactivateButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="24dp"
        android:backgroundTint="#B3261E"
        android:text="Confirm deactivation" />

    <Button
        android:id="@+id/cancelDeactivateButton"
        style="@style/Widget.SmartSolar.Button.Text"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:text="Cancel" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_home.xml M3 · 8 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android" android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical" android:padding="24dp">
    <TextView android:id="@+id/homeTitle" android:layout_width="wrap_content" android:layout_height="wrap_content" android:textSize="28sp"
    android:textStyle="bold" android:textColor="@color/solar_green_dark" />
    <TextView android:id="@+id/homeMessage" android:layout_width="match_parent" android:layout_height="wrap_content" android:layout_marginTop="12dp"
    android:textSize="17sp" />
    <Button android:id="@+id/bookButton" android:layout_width="match_parent" android:layout_height="wrap_content" android:layout_marginTop="24dp"
    android:text="Book a slot" android:visibility="gone" />
    <Button android:id="@+id/profileButton" android:layout_width="match_parent" android:layout_height="wrap_content" android:layout_marginTop="8dp"
    android:text="My profile" />
    <Button android:id="@+id/logoutButton" style="@style/Widget.SmartSolar.Button.Outlined" android:layout_width="match_parent"
    android:layout_height="wrap_content" android:layout_marginTop="8dp" android:text="Log out" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_login.xml M1 · 103 lines

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background"
    android:fillViewport="true">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center_vertical"
        android:orientation="vertical"
        android:padding="24dp">

```

```

<ImageView
    android:layout_width="20dp"
    android:layout_height="20dp"
    android:contentDescription="@null"
    android:src="@drawable/sun_dot" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Welcome back"
    android:textAppearance="@style/TextAppearance.SmartSolar.Title" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="Log in with your username or NIC."
    android:textSize="15sp" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="24dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="2dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <com.google.android.material.textfield.TextInputLayout
            style="@style/Widget.SmartSolar.TextInput"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="0dp"
            android:hint="Username or NIC">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/loginIdentifier"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="text" />
        </com.google.android.material.textfield.TextInputLayout>

        <com.google.android.material.textfield.TextInputLayout
            style="@style/Widget.SmartSolar.TextInput"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:hint="Password"
            app:enableIconMode="password_toggle">

            <com.google.android.material.textfield.TextInputEditText
                android:id="@+id/loginPassword"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:inputType="textPassword" />
        </com.google.android.material.textfield.TextInputLayout>

        <Button
            android:id="@+id/loginButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="20dp"
            android:text="Log in" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>

<Button
    android:id="@+id/registerButton"
    style="@style/Widget.SmartSolar.Button.Outlined"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="Create prosumer account" />

<Button
    android:id="@+id/settingsButton"
    style="@style/Widget.SmartSolar.Button.Text"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:layout_marginTop="8dp"
    android:text="API settings" />
</LinearLayout>
</ScrollView>

```

MobileApp/app/src/main/res/layout/activity_operator_home.xml M3 · 57 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"

```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@color/solar_background"
android:orientation="vertical"
android:padding="16dp">

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Operator"
    android:textColor="@color/solar_green_dark"
    android:textSize="28sp"
    android:textStyle="bold" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:cardBackgroundColor="@color/solar_green_dark"
    app:cardCornerRadius="16dp"
    app:cardElevation="2dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="20dp"
        android:text="Approve a booking, then scan the prosumer's QR code and finalise the job."
        android:textColor="@color/white"
        android:textSize="16sp" />
</com.google.android.material.card.MaterialCardView>

<Button
    android:id="@+id/scanButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="Scan QR" />

<Button
    android:id="@+id/pendingButton"
    style="@style/Widget.SmartSolar.Button.Outlined"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Pending approvals" />

<Button
    android:id="@+id/logoutButton"
    style="@style/Widget.SmartSolar.Button.Text"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="Log out" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_pending_queue.xml M3 · 31 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Pending approvals"
        android:textColor="@color/solar_green_dark"
        android:textSize="28sp"
        android:textStyle="bold" />

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="16dp"
        app:cardCornerRadius="16dp"
        app:cardElevation="2dp">

        <ListView
            android:id="@+id/pendingList"
            android:layout_width="match_parent"
            android:layout_height="match_parent"
            android:padding="8dp" />
    </com.google.android.material.card.MaterialCardView>
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_profile.xml M3 · 157 lines

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"

```

```

xmlns:app="http://schemas.android.com/apk/res-auto"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:background="@color/solar_background">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

```

```

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="My profile"
    android:textColor="@color/solar_green_dark"
    android:textSize="28sp"
    android:textStyle="bold" />

```

```

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="Update your details. Your NIC cannot be changed."
    android:textSize="14sp" />

```

```

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:cardBackgroundColor="@color/solar_green_dark"
    app:cardCornerRadius="16dp"
    app:cardElevation="2dp">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

```

```

<TextView
    android:id="@+id/profileNic"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:textColor="@color/white"
    android:textSize="18sp"
    android:textStyle="bold" />

```

```

<TextView
    android:id="@+id/profileStatus"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:textColor="#C8E6D5"
    android:textSize="15sp" />

```

```

</LinearLayout>

```

```

</com.google.android.material.card.MaterialCardView>

```

```

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    app:cardCornerRadius="16dp"
    app:cardElevation="2dp">

```

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

```

```

<com.google.android.material.textfield.TextInputLayout
    style="@style/Widget.SmartSolar.TextInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="0dp"
    android:hint="Username">

```

```

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/profileUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content" />

```

```

</com.google.android.material.textfield.TextInputLayout>

```

```

<com.google.android.material.textfield.TextInputLayout
    style="@style/Widget.SmartSolar.TextInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Full name">

```

```

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/profileFullName"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPersonName" />

```

```

</com.google.android.material.textfield.TextInputLayout>

```

```

<com.google.android.material.textfield.TextInputLayout
    style="@style/Widget.SmartSolar.TextInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/profileEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textEmailAddress" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    style="@style/Widget.SmartSolar.TextInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Phone">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/profilePhone"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="phone" />
</com.google.android.material.textfield.TextInputLayout>

<com.google.android.material.textfield.TextInputLayout
    style="@style/Widget.SmartSolar.TextInput"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="New password (optional)"
    app:passwordToggle="password_toggle">

    <com.google.android.material.textfield.TextInputEditText
        android:id="@+id/profilePassword"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="textPassword" />
</com.google.android.material.textfield.TextInputLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<Button
    android:id="@+id/saveProfileButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="Save profile" />

<Button
    android:id="@+id/deactivateButton"
    style="@style/Widget.SmartSolar.Button.Outlined"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Request deactivation" />
</LinearLayout>
</ScrollView>

```

MobileApp/app/src/main/res/layout/activity_prosumer_dashboard.xml M2 · 144 lines

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.swiperefreshlayout.widget.SwipeRefreshLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/refresh"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:background="@color/solar_background">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Your dashboard"
                android:textAppearance="@style/TextAppearance.SmartSolar.Title" />

            <TextView
                android:id="@+id/updatedLabel"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="4dp"
                android:textSize="13sp" />

        </LinearLayout>
    </ScrollView>

```

```

android:layout_width="match_parent"
android:layout_height="wrap_content"
android:layout_marginTop="16dp"
android:orientation="horizontal">

<com.google.android.material.card.MaterialCardView
    style="@style/Widget.SmartSolar.Card"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginEnd="6dp"
    android:layout_weight="1"
    app:cardBackgroundColor="@color/status_pending_fill">

    <TextView
        android:id="@+id/pendingCount"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp"
        android:textAppearance="@style/TextAppearance.SmartSolar.Section"
        android:textColor="@color/status_pending" />
</com.google.android.material.card.MaterialCardView>

<com.google.android.material.card.MaterialCardView
    style="@style/Widget.SmartSolar.Card"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_marginStart="6dp"
    android:layout_weight="1"
    app:cardBackgroundColor="@color/status_approved_fill">

    <TextView
        android:id="@+id/approvedCount"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="16dp"
        android:textAppearance="@style/TextAppearance.SmartSolar.Section"
        android:textColor="@color/status_approved" />
</com.google.android.material.card.MaterialCardView>
</LinearLayout>

<TextView
    android:id="@+id/activeCount"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:textAppearance="@style/TextAppearance.SmartSolar.Caption" />

<com.google.android.material.card.MaterialCardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="12dp"
    app:cardBackgroundColor="@color/solar_green_dark"
    app:cardCornerRadius="16dp"
    app:cardElevation="2dp">

    <TextView
        android:id="@+id/nextBooking"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:padding="20dp"
        android:textColor="@color/white"
        android:textSize="17sp" />
</com.google.android.material.card.MaterialCardView>

<Button
    android:id="@+id/bookSlotButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="20dp"
    android:text="Book a slot" />

<Button
    android:id="@+id/nearbyStationsButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Nearby stations" />

<Button
    android:id="@+id/stationListButton"
    style="@style/Widget.SmartSolar.Button.Outlined"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="Station list" />

<Button
    android:id="@+id/bookingsButton"
    style="@style/Widget.SmartSolar.Button.Outlined"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:text="My bookings" />

<Button
    android:id="@+id/profileButton"

```

```

        style="@style/Widget.SmartSolar.Button.Outlined"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="My profile" />

<Button
    android:id="@+id/logoutButton"
    style="@style/Widget.SmartSolar.Button.Text"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="Log out" />
</LinearLayout>
</ScrollView>
</androidx.swiperefreshlayout.widget.SwipeRefreshLayout>

```

MobileApp/app/src/main/res/layout/activity_qr_details.xml M3 · 41 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/resultTitle"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textColor="@color/solar_green_dark"
        android:textSize="28sp"
        android:textStyle="bold" />

    <com.google.android.material.card.MaterialCardView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        app:cardCornerRadius="16dp"
        app:cardElevation="2dp">

        <TextView
            android:id="@+id/resultBody"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:lineSpacingExtra="6dp"
            android:padding="20dp"
            android:textColor="@color/black"
            android:textSize="17sp" />
    </com.google.android.material.card.MaterialCardView>

    <Button
        android:id="@+id/finaliseButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="20dp"
        android:text="Finalise job" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_register.xml M1 · 131 lines

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Create account"
            android:textColor="@color/solar_green_dark"
            android:textSize="28sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="Register with your NIC. A Backoffice officer activates new accounts before you can log in."
            android:textSize="14sp" />

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"

```

```

android:layout_marginTop="16dp"
app:cardCornerRadius="16dp"
app:cardElevation="2dp">

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="16dp">

    <com.google.android.material.textfield.TextInputLayout
        style="@style/Widget.SmartSolar.TextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="0dp"
        android:hint="NIC"
        app:helperText="12 digits, or 9 digits and V">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/registerNic"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textCapCharacters" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        style="@style/Widget.SmartSolar.TextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Username">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/registerUsername"
            android:layout_width="match_parent"
            android:layout_height="wrap_content" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        style="@style/Widget.SmartSolar.TextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Full name">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/registerFullName"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPersonName" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        style="@style/Widget.SmartSolar.TextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email (optional)">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/registerEmail"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textEmailAddress" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        style="@style/Widget.SmartSolar.TextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Phone (optional)">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/registerPhone"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="phone" />
    </com.google.android.material.textfield.TextInputLayout>

    <com.google.android.material.textfield.TextInputLayout
        style="@style/Widget.SmartSolar.TextInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Password (8+ characters)"
        app:passwordToggle="password_toggle">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/registerPassword"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPassword" />
    </com.google.android.material.textfield.TextInputLayout>
</LinearLayout>
</com.google.android.material.card.MaterialCardView>

<Button
    android:id="@+id/createAccountButton"
    android:layout_width="match_parent"

```

```

        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="Register" />
    </LinearLayout>
</ScrollView>

```

MobileApp/app/src/main/res/layout/activity_settings.xml M3 · 65 lines

```

<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="API settings"
            android:textColor="@color/solar_green_dark"
            android:textSize="28sp"
            android:textStyle="bold" />

        <TextView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="4dp"
            android:text="On a real phone use your computer's LAN address, for example http://192.168.1.10:5080/api/"
            android:textSize="14sp" />

        <com.google.android.material.card.MaterialCardView
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            app:cardCornerRadius="16dp"
            app:cardElevation="2dp">

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:orientation="vertical"
                android:padding="16dp">

                <com.google.android.material.textfield.TextInputLayout
                    style="@style/Widget.SmartSolar.TextInput"
                    android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                    android:layout_marginTop="0dp"
                    android:hint="API base URL">

                    <com.google.android.material.textfield.TextInputEditText
                        android:id="@+id/apiBaseUrl"
                        android:layout_width="match_parent"
                        android:layout_height="wrap_content"
                        android:inputType="textUri" />
                </com.google.android.material.textfield.TextInputLayout>
            </LinearLayout>
        </com.google.android.material.card.MaterialCardView>

        <Button
            android:id="@+id/saveSettingsButton"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:layout_marginTop="16dp"
            android:text="Save" />
    </LinearLayout>
</ScrollView>

```

MobileApp/app/src/main/res/layout/activity_splash.xml M1 · 38 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_green_dark"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="32dp">

    <ImageView
        android:layout_width="48dp"
        android:layout_height="48dp"
        android:contentDescription="@null"
        android:src="@drawable/sun_dot" />

    <TextView
        android:layout_width="wrap_content"

```

```

        android:layout_height="wrap_content"
        android:layout_marginTop="8dp"
        android:text="Smart Solar"
        android:textColor="@color/white"
        android:textSize="34sp"
        android:textStyle="bold" />

<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:text="Microgrid energy trading"
    android:textColor="#C8E6D5"
    android:textSize="17sp" />

<ProgressBar
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginTop="40dp"
    android:indeterminateTint="@color/solar_gold" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_station_list.xml M2 · 30 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/solar_background"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Nearby stations"
        android:textColor="@color/solar_green_dark"
        android:textSize="28sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/offlineLabel"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="4dp"
        android:textSize="13sp"
        android:visibility="gone" />

    <ListView
        android:id="@+id/stationList"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="12dp" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/activity_stations_map.xml M2 · 22 lines

```

<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/map"
        android:name="com.google.android.gms.maps.SupportMapFragment"
        android:layout_width="match_parent"
        android:layout_height="match_parent" />

    <TextView
        android:id="@+id/offlineLabel"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom"
        android:background="@color/white"
        android:padding="12dp"
        android:textColor="@color/black"
        android:textSize="14sp"
        android:visibility="gone" />
</FrameLayout>

```

MobileApp/app/src/main/res/layout/bottom_sheet_station.xml M2 · 50 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="vertical"
    android:padding="20dp">

    <TextView
        android:id="@+id/stationName"
        android:layout_width="match_parent"

```

```

        android:layout_height="wrap_content"
        android:textColor="@color/solar_green_dark"
        android:textSize="20sp"
        android:textStyle="bold" />

<TextView
    android:id="@+id/stationDistance"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:textSize="16sp" />

<TextView
    android:id="@+id/stationCapacity"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="8dp"
    android:textSize="16sp" />

<TextView
    android:id="@+id/stationSlots"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:textSize="16sp" />

<TextView
    android:id="@+id/stationFreeSlots"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="4dp"
    android:textSize="16sp" />

<Button
    android:id="@+id/bookHereButton"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="16dp"
    android:text="Book here" />
</LinearLayout>

```

MobileApp/app/src/main/res/layout/item_slot.xml M3 · 12 lines

```

<?xml version="1.0" encoding="utf-8"?>
<CheckedTextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:background="@drawable/bg_slot"
    android:checkMark="?android:attr/listChoiceIndicatorSingle"
    android:gravity="center_vertical"
    android:minHeight="64dp"
    android:paddingStart="16dp"
    android:paddingEnd="12dp"
    android:textColor="@color/black"
    android:textSize="16sp" />

```

MobileApp/app/src/main/res/layout/item_summary_row.xml M3 · 20 lines

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:paddingVertical="8dp">

    <TextView
        android:id="@+id/label"
        android:layout_width="110dp"
        android:layout_height="wrap_content"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/value"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:textIsSelectable="true" />
</LinearLayout>

```

MobileApp/app/src/main/res/values/colors.xml M1 · 20 lines

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="solar_green">#176B45</color>
    <color name="solar_green_dark">#0D4C31</color>
    <color name="solar_gold">#F5B700</color>
    <color name="solar_background">#F4F7F5</color>
    <color name="white">#FFFFFF</color>
    <color name="black">#17211C</color>
    <color name="solar_line">#DDE5E0</color>
    <color name="ink_secondary">#5B6B62</color>
    <color name="danger">#B42318</color>

```

```

<color name="status_pending">#8A6200</color>
<color name="status_pending_fill">#FF4D6</color>
<color name="status_approved">#1E7D4F</color>
<color name="status_approved_fill">#E3F4EA</color>
<color name="status_cancelled">#55606A</color>
<color name="status_cancelled_fill">#EEF0F2</color>
<color name="status_completed">#0BSED7</color>
<color name="status_completed_fill">#E7F0FF</color>
</resources>

```

MobileApp/app/src/main/res/values/strings.xml M1 · 4 lines

```

<?xml version="1.0" encoding="utf-8"?>
<resources>
  <string name="app_name">Smart Solar</string>
</resources>

```

MobileApp/app/src/main/res/values/styles.xml M1 · 80 lines

```

<?xml version="1.0" encoding="utf-8"?>
<!-- Shared look for buttons and text fields so every screen matches. -->
<resources>
  <style name="Widget.SmartSolar.Button" parent="Widget.MaterialComponents.Button">
    <item name="android:minHeight">56dp</item>
    <item name="android:textAllCaps">false</item>
    <item name="android:textSize">16sp</item>
    <item name="android:letterSpacing">0</item>
    <item name="cornerRadius">12dp</item>
  </style>

  <style name="Widget.SmartSolar.Button.Outlined" parent="Widget.MaterialComponents.Button.OutlinedButton">
    <item name="android:minHeight">56dp</item>
    <item name="android:textAllCaps">false</item>
    <item name="android:textSize">16sp</item>
    <item name="android:letterSpacing">0</item>
    <item name="cornerRadius">12dp</item>
    <item name="strokeColor">@color/solar_green</item>
  </style>

  <style name="Widget.SmartSolar.Button.Text" parent="Widget.MaterialComponents.Button.TextButton">
    <item name="android:textAllCaps">false</item>
    <item name="android:textSize">15sp</item>
    <item name="android:letterSpacing">0</item>
  </style>

  <style name="TextAppearance.SmartSolar.Title" parent="TextAppearance.MaterialComponents.Headline6">
    <item name="android:textSize">24sp</item>
    <item name="android:textStyle">bold</item>
    <item name="android:textColor">@color/solar_green_dark</item>
  </style>

  <style name="TextAppearance.SmartSolar.Section" parent="TextAppearance.MaterialComponents.Subtitle1">
    <item name="android:textSize">16sp</item>
    <item name="android:textColor">@color/black</item>
  </style>

  <style name="TextAppearance.SmartSolar.Body" parent="TextAppearance.MaterialComponents.Body1">
    <item name="android:textSize">16sp</item>
    <item name="android:textColor">@color/black</item>
  </style>

  <style name="TextAppearance.SmartSolar.Caption" parent="TextAppearance.MaterialComponents.Caption">
    <item name="android:textSize">13sp</item>
    <item name="android:textColor">@color/ink_secondary</item>
  </style>

  <style name="Widget.SmartSolar.Card" parent="Widget.MaterialComponents.CardView">
    <item name="cardElevation">0dp</item>
    <item name="strokeWidth">1dp</item>
    <item name="strokeColor">@color/solar_line</item>
    <item name="cardCornerRadius">16dp</item>
    <item name="cardBackgroundColor">@color/white</item>
  </style>

  <style name="ShapeAppearance.SmartSolar.Dialog" parent="">
    <item name="cornerFamily">rounded</item>
    <item name="cornerSize">16dp</item>
  </style>

  <style name="ThemeOverlay.SmartSolar.MaterialAlertDialog" parent="ThemeOverlay.MaterialComponents.MaterialAlertDialog">
    <item name="shapeAppearanceOverlay">@style/ShapeAppearance.SmartSolar.Dialog</item>
    <item name="buttonBarPositiveButtonStyle">@style/Widget.SmartSolar.Dialog.Button</item>
    <item name="buttonBarNegativeButtonStyle">@style/Widget.SmartSolar.Dialog.Button</item>
  </style>

  <style name="Widget.SmartSolar.Dialog.Button" parent="Widget.MaterialComponents.Button.TextButton.Dialog">
    <item name="android:textAllCaps">false</item>
    <item name="android:textColor">@color/solar_green</item>
  </style>

  <style name="Widget.SmartSolar.TextInput" parent="Widget.MaterialComponents.TextInputLayout.OutlinedBox">
    <item name="boxCornerRadiusTopStart">12dp</item>
    <item name="boxCornerRadiusTopEnd">12dp</item>

```

```

    <item name="boxCornerRadiusBottomStart">12dp</item>
    <item name="boxCornerRadiusBottomEnd">12dp</item>
    <item name="boxStrokeColor">@color/solar_green</item>
    <item name="android:layout_marginTop">12dp</item>
</style>
</resources>

```

MobileApp/app/src/main/res/values/themes.xml M1 · 21 lines

```

<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Theme.SmartSolar" parent="Theme.MaterialComponents.Light.NoActionBar">
        <item name="colorPrimary">@color/solar_green</item>
        <item name="colorPrimaryVariant">@color/solar_green_dark</item>
        <item name="colorSecondary">@color/solar_gold</item>
        <item name="colorSurface">@color/white</item>
        <item name="colorOnSurface">@color/black</item>
        <item name="colorError">@color/danger</item>
        <item name="android:colorBackground">@color/solar_background</item>
        <item name="android:textColor">@color/black</item>
        <item name="android:textColorSecondary">@color/ink_secondary</item>
        <item name="android:fontFamily">sans</item>
        <item name="android:statusBarColor">@color/solar_green_dark</item>
        <item name="android:navigationBarColor">@color/solar_green_dark</item>
        <item name="android:windowBackground">@color/solar_background</item>
        <item name="materialButtonStyle">@style/Widget.SmartSolar.Button</item>
        <item name="materialAlertDialogTheme">@style/ThemeOverlay.SmartSolar.MaterialAlertDialog</item>
        <item name="android:windowOptOutEdgeToEdgeEnforcement" tools:targetApi="35">true</item>
    </style>
</resources>

```